

DAFTAR FILE SOURCE CODE (.TS / .TSX)

1. **File** : badge.tsx
Folder: src\data-display
2. **File** : card-panel.tsx
Folder: src\data-display
3. **File** : grid.tsx
Folder: src\data-display
4. **File** : date-edit.tsx
Folder: src\data-entry
5. **File** : file-upload.tsx
Folder: src\data-entry
6. **File** : html-memo-box.tsx
Folder: src\data-entry
7. **File** : memo-box.tsx
Folder: src\data-entry
8. **File** : numeric-edit.tsx
Folder: src\data-entry
9. **File** : text-box.tsx
Folder: src\data-entry
10. **File** : confirmation-box.tsx
Folder: src\feedback
11. **File** : drawer.tsx
Folder: src\feedback
12. **File** : hint-box.tsx
Folder: src\feedback
13. **File** : image-viewer.tsx
Folder: src\feedback
14. **File** : message-box.tsx
Folder: src\feedback
15. **File** : pdf-viewer.tsx
Folder: src\feedback
16. **File** : popup-form.tsx
Folder: src\feedback
17. **File** : popup-menu.tsx
Folder: src\feedback
18. **File** : report-plugins-core.ts
Folder: src\feedback

19. **File** : report-workspace.tsx
Folder: src\feedback

20. **File** : skeleton.tsx
Folder: src\feedback

21. **File** : standalone-pagination.tsx
Folder: src\feedback

22. **File** : status-indicator.tsx
Folder: src\feedback

23. **File** : toast.tsx
Folder: src\feedback

24. **File** : report-plugins-core.ts
Folder: src\feedback\report-workspace

25. **File** : report-types.ts
Folder: src\feedback\report-workspace

26. **File** : dom-engine.ts
Folder: src\feedback\report-workspace\engines

27. **File** : avatar.tsx
Folder: src\general

28. **File** : button.tsx
Folder: src\general

29. **File** : label.tsx
Folder: src\general

30. **File** : progress.tsx
Folder: src\general

31. **File** : tabs.tsx
Folder: src\general

32. **File** : thumbnail.tsx
Folder: src\general

33. **File** : checkbox.tsx
Folder: src\selector

34. **File** : combo-box.tsx
Folder: src\selector

35. **File** : custom-dropdown-column-config.ts
Folder: src\selector

36. **File** : list-box.tsx
Folder: src\selector

37. **File** : radio-button.tsx
Folder: src\selector

38. **File** : slider.tsx
Folder: src\selector

39. **File** : switch.tsx
Folder: src\selector

40. **File** : button.tsx
Folder: src\ui

41. **File** : calendar.tsx
Folder: src\ui

42. **File** : checkbox.tsx
Folder: src\ui

43. **File** : command.tsx
Folder: src\ui

44. **File** : dialog.tsx
Folder: src\ui

45. **File** : input-group.tsx
Folder: src\ui

46. **File** : input.tsx
Folder: src\ui

47. **File** : label.tsx
Folder: src\ui

48. **File** : popover.tsx
Folder: src\ui

49. **File** : skeleton.tsx
Folder: src\ui

50. **File** : slider.tsx
Folder: src\ui

51. **File** : switch.tsx
Folder: src\ui

52. **File** : textarea.tsx
Folder: src\ui

53. **File** : tooltip.tsx
Folder: src\ui

FILE: badge.tsx
LOKASI: src\data-display

```
import React, { forwardRef } from "react";
import { cn } from "@lib/utils";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export type BadgeVariant = "brand" | "success" | "warning" | "danger" | "muted";

export interface BadgeProps extends React.HTMLAttributes<HTMLSpanElement> {
  variant?: BadgeVariant;
}

const variantStyles: Record<BadgeVariant, string> = {
  brand: "bg-brand-primary/10 text-brand-primary border-brand-primary/20",
  success: "bg-emerald-500/10 text-emerald-500 border-emerald-500/20",
  warning: "bg-amber-500/10 text-amber-500 border-amber-500/20",
  danger: "bg-red-500/10 text-red-500 border-red-500/20",
  muted: "bg-muted/50 text-muted-foreground border-transparent",
};

const baseStyles =
  "inline-flex items-center w-max text-xs font-medium tabular-nums rounded-full px-2.5 py-0.5 border transition-colors duration-200 ease-out";

export const Badge = forwardRef<HTMLSpanElement, BadgeProps>(
  ({ variant = "muted", className, children, ...props }, ref) => {
    const variantClass = variantStyles[variant];

    return (
      <span
        ref={ref}
        className={cn(baseStyles, variantClass, className)}
        {...props}>
        >
        {children}
      </span>
    );
  },
);

Badge.displayName = "Badge";

export default Badge;
```

FILE: card-panel.tsx
LOKASI: src\data-display

```
import React, { forwardRef } from "react";
import { cn } from "@lib/utils";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export interface CardPanelProps extends React.HTMLAttributes<HTMLDivElement> {
  isHoverable?: boolean;
}

export type CardPanelHeaderProps = React.HTMLAttributes<HTMLDivElement>;

export type CardPanelBodyProps = React.HTMLAttributes<HTMLDivElement>;

export type CardPanelFooterProps = React.HTMLAttributes<HTMLDivElement>;

// ðŸŽ“ BASE CONTAINER STYLES - Premium Surface v2.8 Glassmorphism
const containerBaseStyles =
  "bg-card/90 backdrop-blur-[var(--backdrop-blur)] border border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] rounded-[var(--radius-surface,12px)] transition-all duration-300 ease-out will-change-[box-shadow,border-color]";

const hoverableStyles =
  "hover:shadow-xl hover:border-[color-mix(in_oklch,var(--color-brand-primary)_40%,transparent)]";

const headerBaseStyles =
  "flex items-center justify-between px-6 py-4 border-b border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]";

const bodyBaseStyles = "px-6 py-4";

const footerBaseStyles =
  "flex items-center justify-end gap-3 px-6 py-4 border-t border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]";

// ðŸ’–ï¿½ï¿½ CARD PANEL ROOT - Parent Shell
export const CardPanelRoot = forwardRef<HTMLDivElement, CardPanelProps>(
  ({ isHoverable = false, className, children, ...props }, ref) => {
    return (
      <div
        ref={ref}
        className={cn(
          containerBaseStyles,
          isHoverable && hoverableStyles,
          className,
        )}
        {...props}
      >
        {children}
      </div>
    );
  },
);
```

```
CardPanelRoot.displayName = "CardPanelRoot";
```

```
// ðŸ“‹ CARD PANEL HEADER - Top Row Header Grid Slot
```

```
export const CardPanelHeader = forwardRef<HTMLDivElement, CardPanelHeaderProps>(
  ({ className, children, ...props }, ref) => {
    return (
      <div ref={ref} className={cn(headerBaseStyles, className)} {...props}>
        {children}
      </div>
    );
  },
);
```

```
CardPanelHeader.displayName = "CardPanelHeader";
```

```
// ðŸ“‘! CARD PANEL BODY - Core Data Payload Content Zone
```

```
export const CardPanelBody = forwardRef<HTMLDivElement, CardPanelBodyProps>(
  ({ className, children, ...props }, ref) => {
    return (
      <div ref={ref} className={cn(bodyBaseStyles, className)} {...props}>
        {children}
      </div>
    );
  },
);
```

```
CardPanelBody.displayName = "CardPanelBody";
```

```
// ðŸ”§ CARD PANEL FOOTER - Bottom Slot Toolbar Context Actions
```

```
export const CardPanelFooter = forwardRef<HTMLDivElement, CardPanelFooterProps>(
  ({ className, children, ...props }, ref) => {
    return (
      <div ref={ref} className={cn(footerBaseStyles, className)} {...props}>
        {children}
      </div>
    );
  },
);
```

```
CardPanelFooter.displayName = "CardPanelFooter";
```

```
// ðŸŽˆ COMPOUND COMPONENT EXPORT - Main Entry Point
```

```
export const CardPanel = Object.assign(CardPanelRoot, {
  Header: CardPanelHeader,
  Body: CardPanelBody,
  Footer: CardPanelFooter,
});
```

```
export default CardPanel;
```

FILE: grid.tsx
LOKASI: src\data-display

```
import React, {
  forwardRef,
  useState,
  useEffect,
  useRef,
  useCallback,
  useMemo,
} from "react";
import { ArrowUp, ArrowDown, ArrowUpDown } from "lucide-react";
import { cn } from "@lib/utils";
import { THEME_CONFIG } from "@config/theme-constants";

// ðŸ”’ THE SACRED OFFICIAL TANSTACK CORE & VIRTUAL IMPORTERS SECURED!
import {
  createTable,
  type ColumnDef,
  type RowSelectionState,
  type SortingState,
  type ColumnSizingState,
  type Row,
  type Column,
  getCoreRowModel,
  getSortedRowModel,
  getFilteredRowModel,
} from "@tanstack/table-core";
import { useVirtualizer } from "@tanstack/react-virtual";

import { TextBox } from "../data-entry/text-box";
import { NumericEdit } from "../data-entry/numeric-edit";
import { DateEdit } from "../data-entry/date-edit";
import { Checkbox } from "../selector/check-box";
import { ComboBox } from "../selector/combo-box";
import { HintBox } from "../feedback/hint-box";
import { StandalonePagination } from "../feedback/standalone-pagination";
import type { PopupMenuConfig } from "../feedback/popup-menu";

export interface GridColumnLayout {
  key: string;
  headerLabel: string;
  widthPercent: number;
  align?: "left" | "center" | "right";
  editType?: "text" | "numeric" | "date" | "checkbox" | "combo";
  editOptions?: Record<string, unknown>[];
  enableSorting?: boolean;
  enableFiltering?: boolean;
  // Cell-level hint function
  cellHint?: (value: unknown, recordRow: Record<string, unknown>) => string;
  // Icon hint for accessory icons
  iconHint?: string;
  // Cell-level popup menu override
  cellPopupMenu?: PopupMenuConfig;
}
```

```

export interface GridDataChangePayload {
  rowIndex: number;
  columnKey: string;
  oldValue: unknown;
  newValue: unknown;
}

export interface GridProps extends Omit<
  React.HTMLAttributes<HTMLDivElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  columns: GridColumnLayout[];
  data: Record<string, unknown>[];
  showSelectionCheckbox?: boolean;
  isEditable?: boolean;
  isRowMovable?: boolean;
  isColumnMovable?: boolean;
  rowIdentifierKey?: string;
  rowHeightPx?: number;
  activeRowKey?: string;
  disabled?: boolean;
  onRowClick?: (row: Record<string, unknown>, index: number) => void;
  onDataChange?: (
    updatedData: Record<string, unknown>[],
    changedInfo: GridDataChangePayload,
  ) => void;
  onRowMove?: (fromIndex: number, toIndex: number) => void;
  onColumnMove?: (fromIndex: number, toIndex: number) => void;
}

export const Grid = forwardRef<HTMLDivElement, GridProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      columns = [],
      data = [],
      showSelectionCheckbox = false,
      isEditable = false,
      isRowMovable = false,
      isColumnMovable = false,
      rowIdentifierKey = "id",
      rowHeightPx = THEME_CONFIG.layout.virtualRowHeight,
      activeRowKey,
      onRowClick,
      onDataChange,
      onRowMove,
      onColumnMove,
      className,
      ...props
    },
  ),

```



```

    ref,
  ) => {
    const parentRef = useRef<HTMLDivElement>(null);
    const tableRef = useRef<HTMLTableElement>(null);
    const [sorting, setSorting] = useState<SortingState>([]);
    const [columnFilters, setColumnFilters] = useState<
      import("@tanstack/table-core").ColumnFiltersState
    >([]);
    const [globalFilter, setGlobalFilter] = useState<string>("");
    const [columnOrder, setColumnOrder] = useState<string[]>([]);
    const [columnVisibility, setColumnVisibility] = useState<
      Record<string, boolean>
    >({});
    const [rowSelection, setRowSelection] = useState<RowSelectionState>({});
    const [editingCell, setEditingCell] = useState<{
      rowIndex: number;
      columnKey: string;
    } | null>(null);
    const [editValue, setEditValue] = useState<unknown>("");
    const [columnSizing, setColumnSizing] = useState<ColumnSizingState>({});
    const [isResizing, setIsResizing] = useState<{
      columnKey: string;
      startX: number;
      startWidth: number;
    } | null>(null);
    const [draggedColumn, setDraggedColumn] = useState<string | null>(null);
    const [draggedRow, setDraggedRow] = useState<number | null>(null);
    const [filterInputs, setFilterInputs] = useState<Record<string, string>>(
      {},
    );
  };

  const columnDefs = useMemo(() : ColumnDef<Record<string, unknown>>[] => {
    const defs: ColumnDef<Record<string, unknown>>[] = [];

    if (showSelectionCheckbox) {
      defs.push({
        id: "__selection__",
        header: ({ table }) => (
          <Checkbox
            checked={table.getIsAllRowsSelected()}
            isIndeterminate={table.getIsSomeRowsSelected()}
            onChange={(checked) => table.toggleAllRowsSelected(!checked)}
            className="h-4 w-4"
          />
        ),
        cell: ({ row }) => (
          <Checkbox
            checked={row.getIsSelected()}
            onChange={(checked) => row.toggleSelected(!checked)}
            className="h-4 w-4"
          />
        ),
        size: 48,
        enableSorting: false,
        enableColumnFilter: false,
        enableResizing: false,
      });
    }
  });
}

```

```

columns.forEach((col) => {
  defs.push({
    id: col.key,
    accessorKey: col.key,
    header: col.headerLabel,
    size: Math.max(
      80,
      (col.widthPercent / 100) * THEME_CONFIG.layout.desktopMaxWidth,
    ),
    minSize: 60,
    maxSize: 800,
    enableSorting: col.enableSorting !== false,
    enableColumnFilter: col.enableFiltering !== false,
    enableResizing: true,
    enableHiding: true,
  });
});

return defs;
}, [columns, showSelectionCheckbox]);

const table = useMemo(() => {
  const tableInstance = createTable<Record<string, unknown>>({
    data,
    columns: columnDefs,
    state: {
      sorting,
      columnFilters,
      globalFilter,
      columnOrder,
      columnVisibility,
      rowSelection,
      columnSizing,
    },
    onSortingChange: setSorting,
    onColumnFiltersChange: setColumnFilters,
    onGlobalFilterChange: setGlobalFilter,
    onColumnOrderChange: setColumnOrder,
    onColumnVisibilityChange: setColumnVisibility,
    onRowSelectionChange: setRowSelection,
    onColumnSizingChange: setColumnSizing,
    getRowId: (row) =>
      String(row[rowIdentifierKey] ?? row.id ?? Math.random()),
    manualSorting: true,
    manualFiltering: true,
    enableRowSelection: true,
    getCoreRowModel: getCoreRowModel(),
    getSortedRowModel: getSortedRowModel(),
    getFilteredRowModel: getFilteredRowModel(),
    onStateChange: () => {},
    renderFallbackValue: () => null,
  });
  return tableInstance;
}, [
  data,
  columnDefs,
  sorting,

```

```

    columnFilters,
    globalFilter,
    columnOrder,
    columnVisibility,
    rowSelection,
    columnSizing,
    rowIdentifierKey,
  ]);

const virtualizer = useVirtualizer({
  count: table.getRowModel().rows.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => rowHeightPx,
  overscan: 5,
});

// Debounce filter inputs for TanStack table (300ms delay)
useEffect(() => {
  const handlers: Record<string, ReturnType<typeof setTimeout>> = {};
  Object.entries(filterInputs).forEach(([key, value]) => {
    handlers[key] = setTimeout(() => {
      table.setColumnFilters((prev) => {
        const existing = prev.find((f) => f.id === key);
        if (value === "") {
          return prev.filter((f) => f.id !== key);
        }
        if (existing) {
          return prev.map((f) => (f.id === key ? { ...f, value } : f));
        }
        return [...prev, { id: key, value }];
      });
    }, 300);
  });
  return () => {
    Object.values(handlers).forEach((handler) => clearTimeout(handler));
  };
}, [filterInputs, table]);

const handleEditChange = useCallback((newValue: unknown) => {
  setEditValue(newValue);
}, []);

const handleEditBlur = useCallback(
  (rowIndex: number, columnKey: string, oldValue: unknown) => {
    if (
      editingCell &&
      editingCell.rowIndex === rowIndex &&
      editingCell.columnKey === columnKey
    ) {
      if (editValue !== oldValue) {
        const updatedData = [...data];
        updatedData[rowIndex] = {
          ...updatedData[rowIndex],
          [columnKey]: editValue,
        };
      }

      onDataChange?.(updatedData, {
        rowIndex,

```

```

        columnKey,
        oldValue,
        newValue: editValue,
      });
    }
    setEditingCell(null);
    setEditValue("");
  }
},
[editingCell, editValue, data, onDataChange],
);

const handleKeyDown = useCallback(
  (
    e: React.KeyboardEvent,
    rowIndex: number,
    columnKey: string,
    value: unknown,
  ) => {
    if (e.key === "Enter" && !e.shiftKey) {
      e.preventDefault();
      if (!isEditable) return;
      setEditingCell({ rowIndex, columnKey });
      setEditValue(value);
    } else if (e.key === "Escape" && editingCell) {
      setEditingCell(null);
      setEditValue("");
    } else if (e.key === "Tab") {
      if (editingCell) {
        handleEditBlur(
          editingCell.rowIndex,
          editingCell.columnKey,
          data[editingCell.rowIndex]?.[editingCell.columnKey] as unknown,
        );
      }
    }
  },
  [isEditable, editingCell, data, handleEditBlur],
);

const handleClick = useCallback(
  (row: Record<string, unknown>, index: number) => {
    onRowClick?.(row, index);
  },
  [onRowClick],
);

const handleContextMenu = useCallback(
  (e: React.MouseEvent, row: Record<string, unknown>) => {
    e.preventDefault();
    popupMenu?.trigger(e, row);
  },
  [popupMenu],
);

const handleSort = useCallback(
  (column: Column<Record<string, unknown>>) => {
    const isSorted = column.getIsSorted();

```

```

    column.toggleSorting(
      isSorted === "asc" ? false : isSorted === "desc" ? undefined : true,
    );
  },
  [],
);

const handleFilterChange = useCallback(
  (columnKey: string, value: string) => {
    setFilterInputs((prev) => ({ ...prev, [columnKey]: value }));
  },
  [],
);

const handleColumnResizeStart = useCallback(
  (e: React.MouseEvent, columnKey: string) => {
    e.preventDefault();
    e.stopPropagation();
    const column = table.getColumn(columnKey);
    if (column) {
      setIsResizing({
        columnKey,
        startX: e.clientX,
        startWidth: column.getSize(),
      });
    }
  },
  [table],
);

const handleColumnResize = useCallback(
  (e: MouseEvent) => {
    if (isResizing) {
      const newWidth = Math.max(
        60,
        isResizing.startWidth + (e.clientX - isResizing.startX),
      );
      table.setColumnSizing((prev) => ({
        ...prev,
        [isResizing.columnKey]: newWidth,
      }));
    }
  },
  [isResizing, table],
);

const handleColumnResizeEnd = useCallback(() => {
  setIsResizing(null);
}, []);

const handleDragStart = useCallback(
  (e: React.DragEvent, columnKey: string) => {
    if (!isColumnMovable) return;
    setDraggedColumn(columnKey);
    e.dataTransfer.effectAllowed = "move";
  },
  [isColumnMovable],
);

```

```

const handleDragOver = useCallback((e: React.DragEvent) => {
  e.preventDefault();
  e.dataTransfer.dropEffect = "move";
}, []);

const handleDrop = useCallback(
  (e: React.DragEvent, targetColumnKey: string) => {
    e.preventDefault();
    if (draggedColumn && draggedColumn !== targetColumnKey) {
      const columns = table.getAllColumns();
      const fromIndex = columns.findIndex(
        (c: Column<Record<string, unknown>>) => c.id === draggedColumn,
      );
      const toIndex = columns.findIndex(
        (c: Column<Record<string, unknown>>) => c.id === targetColumnKey,
      );
      if (fromIndex !== -1 && toIndex !== -1) {
        const newOrder = [...columnOrder];
        const [removed] = newOrder.splice(fromIndex, 1);
        newOrder.splice(toIndex, 0, removed);
        setColumnOrder(newOrder);
        onColumnMove?.(fromIndex, toIndex);
      }
    }
    setDraggedColumn(null);
  },
  [draggedColumn, columnOrder, table, onColumnMove],
);

const handleRowDragStart = useCallback(
  (e: React.DragEvent, rowIndex: number) => {
    if (!isRowMovable) return;
    setDraggedRow(rowIndex);
    e.dataTransfer.effectAllowed = "move";
  },
  [isRowMovable],
);

const handleRowDragOver = useCallback((e: React.DragEvent) => {
  e.preventDefault();
  e.dataTransfer.dropEffect = "move";
}, []);

const handleRowDrop = useCallback(
  (e: React.DragEvent, targetRowIndex: number) => {
    e.preventDefault();
    if (draggedRow !== null && draggedRow !== targetRowIndex) {
      onRowMove?.(draggedRow, targetRowIndex);
    }
    setDraggedRow(null);
  },
  [draggedRow, onRowMove],
);

useEffect(() => {
  const handleMouseMove = (e: MouseEvent) => handleColumnResize(e);
  const handleMouseUp = () => handleColumnResizeEnd();

```

```

    if (isResizing) {
        window.addEventListener("mousemove", handleMouseMove);
        window.addEventListener("mouseup", handleMouseUp);
    }

    return () => {
        window.removeEventListener("mousemove", handleMouseMove);
        window.removeEventListener("mouseup", handleMouseUp);
    };
}, [isResizing, handleColumnResize, handleColumnResizeEnd]);

const sortedColumns = useMemo(() => {
    const allColumns = table.getAllColumns();
    if (columnOrder.length > 0) {
        return columnOrder
            .map((id) =>
                allColumns.find(
                    (c: Column<Record<string, unknown>>) => c.id === id,
                ),
            )
            .filter(Boolean) as Column<Record<string, unknown>>[];
    }
    return allColumns;
}, [table, columnOrder]);

const headerGroups = table.getHeaderGroups();
const rows = table.getRowModel().rows;

const getAlignmentClass = (align?: string) => {
    switch (align) {
        case "center":
            return "text-center";
        case "right":
            return "text-right";
        default:
            return "text-left";
    }
};

const renderCellEditor = (
    column: Column<Record<string, unknown>>,
    row: Row<Record<string, unknown>>,
    value: unknown,
) => {
    const colConfig = columns.find((c) => c.key === column.id);
    if (!colConfig || !colConfig.editType) return null;

    const isEditing =
        editingCell?.rowIndex === row.index &&
        editingCell?.columnKey === column.id;

    if (!isEditing) return null;

    const commonProps = {
        value: editValue,
        onChange: handleEditChange,
        onBlur: () => handleEditBlur(row.index, column.id, value),
    };

```

```

    onKeyDown: (e: React.KeyboardEvent) =>
        handleKeyDown(e, row.index, column.id, value),
    className: "w-full h-full min-h-[36px]",
    error: error,
  });

  switch (colConfig.editType) {
    case "text":
      return <TextBox {...commonProps} value={String(editValue ?? "")} />;
    case "numeric":
      return (
        <NumericEdit {...commonProps} value={Number(editValue ?? 0)} />
      );
    case "date":
      return <DateEdit {...commonProps} value={String(editValue ?? "")} />;
    case "checkbox":
      return (
        <Checkbox
          checked={editValue as boolean}
          onChange={(checked) => handleEditChange(checked)}
          className="h-4 w-4"
        />
      );
    case "combo":
      return (
        <ComboBox
          options={colConfig.editOptions || []}
          value={String(editValue ?? "")}
          onChange={handleEditChange}
          className="w-full h-full min-h-[36px]"
        />
      );
    default:
      return null;
  }
};

const renderCellContent = (
  column: Column<Record<string, unknown>>,
  row: Row<Record<string, unknown>>,
) => {
  const value = row.getValue(column.id);
  const colConfig = columns.find((c) => c.key === column.id);
  const isEditing =
    editingCell?.rowIndex === row.index &&
    editingCell?.columnKey === column.id;

  if (isEditing) {
    return renderCellEditor(column, row, value);
  }

  if (colConfig?.editType === "checkbox") {
    return (
      <Checkbox
        checked={value as boolean}
        disabled={!isEditable}
        onChange={
          isEditable

```



```

    ? (checked) => {
      const updatedData = [...data];
      updatedData[row.index] = {
        ...updatedData[row.index],
        [column.id]: checked,
      };
      onDataChange?.(updatedData, {
        rowIndex: row.index,
        columnKey: column.id,
        oldValue: value,
        newValue: checked,
      });
    }
    : undefined
  }
  className="h-4 w-4"
/>
);
}

// Cell-level hint and icon hint rendering
const cellHintText = colConfig?.cellHint
  ? colConfig.cellHint(value, row.original)
  : undefined;
const iconHintText = colConfig?.iconHint;

return (
  <div
    className={cn(
      "relative truncate",
      getAlignmentClass(colConfig?.align),
    )}
  >
    {cellHintText && (
      <HintBox
        content={cellHintText}
        position="top"
        className="inline-block"
      >
        <span className="underline underline-offset-2 decoration-dotted decoration-text-muted
cursor-help">
          {String(value ?? "")}
        </span>
      </HintBox>
    )}
    {!cellHintText && <span>{String(value ?? "")}</span>}
    {iconHintText && (
      <HintBox
        content={iconHintText}
        position="top"
        className="ml-1 inline-block"
      >
        <span
          className="text-text-muted cursor-help"
          aria-label={iconHintText}
        >
          â„¹i
        </span>
      </HintBox>
    )}
  </div>
);

```

```

        </HintBox>
      )}
    </div>
  );
};

return (
  <div
    ref={(el) => {
      parentRef.current = el;
      if (ref) {
        if (typeof ref === "function") ref(el);
        else ref.current = el;
      }
    }}
    className={cn(
      "relative w-full overflow-hidden rounded-lg border border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)] bg-card",
      className,
    )}
    {...props}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}

    {label && (
      <div className="px-4 py-2 border-b border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)] bg-card/40">
        <span className="font-semibold text-text-main">{label}</span>
      </div>
    )}

    <div
      className="relative overflow-auto"
      style={{ maxHeight: THEME_CONFIG.layout.maxGridHeight }}
      onKeyDown={(e) => {
        if (e.key === "ArrowDown" || e.key === "ArrowUp") {
          e.preventDefault();
        }
      }}
    >
      <table
        ref={tableRef}
        className="w-full border-collapse"
        style={{
          minWidth: sortedColumns.reduce(
            (sum: number, col: Column<Record<string, unknown>>) =>
              sum + (col.getSize() || 100),
            0,
          ),
        }}
        role="grid"
      >
        <thead className="sticky top-0 z-10">
          {headerGroups.map(

```

```

(
  headerGroup: import("@tanstack/table-core").HeaderGroup<
    Record<string, unknown>
  >,
) => (
  <tr
    key={headerGroup.id}
    className="bg-card/40 border-b border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)]"
  >
    {headerGroup.headers.map(
      (
        header: import("@tanstack/table-core").Header<
          Record<string, unknown>,
          unknown
        >,
      ) => {
        const column = header.column;
        const isSorted = column.getIsSorted();
        const canSort = column.getCanSort();
        const isResizing = column.getIsResizing();
        const isDragged = draggedColumn === column.id;

        return (
          <th
            key={column.id}
            className={cn(
              "relative px-3 py-2 font-semibold text-text-main whitespace-nowrap",
              "border-r border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)]",
              "last:border-r-0",
              "select-none",
              isDragged && "opacity-50",
              isResizing && "bg-brand-primary/10",
            )}
            style={{ width: column.getSize() }}
            draggable={
              isColumnMovable && column.id !== "__selection__"
            }
            onDragStart={(e) => handleDragStart(e, column.id)}
            onDragOver={handleDragOver}
            onDrop={(e) => handleDrop(e, column.id)}
            onMouseDown={(e) => {
              if (
                e.target === e.currentTarget &&
                column.getCanResize()
              ) {
                handleColumnResizeStart(e, column.id);
              }
            }}
          >
            <div className="flex items-center gap-2">
              {column.id === "__selection__" ? (
                typeof header.column.columnDef.header ===
                "function" ? (
                  header.column.columnDef.header({
                    table,
                    column: header.column,

```

```

        header: header.column.columnDef.header,
      } as unknown as import("@tanstack/table-core").HeaderContext<
        Record<string, unknown>,
        unknown
      >)
    ) : (
      header.column.columnDef.header
    )
  ) : (
    <>
    {canSort && (
      <div
        onClick={() => handleSort(column)}
        className={cn(
          "flex items-center gap-1 p-1 rounded transition-all duration-
100 active:scale-95 hover:bg-[color-mix(in_oklch,var(--color-brand-primary)_10%,transparent)] cursor-
pointer",
          isSorted && "text-brand-primary",
        )}
        aria-label={`Sort by ${column.id}`}
        role="button"
        tabIndex={0}
        onKeyDown={(e) => {
          if (
            e.key === "Enter" ||
            e.key === " "
          ) {
            e.preventDefault();
            handleSort(column);
          }
        }}
      >
      {typeof header.column.columnDef.header ===
"function"
        ? header.column.columnDef.header({
            table,
            column: header.column,
            header:
              header.column.columnDef.header,
          } as unknown as import("@tanstack/table-core").HeaderContext<
            Record<string, unknown>,
            unknown
          >)
            : header.column.columnDef.header}
      {isSorted === "asc" && (
        <ArrowUp className="h-3.5 w-3.5" />
      )}
      {isSorted === "desc" && (
        <ArrowDown className="h-3.5 w-3.5" />
      )}
      {!isSorted && (
        <ArrowUpDown className="h-3.5 w-3.5 text-text-muted" />
      )}
    </div>
  )}
  {!canSort &&
    (typeof header.column.columnDef.header ===
      "function"

```

```

      ? header.column.columnDef.header({
        table,
        column: header.column,
        header:
          header.column.columnDef.header,
      }) as unknown as import("@tanstack/table-core").HeaderContext<
        Record<string, unknown>,
        unknown
      >)
      : header.column.columnDef.header))
    </>
  })
  {column.getCanFilter() &&
    column.id !== "__selection__" && (
      <div className="relative flex-1 min-w-0">
        <input
          type="text"
          placeholder="Filter..."
          value={filterInputs[column.id] || ""}
          onChange={(e) =>
            handleFilterChange(
              column.id,
              e.target.value,
            )
          }
          onClick={(e) => e.stopPropagation()}
          className="w-full px-2 py-1 text-xs bg-background border border-
[color-mix(in_oklch,var(--color-border)_40%,transparent)] rounded focus:outline-none focus:ring-1
focus:ring-brand-primary"
        />
      </div>
    )}
</div>
{column.getCanResize() && (
  <div
    className="absolute right-0 top-0 h-full w-1 cursor-col-resize hover:w-
2 hover:bg-brand-primary/30 transition-all"
    onMouseDown={(e) =>
      handleColumnResizeStart(e, column.id)
    }
  />
)}
</th>
);
},
)}
</tr>
),
)}
</thead>
<tbody>
  {virtualizer.getVirtualItems().map((virtualRow) => {
    const row = rows[virtualRow.index];
    const isSelected = row.getIsSelected();
    const isActive =
      activeRowKey &&
      row.getValue(rowIdentifierKey) === activeRowKey;
    const isEven = virtualRow.index % 2 === 0;

```

```

return (
  <tr
    key={row.id}
    className={cn(
      "transition-colors duration-150",
      "hover:bg-brand-primary/3",
      isSelected &&
        "bg-brand-primary/5 text-text-main font-semibold border-l-2 border-brand-
primary",
      isActive && "bg-brand-primary/10",
      isEven &&
        "bg-[color-mix(in_oklch,var(--color-border)_5%,transparent)]",
        "border-b border-[color-mix(in_oklch,var(--color-border)_20%,transparent)]",
        "last:border-b-0",
    )}
    style={{
      height: rowHeightPx,
      transform: `translateY(${virtualRow.start}px)`,
    }}
    draggable={isRowMovable}
    onDragStart={(e) => handleRowDragStart(e, virtualRow.index)}
    onDragOver={handleRowDragOver}
    onDrop={(e) => handleRowDrop(e, virtualRow.index)}
    onClick={() =>
      handleRowClick(row.original, virtualRow.index)
    }
    onContextMenu={(e) => handleContextMenu(e, row.original)}
  >
    {sortedColumns.map((column) => (
      <td
        key={column.id}
        className={cn(
          "px-3 py-2 whitespace-nowrap overflow-hidden",
          "border-r border-[color-mix(in_oklch,var(--color-border)_20%,transparent)]",
          "last:border-r-0",
          getAlignmentClass(
            columns.find((c) => c.key === column.id)?.align,
          ),
        )}
        style={{ width: column.getSize() }}
        onContextMenu={(e) => {
          const colConfig = columns.find(
            (c) => c.key === column.id,
          );
          if (colConfig?.cellPopupMenu) {
            e.stopPropagation();
            colConfig.cellPopupMenu.trigger(e, row.original);
          }
        }}
      >
        {renderCellContent(column, row)}
      </td>
    )}}
  </tr>
);
)}}
{rows.length === 0 && (

```

```

        <tr>
          <td
            colSpan={sortedColumns.length}
            className="px-4 py-8 text-center text-text-muted"
          >
            No data available
          </td>
        </tr>
      </tbody>
    </table>
  </div>

  <div className="border-t border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] p-3
flex justify-between items-center bg-card/40 shrink-0">
    <div className="text-xs text-text-muted">
      Showing {table.getRowModel().rows.length} records
    </div>
    <StandalonePagination
      currentPage={table.getState().pagination.pageIndex + 1}
      totalPages={table.getPageCount()}
      onPageChange={({targetPage}) => table.setPageIndex(targetPage - 1)}
      disabled={props.disabled}
    />
  </div>

  {error && (
    <div className="px-4 py-2 border-t border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)] bg-destructive/5">
      <span className="text-sm text-destructive">{error}</span>
    </div>
  )}
</div>
);
},
);
Grid.displayName = "Grid";

```

FILE: date-edit.tsx
LOKASI: src\data-entry

```
import React, {
  forwardRef,
  useRef,
  useCallback,
  useState,
  useEffect,
} from "react";
import { LucideIcon, Calendar } from "lucide-react";
import { cn } from "@lib/utils";
import { Button } from "../general/button";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";
import {
  Popover,
  PopoverTrigger,
  PopoverContent,
} from "@components/ui/popover";
import { Calendar as CalendarComponent } from "@components/ui/calendar";

export interface DateEditInnerIconConfig {
  icon: LucideIcon;
  className?: string;
}

export interface DateEditActionButtonConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>, value: string) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface DateEditProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  dbFormat?: string;
  innerIcons?: DateEditInnerIconConfig[];
  actionButtons?: DateEditActionButtonConfig[];
  onChange?: (value: string, dbValue: string) => void;
}

const DEFAULT_DB_FORMAT = "YYYY-MM-DD";

const formatDateForDb = (dateString: string, dbFormat: string): string => {
  if (!dateString) return "";
  const date = new Date(dateString);
  if (isNaN(date.getTime())) return dateString;
}
```



```

const year = date.getFullYear();
const month = String(date.getMonth() + 1).padStart(2, "0");
const day = String(date.getDate()).padStart(2, "0");

return dbFormat
  .replace("YYYY", String(year))
  .replace("MM", month)
  .replace("DD", day);
};

const parseDateFromDb = (dbValue: string, dbFormat: string): string => {
  if (!dbValue) return "";
  const yearMatch = dbFormat.indexOf("YYYY");
  const monthMatch = dbFormat.indexOf("MM");
  const dayMatch = dbFormat.indexOf("DD");

  if (yearMatch === -1 || monthMatch === -1 || dayMatch === -1) return dbValue;

  const year = dbValue.substring(yearMatch, yearMatch + 4);
  const month = dbValue.substring(monthMatch, monthMatch + 2);
  const day = dbValue.substring(dayMatch, dayMatch + 2);

  return `${year}-${month}-${day}`;
};

export const DateEdit = forwardRef<HTMLInputElement, DateEditProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      dbFormat = DEFAULT_DB_FORMAT,
      innerIcons = [],
      actionButtons = [],
      className,
      disabled,
      required,
      placeholder,
      value,
      defaultValue,
      onChange,
      onBlur,
      ...props
    },
    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const hasError = Boolean(error);
    const [displayValue, setDisplayValue] = useState<string>("");
    const [isFocused, setIsFocused] = useState(false);
    const [isOpen, setIsOpen] = useState(false);
    const [selectedDate, setSelectedDate] = useState<Date | undefined>();

    const handleRef = useCallback(
      (node: HTMLInputElement | null) => {
        inputRef.current = node;
        if (ref) {

```

```

        if (typeof ref === "function") {
            ref(node);
        } else {
            ref.current = node;
        }
    }
},
[ref],
);

useEffect(() => {
    const initialValue =
        value !== undefined
        ? String(value)
        : defaultValue !== undefined
        ? String(defaultValue)
        : "";
    if (initialValue) {
        const parsed = parseDateFromDb(initialValue, dbFormat);
        setDisplayValue(parsed);
        setSelectedDate(new Date(parsed));
    } else {
        setDisplayValue("");
        setSelectedDate(undefined);
    }
}, [value, defaultValue, dbFormat]);

const handleBlur = useCallback(
    (e: React.FocusEvent<HTMLInputElement>) => {
        setIsFocused(false);
        onBlur?.(e);
    },
    [onBlur],
);

const handleFocus = useCallback(() => {
    setIsFocused(true);
}, []);

const handleCalendarSelect = useCallback(
    (date: Date | undefined) => {
        if (date) {
            const formatted = date.toISOString().split("T")[0];
            setDisplayValue(formatted);
            setSelectedDate(date);
            const dbValue = formatDateForDb(formatted, dbFormat);
            onChange?.(formatted, dbValue);
        } else {
            setDisplayValue("");
            setSelectedDate(undefined);
            onChange?.("", "");
        }
        setIsOpen(false);
        inputRef.current?.focus();
    },
    [dbFormat, onChange],
);

```

```
const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLInputElement>) => {
    if (e.key === "Enter" || e.key === " ") {
      e.preventDefault();
      setIsOpen(true);
    } else if (e.key === "Escape") {
      setIsOpen(false);
    }
  },
  [],
);

const limitedInnerIcons = innerIcons.slice(0, 4);
const limitedActionButtons = actionButtons.slice(0, 4);

return (
  <div
    className={cn("w-full", className)}
    onContextMenu={(e) => popupMenu?.trigger(e)}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
    <div className="flex items-center w-full gap-2">
      <Popover open={isOpen} onOpenChange={setIsOpen}>
        <PopoverTrigger asChild>
          <div
            className={cn(
              "relative flex items-center flex-1 min-w-0",
              "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
              "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
              "rounded-surface",
              "text-text-main placeholder:text-muted-foreground/60",
              "focus-within:ring-2 focus-within:ring-amber-500/20 focus-within:border-amber-500",
              "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
              "transition-all duration-200 ease-out",
              hasError &&
                "border-destructive/50 focus-within:ring-destructive/50 focus-within:border-destructive/50",
              disabled && "bg-muted/50",
            )}
          </div>
        </PopoverTrigger>
      </div>
    </div>
  );
```

```

        isFocused && "ring-2 ring-amber-500/20 border-amber-500",
      )}
    >
    <input
      ref={handleRef}
      type="text"
      readOnly
      className={cn(
        "flex-1 bg-transparent border-none outline-none",
        "px-4 py-2.5",
        "text-text-main placeholder:text-muted-foreground/60",
        "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
        "w-full min-w-0",
        limitedInnerIcons.length > 0 ? "pr-12" : "pr-12",
      )}
      disabled={disabled}
      required={required}
      placeholder={placeholder}
      value={displayValue}
      onBlur={handleBlur}
      onFocus={handleFocus}
      onKeyDown={handleKeyDown}
      {...props}
    />
    <div className="absolute right-3 flex items-center gap-1">
      <Calendar
        className="w-4 h-4 text-muted-foreground/60"
        aria-hidden="true"
      />
      {limitedInnerIcons.length > 0 && (
        <>
          {limitedInnerIcons.map((iconConfig, index) => (
            <span
              key={index}
              className={cn(
                "flex items-center justify-center",
                "w-5 h-5",
                "text-muted-foreground/60",
                iconConfig.className,
              )}
              aria-hidden="true"
            >
              <iconConfig.icon
                className="w-4 h-4"
                aria-hidden="true"
              />
            </span>
          )}}
        </>
      )}
    </div>
  </PopoverTrigger>
  <PopoverContent className="w-auto p-0" align="start" sideOffset={5}>
    <CalendarComponent
      mode="single"
      selected={selectedDate}
      onSelect={handleCalendarSelect}
    />
  </PopoverContent>

```

```

        className="bg-popover border border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)] rounded-lg shadow-xl"
      />
    </PopoverContent>
  </Popover>
  {limitedActionButtons.length > 0 && (
    <div className="flex items-center gap-1.5 shrink-0">
      {limitedActionButtons.map((action, index) => (
        <Button
          key={index}
          variant="ghost"
          size="icon"
          disabled={disabled || action.disabled}
          onClick={(e) => action.onClick(e, displayValue)}
          className="active:scale-95 transition-all duration-100"
          aria-label={action.tooltipText}
        >
          <action.icon className="w-4 h-4" />
        </Button>
      )}}
    </div>
  )}
</div>
{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>
);
},
);

DateEdit.displayName = "DateEdit";

```

FILE: file-upload.tsx
LOKASI: src\data-entry

```
import React, {
  forwardRef,
  useState,
  useRef,
  useCallback,
  useImperativeHandle,
} from "react";
import { UploadCloud, File, Trash2, LucideIcon } from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";
import { Button } from "../general/button";

export interface FileUploadActionButtonConfig {
  icon: LucideIcon;
  onClick: (file: File) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface FileUploadProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  maxSizeInBytes?: number;
  acceptTypes?: string[];
  onChange?: (files: File[]) => void;
  actionButtons?: FileUploadActionButtonConfig[];
}

export interface FileUploadRef {
  clearFiles: () => void;
  getFiles: () => File[];
}

export interface UploadedFile {
  file: File;
  id: string;
  preview?: string;
}

export const FileUpload = forwardRef<FileUploadRef, FileUploadProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
```

```

    maxSizeInBytes = 10 * 1024 * 1024,
    acceptTypes = [],
    onChange,
    actionButtons = [],
    className,
    disabled,
    required,
    ...props
  },
  ref,
) => {
  const fileInputRef = useRef<HTMLInputElement>(null);
  const dropZoneRef = useRef<HTMLDivElement>(null);
  const hasError = Boolean(error);
  const [files, setFiles] = useState<UploadedFile[]>([]);
  const [isDragOver, setIsDragOver] = useState(false);
  const [isFocused, setIsFocused] = useState(false);

  useImperativeHandle(ref, () => ({
    clearFiles: () => {
      setFiles([]);
      if (fileInputRef.current) {
        fileInputRef.current.value = "";
      }
    },
    getFiles: () => files.map((f) => f.file),
  }));

  const formatFileSize = useCallback((bytes: number): string => {
    if (bytes === 0) return "0 Bytes";
    const k = 1024;
    const sizes = ["Bytes", "KB", "MB", "GB", "TB"];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + " " + sizes[i];
  }, []);

  const validateFile = useCallback(
    (file: File): string | null => {
      if (maxSizeInBytes && file.size > maxSizeInBytes) {
        return `File size exceeds maximum allowed size of ${formatFileSize(maxSizeInBytes)}`;
      }
      if (
        acceptTypes.length > 0 &&
        !acceptTypes.some((type) => file.type.match(type.replace("/*", ".*")))
      ) {
        return `File type ${file.type} is not allowed`;
      }
      return null;
    },
    [maxSizeInBytes, acceptTypes, formatFileSize],
  );

  const processFiles = useCallback(
    (fileList: FileList | File[]) => {
      const newFiles: UploadedFile[] = [];
      const validFiles: File[] = [];

      Array.from(fileList).forEach((file) => {

```

```

    const validationError = validateFile(file);
    if (validationError) {
      return;
    }

    const id = `${file.name}-${file.size}-${Date.now()}-${Math.random().toString(36).substr(2,
9)}}`;

    let preview: string | undefined;

    if (file.type.startsWith("image/")) {
      preview = URL.createObjectURL(file);
    }

    newFiles.push({ file, id, preview });
    validFiles.push(file);
  });

  if (newFiles.length > 0) {
    setFiles((prev) => [...prev, ...newFiles]);
    onChange?.(validFiles);
  }
},
[validateFile, onChange],
);

const handleDragOver = useCallback(
  (e: React.DragEvent<HTMLDivElement>) => {
    e.preventDefault();
    e.stopPropagation();
    if (!disabled) {
      setIsDragOver(true);
    }
  },
  [disabled],
);

const handleDragLeave = useCallback(
  (e: React.DragEvent<HTMLDivElement>) => {
    e.preventDefault();
    e.stopPropagation();
    if (!e.currentTarget.contains(e.relatedTarget as Node)) {
      setIsDragOver(false);
    }
  },
  [],
);

const handleDrop = useCallback(
  (e: React.DragEvent<HTMLDivElement>) => {
    e.preventDefault();
    e.stopPropagation();
    setIsDragOver(false);

    if (disabled) return;

    if (e.dataTransfer.files.length > 0) {
      processFiles(e.dataTransfer.files);
    }
  }

```



```

    },
    [disabled, processFiles],
  );

  const handleFileSelect = useCallback(
    (e: React.ChangeEvent<HTMLInputElement>) => {
      if (e.target.files && e.target.files.length > 0) {
        processFiles(e.target.files);
        e.target.value = "";
      }
    },
    [processFiles],
  );

  const handleFocus = useCallback(() => {
    setIsFocused(true);
  }, []);

  const handleBlur = useCallback(() => {
    setIsFocused(false);
  }, []);

  const removeFile = useCallback((fileId: string) => {
    setFiles((prev) => {
      const fileToRemove = prev.find((f) => f.id === fileId);
      if (fileToRemove?.preview) {
        URL.revokeObjectURL(fileToRemove.preview);
      }
      return prev.filter((f) => f.id !== fileId);
    });
  }, []);

  const limitedActionButtons = actionButtons.slice(0, 4);

  const openFileDialog = useCallback(() => {
    if (!disabled && fileInputRef.current) {
      fileInputRef.current.click();
    }
  }, [disabled]);

  return (
    <div
      className={cn("w-full", className)}
      onContextMenu={(e) => popupMenu?.trigger(e)}
    >
      {hint && (
        <HintBox content={hint} className="mb-1.5">
          <span className="text-sm text-text-muted" />
        </HintBox>
      )}
      {label && (
        <label
          className={cn(
            "block text-sm font-medium text-text-main mb-1.5",
            disabled && "opacity-50",
          )}
        >
          {label}
        </label>
      )}
    </div>
  );

```

```

      {required && (
        <span className="text-destructive ml-0.5" aria-hidden="true">
          *
        </span>
      )}
    </label>
  )}
<div
  ref={dropZoneRef}
  className={cn(
    "relative",
    "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
    "border-2 border-dashed border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
    "rounded-surface",
    "transition-all duration-200 ease-out",
    "focus-within:ring-2 focus-within:ring-amber-500/20 focus-within:border-amber-500 focus-
within:bg-amber-500/5",
    "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
    hasError &&
    "border-destructive/50 focus-within:ring-destructive/50 focus-within:border-
destructive/50",
    disabled && "bg-muted/50",
    isDragOver && "border-amber-500 bg-amber-500/5",
    isFocused && "ring-2 ring-amber-500/20 border-amber-500",
  )}
  onDragOver={handleDragOver}
  onDragLeave={handleDragLeave}
  onDrop={handleDrop}
  onClick={openFileDialog}
  onFocus={handleFocus}
  onBlur={handleBlur}
  tabIndex={disabled ? -1 : 0}
  role="button"
  aria-label={label || "File upload drop zone"}
  aria-disabled={disabled}
>
  <input
    ref={fileInputRef}
    type="file"
    className="absolute inset-0 w-full h-full opacity-0 cursor-pointer"
    disabled={disabled}
    required={required}
    accept={acceptTypes.join(",")}
    onChange={handleFileSelect}
    multiple
    {...props}
  />
<div
  className={cn(
    "flex flex-col items-center justify-center p-8",
    "text-center",
    files.length > 0 && "py-4",
  )}
>
  {files.length === 0 && (
    <>
      <UploadCloud
        className="w-10 h-10 text-muted-foreground/50 mb-3"

```

```

        aria-hidden="true"
      />
    <p className="text-text-main font-medium mb-1">
      Drag & drop files here, or click to browse
    </p>
    <p className="text-sm text-muted-foreground/60">
      {acceptTypes.length > 0
        ? `Accepted: ${acceptTypes.join(", ")}`
        : "All file types accepted"}
      {maxSizeInBytes &&
        ` â€¢ Max size: ${formatFileSize(maxSizeInBytes)}` }
    </p>
  </>
)}
{files.length > 0 && (
  <div className="w-full max-w-md space-y-2">
    {files.map((fileData) => (
      <div
        key={fileData.id}
        className={cn(
          "flex items-center justify-between p-3",
          "bg-background/50",
          "rounded-md",
          "border border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
          "transition-all duration-150",
        )}
      >
        <div className="flex items-center gap-3 min-w-0 flex-1">
          {fileData.preview ? (
            <img
              src={fileData.preview}
              alt={fileData.file.name}
              className="w-10 h-10 rounded-md object-cover"
            />
          ) : (
            <File
              className="w-10 h-10 text-muted-foreground/60 flex-shrink-0"
              aria-hidden="true"
            />
          )}
          <div className="min-w-0">
            <p className="text-sm font-medium text-text-main truncate">
              {fileData.file.name}
            </p>
            <p className="text-xs text-muted-foreground/60">
              {fileData.file.type || "Unknown type"} â€¢ {" "}
              {formatFileSize(fileData.file.size)}
            </p>
          </div>
        </div>
        <div className="flex items-center gap-1.5 shrink-0">
          {limitedActionButtons.length > 0 &&
            limitedActionButtons.map((action, index) => (
              <Button
                key={index}
                type="button"
                variant="ghost"
                size="icon"

```

```

        className="active:scale-95 transition-all duration-100"
        aria-label={action.tooltipText}
        disabled={disabled || action.disabled}
        onClick={(e) => {
            e.stopPropagation();
            action.onClick(fileData.file);
        }}
    >
        <action.icon
            className="w-4 h-4"
            aria-hidden="true"
        />
    </Button>
    )})
<Button
    type="button"
    variant="ghost"
    size="icon"
    className="active:scale-95 transition-all duration-100"
    aria-label="Remove file"
    disabled={disabled}
    onClick={(e) => {
        e.stopPropagation();
        removeFile(fileData.id);
    }}
    >
        <Trash2 className="w-4 h-4" aria-hidden="true" />
    </Button>
</div>
</div>
    )})
</div>
    )}
</div>
{error && (
    <p
        className={cn(
            "mt-1.5 text-sm",
            "text-destructive/90",
            "font-medium",
        )}
        role="alert"
    >
        {error}
    </p>
    )}
</div>
);
},
);

FileUpload.displayName = "FileUpload";

```

FILE: html-memo-box.tsx
LOKASI: src\data-entry

```
import React, {
  forwardRef,
  useImperativeHandle,
  useState,
  useEffect,
} from "react";
import {
  Bold,
  Italic,
  Underline as UnderlineIcon,
  List,
  LucideIcon,
} from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";
import { Button } from "../general/button";
import { useEditor, EditorContent } from "@tiptap/react";
import StarterKit from "@tiptap/starter-kit";
import UnderlineExtension from "@tiptap/extension-underline";

export interface HtmlMemoActionConfig {
  icon: LucideIcon;
  onClick: (htmlContent: string) => void;
  tooltipText?: string;
}

export interface HtmlMemoBoxProps extends Omit<
  React.HTMLAttributes<HTMLDivElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  defaultValue?: string;
  onChange?: (html: string) => void;
  actionIcons?: HtmlMemoActionConfig[];
  disabled?: boolean;
  required?: boolean;
}

export interface HtmlMemoBoxRef {
  getHtml: () => string;
  setHtml: (html: string) => void;
  focus: () => void;
}

export const HtmlMemoBox = forwardRef<HtmlMemoBoxRef, HtmlMemoBoxProps>(
  (
    {
      label,
```

```

    error,
    hint,
    popupMenu,
    defaultValue = "",
    onChange,
    actionIcons = [],
    disabled = false,
    className,
    required,
    ...props
  },
  ref,
) => {
  const [isFocused, setIsFocused] = useState(false);
  const hasError = Boolean(error);

  const editor = useEditor({
    extensions: [
      StarterKit.configure({
        heading: false,
        codeBlock: false,
        blockquote: false,
        horizontalRule: false,
      }),
      UnderlineExtension,
    ],
    content: defaultValue,
    editable: !disabled,
    immediatelyRender: false,
    onUpdate: ({ editor }) => {
      onChange?.(editor.getHTML());
    },
    onFocus: () => setIsFocused(true),
    onBlur: () => setIsFocused(false),
    editorProps: {
      attributes: {
        class: cn(
          "prose prose-sm max-w-none focus:outline-none",
          "min-h-[120px] p-4",
          "text-text-main placeholder:text-muted-foreground/60",
          disabled && "opacity-50 pointer-events-none cursor-not-allowed",
          hasError && "border-destructive/50",
        ),
      },
    },
  });

  useImperativeHandle(ref, () => ({
    getHtml: () => editor?.getHTML() ?? "",
    setHtml: (html: string) => {
      editor?.commands.setContent(html);
      onChange?.(html);
    },
    focus: () => {
      editor?.commands.focus();
    },
  }));
}

```

```

useEffect(() => {
  if (editor && defaultValue !== editor.getHTML()) {
    const { from, to } = editor.state.selection;
    editor.commands.setContent(defaultValue, {
      parseOptions: { preserveWhitespace: true },
    });
    editor.commands.setTextSelection({ from, to });
  }
}, [defaultValue, editor]);

const limitedActionIcons = actionIcons.slice(0, 4);

const isActive = (mark: string) => editor?.isActive(mark) ?? false;
const isActiveNode = (node: string) => editor?.isActive(node) ?? false;

const toolbarButtons = [
  {
    icon: Bold,
    isActive: () => isActive("bold"),
    onClick: () => editor?.chain().focus().toggleBold().run(),
    tooltip: "Bold (Ctrl+B)",
  },
  {
    icon: Italic,
    isActive: () => isActive("italic"),
    onClick: () => editor?.chain().focus().toggleItalic().run(),
    tooltip: "Italic (Ctrl+I)",
  },
  {
    icon: UnderlineIcon,
    isActive: () => isActive("underline"),
    onClick: () => editor?.chain().focus().toggleUnderline().run(),
    tooltip: "Underline (Ctrl+U)",
  },
  {
    icon: List,
    isActive: () => isActiveNode("bulletList"),
    onClick: () => editor?.chain().focus().toggleBulletList().run(),
    tooltip: "Bullet List",
  },
];

return (
  <div
    className={cn("w-full", className)}
    onContextMenu={(e) => popupMenu?.trigger(e)}
    {...props}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >

```

```

    })
  >
  {label}
  {required && (
    <span className="text-destructive ml-0.5" aria-hidden="true">
      *
    </span>
  )}
</label>
)}
<div className="relative">
  <div
    className={cn(
      "flex items-center gap-1 p-2",
      "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border-b border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-t-surface",
      disabled && "opacity-50 pointer-events-none",
    )}
    role="toolbar"
    aria-label="Text formatting"
  >
    {toolbarButtons.map((btn, index) => (
      <Button
        key={index}
        type="button"
        variant="ghost"
        size="icon"
        className={cn(
          "active:scale-95 transition-all duration-100",
          btn.isActive() &&
            "bg-amber-500/20 text-amber-600 dark:text-amber-400",
        )}
        aria-label={btn.tooltip}
        aria-pressed={btn.isActive()}
        disabled={disabled || !editor}
        onClick={btn.onClick}
      >
        <btn.icon className="w-4 h-4" aria-hidden="true" />
      </Button>
    )})}
  </div>
  <div
    className={cn(
      "w-full bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-b-surface",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2 focus-visible:ring-offset-background",
      "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
      "transition-all duration-200 ease-out",
      "min-h-[120px]",
      "outline-none",
      hasError &&
        "border-destructive/50 focus-visible:ring-destructive/50",
      disabled && "bg-muted/50",
      isFocused &&
        "ring-2 ring-ring ring-offset-2 ring-offset-background",
    )}
  >

```



```

    })
    role="textbox"
    aria-multiline="true"
    aria-label={label}
    aria-invalid={hasError}
    aria-describedby={error ? "html-memo-error" : undefined}
  >
    {editor && <EditorContent editor={editor} />}
  </div>
  {limitedActionIcons.length > 0 && (
    <div className="absolute bottom-2 right-2 flex items-center gap-1.5">
      {limitedActionIcons.map((action, index) => (
        <Button
          key={index}
          type="button"
          variant="ghost"
          size="icon"
          className="active:scale-95 transition-all duration-100"
          aria-label={action.tooltipText}
          disabled={disabled}
          onClick={() => action.onClick(editor?.getHTML() ?? "")}
        >
          <action.icon className="w-4 h-4" aria-hidden="true" />
        </Button>
      ))}
    </div>
  )}
</div>
{error && (
  <p
    id="html-memo-error"
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>
);
},
);

HtmlMemoBox.displayName = "HtmlMemoBox";

```

FILE: memo-box.tsx
LOKASI: src\data-entry

```
import React, { forwardRef, useEffect, useRef, useCallbck } from "react";
import { LucideIcon } from "lucide-react";
import { cn } from "@lib/utills";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";
import { Button } from "../general/button";

export interface MemoBoxActionConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;
  tooltipText?: string;
}

export interface MemoBoxProps extends React.TextareaHTMLAttributes<HTMLTextAreaElement> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  actionIcons?: MemoBoxActionConfig[];
}

export const MemoBox = forwardRef<HTMLTextAreaElement, MemoBoxProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      actionIcons = [],
      className,
      disabled,
      required,
      placeholder,
      rows = 4,
      ...props
    },
    ref,
  ) => {
    const textareaRef = useRef<HTMLTextAreaElement>(null);
    const hasError = Boolean(error);

    const handleRef = (node: HTMLTextAreaElement | null) => {
      textareaRef.current = node;
      if (ref) {
        if (typeof ref === "function") {
          ref(node);
        } else {
          ref.current = node;
        }
      }
    };
  }
);
```

```

const adjustHeight = useCallback(() => {
  const textarea = textareaRef.current;
  if (textarea) {
    textarea.style.height = "auto";
    const newHeight = textarea.scrollHeight;
    textarea.style.height = `${newHeight}px`;
  }
}, []);

useEffect(() => {
  adjustHeight();
}, [adjustHeight]);

const handleChange = (e: React.ChangeEvent<HTMLTextAreaElement>) => {
  adjustHeight();
  props.onChange?.(e);
};

const limitedActionIcons = actionIcons.slice(0, 4);

return (
  <div
    className={cn("w-full", className)}
    onContextMenu={(e) => popupMenu?.trigger(e)}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
    <div className="relative">
      <textarea
        ref={handleRef}
        className={cn(
          "w-full bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
          "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-surface",
          "text-text-main placeholder:text-muted-foreground/60",
          "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 focus-visible:ring-offset-background",
          "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
          "transition-all duration-200 ease-out",
          "resize-none",

```

```

      "px-4 py-3",
      "min-h-[100px]",
      hasError &&
        "border-destructive/50 focus-visible:ring-destructive/50",
      disabled && "bg-muted/50",
    )}
    disabled={disabled}
    required={required}
    placeholder={placeholder}
    rows={rows}
    onChange={handleChange}
    {...props}
  />
  {limitedActionIcons.length > 0 && (
    <div className="absolute bottom-2 right-2 flex items-center gap-1.5">
      {limitedActionIcons.map((action, index) => (
        <Button
          key={index}
          type="button"
          variant="ghost"
          size="icon"
          className="active:scale-95 transition-all duration-100"
          aria-label={action.tooltipText}
          disabled={disabled}
          onClick={action.onClick}
        >
          <action.icon className="w-4 h-4" aria-hidden="true" />
        </Button>
      ))}
    </div>
  )}
</div>
{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>
);
},
);

```

```
MemoBox.displayName = "MemoBox";
```

FILE: numeric-edit.tsx
LOKASI: src\data-entry

```
import React, {
  forwardRef,
  useState,
  useEffect,
  useRef,
  useCallback,
} from "react";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { Button } from "../general/button";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface NumericEditInnerIconConfig {
  icon: LucideIcon;
  className?: string;
}

export interface NumericEditActionButtonConfig {
  icon: LucideIcon;
  onClick: (value: number) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface NumericEditProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value" | "defaultValue"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  prefixSymbol?: string;
  f2Editable?: boolean;
  innerIcons?: NumericEditInnerIconConfig[];
  actionButtons?: NumericEditActionButtonConfig[];
  prefixIcon?: LucideIcon;
  suffixIcon?: LucideIcon;
  value?: number | string;
  defaultValue?: number | string;
  onChange?: (value: number) => void;
  decimalPlaces?: number;
}

export const NumericEdit = forwardRef<HTMLInputElement, NumericEditProps>(
  (
```

```

    {
      label,
      error,
      hint,
      popupMenu,
      prefixSymbol = "",
      f2Editable = false,
      innerIcons = [],
      actionButtons = [],
      prefixIcon,
      suffixIcon,
      className,
      disabled,
      required,
      placeholder,
      value,
      defaultValue,
      onChange,
      decimalPlaces = 2,
      ...props
    },
    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const hasError = Boolean(error);
    const [isEditing, setIsEditing] = useState(!f2Editable);
    const [displayValue, setDisplayValue] = useState<string>("");

    const sanitizeValue = useCallback(
      (val: string): string => {
        if (!val) return "";
        const clean = val.replace(/^[^0-9.-]/g, "");
        const parts = clean.split(".");
        if (parts.length > 2) {
          return parts[0] + "." + parts.slice(1).join("");
        }
        if (decimalPlaces > 0 && parts[1] && parts[1].length > decimalPlaces) {
          return parts[0] + "." + parts[1].slice(0, decimalPlaces);
        }
        return clean;
      },
      [decimalPlaces],
    );

    const parseToNumber = useCallback((val: string): number => {
      if (!val) return 0;
      const num = parseFloat(val);
      return isNaN(num) ? 0 : num;
    }, []);

    const formatDisplay = useCallback(
      (val: number | string): string => {
        if (val === "" || val === null || val === undefined) return "";
        const num = typeof val === "string" ? parseToNumber(val) : val;
        if (decimalPlaces > 0) {
          return num.toFixed(decimalPlaces);
        }
        return Math.round(num).toString();
      }
    );
  }

```

```

    },
    [decimalPlaces, parseToNumber],
  );

  useEffect(() => {
    const initialValue =
      value !== undefined
        ? value
        : defaultValue !== undefined
          ? defaultValue
          : "";
    const formatted = formatDisplay(initialValue);
    setDisplayValue(formatted);
  }, [value, defaultValue, formatDisplay]);

  useEffect(() => {
    if (prefixSymbol) {
      setDisplayValue((prev) => prev);
    }
  }, [prefixSymbol]);

  const handleRef = useCallback(
    (node: HTMLInputElement | null) => {
      inputRef.current = node;
      if (ref) {
        if (typeof ref === "function") {
          ref(node);
        } else {
          ref.current = node;
        }
      }
    },
    [ref],
  );

  const handleKeyDown = useCallback(
    (e: React.KeyboardEvent<HTMLInputElement>) => {
      if (f2Editable && e.key === "F2") {
        e.preventDefault();
        setIsEditing(true);
        setTimeout(() => inputRef.current?.focus(), 0);
      }
      if (e.key === "Escape" && isEditing) {
        e.preventDefault();
        setIsEditing(false);
        inputRef.current?.blur();
      }
      if ((e.ctrlKey || e.metaKey) && e.key === "c") {
        const numValue = parseToNumber(displayValue);
        navigator.clipboard?.writeText(numValue.toString());
      }
    },
    [f2Editable, isEditing, displayValue, parseToNumber],
  );

  const handleChange = useCallback(
    (e: React.ChangeEvent<HTMLInputElement>) => {
      if (!isEditing) return;

```

```

    const sanitized = sanitizeValue(e.target.value);
    setDisplayValue(sanitized);
    const numValue = parseToNumber(sanitized);
    onChange?.(numValue);
  },
  [isEditing, sanitizeValue, parseToNumber, onChange],
);

const handleBlur = useCallback(() => {
  if (f2Editable && isEditing) {
    setIsEditing(false);
  }
}, [f2Editable, isEditing]);

const limitedInnerIcons = innerIcons.slice(0, 4);
const limitedActionButtons = actionButtons.slice(0, 4);

const numericValue = parseToNumber(displayValue);

return (
  <div
    className={cn("w-full", className)}
    onContextMenu={(e) => popupMenu?.trigger(e)}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
    <div className="flex items-center w-full gap-2">
      <div
        className={cn(
          "relative flex items-center flex-1 min-w-0",
          "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
          "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-surface",
          "text-text-main placeholder:text-muted-foreground/60",
          "focus-within:ring-2 focus-within:ring-amber-500/20 focus-within:border-amber-500 focus-within:bg-amber-500/5",
          "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
          "transition-all duration-200 ease-out",
          hasError &&
          "border-destructive/50 focus-within:ring-destructive/50 focus-within:border-"

```



```

destructive/50",
  disabled && "bg-muted/50",
  f2Editable && !isEditing && "bg-muted/30",
  f2Editable && isEditing && "bg-amber-500/5 border-amber-500",
)}
>
{(prefixSymbol || prefixIcon) && (
  <div
    className={cn(
      "flex items-center justify-center px-3",
      "text-muted-foreground/60",
      disabled && "opacity-50",
      f2Editable && isEditing && "text-amber-500",
    )}
    aria-hidden="true"
  >
    {prefixSymbol && (
      <span className="text-sm font-medium">{prefixSymbol}</span>
    )}
    {prefixIcon && (
      <span className="w-4 h-4" aria-hidden="true">
        {React.createElement(prefixIcon, { className: "w-4 h-4" })}
      </span>
    )}
  </div>
)}
<input
  ref={handleRef}
  type="text"
  inputMode="decimal"
  className={cn(
    "flex-1 bg-transparent border-none outline-none",
    "px-4 py-2.5",
    "text-text-main placeholder:text-muted-foreground/60",
    "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
    "w-full min-w-0",
    "text-right tabular-nums",
    "font-mono",
    prefixSymbol || prefixIcon ? "pl-0" : "pl-4",
    suffixIcon || limitedInnerIcons.length > 0 ? "pr-0" : "pr-4",
    f2Editable && !isEditing && "cursor-not-allowed",
    f2Editable && isEditing && "bg-amber-500/5",
  )}
  disabled={disabled || (f2Editable && !isEditing)}
  required={required}
  placeholder={placeholder}
  value={displayValue}
  onChange={handleChange}
  onKeyDown={handleKeyDown}
  onBlur={handleBlur}
  readOnly={f2Editable && !isEditing}
  {...props}
/>
{suffixIcon && (
  <div
    className={cn(
      "flex items-center justify-center px-3",
      "text-muted-foreground/60",

```

```

        disabled && "opacity-50",
      )}
      aria-hidden="true"
    >
    <span className="w-4 h-4" aria-hidden="true">
      {React.createElement(suffixIcon, { className: "w-4 h-4" })}
    </span>
  </div>
)}
{limitedInnerIcons.length > 0 && (
  <div className="absolute right-3 flex items-center gap-1">
    {limitedInnerIcons.map((iconConfig, index) => (
      <span
        key={index}
        className={cn(
          "flex items-center justify-center",
          "w-5 h-5",
          "text-muted-foreground/60",
          iconConfig.className,
        )}
        aria-hidden="true"
      >
        <iconConfig.icon className="w-4 h-4" aria-hidden="true" />
      </span>
    ))}
  </div>
)}
</div>
{limitedActionButtons.length > 0 && (
  <div className="flex items-center gap-1.5 shrink-0">
    {limitedActionButtons.map((action, index) => (
      <Button
        key={index}
        variant="ghost"
        size="icon"
        disabled={disabled || action.disabled}
        onClick={() => action.onClick(numericValue)}
        className="active:scale-95 transition-all duration-100"
        aria-label={action.tooltipText}
      >
        <action.icon className="w-4 h-4" />
      </Button>
    ))}
  </div>
)}
</div>
{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}

```

```
        </div>
    );
},
);

NumericEdit.displayName = "NumericEdit";
```

FILE: text-box.tsx
LOKASI: src\data-entry

```
import React, { forwardRef, useRef, useCallback, useState } from "react";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { Button } from "../general/button";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface TextBoxInnerIconConfig {
  icon: LucideIcon;
  className?: string;
}

export interface TextBoxActionButtonConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>, value: string) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface TextBoxProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  innerIcons?: TextBoxInnerIconConfig[];
  actionButtons?: TextBoxActionButtonConfig[];
  onChange?: (value: string) => void;
}

export const TextBox = forwardRef<HTMLInputElement, TextBoxProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      innerIcons = [],
      actionButtons = [],
      className,
      disabled,
      required,
      placeholder,
      value,
      defaultValue,

```

```

    onChange,
    onBlur,
    ...props
  },
  ref,
) => {
  const inputRef = useRef<HTMLInputElement>(null);
  const hasError = Boolean(error);
  const [isFocused, setIsFocused] = useState(false);

  const handleRef = useCallback(
    (node: HTMLInputElement | null) => {
      inputRef.current = node;
      if (ref) {
        if (typeof ref === "function") {
          ref(node);
        } else {
          ref.current = node;
        }
      }
    },
    [ref],
  );

  const handleChange = useCallback(
    (e: React.ChangeEvent<HTMLInputElement>) => {
      onChange?.(e.target.value);
    },
    [onChange],
  );

  const handleBlur = useCallback(
    (e: React.FocusEvent<HTMLInputElement>) => {
      setIsFocused(false);
      onBlur?.(e);
    },
    [onBlur],
  );

  const handleFocus = useCallback(() => {
    setIsFocused(true);
  }, []);

  const limitedInnerIcons = innerIcons.slice(0, 4);
  const limitedActionButtons = actionButtons.slice(0, 4);

  return (
    <div
      className={cn("w-full", className)}
      onContextMenu={(e) => popupMenu?.trigger(e)}
    >
      {hint && (
        <HintBox content={hint} className="mb-1.5">
          <span className="text-sm text-text-muted" />
        </HintBox>
      )}
      {label && (
        <label

```

```

      className={cn(
        "block text-sm font-medium text-text-main mb-1.5",
        disabled && "opacity-50",
      )}
    >
    {label}
    {required && (
      <span className="text-destructive ml-0.5" aria-hidden="true">
        *
      </span>
    )}
  </label>
)}
<div className="flex items-center w-full gap-2">
  <div
    className={cn(
      "relative flex items-center flex-1 min-w-0",
      "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-surface",
      "text-text-main placeholder:text-muted-foreground/60",
      "focus-within:ring-2 focus-within:ring-amber-500/20 focus-within:border-amber-500 focus-
within:bg-amber-500/5",
      "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
      "transition-all duration-200 ease-out",
      hasError &&
      "border-destructive/50 focus-within:ring-destructive/50 focus-within:border-
destructive/50",
      disabled && "bg-muted/50",
      isFocused && "ring-2 ring-amber-500/20 border-amber-500",
    )}
  >
  <input
    ref={handleRef}
    type="text"
    className={cn(
      "flex-1 bg-transparent border-none outline-none",
      "px-4 py-2.5",
      "text-text-main placeholder:text-muted-foreground/60",
      "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
      "w-full min-w-0",
      limitedInnerIcons.length > 0 ? "pr-12" : "pr-4",
    )}
    disabled={disabled}
    required={required}
    placeholder={placeholder}
    value={value}
    defaultValue={defaultValue}
    onChange={handleChange}
    onBlur={handleBlur}
    onFocus={handleFocus}
    {...props}
  />
  {limitedInnerIcons.length > 0 && (
    <div className="absolute right-3 flex items-center gap-1">
      {limitedInnerIcons.map((iconConfig, index) => (
        <span
          key={index}

```

```

        className={cn(
          "flex items-center justify-center",
          "w-5 h-5",
          "text-muted-foreground/60",
          iconConfig.className,
        )}
        aria-hidden="true"
      >
        <iconConfig.icon className="w-4 h-4" aria-hidden="true" />
      </span>
    )}}
  </div>
)}
</div>
{limitedActionButtons.length > 0 && (
  <div className="flex items-center gap-1.5 shrink-0">
    {limitedActionButtons.map((action, index) => (
      <Button
        key={index}
        variant="ghost"
        size="icon"
        disabled={disabled || action.disabled}
        onClick={(e) =>
          action.onClick(e, inputRef.current?.value || "")
        }
        className="active:scale-95 transition-all duration-100"
        aria-label={action.tooltipText}
      >
        <action.icon className="w-4 h-4" />
      </Button>
    )}}
  </div>
)}
</div>
{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>
);
},
);

TextBox.displayName = "TextBox";

```

FILE: confirmation-box.tsx
LOKASI: src\feedback

```
import React, { useEffect, useRef, useCallback } from "react";
import { X, Check, AlertTriangle } from "lucide-react";
import { cn } from "@lib/utils";

// 🏠 LOCAL TYPE ISOLATION GATEWAY
export interface ConfirmationBoxProps {
  open: boolean;
  onClose: () => void;
  onConfirm: () => void;
  title: string;
  description: string;
  isDestructive?: boolean;
  confirmLabel?: string;
  cancellabel?: string;
}

export function ConfirmationBox({
  open,
  onClose,
  onConfirm,
  title,
  description,
  isDestructive = false,
  confirmLabel = "Confirm",
  cancellabel = "Cancel",
}: ConfirmationBoxProps) {
  const dialogRef = useRef<HTMLDialogElement>(null);
  const cancelButtonRef = useRef<HTMLButtonElement>(null);
  const confirmButtonRef = useRef<HTMLButtonElement>(null);
  const previousActiveElement = useRef<HTMLElement | null>(null);

  const handleKeyDown = useCallback(
    (event: KeyboardEvent) => {
      if (event.key === "Escape") {
        event.preventDefault();
        onClose();
      }
    },
    [onClose],
  );

  const handleFocusTrap = useCallback((event: KeyboardEvent) => {
    if (event.key !== "Tab" || !dialogRef.current) return;

    const focusableElements = dialogRef.current.querySelectorAll<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
    );

    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length - 1];

    if (event.shiftKey && document.activeElement === firstElement) {

```



```

    event.preventDefault();
    lastElement?.focus();
  } else if (!event.shiftKey && document.activeElement === lastElement) {
    event.preventDefault();
    firstElement?.focus();
  }
}, []);

const handleConfirm = useCallback(() => {
  onConfirm();
  onClose();
}, [onConfirm, onClose]);

useEffect(() => {
  if (open) {
    previousActiveElement.current = document.activeElement as HTMLElement;
    document.addEventListener("keydown", handleKeyDown);
    document.addEventListener("keydown", handleFocusTrap);
    document.body.style.overflow = "hidden";

    dialogRef.current?.showModal();

    // CRITICAL: Focus Cancel button by default to prevent accidental Enter key confirmation
    cancelButtonRef.current?.focus();

    return () => {
      document.removeEventListener("keydown", handleKeyDown);
      document.removeEventListener("keydown", handleFocusTrap);
      document.body.style.overflow = "";
      previousActiveElement.current?.focus();
      dialogRef.current?.close();
    };
  }
}, [open, handleKeyDown, handleFocusTrap]);

if (!open) return null;

const confirmBgColor = isDestructive ? "bg-destructive" : "bg-brand-primary";
const confirmHoverColor = isDestructive
  ? "hover:bg-destructive/90"
  : "hover:bg-brand-hover";
const confirmActiveColor = isDestructive
  ? "active:bg-destructive"
  : "active:bg-brand-primary";
const confirmRingColor = isDestructive
  ? "focus-visible:ring-destructive"
  : "focus-visible:ring-brand-primary";
const iconColor = isDestructive ? "text-destructive" : "text-brand-primary";
const bgColor = isDestructive ? "bg-destructive/10" : "bg-brand-primary/10";
const borderColor = isDestructive
  ? "border-destructive/30"
  : "border-brand-primary/30";

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
    <div
      className={cn(
        "fixed inset-0 z-40",

```

```

        "bg-black/50 backdrop-blur-sm",
        "transition-opacity duration-300 ease-out",
        "opacity-100",
    )}
    onClick={onClose}
    aria-hidden="true"
  />
  <dialog
    ref={dialogRef}
    className={cn(
      "relative z-50 w-full max-w-md",
      "bg-card/95 backdrop-blur-[var(--backdrop-blur)]",
      "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-[var(--radius-surface)]",
      "shadow-[var(--shadow-xl)]",
      "overflow-hidden",
      "transition-all duration-200 ease-out",
      open
        ? "opacity-100 scale-100"
        : "opacity-0 scale-95 pointer-events-none",
    )}
    role="alertdialog"
    aria-modal="true"
    aria-labelledby="confirmation-box-title"
    aria-describedby="confirmation-box-description"
  >
    <div className={cn("p-6", bgColor, borderColor, "border-t-4")}>
      <div className="flex items-start gap-4">
        <div
          className={cn(
            "flex-shrink-0 w-10 h-10 rounded-full flex items-center justify-center",
            bgColor,
          )}
          aria-hidden="true"
        >
          <AlertTriangle
            className={cn("w-5 h-5", iconColor)}
            strokeWidth={2.5}
          />
        </div>
        <div className="flex-1 min-w-0">
          <h2
            id="confirmation-box-title"
            className="text-lg font-semibold text-foreground leading-tight"
          >
            {title}
          </h2>
          <p
            id="confirmation-box-description"
            className="mt-2 text-sm text-muted-foreground leading-relaxed"
          >
            {description}
          </p>
        </div>
      </div>
    </div>
    <div className="flex items-center justify-end gap-3 p-4 border-t border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]">

```

```

<button
  ref={cancelButtonRef}
  type="button"
  onClick={onClose}
  className={cn(
    "inline-flex items-center justify-center gap-2",
    "h-10 px-4 py-2 text-base font-medium",
    "rounded-[var(--radius-surface)]",
    "bg-secondary text-secondary-foreground",
    "hover:bg-secondary/80",
    "active:scale-95 active:bg-secondary",
    "transition-all duration-150 ease-out",
    "focus-visible:outline-none",
    "focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 focus-
visible:ring-offset-background",
    "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
  )}
>
  <X className="w-4 h-4" aria-hidden="true" />
  {cancelLabel}
</button>
<button
  ref={confirmButtonRef}
  type="button"
  onClick={handleConfirm}
  className={cn(
    "inline-flex items-center justify-center gap-2",
    "h-10 px-4 py-2 text-base font-medium",
    "rounded-[var(--radius-surface)]",
    confirmBgColor,
    "text-white",
    confirmHoverColor,
    "active:scale-95",
    confirmActiveColor,
    "transition-all duration-150 ease-out",
    "focus-visible:outline-none",
    confirmRingColor,
    "focus-visible:ring-2 focus-visible:ring-offset-2 focus-visible:ring-offset-background",
    "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
  )}
>
  <Check className="w-4 h-4" aria-hidden="true" />
  {confirmLabel}
</button>
</div>
</dialog>
</div>
);
}

export default ConfirmationBox;

```

FILE: drawer.tsx
LOKASI: src\feedback

```
import React, {
  createContext,
  useContext,
  useState,
  useEffect,
  useRef,
  useCallback,
} from "react";
import { X } from "lucide-react";
import { cn } from "@lib/Utils";
import { HintBox } from "../hint-box";
import type { PopupMenuConfig } from "../popup-menu";
import { THEME_CONFIG } from "@config/theme-constants";

// Æ°Å.Å;Å! LOCAL TYPE ISOLATION GATEWAY
export interface DrawerRootProps {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  children: React.ReactNode;
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

export interface DrawerOverlayProps extends React.HTMLAttributes<HTMLDivElement> {
  className?: string;
}

export interface DrawerContentProps extends React.HTMLAttributes<HTMLDivElement> {
  size?: "sm" | "md" | "lg" | "xl" | "full";
}

export interface DrawerHeaderProps extends React.HTMLAttributes<HTMLDivElement> {
  title: string;
  description?: string;
}

interface DrawerContextValue {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  size: DrawerContentProps["size"];
  popupMenu?: PopupMenuConfig;
}

const DrawerContext = createContext<DrawerContextValue | null>(null);

function useDrawerContext() {
  const context = useContext(DrawerContext);
  if (!context) {
    throw new Error("Drawer components must be used within DrawerRoot");
  }
  return context;
}
```

```

const SIZE_CLASSES = {
  sm: `w-[${THEME_CONFIG.layout.dropdownMaxHeight.replace("px", "")}px] max-w-[90vw]`,
  md: "w-[400px] max-w-[90vw]",
  lg: "w-[560px] max-w-[90vw]",
  xl: "w-[720px] max-w-[90vw]",
  full: "w-full max-w-[100vw]",
} as const;

export function DrawerRoot({
  open,
  onOpenChange,
  children,
  hint,
  popupMenu,
}: DrawerRootProps) {
  const [size] = useState<DrawerContentProps["size"]>("md");

  const handleOpenChange = useCallback(
    (newOpen: boolean) => {
      onOpenChange(newOpen);
    },
    [onOpenChange],
  );

  const value: DrawerContextValue = {
    open,
    onOpenChange: handleOpenChange,
    size,
    popupMenu,
  };

  return (
    <DrawerContext.Provider value={value}>
      <div className="relative" onContextMenu={(e) => popupMenu?.trigger(e)}>
        {React.Children.map(children, (child) => {
          if (!React.isValidElement(child)) return child;
          if (child.type === DrawerContent) {
            const childProps = child.props as DrawerContentProps;
            // eslint-disable-next-line @typescript-eslint/no-explicit-any
            return React.cloneElement(child as any, {
              size: childProps.size || size,
            });
          }
          return child;
        })}
        {hint && (
          <HintBox content={hint} className="mb-1.5">
            <span className="sr-only">hint</span>
          </HintBox>
        )}
      </div>
    </DrawerContext.Provider>
  );
}

export function DrawerOverlay({ className, ...props }: DrawerOverlayProps) {
  const { open, onOpenChange } = useDrawerContext();

```

```

    if (!open) return null;

    const handleClick = useCallback(() => {
      onOpenChange(false);
    }, [onOpenChange]);

    return (
      <div
        className={cn(
          "fixed inset-0 z-40",
          "bg-black/50 backdrop-blur-sm",
          "transition-opacity duration-300 ease-out",
          open ? "opacity-100" : "opacity-0 pointer-events-none",
          className,
        )}
        onClick={handleClick}
        aria-hidden="true"
        {...props}
      />
    );
  }

export function DrawerContent({
  className,
  size = "md",
  children,
  ...props
}: DrawerContentProps) {
  const { open, onOpenChange } = useDrawerContext();
  const contentRef = useRef<HTMLDivElement>(null);
  const previousActiveElement = useRef<HTMLElement | null>(null);

  const handleKeyDown = useCallback(
    (event: KeyboardEvent) => {
      if (event.key === "Escape") {
        onOpenChange(false);
      }
    },
    [onOpenChange],
  );

  const handleFocusTrap = useCallback((event: KeyboardEvent) => {
    if (event.key !== "Tab" || !contentRef.current) return;

    const focusableElements = contentRef.current.querySelectorAll<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
    );

    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length - 1];

    if (event.shiftKey && document.activeElement === firstElement) {
      event.preventDefault();
      lastElement?.focus();
    } else if (!event.shiftKey && document.activeElement === lastElement) {
      event.preventDefault();
      firstElement?.focus();
    }
  });

```

```

    }
  }, []);

useEffect(() => {
  if (open) {
    previousActiveElement.current = document.activeElement as HTMLElement;
    document.addEventListener("keydown", handleKeyDown);
    document.addEventListener("keydown", handleFocusTrap);
    document.body.style.overflow = "hidden";

    const focusableElement = contentRef.current?.querySelector<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
    );
    focusableElement?.focus();

    return () => {
      document.removeEventListener("keydown", handleKeyDown);
      document.removeEventListener("keydown", handleFocusTrap);
      document.body.style.overflow = "";
      previousActiveElement.current?.focus();
    };
  }
}, [open, handleKeyDown, handleFocusTrap]);

if (!open) return null;

return (
  <div
    ref={contentRef}
    className={cn(
      "fixed right-0 top-0 z-50 h-full",
      "flex flex-col",
      "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border-1 border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "shadow-[var(--shadow-x1)]",
      "transition-transform duration-300 ease-out",
      open ? "translate-x-0" : "translate-x-full",
      SIZE_CLASSES[size],
      className,
    )}
    role="dialog"
    aria-modal="true"
    aria-label="Drawer"
    {...props}
    onContextMenu={() => {
      // Allow context menu to bubble up to DrawerRoot for popupMenu handling
    }}
  >
    {children}
  </div>
);
}

export function DrawerHeader({
  className,
  title,
  description,
  children,

```

```

    ...props
  }: DrawerHeaderProps) {
    return (
      <div
        className={cn(
          "flex flex-col gap-1.5 p-6 border-b border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)]",
          className,
        )}
        {...props}
      >
        <div className="flex items-start justify-between gap-4">
          <div className="flex-1 min-w-0">
            <h2 className="text-lg font-semibold text-foreground leading-tight">
              {title}
            </h2>
            <div>
              <div>
                <p className="mt-1 text-sm text-muted-foreground leading-relaxed">
                  {description}
                </p>
              </div>
            </div>
          </div>
          <DrawerClose />
        </div>
        {children}
      </div>
    );
  }
}

export function DrawerClose({
  className,
  ...props
}: React.ButtonHTMLAttributes<HTMLButtonElement>) {
  const { onOpenChange } = useDrawerContext();

  const handleClick = useCallback(() => {
    onOpenChange(false);
  }, [onOpenChange]);

  return (
    <button
      type="button"
      onClick={handleClick}
      className={cn(
        "flex-shrink-0 p-1.5 rounded-md",
        "text-muted-foreground hover:text-foreground",
        "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
        "active:scale-95",
        "transition-all duration-150 ease-out",
        "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2 focus-visible:ring-offset-background",
        className,
      )}
      aria-label="Close drawer"
      {...props}
    >
      <X size={18} strokeWidth={2.5} aria-hidden="true" />
    </button>
  );
}

```



```
);  
}
```

```
export default DrawerRoot;
```

FILE: hint-box.tsx
LOKASI: src\feedback

```
import React, { useState, useEffect, useRef, useCallback } from "react";
import { LucideIcon, X } from "lucide-react";
import { cn } from "@lib/utils";

// ðŸŸ¡ LOCAL TYPE ISOLATION GATEWAY
export interface HintActionConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;
  tooltipText?: string;
}

export type HintPosition = "top" | "bottom" | "left" | "right";

export interface HintBoxProps {
  content: React.ReactNode;
  children: React.ReactNode;
  position?: HintPosition;
  leadingIcon?: LucideIcon;
  actions?: HintActionConfig[];
  className?: string;
  enterDelay?: number;
  leaveDelay?: number;
}

interface HintBoxState {
  isOpen: boolean;
  isHoveringTrigger: boolean;
  isHoveringContent: boolean;
}

interface PositionStyles {
  top?: string;
  left?: string;
  right?: string;
  bottom?: string;
  transform?: string;
}

const DEFAULT_ENTER_DELAY = 200;
const DEFAULT_LEAVE_DELAY = 150;
const MAX_ACTIONS = 2;

function getPositionStyles(
  position: HintPosition,
  triggerRect: DOMRect | null,
  contentRect: DOMRect | null,
  viewportPadding = 8,
): PositionStyles {
  if (!triggerRect || !contentRect) {
    return {};
  }
```

```

const viewportWidth = window.innerWidth;
const viewportHeight = window.innerHeight;
const gap = 8;

let top = 0;
let left = 0;
let transform = "";

switch (position) {
  case "top": {
    top = triggerRect.top - contentRect.height - gap;
    left = triggerRect.left + (triggerRect.width - contentRect.width) / 2;

    // Viewport boundary checks
    if (top < viewportPadding) {
      top = triggerRect.bottom + gap;
    }
    if (left < viewportPadding) {
      left = viewportPadding;
    }
    if (left + contentRect.width > viewportWidth - viewportPadding) {
      left = viewportWidth - contentRect.width - viewportPadding;
    }
    transform = "translateX(-50%) translateY(-100%)";
    break;
  }
  case "bottom": {
    top = triggerRect.bottom + gap;
    left = triggerRect.left + (triggerRect.width - contentRect.width) / 2;

    if (top + contentRect.height > viewportHeight - viewportPadding) {
      top = triggerRect.top - contentRect.height - gap;
    }
    if (left < viewportPadding) {
      left = viewportPadding;
    }
    if (left + contentRect.width > viewportWidth - viewportPadding) {
      left = viewportWidth - contentRect.width - viewportPadding;
    }
    transform = "translateX(-50%)";
    break;
  }
  case "left": {
    top = triggerRect.top + (triggerRect.height - contentRect.height) / 2;
    left = triggerRect.left - contentRect.width - gap;

    if (left < viewportPadding) {
      left = triggerRect.right + gap;
    }
    if (top < viewportPadding) {
      top = viewportPadding;
    }
    if (top + contentRect.height > viewportHeight - viewportPadding) {
      top = viewportHeight - contentRect.height - viewportPadding;
    }
    transform = "translateY(-50%) translateX(-100%)";
    break;
  }
}

```

```

    case "right": {
      top = triggerRect.top + (triggerRect.height - contentRect.height) / 2;
      left = triggerRect.right + gap;

      if (left + contentRect.width > viewportWidth - viewportPadding) {
        left = triggerRect.left - contentRect.width - gap;
      }
      if (top < viewportPadding) {
        top = viewportPadding;
      }
      if (top + contentRect.height > viewportHeight - viewportPadding) {
        top = viewportHeight - contentRect.height - viewportPadding;
      }
      transform = "translateY(-50%)";
      break;
    }
  }

  return { top: `${top}px`, left: `${left}px`, transform };
}

```

```

function getArrowStyles(position: HintPosition): React.CSSProperties {
  const baseStyles: React.CSSProperties = {
    width: 0,
    height: 0,
    borderLeft: "6px solid transparent",
    borderRight: "6px solid transparent",
    position: "absolute",
    pointerEvents: "none",
  };
}

```

```

switch (position) {
  case "top":
    return {
      ...baseStyles,
      borderTop: "6px solid",
      bottom: "-6px",
      left: "50%",
      transform: "translateX(-50%) rotate(180deg)",
    };
  case "bottom":
    return {
      ...baseStyles,
      borderBottom: "6px solid",
      top: "-6px",
      left: "50%",
      transform: "translateX(-50%)",
    };
  case "left":
    return {
      ...baseStyles,
      borderLeft: "6px solid",
      right: "-6px",
      top: "50%",
      transform: "translateY(-50%) rotate(90deg)",
    };
  case "right":
    return {

```

```

        ...baseStyles,
        borderRight: "6px solid",
        left: "-6px",
        top: "50%",
        transform: "translateY(-50%) rotate(-90deg)",
    };
}
}

export function HintBox({
    content,
    children,
    position = "top",
    leadingIcon: LeadingIcon,
    actions = [],
    className,
    enterDelay = DEFAULT_ENTER_DELAY,
    leaveDelay = DEFAULT_LEAVE_DELAY,
}: HintBoxProps) {
    const [state, setState] = useState<HintBoxState>({
        isOpen: false,
        isHoveringTrigger: false,
        isHoveringContent: false,
    });

    const triggerRef = useRef<HTMLDivElement>(null);
    const contentRef = useRef<HTMLDivElement>(null);
    const portalRef = useRef<HTMLDivElement>(null);
    const enterTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const leaveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const positionStylesRef = useRef<PositionStyles>({});

    const updatePosition = useCallback(() => {
        if (triggerRef.current && contentRef.current) {
            const trigger = triggerRef.current.getBoundingClientRect();
            const content = contentRef.current.getBoundingClientRect();
            positionStylesRef.current = getPositionStyles(position, trigger, content);
        }
    }, [position]);

    useEffect(() => {
        updatePosition();
        window.addEventListener("resize", updatePosition);
        window.addEventListener("scroll", updatePosition, true);
        return () => {
            window.removeEventListener("resize", updatePosition);
            window.removeEventListener("scroll", updatePosition, true);
        };
    }, [updatePosition]);

    const clearTimeouts = useCallback(() => {
        if (enterTimeoutRef.current) {
            clearTimeout(enterTimeoutRef.current);
        }
        if (leaveTimeoutRef.current) {
            clearTimeout(leaveTimeoutRef.current);
        }
    }, []);
}

```

```

const handleTriggerEnter = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringTrigger: true }));
  enterTimeoutRef.current = setTimeout(() => {
    setState((prev) => ({ ...prev, isOpen: true }));
  }, enterDelay);
}, [clearTimeouts, enterDelay]);

const handleTriggerLeave = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringTrigger: false }));
  leaveTimeoutRef.current = setTimeout(() => {
    setState((prev) => {
      if (!prev.isHoveringContent) {
        return { ...prev, isOpen: false };
      }
      return prev;
    });
  }, leaveDelay);
}, [clearTimeouts, leaveDelay]);

const handleContentEnter = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringContent: true, isOpen: true }));
}, [clearTimeouts]);

const handleContentLeave = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringContent: false }));
  leaveTimeoutRef.current = setTimeout(() => {
    setState((prev) => {
      if (!prev.isHoveringTrigger) {
        return { ...prev, isOpen: false };
      }
      return prev;
    });
  }, leaveDelay);
}, [clearTimeouts, leaveDelay]);

const handleActionClick = useCallback(
  (action: HintActionConfig, e: React.MouseEvent<HTMLButtonElement>) => {
    action.onClick(e);
    if (action.icon === X || action.tooltipText === "Dismiss") {
      setState((prev) => ({ ...prev, isOpen: false }));
    }
  },
  [],
);

const handleKeyDown = useCallback((e: React.KeyboardEvent) => {
  if (e.key === "Escape") {
    setState((prev) => ({ ...prev, isOpen: false }));
  }
}, []);

const triggerElement = React.Children.only(children) as React.ReactElement;
const enhancedTrigger = (

```

```

    <div
      ref={triggerRef}
      className="relative inline-block"
      onMouseEnter={handleTriggerEnter}
      onMouseLeave={handleTriggerLeave}
      onFocus={handleTriggerEnter}
      onBlur={handleTriggerLeave}
      onKeyDown={handleKeyDown}
      aria-describedby={state.isOpen ? "hint-box-content" : undefined}
    >
      {triggerElement}
    </div>
  );

  const limitedActions = actions.slice(0, MAX_ACTIONS);

  const contentElement = (
    <div
      ref={contentRef}
      id="hint-box-content"
      role="tooltip"
      style={positionStylesRef.current as React.CSSProperties}
      className={cn(
        "fixed z-50 pointer-events-auto",
        "bg-card backdrop-blur-[var(--backdrop-blur)]",
        "border border-[color-mix(in_oklch_var(--border)_50%_transparent)]",
        "rounded-surface",
        "shadow-lg",
        "p-3",
        "max-w-xs",
        "text-sm text-card-foreground",
        "font-sans",
        "transition-all duration-200 ease-out",
        "opacity-0 scale-95 data-[state=open]:opacity-100 data-[state=open]:scale-100",
        "data-[side=bottom]:translate-y-2 data-[side=top]:-translate-y-2",
        "data-[side=left]:-translate-x-2 data-[side=right]:translate-x-2",
        className,
      )}
      data-state={state.isOpen ? "open" : "closed"}
      data-side={position}
      onMouseEnter={handleContentEnter}
      onMouseLeave={handleContentLeave}
    >
      <div className="flex items-start gap-2.5">
        {LeadingIcon && (
          <div
            className={cn(
              "flex-shrink-0 flex items-center justify-center",
              "w-5 h-5",
              "text-muted-foreground/80",
            )}
            aria-hidden="true"
          >
            <LeadingIcon className="w-4 h-4" />
          </div>
        )}
        <div className="flex-1 min-w-0">{content}</div>
        {limitedActions.length > 0 && (

```

```

<div className="flex items-center gap-1.5 flex-shrink-0">
  {limitedActions.map((action, index) => (
    <button
      key={index}
      type="button"
      onClick={(e) => handleActionClick(action, e)}
      className={cn(
        "relative flex items-center justify-center",
        "w-7 h-7 rounded-md",
        "text-muted-foreground/60 hover:text-foreground",
        "bg-transparent hover:bg-accent",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2",
        "disabled:opacity-50 disabled:pointer-events-none",
      )}
      aria-label={action.tooltipText || "Action"}
      disabled={!action.onClick}
    >
      <action.icon className="w-3.5 h-3.5" aria-hidden="true" />
    </button>
  )})}
</div>
  )}
</div>
<div
  style={getArrowStyles(position)}
  className="pointer-events-none"
  aria-hidden="true"
/>
</div>
);

return (
  <>
    {enhancedTrigger}
    {state.isOpen && (
      <div
        ref={portalRef}
        className="fixed inset-0 z-40 pointer-events-none"
        aria-hidden="true"
      >
        {contentElement}
      </div>
    )}
  </>
);
}

export default HintBox;

```


FILE: image-viewer.tsx
LOKASI: src\feedback

```
import React, {
  forwardRef,
  useState,
  useEffect,
  useCallback,
  useImperativeHandle,
  useRef,
} from "react";
import {
  ZoomIn,
  ZoomOut,
  RotateCw,
  RefreshCw,
  MoreHorizontal,
  Copy,
  Download,
  ExternalLink,
} from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../hint-box";
import {
  PopupMenu,
  type PopupMenuConfig,
  type PopupMenuItemConfig,
} from "../popup-menu";

export interface ImageViewerProps extends React.HTMLAttributes<HTMLDivElement> {
  src?: string;
  alt?: string;
  onScaleChange?: (scale: number, rotate: number) => void;
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

export interface ImageViewerRef {
  reset: () => void;
  zoomIn: () => void;
  zoomOut: () => void;
  rotate: () => void;
}

const MIN_SCALE = 0.1;
const MAX_SCALE = 5;
const SCALE_STEP = 0.2;

export const ImageViewer = forwardRef<ImageViewerRef, ImageViewerProps>(
  (
    {
      src,
      alt = "Document preview",
      onScaleChange,
      className,

```

```

    style,
    hint,
    popupMenu,
    ...props
  },
  ref,
) => {
  const [scale, setScale] = useState(1);
  const [rotate, setRotate] = useState(0);
  const [isLoading, setIsLoading] = useState(false);
  const [hasError, setHasError] = useState(false);
  const imgRef = useRef<HTMLImageElement>(null);
  const containerRef = useRef<HTMLDivElement>(null);

  const clampScale = useCallback((newScale: number) => {
    return Math.max(MIN_SCALE, Math.min(MAX_SCALE, newScale));
  }, []);

  const handleScaleChange = useCallback(
    (newScale: number) => {
      const clamped = clampScale(newScale);
      setScale(clamped);
      onScaleChange?.(clamped, rotate);
    },
    [clampScale, onScaleChange, rotate],
  );

  const handleRotateChange = useCallback(
    (newRotate: number) => {
      const normalized = ((newRotate % 360) + 360) % 360;
      setRotate(normalized);
      onScaleChange?.(scale, normalized);
    },
    [onScaleChange, scale],
  );

  const reset = useCallback(() => {
    setScale(1);
    setRotate(0);
    onScaleChange?.(1, 0);
  }, [onScaleChange]);

  const zoomIn = useCallback(() => {
    handleScaleChange(scale + SCALE_STEP);
  }, [handleScaleChange, scale]);

  const zoomOut = useCallback(() => {
    handleScaleChange(scale - SCALE_STEP);
  }, [handleScaleChange, scale]);

  const rotateCw = useCallback(() => {
    handleRotateChange(rotate + 90);
  }, [handleRotateChange, rotate]);

  useImperativeHandle(
    ref,
    () => ({
      reset,

```

```

        zoomIn,
        zoomOut,
        rotate: rotateCw,
    })),
    [reset, zoomIn, zoomOut, rotateCw],
);

useEffect(() => {
    const handleKeyDown = (event: KeyboardEvent) => {
        if (event.key === "Escape") {
            reset();
        }
    };

    document.addEventListener("keydown", handleKeyDown);
    return () => document.removeEventListener("keydown", handleKeyDown);
}, [reset]);

const handleImageLoad = useCallback(() => {
    setIsLoading(false);
    setHasError(false);
}, []);

const handleImageError = useCallback(() => {
    setIsLoading(false);
    setHasError(true);
}, []);

const handleDoubleClick = useCallback(() => {
    reset();
}, [reset]);

const handleWheel = useCallback(
    (event: React.WheelEvent<HTMLDivElement>) => {
        if (event.ctrlKey || event.metaKey) {
            event.preventDefault();
            const delta = event.deltaY > 0 ? -SCALE_STEP : SCALE_STEP;
            handleScaleChange(scale + delta);
        }
    },
    [handleScaleChange, scale],
);

const handleCopyImage = useCallback(() => {
    if (src) {
        navigator.clipboard.writeText(src);
    }
}, [src]);

const handleDownloadImage = useCallback(() => {
    if (src) {
        const link = document.createElement("a");
        link.href = src;
        link.download = alt || "image";
        link.target = "_blank";
        document.body.appendChild(link);
        link.click();
        document.body.removeChild(link);
    }
}, [src]);

```

```

    }
  }, [src, alt]);

const handleOpenInNewTab = useCallback(() => {
  if (src) {
    window.open(src, "_blank", "noopener,noreferrer");
  }
}, [src]);

if (!src) {
  return (
    <div
      ref={containerRef}
      className={cn(
        "relative flex items-center justify-center",
        "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
        "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
        "rounded-surface",
        "min-h-[300px]",
        className,
      )}
      style={style}
      {...props}
      onContextMenu={(e) => popupMenu?.trigger(e)}
    >
      {hint && (
        <HintBox content={hint} className="m-3 mb-0">
          <span className="sr-only">hint</span>
        </HintBox>
      )}
      <div className="text-center p-8">
        <RefreshCw
          className="w-12 h-12 text-muted-foreground/40 mx-auto mb-3 animate-spin"
          aria-hidden="true"
        />
        <p className="text-muted-foreground">No image source provided</p>
      </div>
    </div>
  );
}

```

```

const menuItems: PopupMenuItemConfig[] = [
  {
    id: "copy-link",
    label: "Copy Link",
    icon: Copy,
    onClick: handleCopyImage,
    disabled: !src,
  },
  {
    id: "download",
    label: "Download",
    icon: Download,
    onClick: handleDownloadImage,
    disabled: !src,
  },
  {
    id: "open-new-tab",

```

```

        label: "Open in New Tab",
        icon: ExternalLink,
        onClick: handleOpenInNewTab,
        disabled: !src,
    },
    {
        id: "reset",
        label: "Reset View",
        icon: RefreshCw,
        onClick: reset,
    },
];

return (
    <div
        ref={containerRef}
        className={cn(
            "relative",
            "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
            "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
            "rounded-surface",
            "overflow-hidden",
            className,
        )}
        style={style}
        onWheel={handleWheel}
        {...props}
        onContextMenu={(e) => popupMenu?.trigger(e)}
    >
        {hint && (
            <HintBox content={hint} className="m-3 mb-0">
                <span className="sr-only">hint</span>
            </HintBox>
        )}
        <div
            className={cn(
                "flex items-center justify-between p-3",
                "bg-[color-mix(in_oklch,var(--color-card)_95%,transparent)]",
                "backdrop-blur-[var(--backdrop-blur)]",
                "border-b border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
            )}
            role="toolbar"
            aria-label="Image viewer controls"
        >
            <div className="flex items-center gap-2">
                <span className="text-xs text-muted-foreground font-mono">
                    {Math.round(scale * 100)}%
                </span>
                {rotate !== 0 && (
                    <span className="text-xs text-muted-foreground font-mono">
                        {rotate}Â°
                    </span>
                )}
            </div>
            <div className="flex items-center gap-1.5">
                <button
                    type="button"
                    onClick={zoomOut}

```

```

disabled={scale <= MIN_SCALE}
className={cn(
  "relative flex items-center justify-center",
  "w-8 h-8 rounded-md",
  "text-muted-foreground/60 hover:text-foreground",
  "bg-transparent hover:bg-accent",
  "transition-transform duration-100",
  "active:scale-95",
  "focus-visible:outline-none",
  "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
  "disabled:opacity-50 disabled:pointer-events-none",
)}
aria-label="Zoom out"
aria-disabled={scale <= MIN_SCALE}
>
  <ZoomOut className="w-4 h-4" aria-hidden="true" />
</button>
<button
  type="button"
  onClick={reset}
  className={cn(
    "relative flex items-center justify-center",
    "w-8 h-8 rounded-md",
    "text-muted-foreground/60 hover:text-foreground",
    "bg-transparent hover:bg-accent",
    "transition-transform duration-100",
    "active:scale-95",
    "focus-visible:outline-none",
    "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
  )}
  aria-label="Reset view"
>
  <RefreshCw className="w-4 h-4" aria-hidden="true" />
</button>
<button
  type="button"
  onClick={zoomIn}
  disabled={scale >= MAX_SCALE}
  className={cn(
    "relative flex items-center justify-center",
    "w-8 h-8 rounded-md",
    "text-muted-foreground/60 hover:text-foreground",
    "bg-transparent hover:bg-accent",
    "transition-transform duration-100",
    "active:scale-95",
    "focus-visible:outline-none",
    "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
    "disabled:opacity-50 disabled:pointer-events-none",
  )}
  aria-label="Zoom in"
  aria-disabled={scale >= MAX_SCALE}
>
  <ZoomIn className="w-4 h-4" aria-hidden="true" />
</button>
<button
  type="button"
  onClick={rotateCw}
  className={cn(

```

```

        "relative flex items-center justify-center",
        "w-8 h-8 rounded-md",
        "text-muted-foreground/60 hover:text-foreground",
        "bg-transparent hover:bg-accent",
        "transition-transform duration-100",
        "active:scale-95",
        "focus-visible:outline-none",
        "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
    )}
    aria-label="Rotate 90° clockwise"
  >
    <RotateCw className="w-4 h-4" aria-hidden="true" />
  </button>
  <PopupMenu
    items={menuItems}
    trigger={
      <button
        type="button"
        className={cn(
          "relative flex items-center justify-center",
          "w-8 h-8 rounded-md",
          "text-muted-foreground/60 hover:text-foreground",
          "bg-transparent hover:bg-accent",
          "transition-transform duration-100",
          "active:scale-95",
          "focus-visible:outline-none",
          "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
        )}
        aria-label="More options"
      >
        <MoreHorizontal className="w-4 h-4" aria-hidden="true" />
      </button>
    }
    triggerType="click"
  />
</div>
</div>
<div
  className="relative w-full h-full min-h-[300px] overflow-auto"
  style={{
    backgroundColor:
      "color-mix(in oklch, var(--color-muted) 30%, transparent)",
  }}
>
  {isLoading && (
    <div className="absolute inset-0 flex items-center justify-center z-10">
      <RefreshCw
        className="w-10 h-10 text-muted-foreground/40 animate-spin"
        aria-hidden="true"
      />
      <span className="sr-only">Loading image...</span>
    </div>
  )}
  {hasError && (
    <div className="absolute inset-0 flex items-center justify-center z-10 p-8 text-center">
      <div>
        <RefreshCw

```

```

        className="w-12 h-12 text-destructive/60 mx-auto mb-3"
        aria-hidden="true"
      />
      <p className="text-destructive font-medium">
        Failed to load image
      </p>
      <p className="text-sm text-muted-foreground mt-1">
        The image source may be invalid or inaccessible
      </p>
    </div>
  </div>
)}
<img
  ref={imgRef}
  src={src}
  alt={alt}
  className={cn(
    "block max-w-none",
    "transition-transform duration-200 ease-out",
    "origin-center",
    isLoading && "opacity-0",
    hasError && "hidden",
  )}
  style={{
    transform: `scale(${scale}) rotate(${rotate}deg)`,
    transformOrigin: "center center",
  }}
  onLoad={handleImageLoad}
  onError={handleImageError}
  onDoubleClick={handleDoubleClick}
  onLoadStart={() => setIsLoading(true)}
/>
</div>
</div>
);
},
);

ImageViewer.displayName = "ImageViewer";

```


FILE: message-box.tsx
LOKASI: src\feedback

```
import React, { useEffect, useRef, useCallback } from "react";
import { Info, CheckCircle, AlertTriangle, Check } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
// ðŸš€ LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

```
// ðŸŸª LOCAL TYPE ISOLATION GATEWAY
export interface MessageBoxProps {
  open: boolean;
  onClose: () => void;
  title: string;
  description: string;
  variant?: "info" | "success" | "warning";
}
```

```
const variantStyles = {
  info: {
    icon: Info,
    iconColor: "text-blue-500",
    bgColor: "bg-blue-500/10",
    borderColor: "border-blue-500/30",
    ringColor: "focus-visible:ring-blue-500",
  },
  success: {
    icon: CheckCircle,
    iconColor: "text-emerald-500",
    bgColor: "bg-emerald-500/10",
    borderColor: "border-emerald-500/30",
    ringColor: "focus-visible:ring-emerald-500",
  },
  warning: {
    icon: AlertTriangle,
    iconColor: "text-amber-500",
    bgColor: "bg-amber-500/10",
    borderColor: "border-amber-500/30",
    ringColor: "focus-visible:ring-amber-500",
  },
} as const;
```

```
const variantLabels = {
  info: "Information",
  success: "Success",
  warning: "Warning",
} as const;
```

```
export function MessageBox({
  open,
  onClose,
```

```

    title,
    description,
    variant = "info",
  }: MessageBoxProps) {
    const dialogRef = useRef<HTMLDialogElement>(null);
    const previousActiveElement = useRef<HTMLElement | null>(null);
    const {
      icon: Icon,
      iconColor,
      bgColor,
      borderColor,
      ringColor,
    } = variantStyles[variant];

    const handleKeyDown = useCallback(
      (event: KeyboardEvent) => {
        if (event.key === "Escape") {
          event.preventDefault();
          onClose();
        }
      },
      [onClose],
    );

    const handleFocusTrap = useCallback((event: KeyboardEvent) => {
      if (event.key !== "Tab" || !dialogRef.current) return;

      const focusableElements = dialogRef.current.querySelectorAll<HTMLElement>(
        'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
      );

      const firstElement = focusableElements[0];
      const lastElement = focusableElements[focusableElements.length - 1];

      if (event.shiftKey && document.activeElement === firstElement) {
        event.preventDefault();
        lastElement?.focus();
      } else if (!event.shiftKey && document.activeElement === lastElement) {
        event.preventDefault();
        firstElement?.focus();
      }
    }, []);

    useEffect(() => {
      if (open) {
        previousActiveElement.current = document.activeElement as HTMLElement;
        document.addEventListener("keydown", handleKeyDown);
        document.addEventListener("keydown", handleFocusTrap);
        document.body.style.overflow = "hidden";

        dialogRef.current?.showModal();

        const focusableElement = dialogRef.current?.querySelector<HTMLElement>(
          'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
        );
        focusableElement?.focus();

        return () => {

```

```

        document.removeEventListener("keydown", handleKeyDown);
        document.removeEventListener("keydown", handleFocusTrap);
        document.body.style.overflow = "";
        previousActiveElement.current?.focus();
        dialogRef.current?.close();
    };
}
}, [open, handleKeyDown, handleFocusTrap]);

if (!open) return null;

return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
        <div
            className={cn(
                "fixed inset-0 z-40",
                "bg-black/50 backdrop-blur-sm",
                "transition-opacity duration-300 ease-out",
                "opacity-100",
            )}
            onClick={onClose}
            aria-hidden="true"
        />
        <dialog
            ref={dialogRef}
            className={cn(
                "relative z-50 w-full max-w-md",
                "bg-card/95 backdrop-blur-[var(--backdrop-blur)]",
                "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
                "rounded-[var(--radius-surface)]",
                "shadow-[var(--shadow-xl)]",
                "overflow-hidden",
                "transition-all duration-200 ease-out",
                open
                    ? "opacity-100 scale-100"
                    : "opacity-0 scale-95 pointer-events-none",
            )}
            role="alertdialog"
            aria-modal="true"
            aria-labelledby="message-box-title"
            aria-describedby="message-box-description"
        >
            <div className={cn("p-6", bgColor, borderColor, "border-t-4")}>
                <div className="flex items-start gap-4">
                    <div
                        className={cn(
                            "flex-shrink-0 w-10 h-10 rounded-full flex items-center justify-center",
                            bgColor,
                        )}
                        aria-hidden="true"
                    >
                        <Icon className={cn("w-5 h-5", iconColor)} strokeWidth={2.5} />
                    </div>
                    <div className="flex-1 min-w-0">
                        <div className="flex items-start justify-between gap-4">
                            <div>
                                <h2
                                    id="message-box-title"

```

```

        className="text-lg font-semibold text-foreground leading-tight"
      >
        {title}
      </h2>
      <p
        id="message-box-description"
        className="mt-2 text-sm text-muted-foreground leading-relaxed"
      >
        {description}
      </p>
    </div>
    <span
      className={cn(
        "flex-shrink-0 px-2 py-0.5 text-xs font-medium rounded-full",
        bgColor,
        "text-foreground",
      )}
      aria-hidden="true"
    >
      {variantLabels[variant]}
    </span>
  </div>
</div>
</div>
<div className="flex items-center justify-end gap-3 p-4 border-t border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]">
  <button
    type="button"
    onClick={onClose}
    className={cn(
      "inline-flex items-center justify-center gap-2",
      "h-10 px-4 py-2 text-base font-medium",
      "rounded-[var(--radius-surface)]",
      "bg-brand-primary text-white",
      "hover:bg-brand-hover",
      "active:scale-95 active:bg-brand-primary",
      "transition-all duration-150 ease-out",
      "focus-visible:outline-none",
      ringColor,
      "focus-visible:ring-2 focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
    )}
    autoFocus
  >
    <Check className="w-4 h-4" aria-hidden="true" />
    OK
  </button>
</div>
</dialog>
</div>
);
}

export default MessageBox;

```

FILE: pdf-viewer.tsx
LOKASI: src\feedback

```
import React, {
  forwardRef,
  useState,
  useEffect,
  useRef,
  useCallback,
  useImperativeHandle,
} from "react";
import {
  FileText,
  ShieldAlert,
  RefreshCw,
  ExternalLink,
  MoreHorizontal,
} from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../hint-box";
import {
  PopupMenu,
  type PopupMenuConfig,
  type PopupMenuItemConfig,
} from "../popup-menu";
import { THEME_CONFIG } from "@config/theme-constants";

export interface PdfViewerProps extends React.HTMLAttributes<HTMLDivElement> {
  src?: string;
  title?: string;
  fallbackText?: string;
  height?: string | number;
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

export const PdfViewer = forwardRef<HTMLDivElement, PdfViewerProps>(
  (
    {
      src,
      title = "PDF Document",
      fallbackText = "Unable to display PDF. Your browser may not support embedded PDFs.",
      height = THEME_CONFIG.layout.maxGridHeight,
      className,
      style,
      hint,
      popupMenu,
      ...props
    },
    ref,
  ) => {
    const [isLoading, setIsLoading] = useState(false);
    const [hasError, setHasError] = useState(false);
    const [iframeSrc, setIframeSrc] = useState<string | undefined>(src);
    const iframeRef = useRef<HTMLIFrameElement>(null);
```

```

const containerRef = useRef<HTMLDivElement>(null);

useImperativeHandle(ref, () => containerRef.current as HTMLDivElement, []);

useEffect(() => {
  if (src !== iframeSrc) {
    setIframeSrc(src);
    setHasError(false);
    setIsLoading(true);
  }
}, [src, iframeSrc]);

const handleLoad = useCallback(() => {
  setIsLoading(false);
  setHasError(false);
}, []);

const handleError = useCallback(() => {
  setIsLoading(false);
  setHasError(true);
}, []);

const handleRetry = useCallback(() => {
  if (iframeRef.current && iframeSrc) {
    setIsLoading(true);
    setHasError(false);
    iframeRef.current.src = iframeSrc;
  }
}, [iframeSrc]);

const handleOpenInNewTab = useCallback(() => {
  if (src) {
    window.open(src, "_blank", "noopener,noreferrer");
  }
}, [src]);

const containerStyle: React.CSSProperties = {
  height: typeof height === "number" ? `${height}px` : height,
  ...style,
};

const menuItems: PopupMenuItemConfig[] = [
  {
    id: "open-new-tab",
    label: "Open in New Tab",
    icon: ExternalLink,
    onClick: handleOpenInNewTab,
    disabled: !src,
  },
  {
    id: "retry",
    label: "Retry",
    icon: RefreshCw,
    onClick: handleRetry,
    disabled: !hasError,
  },
  {
    id: "copy-link",

```

```

    label: "Copy Link",
    icon: ExternalLink,
    onClick: () => {
      if (src) {
        navigator.clipboard.writeText(src);
      }
    },
    disabled: !src,
  },
];

return (
  <div
    ref={containerRef}
    className={cn(
      "relative",
      "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-surface",
      "overflow-hidden",
      "flex flex-col",
      className,
    )}
    style={containerStyle}
    {...props}
    onContextMenu={(e) => popupMenu?.trigger(e)}
  >
    {hint && (
      <HintBox content={hint} className="m-3 mb-0">
        <span className="sr-only">hint</span>
      </HintBox>
    )}
    <div
      className={cn(
        "flex items-center justify-between p-3",
        "bg-[color-mix(in_oklch,var(--color-card)_95%,transparent)]",
        "backdrop-blur-[var(--backdrop-blur)]",
        "border-b border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
      )}
      role="toolbar"
      aria-label="PDF viewer controls"
    >
      <div className="flex items-center gap-3 min-w-0">
        <FileText
          className="w-5 h-5 text-muted-foreground/60 flex-shrink-0"
          aria-hidden="true"
        />
        <div className="min-w-0">
          <h3 className="text-sm font-medium text-foreground truncate">
            {title}
          </h3>
          {src && (
            <p className="text-xs text-muted-foreground/70 truncate max-w-[300px]">
              {src}
            </p>
          )}
        </div>
      </div>
    </div>
  )

```

```

<div className="flex items-center gap-2">
  {isLoading && (
    <div className="flex items-center gap-2 text-xs text-muted-foreground">
      <RefreshCw
        className="w-4 h-4 animate-spin"
        aria-hidden="true"
      />
      <span>Loading...</span>
    </div>
  )}
  {hasError && (
    <div className="flex items-center gap-2 text-xs text-destructive/80">
      <ShieldAlert
        className="w-4 h-4 flex-shrink-0"
        aria-hidden="true"
      />
      <span>Failed to load</span>
    </div>
  )}
  <PopupMenu
    items={menuItems}
    trigger={
      <button
        type="button"
        className={cn(
          "relative flex items-center justify-center",
          "w-8 h-8 rounded-md",
          "text-muted-foreground/60 hover:text-foreground",
          "bg-transparent hover:bg-accent",
          "transition-transform duration-100",
          "active:scale-95",
          "focus-visible:outline-none",
          "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-
500",
        )}
        aria-label="More options"
      >
        <MoreHorizontal className="w-4 h-4" aria-hidden="true" />
      </button>
    }
    triggerType="click"
  />
  <button
    type="button"
    onClick={handleOpenInNewTab}
    disabled={!src}
    className={cn(
      "relative flex items-center justify-center gap-1.5",
      "px-3 py-1.5 rounded-md text-xs font-medium",
      "text-muted-foreground/60 hover:text-foreground",
      "bg-transparent hover:bg-accent",
      "transition-transform duration-100",
      "active:scale-95",
      "focus-visible:outline-none",
      "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
      "disabled:opacity-50 disabled:pointer-events-none",
    )}
    aria-label="Open PDF in new tab"
  >

```



```

    >
    <ExternalLink className="w-3.5 h-3.5" aria-hidden="true" />
    <span className="hidden sm:inline">Open</span>
  </button>
  {hasError && (
    <button
      type="button"
      onClick={handleRetry}
      className={cn(
        "relative flex items-center justify-center",
        "w-8 h-8 rounded-md",
        "text-muted-foreground/60 hover:text-foreground",
        "bg-transparent hover:bg-accent",
        "transition-transform duration-100",
        "active:scale-95",
        "focus-visible:outline-none",
        "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-
500",
      )}
      aria-label="Retry loading PDF"
    >
    <RefreshCw className="w-4 h-4" aria-hidden="true" />
    </button>
  )}
</div>
</div>
<div
  className="relative flex-1 w-full overflow-hidden"
  style={{ minHeight: 0 }}
>
  {isLoading && (
    <div className="absolute inset-0 flex items-center justify-center z-10 bg-card/90">
      <div className="text-center p-8">
        <RefreshCw
          className="w-12 h-12 text-muted-foreground/40 mx-auto mb-3 animate-spin"
          aria-hidden="true"
        />
        <p className="text-muted-foreground">Loading document...</p>
      </div>
    </div>
  )}
  {hasError && (
    <div className="absolute inset-0 flex items-center justify-center z-10 p-8 bg-card/90">
      <div className="text-center max-w-md">
        <ShieldAlert
          className="w-16 h-16 text-destructive/60 mx-auto mb-4"
          aria-hidden="true"
        />
        <h4 className="text-lg font-medium text-foreground mb-2">
          Unable to Display PDF
        </h4>
        <p className="text-sm text-muted-foreground mb-6">
          {fallbackText}
        </p>
        <div className="flex items-center justify-center gap-3">
          <button
            type="button"
            onClick={handleRetry}

```

```

500",
    className={cn(
      "relative flex items-center justify-center gap-2",
      "px-4 py-2 rounded-md text-sm font-medium",
      "bg-brand-primary text-white",
      "hover:bg-brand-hover",
      "transition-transform duration-100",
      "active:scale-95",
      "focus-visible:outline-none",
      "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-
500",
    )}
  >
    <RefreshCw className="w-4 h-4" aria-hidden="true" />
    <span>Retry</span>
  </button>
  <button
    type="button"
    onClick={handleOpenInNewTab}
    disabled={!src}
    className={cn(
      "relative flex items-center justify-center gap-2",
      "px-4 py-2 rounded-md text-sm font-medium",
      "bg-transparent border border-border",
      "text-foreground hover:bg-accent",
      "transition-transform duration-100",
      "active:scale-95",
      "focus-visible:outline-none",
      "focus-visible:ring-2 focus-visible:ring-amber-500/20 focus-visible:border-amber-
500",
      "disabled:opacity-50 disabled:pointer-events-none",
    )}
  >
    <ExternalLink className="w-4 h-4" aria-hidden="true" />
    <span>Open in New Tab</span>
  </button>
</div>
</div>
</div>
)}
<iframe
  ref={iframeRef}
  src={iframeSrc ?? ""}
  title={title}
  className={cn(
    "absolute inset-0 w-full h-full border-0",
    "bg-transparent",
    isLoading && "opacity-0",
    hasError && "hidden",
  )}
  sandbox="allow-scripts allow-same-origin"
  allow="fullscreen"
  onLoad={handleLoad}
  onError={handleError}
  loading="lazy"
/>
{!iframeSrc && !isLoading && !hasError && (
  <div className="absolute inset-0 flex items-center justify-center p-8 bg-card/90">
    <div className="text-center">

```

```
        <FileText
            className="w-16 h-16 text-muted-foreground/40 mx-auto mb-4"
            aria-hidden="true"
        />
        <p className="text-muted-foreground">No PDF source provided</p>
    </div>
</div>
    )}
</div>
</div>
);
},
);

PdfViewer.displayName = "PdfViewer";
```

FILE: popup-form.tsx
LOKASI: src\feedback

```
import React, { useEffect, useRef, useCallback } from "react";
import { X, LucideIcon } from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export interface PopupActionConfig {
  label: string;
  onClick: () => void | Promise<void>;
  variant?: "brand" | "secondary" | "destructive";
  disabled?: boolean;
}

export interface PopupFormProps {
  isOpen: boolean;
  onClose: () => void;
  title: string;
  description?: string;
  icon?: LucideIcon;
  mode?: "modal" | "non-modal";
  width?: number | string;
  height?: number | string;
  headerActions?: Array<{
    icon: LucideIcon;
    onClick: () => void;
    tooltip?: string;
  }>;
  footerActions?: {
    primary?: PopupActionConfig;
    secondary?: PopupActionConfig;
  };
  children: React.ReactNode;
  hint?: string;
}

export function PopupForm({
  isOpen,
  onClose,
  title,
  description,
  icon: IconHeader,
  mode = "modal",
  width = 540,
  height,
  headerActions = [],
  footerActions,
  children,
  hint,
}: PopupFormProps) {
  const modalRef = useRef<HTMLDivElement>(null);
  const overlayRef = useRef<HTMLDivElement>(null);
  const previousActiveElementRef = useRef<HTMLElement | null>(null);
```

```

const focusableElementsRef = useRef<HTMLInputElement[]>([]);

const getFocusableElements = useCallback(() => {
  if (!modalRef.current) return [];
  const selector = [
    'button:not([disabled]):not([aria-hidden="true"])',
    '[href]:not([disabled]):not([aria-hidden="true"])',
    'input:not([disabled]):not([aria-hidden="true"])',
    'select:not([disabled]):not([aria-hidden="true"])',
    'textarea:not([disabled]):not([aria-hidden="true"])',
    '[tabindex]:not([tabindex="-1"]):not([disabled]):not([aria-hidden="true"])',
    '[contenteditable="true"]:not([disabled]):not([aria-hidden="true"])',
  ].join(",");
  return Array.from(modalRef.current.querySelectorAll<HTMLInputElement>(selector));
}, []);

const trapFocus = useCallback(
  (event: KeyboardEvent) => {
    if (event.key !== "Tab") return;
    const focusableElements = getFocusableElements();
    if (focusableElements.length === 0) return;

    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length - 1];

    if (event.shiftKey) {
      if (document.activeElement === firstElement) {
        event.preventDefault();
        lastElement.focus();
      }
    } else {
      if (document.activeElement === lastElement) {
        event.preventDefault();
        firstElement.focus();
      }
    }
  },
  [getFocusableElements],
);

const handleKeyDown = useCallback(
  (event: KeyboardEvent) => {
    if (event.key === "Escape") {
      event.preventDefault();
      onClose();
      return;
    }
    trapFocus(event);
  },
  [onClose, trapFocus],
);

const handleOverlayClick = useCallback(
  (event: React.MouseEvent<HTMLDivElement>) => {
    if (mode === "modal" && event.target === overlayRef.current) {
      onClose();
    }
  },
);

```

```

    [mode, onClose],
  );

  useEffect(() => {
    if (isOpen) {
      previousActiveElementRef.current = document.activeElement as HTMLElement;
      document.addEventListener("keydown", handleKeyDown);
      document.body.style.overflow = "hidden";

      const focusableElements = getFocusableElements();
      focusableElementsRef.current = focusableElements;

      if (focusableElements.length > 0) {
        focusableElements[0].focus();
      } else {
        modalRef.current?.focus();
      }
    }

    return () => {
      document.removeEventListener("keydown", handleKeyDown);
      document.body.style.overflow = "";
      if (previousActiveElementRef.current) {
        previousActiveElementRef.current.focus();
      }
    };
  }, [isOpen, handleKeyDown, getFocusableElements]);

  useEffect(() => {
    if (isOpen && modalRef.current) {
      const focusableElements = getFocusableElements();
      focusableElementsRef.current = focusableElements;
      if (focusableElements.length > 0) {
        focusableElements[0].focus();
      }
    }
  }, [isOpen, getFocusableElements]);

  if (!isOpen) return null;

  const containerWidth = typeof width === "number" ? `${width}px` : width;
  const containerHeight = height
    ? typeof height === "number"
      ? `${height}px`
      : height
    : "auto";

  const headerActionButtons = headerActions.slice(0, 4).map((action, index) => (
    <button
      key={index}
      type="button"
      onClick={action.onClick}
      className={cn(
        "relative flex items-center justify-center",
        "w-8 h-8 rounded-lg",
        "text-muted-foreground/60 hover:text-foreground",
        "bg-transparent hover:bg-accent",
        "transition-all duration-150 ease-out",

```

```

        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
        "disabled:opacity-50 disabled:pointer-events-none",
    )}
    aria-label={action.tooltip || "Action"}
  >
    <action.icon className="w-4 h-4" aria-hidden="true" />
  </button>
));

const closeButton = (
  <button
    type="button"
    onClick={onClose}
    className={cn(
      "relative flex items-center justify-center",
      "w-8 h-8 rounded-lg",
      "text-muted-foreground/60 hover:text-foreground",
      "bg-transparent hover:bg-accent",
      "transition-all duration-150 ease-out",
      "active:scale-95",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
    )}
    aria-label="Close"
  >
    <X className="w-4 h-4" aria-hidden="true" />
  </button>
);

const renderFooterActions = () => {
  if (!footerActions) return null;

  const secondaryButton = footerActions.secondary ? (
    <button
      type="button"
      onClick={footerActions.secondary.onClick}
      disabled={footerActions.secondary.disabled}
      className={cn(
        "relative inline-flex items-center justify-center",
        "px-4 py-2.5 text-sm font-medium",
        "rounded-lg",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
        "disabled:opacity-50 disabled:pointer-events-none",
        footerActions.secondary.variant === "destructive"
          ? "text-destructive bg-destructive/10 hover:bg-destructive/20"
          : "text-muted-foreground bg-muted hover:bg-muted/80",
      )}
    >
      {footerActions.secondary.label}
    </button>
  ) : null;

  const primaryButton = footerActions.primary ? (
    <button
      type="button"

```

```

onClick={footerActions.primary.onClick}
disabled={footerActions.primary.disabled}
className={cn(
  "relative inline-flex items-center justify-center",
  "px-4 py-2.5 text-sm font-medium",
  "rounded-lg",
  "transition-[var(--transition-fast)]",
  "active:scale-95",
  "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-
2",
  "disabled:opacity-50 disabled:pointer-events-none",
  "bg-brand-primary text-white",
  "hover:bg-brand-hover",
)}
>
  {footerActions.primary.label}
</button>
) : null;

return (
  <div className="flex items-center justify-end gap-3">
    {secondaryButton}
    {primaryButton}
  </div>
);
};

return (
  <div
    ref={overlayRef}
    className={cn(
      "fixed inset-0 z-50 flex items-center justify-center",
      "p-4",
      mode === "modal"
        ? "bg-black/50 backdrop-blur-[var(--backdrop-blur)]"
        : "bg-transparent pointer-events-none",
      "transition-all duration-200 ease-out",
      "opacity-0 data-[state=open]:opacity-100",
    )}
    data-state={isOpen ? "open" : "closed"}
    onClick={handleOverlayClick}
    role={mode === "modal" ? "dialog" : "document"}
    aria-modal={mode === "modal" ? "true" : "false"}
    aria-labelledby="popup-form-title"
    aria-describedby={description ? "popup-form-description" : undefined}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span aria-hidden="true" style={{ display: "none" }} />
      </HintBox>
    )}
    <div
      ref={modalRef}
      tabIndex={-1}
      className={cn(
        "relative w-full max-h-[90vh] overflow-hidden",
        "bg-card backdrop-blur-[var(--backdrop-blur)]",
        "border border-[color-mix(in_oklch_var(--border)_50%_transparent)]",

```



```

    "rounded-surface-lg",
    "shadow-2x1",
    "font-sans",
    "flex flex-col",
    "transition-all duration-300 ease-out",
    "opacity-0 scale-95 data-[state=open]:opacity-100 data-[state=open]:scale-100",
    "translate-y-4 data-[state=open]:translate-y-0",
  )}
  style={{
    width: containerWidth,
    height: containerHeight,
  }}
  data-state={isOpen ? "open" : "closed"}
  role={mode === "modal" ? "dialog" : "document"}
  aria-modal={mode === "modal" ? "true" : "false"}
}
<header
  className={cn(
    "flex items-start justify-between gap-4",
    "px-6 py-4",
    "border-b border-[color-mix(in_oklch_var(--border)_50%_transparent)]",
    "flex-shrink-0",
  )}
>
  <div className="flex items-start gap-4 flex-1 min-w-0">
    {IconHeader && (
      <div
        className={cn(
          "flex-shrink-0 flex items-center justify-center",
          "w-10 h-10 rounded-lg",
          "bg-brand-primary/10",
          "text-brand-primary",
          "ring-1 ring-brand-primary/20",
        )}
        aria-hidden="true"
      >
        <IconHeader className="w-5 h-5" />
      </div>
    )}
    <div className="min-w-0">
      <h2
        id="popup-form-title"
        className={cn(
          "text-lg font-semibold text-card-foreground",
          "truncate",
          "font-sans",
        )}
      >
        {title}
      </h2>
      {description && (
        <p
          id="popup-form-description"
          className={cn(
            "mt-1 text-sm text-muted-foreground",
            "truncate",
            "font-sans",
          )}
        >

```

```

        >
        {description}
      </p>
    )}
  </div>
</div>
<div className="flex items-center gap-2 flex-shrink-0">
  {headerActionButtons}
  {closeButton}
</div>
</header>

<main className={cn("flex-1 overflow-y-auto", "p-6", "font-sans")}>
  {children}
</main>

{footerActions && (
  <footer
    className={cn(
      "flex items-center justify-end gap-3",
      "px-6 py-4",
      "border-t border-[color-mix(in_oklch_var(--border)_50%_transparent)]",
      "bg-card/50 backdrop-blur-[var(--backdrop-blur)]",
      "flex-shrink-0",
    )}
  >
    {renderFooterActions()}
  </footer>
)}
</div>
</div>
);
}

export default PopupForm;

```

FILE: popup-menu.tsx
LOKASI: src\feedback

```
import React, { useRef, useEffect, useState, useCallback } from "react";
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu";
import { LucideIcon } from "lucide-react";
import { cn } from "@lib/Utils";
import { Checkbox } from "@components/selector/checkbox";

export interface PopupMenuConfig {
  trigger: (e: React.MouseEvent, recordRow?: Record<string, unknown>) => void;
  [key: string]: unknown;
}

// ðŸŸ¡ LOCAL TYPE ISOLATION GATEWAY
export interface PopupMenuItemConfig {
  id: string;
  label: string;
  icon?: LucideIcon;
  shortcut?: string;
  disabled?: boolean;
  destructive?: boolean;
  onClick?: () => void;
  children?: PopupMenuItemConfig[]; // Up to 3 layers cascading
  // Toggleable Checkbox capability
  isChecked?: boolean;
  checked?: boolean;
  onCheckedChange?: (checked: boolean) => void;
  // Hint support for menu items
  hint?: string;
}

export interface PopupMenuProps {
  items: PopupMenuItemConfig[];
  trigger: React.ReactNode;
  triggerType?: "click" | "context-menu";
  className?: string;
}

// Internal types for recursive rendering
interface SubMenuContextValue {
  level: number;
  parentRect?: DOMRect;
  openSubMenus: Set<string>;
  registerSubMenu: (id: string, element: HTMLElement | null) => void;
  unregisterSubMenu: (id: string) => void;
  isSubMenuOpen: (id: string) => boolean;
}

const SubMenuContext = React.createContext<SubMenuContextValue | null>(null);

function useSubMenuContext() {
  const context = React.useContext(SubMenuContext);
  if (!context) {
    throw new Error("SubMenu components must be rendered within a PopupMenu");
  }
}
```

```

    return context;
  }

  // Recursive menu item component
  interface MenuItemProps {
    item: PopupMenuItemConfig;
    level: number;
    onClose: () => void;
  }

  function MenuItem({ item, level, onClose }: MenuItemProps) {
    const { openSubMenus, registerSubMenu, unregisterSubMenu, isSubMenuOpen } =
      useSubMenuContext();
    const itemRef = useRef<HTMLDivElement>(null);
    const subMenuRef = useRef<HTMLDivElement>(null);
    const [subMenuPosition, setSubMenuPosition] = useState<{
      top: number;
      left: number;
    } | null>(null);
    const isOpen = isSubMenuOpen(item.id);
    const hasChildren = item.children && item.children.length > 0;
    const isCheckboxItem = item.isCheckbox === true;

    // Register/unregister submenu element
    useEffect(() => {
      registerSubMenu(item.id, subMenuRef.current);
      return () => unregisterSubMenu(item.id);
    }, [item.id, registerSubMenu, unregisterSubMenu]);

    // Calculate submenu position to avoid viewport clipping
    const calculateSubMenuPosition = useCallback(() => {
      if (!itemRef.current) return;

      const rect = itemRef.current.getBoundingClientRect();
      const viewportWidth = window.innerWidth;
      const viewportHeight = window.innerHeight;
      const subMenuWidth = 240; // Approximate submenu width
      const subMenuHeight = 200; // Approximate submenu height

      let left = rect.right;
      let top = rect.top;

      // Check right boundary
      if (left + subMenuWidth > viewportWidth) {
        left = rect.left - subMenuWidth;
      }

      // Check bottom boundary
      if (top + subMenuHeight > viewportHeight) {
        top = viewportHeight - subMenuHeight;
      }

      // Check top boundary
      if (top < 0) {
        top = 0;
      }

      setSubMenuPosition({ top, left });
    }, [itemRef, viewportWidth, viewportHeight, subMenuWidth, subMenuHeight]);
  }

```

```

    }, []);

    useEffect(() => {
      if (isOpen) {
        calculateSubMenuPosition();
      }
    }, [isOpen, calculateSubMenuPosition]);

    const handleKeyDown = (event: React.KeyboardEvent) => {
      switch (event.key) {
        case "Enter":
        case " ":
          event.preventDefault();
          if (!item.disabled && item.onClick) {
            item.onClick();
            onClose();
          }
          break;
        case "ArrowRight":
          event.preventDefault();
          if (hasChildren && !item.disabled) {
            // Open submenu - handled by parent context
          }
          break;
        case "ArrowLeft":
          event.preventDefault();
          // Close submenu - handled by parent context
          break;
      }
    };

    const handleMouseEnter = () => {
      if (hasChildren && !item.disabled) {
        // Submenu opening is handled by the parent DropdownMenuSub
      }
    };

    if (item.id === "divider") {
      return (
        <DropdownMenuPrimitive.Separator
          className="h-px bg-border/50 my-1"
          key={item.id}
        />
      );
    }

    // Render CheckboxItem for toggleable checkbox items
    if (isCheckboxItem) {
      return (
        <DropdownMenuPrimitive.CheckboxItem
          ref={itemRef}
          key={item.id}
          disabled={item.disabled}
          checked={item.checked ?? false}
          onCheckedChange={item.onCheckedChange}
          onSelect={(e) => e.preventDefault()} // ANTI-CLOSE GUARD: Prevent dropdown from closing on
checkbox toggle
          className={cn(

```

```

        "relative flex cursor-default select-none items-center rounded-sm px-3 py-2 text-sm outline-
none transition-all duration-100",
        "focus:bg-brand-primary/10 focus:text-brand-primary active:scale-[0.98]",
        "data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
        item.destructive
          ? "text-destructive focus:bg-destructive/10 focus:text-destructive"
          : "text-text-main",
        level > 0 && "pl-8",
      )}
      onKeyDown={handleKeyDown}
      onMouseEnter={handleMouseEnter}
    >
      {item.icon && (
        <span className="mr-3 h-4 w-4 flex-shrink-0" aria-hidden="true">
          <item.icon className="h-4 w-4 stroke-[2px]" />
        </span>
      )}
      <Checkbox
        checked={item.checked ?? false}
        readOnly
        className="pointer-events-none mr-3 shrink-0 scale-90"
      />
      <span className="flex-1 truncate">{item.label}</span>
      {item.shortcut && (
        <span className="ml-auto text-text-sub text-xs font-mono tracking-wide">
          {item.shortcut}
        </span>
      )}

      {/* Submenu Content for checkbox items with children */}
      {hasChildren && (
        <DropdownMenuPrimitive.SubContent
          ref={subMenuRef}
          sideOffset={4}
          align="start"
          collisionPadding={8}
          className={cn(
            "z-50 min-w-[12rem] overflow-hidden rounded-[12px] border border-[color-
mix(in_oklch,var(--color-border)_40%,transparent)]",
            "bg-card/85 backdrop-blur-[12px] shadow-xl p-1.5 text-text-main",
            "data-[state=open]:animate-in data-[state=closed]:animate-out",
            "data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
            "data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2",
            "data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
          )}
          style={
            subMenuPosition
              ? {
                  position: "fixed",
                  top: subMenuPosition.top,
                  left: subMenuPosition.left,
                }
              : undefined
          }
        >
          <SubMenuContext.Provider
            value={{
              level: level + 1,

```

```

        openSubMenus,
        registerSubMenu,
        unregisterSubMenu,
        isSubMenuOpen,
      }}
    >
    {item.children?.map((child) => (
      <MenuItem
        key={child.id}
        item={child}
        level={level + 1}
        onClose={onClose}
      />
    ))}
    </SubMenuContext.Provider>
  </DropdownMenuPrimitive.SubContent>
)}
</DropdownMenuPrimitive.CheckboxItem>
);
}

// Standard Item rendering
return (
  <DropdownMenuPrimitive.Item
    ref={itemRef}
    key={item.id}
    disabled={item.disabled}
    onSelect={
      item.onClick
      ? () => {
          item.onClick?.();
          onClose();
        }
      : undefined
    }
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm px-3 py-2 text-sm outline-
none transition-all duration-100",
      "focus:bg-brand-primary/10 focus:text-brand-primary active:scale-[0.98]",
      "data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      item.destructive
        ? "text-destructive focus:bg-destructive/10 focus:text-destructive"
        : "text-text-main",
      level > 0 && "pl-8",
    )}
    onKeyDown={handleKeyDown}
    onMouseEnter={handleMouseEnter}
  >
    {item.icon && (
      <span className="mr-3 h-4 w-4 flex-shrink-0" aria-hidden="true">
        <item.icon className="h-4 w-4 stroke-[2px]" />
      </span>
    )}
    <span className="flex-1 truncate">{item.label}</span>
    {item.hint && (
      <span className="ml-2 text-text-sub text-xs font-mono tracking-wide opacity-70">
        {item.hint}
      </span>
    )}
  </DropdownMenuPrimitive.Item>
);

```

```

    })
    {hasChildren && (
      <DropdownMenuPrimitive.SubTrigger
        className="flex items-center ml-auto"
        aria-label="Open submenu"
      >
        <span className="text-text-sub text-xs mr-2">â–¶</span>
      </DropdownMenuPrimitive.SubTrigger>
    })
    {item.shortcut && (
      <span className="ml-auto text-text-sub text-xs font-mono tracking-wide">
        {item.shortcut}
      </span>
    })

    { /* Submenu Content */ }
    {hasChildren && (
      <DropdownMenuPrimitive.SubContent
        ref={subMenuRef}
        sideOffset={4}
        align="start"
        collisionPadding={8}
        className={cn(
          "z-50 min-w-[12rem] overflow-hidden rounded-[12px] border border-[color-mix(in_oklch,var(--
color-border)_40%,transparent)]",
          "bg-card/85 backdrop-blur-[12px] shadow-xl p-1.5 text-text-main",
          "data-[state=open]:animate-in data-[state=closed]:animate-out",
          "data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
          "data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2",
          "data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        )}
        style={
          subMenuPosition
            ? {
                position: "fixed",
                top: subMenuPosition.top,
                left: subMenuPosition.left,
              }
            : undefined
        }
      >
        <SubMenuContext.Provider
          value={{
            level: level + 1,
            openSubMenus,
            registerSubMenu,
            unregisterSubMenu,
            isSubMenuOpen,
          }}
        >
          {item.children?.map((child) => (
            <MenuItem
              key={child.id}
              item={child}
              level={level + 1}
              onClose={onClose}
            />
          ))}
        </SubMenuContext.Provider>
      </DropdownMenuPrimitive.SubContent>
    )}
  )}

```



```

        </SubMenuContext.Provider>
      </DropdownMenuPrimitive.SubContent>
    )}
  </DropdownMenuPrimitive.Item>
);
}

// Main PopupMenu component
export function PopupMenu({
  items,
  trigger,
  triggerType = "context-menu",
  className,
}: PopupMenuProps) {
  const [openSubMenus, setOpenSubMenus] = useState<Set<string>>(new Set());
  const [, setTriggerRef] = useState<HTMLElement | null>(null);
  const menuRef = useRef<HTMLDivElement>(null);

  const registerSubMenu = useCallback(() => {
    // Registration logic for submenu tracking
  }, []);

  const unregisterSubMenu = useCallback(() => {
    // Unregistration logic
  }, []);

  const isSubMenuOpen = useCallback(
    (id: string) => {
      return openSubMenus.has(id);
    },
    [openSubMenus],
  );

  const handleOpenChange = useCallback((open: boolean) => {
    if (!open) {
      setOpenSubMenus(new Set());
    }
  }, []);

  const handleContextMenu = useCallback(
    (event: React.MouseEvent) => {
      if (triggerType === "context-menu") {
        event.preventDefault();
        event.stopPropagation();
      }
    },
    [triggerType],
  );

  const handleKeyDown = useCallback((event: React.KeyboardEvent) => {
    // Global keyboard navigation for the menu
    if (event.key === "Escape") {
      // Close all submenus
      setOpenSubMenus(new Set());
    }
  }, []);

  const contextValue: SubMenuContextValue = {

```

```

    level: 0,
    openSubMenus,
    registerSubMenu,
    unregisterSubMenu,
    isSubMenuOpen,
  };

const renderTrigger = () => {
  if (triggerType === "context-menu") {
    return (
      <DropdownMenuPrimitive.Trigger asChild>
        <div
          ref={setTriggerRef}
          className={cn("relative", className)}
          onContextMenu={handleContextMenu}
          onKeyDown={handleKeyDown}
        >
          {trigger}
        </div>
      </DropdownMenuPrimitive.Trigger>
    );
  }

  return (
    <DropdownMenuPrimitive.Trigger asChild>
      <div
        ref={setTriggerRef}
        className={cn("relative", className)}
        onKeyDown={handleKeyDown}
      >
        {trigger}
      </div>
    </DropdownMenuPrimitive.Trigger>
  );
};

return (
  <DropdownMenuPrimitive.Root onOpenChange={handleOpenChange}>
    <SubMenuContext.Provider value={contextValue}>
      {renderTrigger()}

      <DropdownMenuPrimitive.Portal>
        <DropdownMenuPrimitive.Content
          ref={menuRef}
          side="bottom"
          sideOffset={4}
          align="start"
          collisionPadding={8}
          className={cn(
            "z-50 min-w-[12rem] overflow-hidden rounded-[12px] border border-[color-
mix(in_oklch,var(--color-border)_40%,transparent))",
            "bg-card/85 backdrop-blur-[12px] shadow-xl p-1.5 text-text-main",
            "data-[state=open]:animate-in data-[state=closed]:animate-out",
            "data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
            "data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2",
            "data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
          )}
        >

```

```
        {items.map((item) => (  
            <MenuItem  
                key={item.id}  
                item={item}  
                level={0}  
                onClose={() => handleOpenChange(false)}  
            />  
        ))}  
    </DropdownMenuPrimitive.Content>  
    </DropdownMenuPrimitive.Portal>  
    </SubMenuContext.Provider>  
    </DropdownMenuPrimitive.Root>  
);  
}
```

```

FILE: report-plugins-core.ts
LOKASI: src\feedback

```

```

import React from "react";

/**
 * Core reactive state messenger for the report workspace.
 * Acts as the single source of truth for all plugin interactions.
 */
export interface ReportWorkspaceContext {
  htmlContent: string;
  zoom: number;
  currentPage: number;
  totalPages: number;
  isDarkMode: boolean;
  autoFitScale: number;
  setZoom: React.Dispatch<React.SetStateAction<number>>>;
  setCurrentPage: React.Dispatch<React.SetStateAction<number>>>;
  setIsDarkMode: React.Dispatch<React.SetStateAction<boolean>>>;
  triggerAutoFit: () => void;
}

/**
 * Master blueprint interface for independent plugin components.
 * Defines the complete contract for plugin lifecycle and rendering hooks.
 */
export interface ReportWorkspacePlugin {
  id: string;
  name: string;
  onInit?: (context: ReportWorkspaceContext) => void;
  onDOMRender?: (
    container: HTMLDivElement,
    context: ReportWorkspaceContext,
  ) => void;
  renderToolbarAction?: (context: ReportWorkspaceContext) => React.ReactNode;
  renderContextMenuItems?: (
    context: ReportWorkspaceContext,
    targetElement: HTMLElement,
  ) => React.ReactNode;
}

/**
 * Immutable plugin registry for managing plugin lifecycle.
 * Provides type-safe registration and retrieval of workspace plugins.
 */
export interface ReportPluginRegistry {
  readonly plugins: ReadonlyMap<string, Readonly<ReportWorkspacePlugin>>;
  readonly register: (plugin: Readonly<ReportWorkspacePlugin>) => void;
  readonly unregister: (pluginId: string) => boolean;
  readonly get: (
    pluginId: string,
  ) => Readonly<ReportWorkspacePlugin> | undefined;
  readonly getAll: () => ReadonlyArray<Readonly<ReportWorkspacePlugin>>;
  readonly has: (pluginId: string) => boolean;
  readonly clear: () => void;
}

```

```

}

/**
 * Immutable configuration gateway for plugin initialization.
 * Provides type-safe configuration injection for plugin instances.
 */
export interface ReportPluginConfigGateway<
  Config extends Record<string, unknown>,
> {
  readonly config: Readonly<Config>;
  readonly getConfig: <Key extends keyof Config>(key: Key) => Config[Key];
  readonly hasConfig: (key: string) => boolean;
  readonly mergeConfig: <PartialConfig extends Partial<Config>>>(
    partial: PartialConfig,
  ) => ReportPluginConfigGateway<Config & PartialConfig>;
}

/**
 * Immutable plugin execution context for runtime operations.
 * Provides controlled access to workspace state and plugin utilities.
 */
export interface ReportPluginExecutionContext<
  Config extends Record<string, unknown> = Record<string, unknown>,
> {
  readonly workspace: Readonly<ReportWorkspaceContext>;
  readonly config: ReportPluginConfigGateway<Config>;
  readonly registry: Readonly<ReportPluginRegistry>;
  readonly emitEvent: <
    EventName extends string,
    Payload extends Record<string, unknown>,
  >(
    eventName: EventName,
    payload: Payload,
  ) => void;
  readonly onEvent: <
    EventName extends string,
    Payload extends Record<string, unknown>,
  >(
    eventName: EventName,
    handler: (payload: Payload) => void,
  ) => () => void;
}

/**
 * Immutable plugin factory for creating configured plugin instances.
 * Ensures type-safe plugin instantiation with configuration validation.
 */
export interface ReportPluginFactory<
  Plugin extends ReportWorkspacePlugin,
  Config extends Record<string, unknown> = Record<string, unknown>,
> {
  readonly create: (
    config: Readonly<Config>,
    context: Readonly<ReportPluginExecutionContext<Config>>,
  ) => Readonly<Plugin>;
  readonly validateConfig: (config: unknown) => config is Config;
  readonly defaultConfig: Readonly<Config>;
}

```

```

/**
 * Immutable plugin manager for orchestrating plugin lifecycle.
 * Coordinates initialization, rendering, and event propagation.
 */
export interface ReportPluginManager<
  Config extends Record<string, unknown> = Record<string, unknown>,
> {
  readonly initialize: (
    context: Readonly<ReportWorkspaceContext>,
    config: Readonly<Config>,
  ) => Promise<void>;
  readonly destroy: () => Promise<void>;
  readonly renderToolBarActions: (
    context: Readonly<ReportWorkspaceContext>,
  ) => React.ReactNode[];
  readonly renderContextMenuItems: (
    context: Readonly<ReportWorkspaceContext>,
    targetElement: HTMLElement,
  ) => React.ReactNode[];
  readonly getPlugin: <Plugin extends ReportWorkspacePlugin>(
    pluginId: string,
  ) => Readonly<Plugin> | undefined;
  readonly getAllPlugins: () => ReadonlyArray<Readonly<ReportWorkspacePlugin>>;
}

/**
 * Immutable configuration schema for plugin validation.
 * Defines the contract for plugin configuration structure.
 */
export interface ReportPluginConfigSchema<
  Config extends Record<string, unknown> = Record<string, unknown>,
> {
  readonly properties: Readonly<
    Record<keyof Config, ReportPluginPropertySchema>
  >;
  readonly required: ReadonlyArray<keyof Config>;
  readonly additionalProperties: boolean;
}

/**
 * Individual property schema for plugin configuration validation.
 */
export interface ReportPluginPropertySchema {
  readonly type: "string" | "number" | "boolean" | "object" | "array";
  readonly description?: string;
  readonly default?: unknown;
  readonly enum?: ReadonlyArray<unknown>;
  readonly minimum?: number;
  readonly maximum?: number;
  readonly minLength?: number;
  readonly maxLength?: number;
  readonly pattern?: string;
}

/**
 * Immutable event payload for plugin communication.
 * Ensures type-safe event propagation across plugins.

```

```

    */
export interface ReportPluginEvent<
  EventName extends string = string,
  Payload extends Record<string, unknown> = Record<string, unknown>,
> {
  readonly eventName: EventName;
  readonly payload: Readonly<Payload>;
  readonly timestamp: number;
  readonly sourcePluginId: string;
}

/**
 * Immutable plugin metadata for registry and discovery.
 */
export interface ReportPluginMetadata {
  readonly id: string;
  readonly name: string;
  readonly version: string;
  readonly description: string;
  readonly author: string;
  readonly dependencies: ReadonlyArray<string>;
  readonly tags: ReadonlyArray<string>;
  readonly isEnabled: boolean;
}

/**
 * Immutable plugin state for persistence and hydration.
 */
export interface ReportPluginState<
  State extends Record<string, unknown> = Record<string, unknown>,
> {
  readonly pluginId: string;
  readonly state: Readonly<State>;
  readonly version: number;
  readonly lastUpdated: number;
}

/**
 * Immutable plugin lifecycle hooks for advanced control.
 */
export interface ReportPluginLifecycleHooks<
  Config extends Record<string, unknown> = Record<string, unknown>,
  State extends Record<string, unknown> = Record<string, unknown>,
> {
  readonly onBeforeInit?: (
    context: Readonly<ReportPluginExecutionContext<Config>>,
  ) => Promise<void> | void;
  readonly onAfterInit?: (
    context: Readonly<ReportPluginExecutionContext<Config>>,
  ) => Promise<void> | void;
  readonly onBeforeRender?: (
    context: Readonly<ReportPluginExecutionContext<Config>>,
  ) => Promise<void> | void;
  readonly onAfterRender?: (
    context: Readonly<ReportPluginExecutionContext<Config>>,
  ) => Promise<void> | void;
  readonly onBeforeDestroy?: (
    context: Readonly<ReportPluginExecutionContext<Config>>,
  ) => Promise<void> | void;
}

```

```

    ) => Promise<void> | void;
    readonly onAfterDestroy?: (
      context: Readonly<ReportPluginExecutionContext<Config>>,
    ) => Promise<void> | void;
    readonly onStateChange?: (
      previousState: Readonly<State>,
      nextState: Readonly<State>,
      context: Readonly<ReportPluginExecutionContext<Config>>,
    ) => Promise<void> | void;
    readonly onConfigChange?: (
      previousConfig: Readonly<Config>,
      nextConfig: Readonly<Config>,
      context: Readonly<ReportPluginExecutionContext<Config>>,
    ) => Promise<void> | void;
  }

  /**
   * Immutable plugin definition combining all contracts.
   * Represents a complete, self-contained plugin module.
   */
  export interface ReportPluginDefinition<
    Plugin extends ReportWorkspacePlugin,
    Config extends Record<string, unknown> = Record<string, unknown>,
    State extends Record<string, unknown> = Record<string, unknown>,
  > {
    readonly metadata: Readonly<ReportPluginMetadata>;
    readonly configSchema: Readonly<ReportPluginConfigSchema<Config>>;
    readonly factory: Readonly<ReportPluginFactory<Plugin, Config>>;
    readonly lifecycle: Readonly<ReportPluginLifecycleHooks<Config, State>>;
    readonly defaultState: Readonly<State>;
  }

  /**
   * Immutable plugin registry builder for fluent configuration.
   */
  export interface ReportPluginRegistryBuilder {
    readonly withPlugin: <
      Plugin extends ReportWorkspacePlugin,
      Config extends Record<string, unknown> = Record<string, unknown>,
      State extends Record<string, unknown> = Record<string, unknown>,
    >(
      definition: Readonly<ReportPluginDefinition<Plugin, Config, State>>,
    ) => ReportPluginRegistryBuilder;
    readonly withConfig: <Config extends Record<string, unknown>>(
      config: Readonly<Config>,
    ) => ReportPluginRegistryBuilder;
    readonly build: () => Readonly<ReportPluginRegistry>;
  }

  /**
   * Immutable workspace plugin context for React integration.
   * Provides React context integration for plugin system.
   */
  export interface ReportWorkspacePluginContextValue<
    Config extends Record<string, unknown> = Record<string, unknown>,
  > {
    readonly manager: Readonly<ReportPluginManager<Config>>;
    readonly context: Readonly<ReportWorkspaceContext>;
  }

```



```

    readonly registry: Readonly<ReportPluginRegistry>;
}

/**
 * Type guard for validating plugin configuration objects.
 */
export function isValidPluginConfig<Config extends Record<string, unknown>>(
    config: unknown,
    schema: Readonly<ReportPluginConfigSchema<Config>>,
): config is Config {
    if (typeof config !== "object" || config === null) {
        return false;
    }

    const configRecord = config as Record<string, unknown>;
    const requiredKeys = schema.required as ReadonlyArray<string>;

    for (const key of requiredKeys) {
        if (!(key in configRecord)) {
            return false;
        }
    }

    if (!schema.additionalProperties) {
        const allowedKeys = new Set(Object.keys(schema.properties));
        for (const key of Object.keys(configRecord)) {
            if (!allowedKeys.has(key)) {
                return false;
            }
        }
    }

    for (const [key, propertySchema] of Object.entries(schema.properties)) {
        const value = configRecord[key];
        if (value !== undefined && !validateProperty(value, propertySchema)) {
            return false;
        }
    }

    return true;
}

function validateProperty(
    value: unknown,
    schema: Readonly<ReportPluginPropertySchema>,
): boolean {
    switch (schema.type) {
        case "string":
            if (typeof value !== "string") return false;
            if (schema.minLength !== undefined && value.length < schema.minLength)
                return false;
            if (schema.maxLength !== undefined && value.length > schema.maxLength)
                return false;
            if (
                schema.pattern !== undefined &&
                !new RegExp(schema.pattern).test(value)
            )
                return false;
    }
}

```

```

    if (schema.enum !== undefined && !schema.enum.includes(value))
        return false;
    return true;
case "number":
    if (typeof value !== "number" || Number.isNaN(value)) return false;
    if (schema.minimum !== undefined && value < schema.minimum) return false;
    if (schema.maximum !== undefined && value > schema.maximum) return false;
    if (schema.enum !== undefined && !schema.enum.includes(value))
        return false;
    return true;
case "boolean":
    return typeof value === "boolean";
case "object":
    if (typeof value !== "object" || value === null || Array.isArray(value))
        return false;
    return true;
case "array":
    if (!Array.isArray(value)) return false;
    return true;
default:
    return false;
}
}

/**
 * Creates an immutable plugin registry with type-safe operations.
 */
export function createPluginRegistry(): Readonly<ReportPluginRegistry> {
    const plugins = new Map<string, Readonly<ReportWorkspacePlugin>>();

    return Object.freeze({
        get plugins() {
            return plugins;
        },
        register(plugin: Readonly<ReportWorkspacePlugin>): void {
            if (plugins.has(plugin.id)) {
                throw new Error(`Plugin with id "${plugin.id}" is already registered`);
            }
            plugins.set(plugin.id, plugin);
        },
        unregister(pluginId: string): boolean {
            return plugins.delete(pluginId);
        },
        get(pluginId: string): Readonly<ReportWorkspacePlugin> | undefined {
            return plugins.get(pluginId);
        },
        getAll(): ReadonlyArray<Readonly<ReportWorkspacePlugin>> {
            return Array.from(plugins.values());
        },
        has(pluginId: string): boolean {
            return plugins.has(pluginId);
        },
        clear(): void {
            plugins.clear();
        },
    });
}

```

```

/**
 * Creates an immutable plugin configuration gateway.
 */
export function createPluginConfigGateway<
  Config extends Record<string, unknown>,
>(config: Readonly<Config>): Readonly<ReportPluginConfigGateway<Config>> {
  const gateway: ReportPluginConfigGateway<Config> = {
    get config() {
      return config;
    },
    getConfig<Key extends keyof Config>(key: Key): Config[Key] {
      return config[key];
    },
    hasConfig(key: string): boolean {
      return key in config;
    },
    mergeConfig<PartialConfig extends Partial<Config>>(
      partial: PartialConfig,
    ): ReportPluginConfigGateway<Config & PartialConfig> {
      return createPluginConfigGateway({ ...config, ...partial } as Config &
        PartialConfig) as ReportPluginConfigGateway<Config & PartialConfig>;
    },
  };
  return Object.freeze(gateway);
}

```

FILE: report-workspace.tsx
LOKASI: src\feedback

```
import React, {
  useState,
  useEffect,
  useRef,
  useCallback,
  useMemo,
  useLayoutEffect,
} from "react";
import {
  LucideIcon,
  Copy,
  Minimize2,
  Maximize2,
  AlertTriangle,
  X,
  ChevronDown,
  ChevronRight,
} from "lucide-react";
import { cn } from "@lib/utils";
import { Button } from "../general/button";
import { HintBox } from "../hint-box";
import {
  PopupMenu,
  type PopupMenuConfig,
  type PopupMenuItem,
} from "../popup-menu";
import { THEME_CONFIG, type TactileConfig } from "@config/theme-constants";
import type {
  ReportWorkspaceContext,
  ReportWorkspacePlugin,
} from "../report-plugins-core";

// â€œ LOCAL TYPE ISOLATION GATEWAY
export interface ReportWorkspaceProps extends React.HTMLAttributes<HTMLDivElement> {
  htmlReportStream: string;
  reportTitle: string;
  hint?: string;
  plugins?: ReadonlyArray<Readonly<ReportWorkspacePlugin>>;
  onLinkClick?: (type: string, id: string) => void;
}

interface ReportPageMetrics {
  pageElement: HTMLDivElement;
  scrollHeight: number;
  clientHeight: number;
  footerElement: HTMLElement | null;
  footerRect: DOMRect | null;
  pageRect: DOMRect;
}

interface AutoFitState {
  scale: number;
```

```

    isAutoFitting: boolean;
    showOverflowWarning: boolean;
    overflowWarningMessage: string;
}

interface ContextMenuState {
    isOpen: boolean;
    position: { x: number; y: number };
    targetElement: HTMLElement | null;
}

interface TooltipState {
    isOpen: boolean;
    position: { x: number; y: number };
    content: React.ReactNode;
    targetElement: HTMLElement | null;
}

interface GroupToggleState {
    expandedGroups: Set<string>;
}

const TACTILE = THEME_CONFIG.tactile as TactileConfig;
const MIN_AUTO_FIT_SCALE = 0.95;
const AUTO_FIT_STEP = 0.01;
const MAX_AUTO_FIT_ITERATIONS = 50;
const AUTO_FIT_DEBOUNCE_MS = 16;

const DEFAULT_CONTEXT_MENU_ITEMS: ReadonlyArray<PopupMenuItem> = [
    {
        id: "copy-cell-value",
        label: "Copy Cell Value",
        icon: Copy,
        shortcut: "Ctrl+C",
    },
    {
        id: "copy-full-line",
        label: "Copy Full Line Data",
        icon: Copy,
        shortcut: "Ctrl+Shift+C",
    },
    {
        id: "toggle-hidden-columns",
        label: "Toggle Hidden Column Mode",
        icon: Minimize2,
    },
    {
        id: "freeze-header",
        label: "Freeze Header Layout",
        icon: Maximize2,
    },
] as const;

function generatePageId(pageIndex: number): string {
    return `report-page-${pageIndex}`;
}

function generateGroupId(groupValue: string): string {

```

```

    return `group-${groupValue.replace(/^[a-zA-Z0-9]/g, "-")}`;
  }

export function ReportWorkspace({
  htmlReportStream,
  reportTitle,
  hint,
  plugins = [],
  onLinkClick,
  className,
  style,
  children,
  ...props
}: ReportWorkspaceProps) {
  const containerRef = useRef<HTMLDivElement>(null);
  const pagesContainerRef = useRef<HTMLDivElement>(null);
  const scaleContainerRef = useRef<HTMLDivElement>(null);
  const footerWarningRef = useRef<HTMLDivElement>(null);
  const contextMenuRef = useRef<HTMLDivElement>(null);
  const tooltipRef = useRef<HTMLDivElement>(null);

  const [zoom, setZoom] = useState<number>(1);
  const [currentPage, setCurrentPage] = useState<number>(1);
  const [totalPages, setTotalPages] = useState<number>(1);
  const [isDarkMode, setIsDarkMode] = useState<boolean>(false);
  const [autoFitScale, setAutoFitScale] = useState<number>(1);
  const [autoFitState, setAutoFitState] = useState<AutoFitState>({
    scale: 1,
    isAutoFitting: false,
    showOverflowWarning: false,
    overflowWarningMessage: "",
  });
  const [contextMenuState, setContextMenuState] = useState<ContextMenuState>({
    isOpen: false,
    position: { x: 0, y: 0 },
    targetElement: null,
  });
  const [tooltipState, setTooltipState] = useState<TooltipState>({
    isOpen: false,
    position: { x: 0, y: 0 },
    content: null,
    targetElement: null,
  });
  const [groupToggleState, setGroupToggleState] = useState<GroupToggleState>({
    expandedGroups: new Set<string>(),
  });
  const [mounted, setMounted] = useState(false);

  const autoFitRafRef = useRef<number | null>(null);
  const autoFitIterationRef = useRef<number>(0);
  const resizeObserverRef = useRef<ResizeObserver | null>(null);
  const mutationObserverRef = useRef<MutationObserver | null>(null);
  const clickOutsideHandlerRef = useRef<((event: MouseEvent) => void) | null>(
    null,
  );

  const workspaceContext = useMemo<ReportWorkspaceContext>(
    () => ({

```

```

    htmlContent: htmlReportStream,
    zoom,
    currentPage,
    totalPages,
    isDarkMode,
    autoFitScale,
    setZoom,
    setCurrentPage,
    setIsDarkMode,
    triggerAutoFit,
  )),
  [htmlReportStream, zoom, currentPage, totalPages, isDarkMode, autoFitScale],
);

const triggerAutoFit = useCallback(() => {
  if (
    autoFitState.isAutoFitting ||
    !pagesContainerRef.current ||
    !scaleContainerRef.current
  ) {
    return;
  }

  setAutoFitState((prev) => ({
    ...prev,
    isAutoFitting: true,
    showOverflowWarning: false,
  }));
  autoFitIterationRef.current = 0;

  const performAutoFit = () => {
    if (!pagesContainerRef.current || !scaleContainerRef.current) {
      setAutoFitState((prev) => ({ ...prev, isAutoFitting: false }));
      return;
    }

    const pages = Array.from(
      pagesContainerRef.current.querySelectorAll<HTMLDivElement>(
        "[data-report-page]",
      ),
    );
    if (pages.length === 0) {
      setAutoFitState((prev) => ({ ...prev, isAutoFitting: false }));
      return;
    }

    let hasOverflow = false;
    let maxScale = 1;

    for (const page of pages) {
      const pageRect = page.getBoundingClientRect();
      const footer = page.querySelector<HTMLElement>("[data-report-footer]");

      if (footer) {
        const footerRect = footer.getBoundingClientRect();
        const pageBottom = pageRect.bottom;

        if (footerRect.bottom > pageBottom + 2) {

```

```

        hasOverflow = true;
        const requiredScale = pageBottom / footerRect.bottom;
        maxScale = Math.min(
            maxScale,
            Math.max(requiredScale, MIN_AUTO_FIT_SCALE),
        );
    }
}

if (page.scrollHeight > page.clientHeight + 2) {
    hasOverflow = true;
    const requiredScale = page.clientHeight / page.scrollHeight;
    maxScale = Math.min(
        maxScale,
        Math.max(requiredScale, MIN_AUTO_FIT_SCALE),
    );
}
}

const newScale = Math.max(maxScale, MIN_AUTO_FIT_SCALE);
const clampedScale = Math.min(newScale, 1);

if (
    hasOverflow &&
    clampedScale < zoom - 0.001 &&
    autoFitIterationRef.current < MAX_AUTO_FIT_ITERATIONS
) {
    autoFitIterationRef.current++;
    const nextScale = Math.max(zoom - AUTO_FIT_STEP, clampedScale);
    setZoom(nextScale);
    setAutoFitScale(nextScale);

    autoFitRafRef.current = requestAnimationFrame(() => {
        setTimeout(performAutoFit, AUTO_FIT_DEBOUNCE_MS);
    });
} else {
    if (hasOverflow && clampedScale <= MIN_AUTO_FIT_SCALE + 0.001) {
        setAutoFitState((prev) => ({
            ...prev,
            isAutoFitting: false,
            showOverflowWarning: true,
            overflowWarningMessage:
                "Footer overflow detected. Content may be clipped at minimum scale.",
            scale: clampedScale,
        }));
    } else {
        setAutoFitState((prev) => ({
            ...prev,
            isAutoFitting: false,
            showOverflowWarning: false,
            scale: clampedScale,
        }));
    }
    setZoom(clampedScale);
    setAutoFitScale(clampedScale);
}
};

```



```

    performAutoFit();
  }, [autoFitState.isAutoFitting, zoom]);

  useEffect(() => {
    if (!mounted) return;

    const handleResize = () => {
      if (autoFitScale < 1 && !autoFitState.isAutoFitting) {
        triggerAutoFit();
      }
    };

    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, [mounted, autoFitScale, autoFitState.isAutoFitting, triggerAutoFit]);

  useEffect(() => {
    setMounted(true);

    const container = containerRef.current;
    if (!container) return;

    const handleClickOutside = (event: MouseEvent) => {
      if (
        contextMenuState.isOpen &&
        contextMenuRef.current &&
        !contextMenuRef.current.contains(event.target as Node)
      ) {
        setContextMenuState((prev) => ({
          ...prev,
          isOpen: false,
          targetElement: null,
        }));
      }
      if (tooltipState.isOpen) {
        setTooltipState((prev) => ({
          ...prev,
          isOpen: false,
          targetElement: null,
          content: null,
        }));
      }
    };

    clickOutsideHandlerRef.current = handleClickOutside;
    document.addEventListener("mousedown", handleClickOutside, true);
    document.addEventListener("contextmenu", handleClickOutside, true);

    return () => {
      if (clickOutsideHandlerRef.current) {
        document.removeEventListener(
          "mousedown",
          clickOutsideHandlerRef.current,
          true,
        );
      }
      document.removeEventListener(
        "contextmenu",
        clickOutsideHandlerRef.current,
      );
    };
  }, [
    clickOutsideHandlerRef,
    contextMenuRef,
    contextMenuState,
    tooltipState,
  ]);

```

```

        true,
    );
}
if (autoFitRafRef.current) {
    cancelAnimationFrame(autoFitRafRef.current);
}
if (resizeObserverRef.current) {
    resizeObserverRef.current.disconnect();
}
if (mutationObserverRef.current) {
    mutationObserverRef.current.disconnect();
}
};
}, [contextMenuState.isOpen, tooltipState.isOpen]);

useEffect(() => {
    if (!pagesContainerRef.current) return;

    const pages =
        pagesContainerRef.current.querySelectorAll<HTMLDivElement>(
            "[data-report-page]",
        );
    setTotalPages(pages.length);

    const setupPageObservers = () => {
        if (resizeObserverRef.current) {
            resizeObserverRef.current.disconnect();
        }

        resizeObserverRef.current = new ResizeObserver((entries) => {
            for (const entry of entries) {
                if (entry.target.hasAttribute("data-report-page")) {
                    if (autoFitScale < 1 && !autoFitState.isAutoFitting) {
                        triggerAutoFit();
                    }
                }
            }
        });
    };

    pages.forEach((page) => {
        if (resizeObserverRef.current) {
            resizeObserverRef.current.observe(page);
        }
    });
});

if (mutationObserverRef.current) {
    mutationObserverRef.current.disconnect();
}

mutationObserverRef.current = new MutationObserver((mutations) => {
    let shouldReobserve = false;
    for (const mutation of mutations) {
        if (mutation.type === "childList") {
            shouldReobserve = true;
            break;
        }
    }
});

```

```

        if (shouldReobserve) {
            setupPageObservers();
        }
    });

    mutationObserverRef.current.observe(pagesContainerRef.current, {
        childList: true,
        subtree: true,
    });

    setupPageObservers();

    return () => {
        if (resizeObserverRef.current) {
            resizeObserverRef.current.disconnect();
        }
        if (mutationObserverRef.current) {
            mutationObserverRef.current.disconnect();
        }
    };
}, [
    htmlReportStream,
    autoFitScale,
    autoFitState.isAutoFitting,
    triggerAutoFit,
]);

useEffect(() => {
    if (!containerRef.current) return;

    const handleLinkClick = (event: MouseEvent) => {
        const target = event.target as HTMLElement;
        const linkElement = target.closest("[data-link][data-id]");

        if (linkElement) {
            event.preventDefault();
            event.stopPropagation();
            const type = linkElement.getAttribute("data-link") || "";
            const id = linkElement.getAttribute("data-id") || "";
            if (type && id && onLinkClick) {
                onLinkClick(type, id);
            }
        }
    };

    const handleTooltipMouseEnter = (event: MouseEvent) => {
        const target = event.target as HTMLElement;
        const tooltipElement = target.closest("[data-tooltip]");

        if (tooltipElement) {
            const content = tooltipElement.getAttribute("data-tooltip");
            if (content) {
                const rect = tooltipElement.getBoundingClientRect();
                setTooltipState({
                    isOpen: true,
                    position: { x: rect.left + rect.width / 2, y: rect.top },
                    content,
                    targetElement: tooltipElement,
                });
            }
        }
    };

```

```

    });
  }
}
};

const handleTooltipMouseLeave = (event: MouseEvent) => {
  const target = event.target as HTMLElement;
  const tooltipElement = target.closest("[data-tooltip]");

  if (
    tooltipElement &&
    !tooltipElement.contains(event.relatedTarget as Node)
  ) {
    setTooltipState((prev) => ({
      ...prev,
      isOpen: false,
      targetElement: null,
      content: null,
    }));
  }
};

const handleGroupToggle = (event: MouseEvent) => {
  const target = event.target as HTMLElement;
  const groupTrigger = target.closest("[data-group-trigger]");

  if (groupTrigger) {
    event.preventDefault();
    event.stopPropagation();
    const groupValue = groupTrigger.getAttribute("data-group-trigger");
    if (groupValue) {
      const groupId = generateGroupId(groupValue);
      setGroupToggleState((prev) => {
        const newExpanded = new Set(prev.expandedGroups);
        if (newExpanded.has(groupId)) {
          newExpanded.delete(groupId);
        } else {
          newExpanded.add(groupId);
        }
        return { expandedGroups: newExpanded };
      });
    }
  }
};

const handleContextMenu = (event: MouseEvent) => {
  const target = event.target as HTMLElement;
  const reportArea = target.closest("[data-report-workspace]");

  if (
    reportArea &&
    (event.target === reportArea || reportArea.contains(target))
  ) {
    event.preventDefault();
    event.stopPropagation();
    setContextMenuState({
      isOpen: true,
      position: { x: event.clientX, y: event.clientY },
    });
  }
};

```

```

        targetElement: target,
    });
}
};

containerRef.current.addEventListener("click", handleLinkClick, true);
containerRef.current.addEventListener(
    "mouseenter",
    handleTooltipMouseEnter,
    true,
);
containerRef.current.addEventListener(
    "mouseleave",
    handleTooltipMouseLeave,
    true,
);
containerRef.current.addEventListener("click", handleGroupToggle, true);
containerRef.current.addEventListener("contextmenu", handleContextMenu);

return () => {
    if (containerRef.current) {
        containerRef.current.removeEventListener(
            "click",
            handleLinkClick,
            true,
        );
        containerRef.current.removeEventListener(
            "mouseenter",
            handleTooltipMouseEnter,
            true,
        );
        containerRef.current.removeEventListener(
            "mouseleave",
            handleTooltipMouseLeave,
            true,
        );
        containerRef.current.removeEventListener(
            "click",
            handleGroupToggle,
            true,
        );
        containerRef.current.removeEventListener(
            "contextmenu",
            handleContextMenu,
        );
    }
};
}, [onLinkClick]);

const handleGroupRowVisibility = useCallback(
    (groupValue: string, isExpanded: boolean) => {
        if (!pagesContainerRef.current) return;

        const groupRows = pagesContainerRef.current.querySelectorAll<HTMLElement>(
            `[data-group="${groupValue}"]`,
        );
        groupRows.forEach((row) => {
            row.style.display = isExpanded ? "" : "none";

```

```

    });
  },
  [],
);

useEffect(() => {
  groupToggleState.expandedGroups.forEach((groupId) => {
    const groupValue = groupId.replace("group-", "").replace(/-/g, " ");
    handleGroupRowVisibility(groupValue, true);
  });

  const allGroupValues = new Set<string>();
  if (pagesContainerRef.current) {
    pagesContainerRef.current
      .querySelectorAll("[data-group-trigger]")
      .forEach((el) => {
        const val = el.getAttribute("data-group-trigger");
        if (val) allGroupValues.add(val);
      });
  }

  allGroupValues.forEach((groupValue) => {
    const groupId = generateGroupId(groupValue);
    if (!groupToggleState.expandedGroups.has(groupId)) {
      handleGroupRowVisibility(groupValue, false);
    }
  });
}, [groupToggleState.expandedGroups, handleGroupRowVisibility]);

const buildContextMenuItems = useCallback(): PopupMenuConfig => {
  const pluginActions: PopupMenuItem[] = [];

  plugins.forEach((plugin) => {
    if (plugin.renderContextMenuItems) {
      const pluginItems = plugin.renderContextMenuItems(
        workspaceContext,
        contextMenuState.targetElement!,
      );
      if (pluginItems) {
        const itemsArray = React.Children.toArray(pluginItems);
        itemsArray.forEach((item, index) => {
          if (React.isValidElement(item)) {
            pluginActions.push({
              id: `plugin-${plugin.id}-${index}`,
              label:
                item.props.children?.toString() ||
                `Plugin Action ${index + 1}`,
              icon: item.props.icon,
              onClick: item.props.onClick,
            });
          }
        });
      }
    }
  });

  return {
    items: pluginActions,
  };
};

const defaultItems: PopupMenuItem[] = DEFAULT_CONTEXT_MENU_ITEMS.map(
  (item) => ({

```

```

...item,
onClick: () => {
  switch (item.id) {
    case "copy-cell-value":
      if (contextMenuState.targetElement) {
        const cellText =
          contextMenuState.targetElement.textContent?.trim() || "";
        navigator.clipboard.writeText(cellText);
      }
      break;
    case "copy-full-line":
      if (contextMenuState.targetElement) {
        const row = contextMenuState.targetElement.closest(
          "tr, [data-report-row]",
        );
        const lineText = row?.textContent?.trim() || "";
        navigator.clipboard.writeText(lineText);
      }
      break;
    case "toggle-hidden-columns":
      if (pagesContainerRef.current) {
        const hiddenColumns =
          pagesContainerRef.current.querySelectorAll(
            "[data-hidden-column]",
          );
        hiddenColumns.forEach((col) => {
          const isHidden =
            col.getAttribute("data-hidden-column") === "true";
          col.setAttribute(
            "data-hidden-column",
            (!isHidden).toString(),
          );
          (col as HTMLElement).style.display = isHidden ? "" : "none";
        });
      }
      break;
    case "freeze-header":
      if (pagesContainerRef.current) {
        const headers = pagesContainerRef.current.querySelectorAll(
          "[data-report-header]",
        );
        headers.forEach((header) => {
          const isFrozen =
            header.getAttribute("data-frozen") === "true";
          header.setAttribute("data-frozen", (!isFrozen).toString());
          (header as HTMLElement).style.position = isFrozen
            ? ""
            : "sticky";
          (header as HTMLElement).style.top = isFrozen ? "" : "0";
          (header as HTMLElement).style.zIndex = isFrozen ? "" : "10";
        });
      }
      break;
  }
  setContextMenuState((prev) => ({ ...prev, isOpen: false }));
},
}),
);

```

```

    return {
      items: [...defaultItems, ...pluginActions],
      trigger: null,
      triggerType: "context-menu",
    };
  }, [plugins, workspaceContext, contextMenuState.targetElement]);

const renderToolbarActions = useCallback(() => {
  const actions: React.ReactNode[] = [];

  plugins.forEach((plugin) => {
    if (plugin.renderToolbarAction) {
      const action = plugin.renderToolbarAction(workspaceContext);
      if (action) {
        actions.push(action);
      }
    }
  });

  return actions;
}, [plugins, workspaceContext]);

const handleZoomIn = useCallback(() => {
  setZoom((prev) => Math.min(prev + 0.1, 3));
}, []);

const handleZoomOut = useCallback(() => {
  setZoom((prev) => Math.max(prev - 0.1, 0.25));
}, []);

const handleZoomReset = useCallback(() => {
  setZoom(1);
  setAutoFitScale(1);
}, []);

const handlePageChange = useCallback(
  (delta: number) => {
    setCurrentPage((prev) => Math.max(1, Math.min(totalPages, prev + delta)));
  },
  [totalPages],
);

const handleDarkModeToggle = useCallback(() => {
  setIsDarkMode((prev) => !prev);
}, []);

const handleAutoFitClick = useCallback(() => {
  triggerAutoFit();
}, [triggerAutoFit]);

const handleKeepOriginalClick = useCallback(() => {
  setAutoFitState((prev) => ({ ...prev, showOverflowWarning: false }));
  setZoom(1);
  setAutoFitScale(1);
}, []);

const transformStyle = useMemo(

```



```

    () => ({
      transform: `scale(${zoom})`,
      transformOrigin: "top center",
      width: `${100 / zoom}%`,
    }),
    [zoom],
  );

const containerStyle = useMemo(
  () => ({
    ...style,
    width: "100%",
    overflow: "auto",
    position: "relative" as const,
  }),
  [style],
);

const pageElements = useMemo(() => {
  const parser = new DOMParser();
  const doc = parser.parseFromString(htmlReportStream, "text/html");
  const bodyContent = doc.body.innerHTML;

  const tempContainer = document.createElement("div");
  tempContainer.innerHTML = bodyContent;

  const pages = tempContainer.querySelectorAll("[data-report-page]");
  if (pages.length === 0) {
    const singlePage = document.createElement("div");
    singlePage.setAttribute("data-report-page", "1");
    singlePage.innerHTML = bodyContent;
    return [singlePage];
  }

  return Array.from(pages).map((page, index) => {
    const pageDiv = document.createElement("div");
    pageDiv.setAttribute("data-report-page", String(index + 1));
    pageDiv.innerHTML = page.innerHTML;
    return pageDiv;
  }));
}, [htmlReportStream]);

return (
  <div
    ref={containerRef}
    className={cn(
      "relative flex flex-col min-h-0 bg-background",
      "data-[dark-mode=true]:bg-background data-[dark-mode=true]:text-foreground",
      className,
    )}
    style={containerStyle}
    data-report-workspace=""
    data-dark-mode={isDarkMode}
    {...props}
  >
    <div
      className={cn(
        "flex items-center justify-between gap-3 p-3 border-b border-[color-mix(in_oklch,var(--color-

```

```

border)_40%,transparent)]",
    "bg-card/50 backdrop-blur-sm",
    TACTILE.flex,
    TACTILE.itemsCenter,
    TACTILE.justifyBetween,
    TACTILE.gap3,
    TACTILE.p3,
  )}
}
>
<div
  className={cn(
    "flex items-center gap-2",
    TACTILE.flex,
    TACTILE.itemsCenter,
    TACTILE.gap2,
  )}
>
  <h1
    className={cn(
      "font-semibold text-text-main truncate max-w-[300px]",
      TACTILE.textLg,
      TACTILE.fontSemibold,
      TACTILE.textForeground,
      TACTILE.truncate,
      TACTILE.maxW300px,
    )}
  >
    {reportTitle}
  </h1>
  {hint && (
    <HintBox content={hint} position="top">
      <span className="text-sm text-text-muted" />
    </HintBox>
  )}
</div>

<div
  className={cn(
    "flex items-center gap-2",
    TACTILE.flex,
    TACTILE.itemsCenter,
    TACTILE.gap2,
  )}
>
  {renderToolbarActions()}

  <div
    className={cn(
      "flex items-center gap-1 border border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)] rounded-surface overflow-hidden",
      TACTILE.border,
      TACTILE.roundedSurface,
      TACTILE.flex,
      TACTILE.itemsCenter,
      TACTILE.gap1,
    )}
  >
    <Button

```

```

        variant="ghost"
        size="icon"
        onClick={handleZoomOut}
        aria-label="Zoom out"
        className={TACTILE.activeScale95}
      >
        <Minimize2 className="w-4 h-4" />
      </Button>
    <span
      className={cn(
        "px-2 text-sm font-mono text-text-main min-w-[48px] text-center",
        TACTILE.textSm,
        TACTILE.fontMono,
        TACTILE.textForeground,
        TACTILE.px2,
      )}
    >
      {Math.round(zoom * 100)}%
    </span>
    <Button
      variant="ghost"
      size="icon"
      onClick={handleZoomIn}
      aria-label="Zoom in"
      className={TACTILE.activeScale95}
    >
      <Maximize2 className="w-4 h-4" />
    </Button>
    <Button
      variant="ghost"
      size="icon"
      onClick={handleZoomReset}
      aria-label="Reset zoom"
      className={TACTILE.activeScale95}
    >
      <span className="text-xs font-medium">100%</span>
    </Button>
  </div>

  <div
    className={cn(
      "flex items-center gap-1",
      TACTILE.flex,
      TACTILE.itemsCenter,
      TACTILE.gap1,
    )}
  >
    <Button
      variant="ghost"
      size="icon"
      onClick={() => handlePageChange(-1)}
      disabled={currentPage <= 1}
      aria-label="Previous page"
      className={TACTILE.activeScale95}
    >
      <ChevronLeft className="w-4 h-4" />
    </Button>
    <span

```

```

        className={cn(
          "px-2 text-sm font-mono text-text-main min-w-[60px] text-center",
          TACTILE.textSm,
          TACTILE.fontMono,
          TACTILE.textForeground,
          TACTILE.px2,
        )}
      >
        {currentPage} / {totalPages}
      </span>
      <Button
        variant="ghost"
        size="icon"
        onClick={() => handlePageChange(1)}
        disabled={currentPage >= totalPages}
        aria-label="Next page"
        className={TACTILE.activeScale95}
      >
        <ChevronRight className="w-4 h-4" />
      </Button>
    </div>

    <Button
      variant="ghost"
      size="icon"
      onClick={handleDarkModeToggle}
      aria-label={
        isDarkMode ? "Switch to light mode" : "Switch to dark mode"
      }
      className={TACTILE.activeScale95}
    >
      {isDarkMode ? (
        <Sun className="w-4 h-4" />
      ) : (
        <Moon className="w-4 h-4" />
      )}
    </Button>

    <Button
      variant="ghost"
      size="sm"
      onClick={handleAutoFitClick}
      aria-label="Auto-fit content"
      className={TACTILE.activeScale95}
    >
      <Maximize2 className="w-4 h-4 mr-1" />
      <span className="hidden sm:inline">Auto Fit</span>
    </Button>
  </div>
</div>

<div
  ref={pagesContainerRef}
  className={cn(
    "flex-1 overflow-auto relative",
    "data-[dark-mode=true]:bg-background",
    TACTILE.flex1,
    TACTILE.overflowAuto,
  )}

```

```

    TACTILE.relative,
  )}
  style={{ width: "100%", height: "100%" }}
  data-dark-mode={isDarkMode}
>
<div
  ref={scaleContainerRef}
  className={cn("relative", TACTILE.relative)}
  style={transformStyle}
>
  {pageElements.map((pageElement, index) => (
    <div
      key={index}
      className={cn(
        "relative mx-auto my-4 bg-white shadow-lg",
        "data-[dark-mode=true]:bg-card data-[dark-mode=true]:shadow-[var(--shadow-xl)]",
        "min-h-[297mm] w-[210mm]",
        "print:page-break-after-always print:shadow-none print:my-0",
        TACTILE.mxAuto,
        TACTILE.my4,
        TACTILE.shadowXl,
      )}
      style={{
        width: "210mm",
        minHeight: "297mm",
        boxSizing: "border-box",
      }}
      data-report-page={index + 1}
      data-dark-mode={isDarkMode}
    >
      {pageElement}
    </div>
  )}}
</div>
</div>

{autoFitState.showOverflowWarning && (
  <div
    ref={footerWarningRef}
    className={cn(
      "fixed bottom-4 left-1/2 -translate-x-1/2 z-50",
      "bg-destructive/95 text-destructive-foreground",
      "border border-destructive/50 rounded-surface",
      "shadow-xl px-4 py-3 flex items-center gap-3",
      "animate-in slide-in-from-bottom-4 fade-in duration-200",
      "max-w-[90vw]",
      TACTILE.fixed,
      TACTILE.bottom4,
      TACTILE.left1_2,
      TACTILE.translateXFullNegative,
      TACTILE.z50,
      TACTILE.roundedSurface,
      TACTILE.shadowXl,
      TACTILE.px4,
      TACTILE.py3,
      TACTILE.flex,
      TACTILE.itemsCenter,
      TACTILE.gap3,
    )}
  >
    {footerWarning}
  </div>
)}

```

```

    })
    role="alert"
    aria-live="assertive"
  >
  <AlertTriangle className="w-5 h-5 flex-shrink-0 text-destructive-foreground" />
  <span className="text-sm font-medium flex-1">
    {autoFitState.overflowWarningMessage}
  </span>
  <Button
    variant="destructive"
    size="sm"
    onClick={handleAutoFitClick}
    className={TACTILE.activeScale95}
  >
    Auto Fit View
  </Button>
  <Button
    variant="ghost"
    size="sm"
    onClick={handleKeepOriginalClick}
    className={TACTILE.activeScale95}
  >
    Keep Original
  </Button>
  <Button
    variant="ghost"
    size="icon"
    onClick={() =>
      setAutoFitState((prev) => ({
        ...prev,
        showOverflowWarning: false,
      }))
    }
    aria-label="Dismiss warning"
    className={TACTILE.activeScale95}
  >
    <X className="w-4 h-4" />
  </Button>
</div>
)}}

```

```

{tooltipState.isOpen && tooltipState.content && (
  <div
    ref={tooltipRef}
    className={cn(
      "fixed z-50 pointer-events-none",
      "bg-card backdrop-blur-[var(--backdrop-blur)]",
      "border border-[color-mix(in_oklch,var(--color-border)_50%,transparent)]",
      "rounded-surface shadow-lg px-3 py-2",
      "max-w-xs text-sm text-card-foreground font-sans",
      "animate-in fade-in-0 duration-150",
      TACTILE.fixed,
      TACTILE.z50,
      TACTILE.roundedSurface,
      TACTILE.shadowX1,
      TACTILE.px3,
      TACTILE.py2,
      TACTILE.textSm,
    )}
  >
    {tooltipState.content}
  </div>
)}

```

```

        TACTILE.maxW300px,
      )}
      style={{
        left: tooltipState.position.x,
        top: tooltipState.position.y,
        transform: "translateX(-50%) translateY(-100%)",
      }}
      role="tooltip"
    >
      {tooltipState.content}
    </div>
  )}

  {contextMenuState.isOpen && (
    <PopupMenu ref={contextMenuRef} {...buildContextMenuItems()} />
  )}

  {children}
</div>
);
}

import { Sun, Moon, ChevronLeft } from "lucide-react";

ReportWorkspace.displayName = "ReportWorkspace";

export default ReportWorkspace;

```

FILE: skeleton.tsx
LOKASI: src\feedback

```
import React from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface SkeletonProps extends React.HTMLAttributes<HTMLDivElement> {
  /**
   * Visual variant of the skeleton
   * @default "text"
   */
  variant?: "text" | "circular" | "rectangular" | "rounded";
  /**
   * Width of the skeleton
   * @default "100%"
   */
  width?: string | number;
  /**
   * Height of the skeleton
   * @default "1rem"
   */
  height?: string | number;
  /**
   * Whether the skeleton should animate
   * @default "pulse"
   */
  animation?: "pulse" | "wave" | "none";
  /**
   * Border radius for rectangular variant
   * @default "0.375rem"
   */
  radius?: string | number;
  /**
   * Base color for the skeleton
   * @default "muted"
   */
  baseColor?: "muted" | "muted-foreground/10" | "border" | "primary/10";
}

/**
 * Skeleton - A versatile skeleton loading placeholder component
 *
 * Provides multiple visual variants for different content types:
 * - text: Simulates text lines with varying widths
 * - circular: Circular placeholder for avatars, badges
 * - rectangular: Rectangular placeholder for cards, images
 * - rounded: Rounded rectangular placeholder for buttons, chips
 *
 * Supports multiple animation types:
 * - pulse: Subtle opacity pulse animation (default)
```



```

* - wave: Shimmer wave animation
* - none: Static placeholder without animation
*
* @example
* ```tsx
* // Text skeleton
* <Skeleton variant="text" width="80%" />
*
* // Circular avatar skeleton
* <Skeleton variant="circular" width={40} height={40} />
*
* // Card skeleton with wave animation
* <Skeleton variant="rectangular" width="100%" height={200} animation="wave" />
*
* // Button skeleton
* <Skeleton variant="rounded" width={120} height={40} radius="9999px" />
* ```
*/
export const Skeleton = React.forwardRef<HTMLDivElement, SkeletonProps>(
  (
    {
      className,
      variant = "text",
      width = "100%",
      height = "1rem",
      animation = "pulse",
      radius = "0.375rem",
      baseColor = "muted",
      style,
      ...props
    },
    ref,
  ) => {
    const widthValue = typeof width === "number" ? `${width}px` : width;
    const heightValue = typeof height === "number" ? `${height}px` : height;
    const radiusValue = typeof radius === "number" ? `${radius}px` : radius;

    const variantClasses = {
      circular: "rounded-full",
      rounded: `rounded-[${radiusValue}]`,
      rectangular: `rounded-[${radiusValue}]`,
      text: "rounded-[0.25rem]",
    }[variant];

    const animationClasses = {
      pulse: "animate-pulse duration-1000",
      wave: "animate-[skeleton-wave_1.5s_ease-in-out_infinite] bg-gradient-to-r from-[hsl(var(--muted))] via-[hsl(var(--muted-foreground/10))] to-[hsl(var(--muted))] bg-[length:200%_100%]",
      none: "",
    }[animation];

    const baseColorClass = `bg-[hsl(var(--${baseColor}))]`;

    return (
      <div
        ref={ref}
        className={cn(
          "skeleton",

```

```

        variantClasses,
        animationClasses,
        baseColorClass,
        className,
    )}
    style={{
        width: widthValue,
        height: heightValue,
        ...style,
    }}
    {...props}
  />
);
},
);

Skeleton.displayName = "Skeleton";

/**
 * SkeletonText - Pre-configured skeleton for text content
 * Renders multiple lines with varying widths to simulate paragraph text
 */
export interface SkeletonTextProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Number of text lines to render
   * @default 3
   */
  lines?: number;
  /**
   * Maximum width of the text block
   * @default "100%"
   */
  maxWidth?: string | number;
  /**
   * Spacing between lines
   * @default "0.5rem"
   */
  lineGap?: string | number;
  /**
   * Whether the last line should be shorter (more realistic)
   * @default true
   */
  lastLineShort?: boolean;
  /**
   * Approximate width of the last line when lastLineShort is true
   * @default "60%"
   */
  lastLineWidth?: string | number;
}

export const SkeletonText = React.forwardRef<HTMLDivElement, SkeletonTextProps>(
  (
    {
      className,
      lines = 3,

```

```

    maxWidth = "100%",
    lineGap = "0.5rem",
    lastLineShort = true,
    lastLineWidth = "60%",
    animation = "pulse",
    baseColor = "muted",
    radius = "0.25rem",
    style,
    ...props
  },
  ref,
) => {
  const maxWidthValue =
    typeof maxWidth === "number" ? `${maxWidth}px` : maxWidth;
  const lineGapValue = typeof lineGap === "number" ? `${lineGap}px` : lineGap;
  const lastLineWidthValue =
    typeof lastLineWidth === "number" ? `${lastLineWidth}px` : lastLineWidth;

  const lineHeight = "1rem";

  return (
    <div
      ref={ref}
      className={cn("skeleton-text-container", className)}
      style={{
        maxWidth: maxWidthValue,
        display: "flex",
        flexDirection: "column",
        gap: lineGapValue,
        ...style,
      }}
      {...props}
    >
      {Array.from({ length: lines }).map((_, index) => {
        const isLastLine = index === lines - 1;
        const width =
          isLastLine && lastLineShort ? lastLineWidthValue : "100%";

        return (
          <Skeleton
            key={index}
            variant="text"
            width={width}
            height={lineHeight}
            animation={animation}
            baseColor={baseColor}
            radius={radius}
          />
        );
      })}
    </div>
  );
},
);

SkeletonText.displayName = "SkeletonText";

/**

```

```

    * SkeletonCard - Pre-configured skeleton for card layouts
    * Renders a card skeleton with image placeholder, title, and text lines
    */
export interface SkeletonCardProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Whether to show image placeholder
   * @default true
   */
  withImage?: boolean;
  /**
   * Height of the image placeholder
   * @default "200px"
   */
  imageHeight?: string | number;
  /**
   * Number of text lines in the card body
   * @default 3
   */
  lines?: number;
  /**
   * Whether to show action buttons placeholder
   * @default false
   */
  withActions?: boolean;
  /**
   * Card border radius
   * @default "0.5rem"
   */
  radius?: string | number;
}

export const SkeletonCard = React.forwardRef<HTMLDivElement, SkeletonCardProps>(
  (
    {
      className,
      withImage = true,
      imageHeight = "200px",
      lines = 3,
      withActions = false,
      radius = "0.5rem",
      animation = "pulse",
      baseColor = "muted",
      style,
      ...props
    },
    ref,
  ) => {
    const radiusValue = typeof radius === "number" ? `${radius}px` : radius;
    const imageHeightValue =
      typeof imageHeight === "number" ? `${imageHeight}px` : imageHeight;

    return (
      <div
        ref={ref}
        className={cn("skeleton-card", className)}

```

```

    style={{
      borderRadius: radiusValue,
      overflow: "hidden",
      backgroundColor: `hsl(var(--${baseColor}))`,
      ...style,
    }}
    {...props}
  >
  {withImage && (
    <Skeleton
      variant="rectangular"
      width="100%"
      height={imageHeightValue}
      animation={animation}
      baseColor={baseColor}
      radius="0"
    />
  )}
  <div className="skeleton-card-content" style={{ padding: "1rem" }}>
    <Skeleton
      variant="text"
      width="60%"
      height="1.25rem"
      animation={animation}
      baseColor={baseColor}
      radius={radius}
    />
    <SkeletonText
      lines={lines}
      animation={animation}
      baseColor={baseColor}
      radius={radius}
      lineGap="0.375rem"
    />
    {withActions && (
      <div
        className="skeleton-card-actions"
        style={{ marginTop: "1rem", display: "flex", gap: "0.5rem" }}
      >
        <Skeleton
          variant="rounded"
          width={100}
          height={40}
          animation={animation}
          baseColor={baseColor}
          radius="9999px"
        />
        <Skeleton
          variant="rounded"
          width={100}
          height={40}
          animation={animation}
          baseColor={baseColor}
          radius="9999px"
        />
      </div>
    )}
  </div>

```

```

    </div>
  );
},
);

SkeletonCard.displayName = "SkeletonCard";

/**
 * SkeletonAvatar - Pre-configured skeleton for avatar placeholders
 */
export interface SkeletonAvatarProps extends Omit<
  SkeletonProps,
  "variant" | "width" | "height"
> {
  /**
   * Size of the avatar
   * @default "md"
   */
  size?: "xs" | "sm" | "md" | "lg" | "xl" | "2xl";
  /**
   * Custom size in pixels (overrides size prop)
   */
  sizePx?: number;
}

const avatarSizes = {
  xs: 24,
  sm: 32,
  md: 40,
  lg: 48,
  xl: 64,
  "2xl": 96,
} as const;

export const SkeletonAvatar = React.forwardRef<
  HTMLDivElement,
  SkeletonAvatarProps
>(
  (
    {
      className,
      size = "md",
      sizePx,
      animation = "pulse",
      baseColor = "muted",
      style,
      ...props
    },
    ref,
  ) => {
    const sizeValue = sizePx ?? avatarSizes[size];

    return (
      <Skeleton
        ref={ref}
        className={cn("skeleton-avatar", className)}
        variant="circular"
        width={sizeValue}

```

```

        height={sizeValue}
        animation={animation}
        baseColor={baseColor}
        style={style}
        {...props}
      />
    );
  },
);

SkeletonAvatar.displayName = "SkeletonAvatar";

/**
 * SkeletonButton - Pre-configured skeleton for button placeholders
 */
export interface SkeletonButtonProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Button size variant
   * @default "md"
   */
  size?: "sm" | "md" | "lg" | "icon";
  /**
   * Custom width (overrides size)
   */
  width?: string | number;
  /**
   * Custom height (overrides size)
   */
  height?: string | number;
  /**
   * Border radius
   * @default "0.375rem"
   */
  radius?: string | number;
}

const buttonSizes = {
  sm: { height: 32, width: "auto", minWidth: 64 },
  md: { height: 40, width: "auto", minWidth: 80 },
  lg: { height: 48, width: "auto", minWidth: 96 },
  icon: { height: 40, width: 40, minWidth: 40 },
} as const;

export const SkeletonButton = React.forwardRef<
  HTMLDivElement,
  SkeletonButtonProps
>((
  {
    className,
    size = "md",
    width,
    height,
    animation = "pulse",
    baseColor = "muted",

```

```

    radius = "0.375rem",
    style,
    ...props
  },
  ref,
) => {
  const sizeConfig = buttonSizes[size];
  const widthValue = width ?? sizeConfig.width;
  const heightValue = height ?? sizeConfig.height;
  const minWidthValue = sizeConfig.minWidth;

  return (
    <Skeleton
      ref={ref}
      className={cn("skeleton-button", className)}
      variant="rounded"
      width={widthValue}
      height={heightValue}
      animation={animation}
      baseColor={baseColor}
      radius={radius}
      style={{
        minWidth:
          typeof minWidthValue === "number"
            ? `${minWidthValue}px`
            : minWidthValue,
        ...style,
      }}
      {...props}
    />
  );
},
);

SkeletonButton.displayName = "SkeletonButton";

/**
 * SkeletonInput - Pre-configured skeleton for input field placeholders
 */
export interface SkeletonInputProps extends Omit<
  SkeletonProps,
  "variant" | "height"
> {
  /**
   * Input size variant
   * @default "md"
   */
  size?: "sm" | "md" | "lg";
  /**
   * Custom width
   * @default "100%"
   */
  width?: string | number;
}

const inputSizes = {
  sm: { height: 32 },
  md: { height: 40 },

```



```

    lg: { height: 48 },
  } as const;

export const SkeletonInput = React.forwardRef<
  HTMLDivElement,
  SkeletonInputProps
>(
  (
    {
      className,
      size = "md",
      width = "100%",
      animation = "pulse",
      baseColor = "muted",
      radius = "0.375rem",
      style,
      ...props
    },
    ref,
  ) => {
    const heightValue = inputSizes[size].height;

    return (
      <Skeleton
        ref={ref}
        className={cn("skeleton-input", className)}
        variant="rounded"
        width={width}
        height={heightValue}
        animation={animation}
        baseColor={baseColor}
        radius={radius}
        style={style}
        {...props}
      />
    );
  },
);

SkeletonInput.displayName = "SkeletonInput";

/**
 * SkeletonList - Pre-configured skeleton for list item placeholders
 */
export interface SkeletonListProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Number of list items to render
   * @default 5
   */
  items?: number;
  /**
   * Whether each item has an avatar/icon placeholder
   * @default true
   */
  withAvatar?: boolean;

```

```

/**
 * Number of text lines per item
 * @default 2
 */
linesPerItem?: number;
/**
 * Gap between list items
 * @default "0.75rem"
 */
itemGap?: string | number;
}

export const SkeletonList = React.forwardRef<HTMLDivElement, SkeletonListProps>(
  (
    {
      className,
      items = 5,
      withAvatar = true,
      linesPerItem = 2,
      itemGap = "0.75rem",
      animation = "pulse",
      baseColor = "muted",
      radius = "0.375rem",
      style,
      ...props
    },
    ref,
  ) => {
    const itemGapValue = typeof itemGap === "number" ? `${itemGap}px` : itemGap;

    return (
      <div
        ref={ref}
        className={cn("skeleton-list", className)}
        style={{
          display: "flex",
          flexDirection: "column",
          gap: itemGapValue,
          ...style,
        }}
        {...props}
      >
        {Array.from({ length: items }).map((_, index) => (
          <div
            key={index}
            className="skeleton-list-item"
            style={{
              display: "flex",
              alignItems: "flex-start",
              gap: "0.75rem",
            }}
          >
            {withAvatar && (
              <SkeletonAvatar
                size="md"
                animation={animation}
                baseColor={baseColor}
              />

```

```

    })
    <div style={{ flex: 1, minWidth: 0 }}>
      <SkeletonText
        lines={linesPerItem}
        animation={animation}
        baseColor={baseColor}
        radius={radius}
        lineGap="0.25rem"
        lastLineShort
        lastLineWidth="70%"
      />
    </div>
  </div>
  )))
</div>
);
},
);

SkeletonList.displayName = "SkeletonList";

/**
 * SkeletonTable - Pre-configured skeleton for table row placeholders
 */
export interface SkeletonTableProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Number of rows to render
   * @default 5
   */
  rows?: number;
  /**
   * Number of columns
   * @default 4
   */
  columns?: number;
  /**
   * Whether to show header row skeleton
   * @default true
   */
  withHeader?: boolean;
  /**
   * Column widths (percentage or pixel values)
   */
  columnWidths?: (string | number)[];
}

export const SkeletonTable = React.forwardRef<
  HTMLDivElement,
  SkeletonTableProps
>(
  (
    {
      className,
      rows = 5,
      columns = 4,

```

```

    withHeader = true,
    columnWidths,
    animation = "pulse",
    baseColor = "muted",
    radius = "0.25rem",
    style,
    ...props
  },
  ref,
) => {
  const defaultColumnWidth = 100 / columns;
  const colWidths =
    columnWidths?.length === columns
      ? columnWidths
      : Array(columns).fill(defaultColumnWidth);

  const renderRow = (isHeader: boolean, rowIndex: number) => (
    <div
      key={rowIndex}
      className={cn(
        "skeleton-table-row",
        isHeader && "skeleton-table-header",
      )}
      style={{
        display: "grid",
        gridTemplateColumns: colWidths
          .map((w) => (typeof w === "number" ? `${w}%` : w))
          .join(" "),
        gap: "1rem",
        padding: "0.75rem 1rem",
        ...(isHeader && { fontWeight: 600 }),
      }}
    >
      {colWidths.map((_, colIndex) => (
        <Skeleton
          key={colIndex}
          variant="text"
          width="100%"
          height={isHeader ? "1rem" : "0.875rem"}
          animation={animation}
          baseColor={baseColor}
          radius={radius}
        />
      ))}
    </div>
  );

  return (
    <div
      ref={ref}
      className={cn("skeleton-table", className)}
      style={{
        borderRadius: radius,
        overflow: "hidden",
        backgroundColor: `hsl(var(--${baseColor}))`,
        ...style,
      }}
      {...props}
    >

```

```
    >
    {withHeader && renderRow(true, -1)}
    {Array.from({ length: rows }).map((_, index) =>
      renderRow(false, index),
    )}
  </div>
);
},
);

SkeletonTable.displayName = "SkeletonTable";

export default Skeleton;
```

FILE: standalone-pagination.tsx
LOKASI: src\feedback

```
import React, { useCallback, useMemo as useMemoReact } from "react";
import {
  ChevronLeft,
  ChevronRight,
  ChevronFirst,
  ChevronLast,
  MoreHorizontal,
} from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";

// ðŸš¡ LOCAL TYPE ISOLATION GATEWAY
export interface PaginationItem {
  type: "page" | "ellipsis" | "first" | "last" | "prev" | "next";
  page?: number;
  label: string;
  ariaLabel: string;
  isActive?: boolean;
  isDisabled?: boolean;
}

export interface StandalonePaginationProps {
  /** Current active page (1-indexed) */
  currentPage: number;
  /** Total number of pages */
  totalPages: number;
  /** Number of sibling pages to show on each side of current page */
  siblingCount?: number;
  /** Number of pages to show at the start/end boundaries */
  boundaryCount?: number;
  /** Callback when page changes */
  onPageChange: (page: number) => void;
  /** Custom className for the pagination container */
  className?: string;
  /** Custom className for individual page items */
  itemClassName?: string;
  /** Custom className for active page item */
  activeItemClassName?: string;
  /** Custom className for disabled items */
  disabledItemClassName?: string;
  /** Show first/last page buttons */
  showFirstLast?: boolean;
  /** Show previous/next buttons */
  showPrevNext?: boolean;
  /** Custom labels */
  labels?: {
    first?: { label?: string; ariaLabel?: string };
    last?: { label?: string; ariaLabel?: string };
    previous?: { label?: string; ariaLabel?: string };
    next?: { label?: string; ariaLabel?: string };
    ellipsis?: string;
    page?: (page: number) => string;
  };
}
```

```

    pageAriaLabel?: (page: number) => string;
  };
  /** ARIA label for the pagination nav */
  ariaLabel?: string;
  /** Disable all interactions */
  disabled?: boolean;
  /** Size variant */
  size?: "sm" | "md" | "lg";
  /** Variant style */
  variant?: "default" | "outline" | "ghost";
  /** Optional hint text to display below pagination */
  hint?: string;
}

// ðŸŽˆ SIBLING WINDOW MATH ENGINE
function calculatePaginationItems(
  clampedPage: number,
  totalPages: number,
  siblingCount: number = 1,
  boundaryCount: number = 1,
): PaginationItem[] {
  const items: PaginationItem[] = [];

  // Calculate sibling range using pre-clamped page
  const leftSiblingStart = Math.max(2, clampedPage - siblingCount);
  const rightSiblingEnd = Math.min(totalPages - 1, clampedPage + siblingCount);

  // Calculate boundary ranges
  const leftBoundaryEnd = Math.min(boundaryCount, totalPages);
  const rightBoundaryStart = Math.max(totalPages - boundaryCount + 1, 1);

  // Build page numbers to show
  const pagesToShow = new Set<number>();

  // Add left boundary pages
  for (let i = 1; i <= leftBoundaryEnd; i++) {
    pagesToShow.add(i);
  }

  // Add sibling pages around current
  for (let i = leftSiblingStart; i <= rightSiblingEnd; i++) {
    if (i >= 1 && i <= totalPages) {
      pagesToShow.add(i);
    }
  }

  // Add right boundary pages
  for (let i = rightBoundaryStart; i <= totalPages; i++) {
    pagesToShow.add(i);
  }

  // Convert to sorted array
  const sortedPages = Array.from(pagesToShow).sort((a, b) => a - b);

  // Build final items with ellipsis injection
  let lastPage = 0;
  for (const page of sortedPages) {
    // Inject ellipsis if there's a gap

```

```

    if (lastPage > 0 && page > lastPage + 1) {
      items.push({
        type: "ellipsis",
        label: "â€¦",
        ariaLabel: "More pages",
      });
    }
    items.push({
      type: "page",
      page,
      label: page.toString(),
      ariaLabel: `Page ${page}`,
      isActive: page === clampedPage,
    });
    lastPage = page;
  }

  return items;
}

// ðŸŽ“ SIZE VARIANT TOKENS
const SIZE_CLASSES = {
  sm: "h-8 px-2.5 text-xs gap-0.5",
  md: "h-10 px-3 text-sm gap-1",
  lg: "h-12 px-4 text-base gap-1.5",
} as const;

const ITEM_SIZE_CLASSES = {
  sm: "h-8 w-8 text-xs",
  md: "h-10 w-10 text-sm",
  lg: "h-12 w-12 text-base",
} as const;

const VARIANT_CLASSES = {
  default:
    "bg-background border-border hover:bg-accent hover:text-accent-foreground",
  outline:
    "bg-transparent border-border hover:bg-accent hover:text-accent-foreground border",
  ghost: "bg-transparent hover:bg-accent hover:text-accent-foreground",
} as const;

const ACTIVE_VARIANT_CLASSES = {
  default:
    "bg-brand-primary text-white border-brand-primary hover:bg-brand-hover",
  outline: "bg-brand-primary text-white border-brand-primary",
  ghost: "bg-brand-primary text-white",
} as const;

const DISABLED_CLASSES = "opacity-50 pointer-events-none cursor-not-allowed";

// ðŸŽ“ MAIN COMPONENT
export function StandalonePagination({
  currentPage,
  totalPages,
  siblingCount = 1,
  boundaryCount = 1,
  onPageChange,
  className,

```



```

    itemClassName,
    activeItemClassName,
    disabledItemClassName,
    showFirstLast = true,
    showPrevNext = true,
    labels = {},
    ariaLabel = "Pagination",
    disabled = false,
    size = "md",
    variant = "default",
    hint,
  }: StandalonePaginationProps) {
    // Clamp current page
    const clampedPage = useMemoReact(
      () => Math.max(1, Math.min(currentPage, totalPages)),
      [currentPage, totalPages],
    );

    // Memoized pagination items calculation
    const paginationItems = useMemoReact(
      () =>
        calculatePaginationItems(
          clampedPage,
          totalPages,
          siblingCount,
          boundaryCount,
        ),
      [clampedPage, totalPages, siblingCount, boundaryCount],
    );

    // Memoized click handler
    const handleClick = useCallback(
      (page: number) => {
        if (
          !disabled &&
          page >= 1 &&
          page <= totalPages &&
          page !== clampedPage
        ) {
          onPageChange(page);
        }
      },
      [disabled, totalPages, clampedPage, onPageChange],
    );

    // Memoized keyboard handler
    const handleKeyDown = useCallback(
      (event: React.KeyboardEvent, page?: number) => {
        if (disabled) return;

        let targetPage: number | null = null;

        switch (event.key) {
          case "Enter":
          case " ":
            event.preventDefault();
            if (page !== undefined) {
              targetPage = page;
            }

```

```

    }
    break;
  case "ArrowLeft":
    event.preventDefault();
    targetPage = Math.max(1, clampedPage - 1);
    break;
  case "ArrowRight":
    event.preventDefault();
    targetPage = Math.min(totalPages, clampedPage + 1);
    break;
  case "Home":
    event.preventDefault();
    targetPage = 1;
    break;
  case "End":
    event.preventDefault();
    targetPage = totalPages;
    break;
}

if (targetPage !== null && targetPage !== clampedPage) {
  onPageChange(targetPage);
}
},
[disabled, clampedPage, totalPages, onPageChange],
);

// Early return if no pagination needed
if (totalPages <= 1) {
  return null;
}

const sizeClass = SIZE_CLASSES[size];
const itemSizeClass = ITEM_SIZE_CLASSES[size];
const variantClass = VARIANT_CLASSES[variant];
const activeVariantClass = ACTIVE_VARIANT_CLASSES[variant];

return (
  <>
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span aria-hidden="true" style={{ display: "none" }} />
      </HintBox>
    )}
    <nav
      role="navigation"
      aria-label={ariaLabel}
      className={cn(
        "flex items-center justify-center gap-1.5",
        "font-medium",
        className,
      )}
    >
      {/* First Page Button */}
      {showFirstLast && clampedPage > 1 && (
        <button
          type="button"
          onClick={() => handlePageClick(1)}

```

```

onKeyDown={(e) => handleKeyDown(e, 1)}
disabled={disabled || clampedPage === 1}
className={cn(
  "inline-flex items-center justify-center",
  "rounded-md border",
  "font-medium transition-all duration-150 ease-out",
  "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-
visible:ring-offset-2 focus-visible:ring-offset-background",
  "active:scale-[0.98]",
  sizeClass,
  variantClass,
  disabled && DISABLED_CLASSES,
  disabledItemClassName,
)}
aria-label={labels.first?.ariaLabel ?? "Go to first page"}
title={labels.first?.label ?? "First page"}
>
<ChevronFirst
  size={size === "sm" ? 14 : size === "md" ? 16 : 18}
  strokeWidth={2.5}
  aria-hidden="true"
/>
</button>
)}

{/* Previous Page Button */}
{showPrevNext && clampedPage > 1 && (
  <button
    type="button"
    onClick={() => handlePageClick(clampedPage - 1)}
    onKeyDown={(e) => handleKeyDown(e, clampedPage - 1)}
    disabled={disabled || clampedPage === 1}
    className={cn(
      "inline-flex items-center justify-center",
      "rounded-md border",
      "font-medium transition-all duration-150 ease-out",
      "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-
visible:ring-offset-2 focus-visible:ring-offset-background",
      "active:scale-[0.98]",
      sizeClass,
      variantClass,
      disabled && DISABLED_CLASSES,
      disabledItemClassName,
    )}
    aria-label={labels.previous?.ariaLabel ?? "Go to previous page"}
    title={labels.previous?.label ?? "Previous page"}
  >
    <ChevronLeft
      size={size === "sm" ? 14 : size === "md" ? 16 : 18}
      strokeWidth={2.5}
      aria-hidden="true"
    />
  </button>
)}

{/* Page Numbers & Ellipsis */}
<div
  className="flex items-center gap-0.5"

```

```

    role="group"
    aria-label="Page numbers"
  >
  {paginationItems.map((item, index) => {
    const isActive = item.isActive;
    const isEllipsis = item.type === "ellipsis";

    if (isEllipsis) {
      return (
        <span
          key={`ellipsis-${index}`}
          className={cn(
            "inline-flex items-center justify-center",
            "text-muted-foreground",
            "select-none",
            itemSizeClass,
            itemClassName,
          )}
          aria-hidden="true"
        >
          <MoreHorizontal
            size={size === "sm" ? 14 : size === "md" ? 16 : 18}
            strokeWidth={2}
            aria-hidden="true"
          />
        </span>
      );
    }
  })

  return (
    <button
      key={`page-${item.page}`}
      type="button"
      onClick={() => item.page && handlePageClick(item.page)}
      onKeyDown={(e) => handleKeyDown(e, item.page)}
      disabled={disabled || isActive}
      className={cn(
        "inline-flex items-center justify-center",
        "rounded-md border",
        "font-medium transition-all duration-150 ease-out",
        "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 focus-visible:ring-offset-background",
        "active:scale-[0.98]",
        itemSizeClass,
        isActive ? activeVariantClass : variantClass,
        isActive && "cursor-default",
        disabled && DISABLED_CLASSES,
        isActive ? activeItemClassName : itemClassName,
      )}
      aria-label={item.ariaLabel}
      aria-current={isActive ? "page" : undefined}
      tabIndex={isActive ? 0 : -1}
    >
      {labels.page ? labels.page(item.page!) : item.label}
    </button>
  );
})}
</div>

```

```

    { /* Next Page Button */
    {showPrevNext && clampedPage < totalPages && (
      <button
        type="button"
        onClick={() => handlePageClick(clampedPage + 1)}
        onKeyDown={(e) => handleKeyDown(e, clampedPage + 1)}
        disabled={disabled || clampedPage === totalPages}
        className={cn(
          "inline-flex items-center justify-center",
          "rounded-md border",
          "font-medium transition-all duration-150 ease-out",
          "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-
visible:ring-offset-2 focus-visible:ring-offset-background",
          "active:scale-[0.98]",
          sizeClass,
          variantClass,
          disabled && DISABLED_CLASSES,
          disabledItemClassName,
        )}
        aria-label={labels.next?.ariaLabel ?? "Go to next page"}
        title={labels.next?.label ?? "Next page"}
      >
        <ChevronRight
          size={size === "sm" ? 14 : size === "md" ? 16 : 18}
          strokeWidth={2.5}
          aria-hidden="true"
        />
      </button>
    )}

    { /* Last Page Button */
    {showFirstLast && clampedPage < totalPages && (
      <button
        type="button"
        onClick={() => handlePageClick(totalPages)}
        onKeyDown={(e) => handleKeyDown(e, totalPages)}
        disabled={disabled || clampedPage === totalPages}
        className={cn(
          "inline-flex items-center justify-center",
          "rounded-md border",
          "font-medium transition-all duration-150 ease-out",
          "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-
visible:ring-offset-2 focus-visible:ring-offset-background",
          "active:scale-[0.98]",
          sizeClass,
          variantClass,
          disabled && DISABLED_CLASSES,
          disabledItemClassName,
        )}
        aria-label={labels.last?.ariaLabel ?? "Go to last page"}
        title={labels.last?.label ?? "Last page"}
      >
        <ChevronLast
          size={size === "sm" ? 14 : size === "md" ? 16 : 18}
          strokeWidth={2.5}
          aria-hidden="true"
        />
      </button>
    )}
  )}

```

```
        </button>
      )}
    </nav>
  </>
);
}

export { StandalonePagination as Pagination };

export default StandalonePagination;
```

```
FILE: status-indicator.tsx
LOKASI: src\feedback
```

```
import React from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { HintBox } from "../hint-box";

// ðŸš€ LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// ðŸš¡ LOCAL TYPE ISOLATION GATEWAY
export type StatusIndicatorVariant =
  "draft" | "pending" | "posted" | "approved" | "void" | "destructive" | "info";
export type StatusIndicatorSize = "sm" | "md" | "lg";

export interface StatusIndicatorProps extends React.HTMLAttributes<HTMLDivElement> {
  variant?: StatusIndicatorVariant;
  size?: StatusIndicatorSize;
  hasPulse?: boolean;
  label: string;
  hint?: string;
}

const variantStyles: Record<StatusIndicatorVariant, string> = {
  draft:
    "bg-[color-mix(in_oklch_var(--muted)_15%_transparent)] text-muted-foreground border-[color-mix(in_oklch_var(--border)_40%_transparent)]",
  pending:
    "bg-[color-mix(in_oklch_oklch(0.828_0.189_84.429)_12%_transparent)] text-[oklch(0.45_0.15_85)] border-[color-mix(in_oklch_oklch(0.828_0.189_84.429)_35%_transparent)]",
  posted:
    "bg-[color-mix(in_oklch_oklch(0.527_0.154_150.013)_12%_transparent)] text-[oklch(0.42_0.12_150)] border-[color-mix(in_oklch_oklch(0.527_0.154_150.013)_35%_transparent)]",
  approved:
    "bg-[color-mix(in_oklch_oklch(0.527_0.154_150.013)_12%_transparent)] text-[oklch(0.42_0.12_150)] border-[color-mix(in_oklch_oklch(0.527_0.154_150.013)_35%_transparent)]",
  void: "bg-[color-mix(in_oklch_var(--destructive)_12%_transparent)] text-destructive border-[color-mix(in_oklch_var(--destructive)_35%_transparent)]",
  destructive:
    "bg-[color-mix(in_oklch_var(--destructive)_12%_transparent)] text-destructive border-[color-mix(in_oklch_var(--destructive)_35%_transparent)]",
  info: "bg-[color-mix(in_oklch_oklch(0.546_0.245_262.881)_12%_transparent)] text-[oklch(0.45_0.18_265)] border-[color-mix(in_oklch_oklch(0.546_0.245_262.881)_35%_transparent)]",
};

const sizeStyles: Record<
  StatusIndicatorSize,
  { container: string; text: string; dot: string; gap: string }
> = {
  sm: {
    container: "px-2 py-0.5 rounded-full",
    text: "text-xs font-medium leading-none",
```

```

    dot: "w-1.5 h-1.5",
    gap: "gap-1.5",
  },
  md: {
    container: "px-2.5 py-1 rounded-full",
    text: "text-sm font-medium leading-none",
    dot: "w-2 h-2",
    gap: "gap-2",
  },
  lg: {
    container: "px-3 py-1.5 rounded-full",
    text: "text-base font-medium leading-none",
    dot: "w-2.5 h-2.5",
    gap: "gap-2.5",
  },
};

const pulseStyles = "animate-pulse duration-1000 ease-in-out";

export function StatusIndicator({
  variant = "draft",
  size = "md",
  hasPulse = false,
  label,
  hint,
  className,
  ...props
}: StatusIndicatorProps) {
  const sizes = sizeStyles[size];
  const variantClass = variantStyles[variant];

  return (
    <>
      {hint && (
        <HintBox content={hint} className="mb-1.5">
          <span aria-hidden="true" style={{ display: "none" }} />
        </HintBox>
      )}
      <div
        className={cn(
          "inline-flex items-center",
          sizes.container,
          sizes.gap,
          variantClass,
          "border",
          "transition-colors duration-200 ease-out",
          className,
        )}
        {...props}
      >
        {hasPulse && (
          <span
            className={cn(
              sizes.dot,
              "rounded-full",
              "bg-current",
              pulseStyles,
              "opacity-70",

```



```
        })
        aria-hidden="true"
      />
    })
    <span className={cn(sizes.text, "whitespace-nowrap")}>{label}</span>
  </div>
</>
);
}

export default StatusIndicator;
```

FILE: toast.tsx
LOKASI: src\feedback

```
import React, { useState, useCallback, useRef, useEffect } from "react";
import { Toaster as SonnerToaster, toast as sonnerToast } from "sonner";
import { X, Pin, LucideIcon } from "lucide-react";
import { cn } from "@lib/Utils";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export interface ToastActionIconConfig {
  icon: LucideIcon;
  onClick: (id: string | number) => void;
  tooltipText?: string;
}

export interface CustomToastProps {
  id: string | number;
  message: string;
  description?: string;
  leadingIcon?: LucideIcon;
  isCloseable?: boolean;
  isPinnable?: boolean;
  actionIcons?: ToastActionIconConfig[];
}

interface ToastState {
  isPinned: boolean;
  isHovered: boolean;
}

const MAX_ACTION_ICONS = 4;

function CustomToastCard({
  id,
  message,
  description,
  leadingIcon,
  isCloseable = false,
  isPinnable = false,
  actionIcons = [],
  onDismiss,
}: CustomToastProps & { onDismiss: (id: string | number) => void }) {
  const [state, setState] = useState<ToastState>({
    isPinned: false,
    isHovered: false,
  });

  const toastRef = useRef<HTMLDivElement>(null);
  const timeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);

  const handleDismiss = useCallback(() => {
    onDismiss(id);
  }, [id, onDismiss]);

  const handlePinToggle = useCallback(() => {
    setState((prev) => ({ ...prev, isPinned: !prev.isPinned }));
  }, [id, onDismiss]);
```

```

    }, []);

    const handleActionClick = useCallback(
      (action: ToastActionIconConfig) => {
        action.onClick(id);
      },
      [id],
    );

    useEffect(() => {
      if (state.isPinned) {
        if (timeoutRef.current) {
          clearTimeout(timeoutRef.current);
          timeoutRef.current = null;
        }
      }
      return () => {
        if (timeoutRef.current) {
          clearTimeout(timeoutRef.current);
        }
      };
    }, [state.isPinned]);

    const handleMouseEnter = useCallback(() => {
      setState((prev) => ({ ...prev, isHovered: true }));
    }, []);

    const handleMouseLeave = useCallback(() => {
      setState((prev) => ({ ...prev, isHovered: false }));
    }, []);

    const limitedActionIcons = actionIcons.slice(0, MAX_ACTION_ICONS);

    return (
      <div
        ref={toastRef}
        className={cn(
          "relative flex flex-col gap-2",
          "bg-card/80 backdrop-blur-[var(--backdrop-blur)]",
          "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-[var(--radius-surface)]",
          "shadow-[var(--shadow-md)]",
          "text-text-main",
          "transition-all duration-200 ease-out",
          "min-w-[320px] max-w-[480px]",
          "p-4",
          state.isHovered &&
            "shadow-[var(--shadow-lg)] border-[color-mix(in_oklch,var(--color-border)_80%,transparent)]",
        )}
        onMouseEnter={handleMouseEnter}
        onMouseLeave={handleMouseLeave}
      >
        <div className="flex items-start gap-3">
          {leadingIcon && (
            <div className="flex-shrink-0 mt-0.5 text-current opacity-80">
              {React.createElement(leadingIcon, {
                size: 20,
                strokeWidth: 2,

```

```

        "aria-hidden": "true",
      }}}
    </div>
  )}
  <div className="flex-1 min-w-0">
    <p className="font-medium text-sm leading-tight text-foreground">
      {message}
    </p>
    {description && (
      <p className="mt-1 text-sm leading-relaxed text-muted-foreground">
        {description}
      </p>
    )}
  </div>
  <div className="flex items-center gap-1.5 flex-shrink-0">
    {isPinnable && (
      <button
        type="button"
        onClick={handlePinToggle}
        className={cn(
          "p-1.5 rounded-md transition-all duration-150 ease-out",
          "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
          "active:scale-95",
          "text-muted-foreground hover:text-foreground",
          state.isPinned &&
            "text-brand-primary bg-[color-mix(in_oklch,var(--brand-primary)_15%,transparent)]",
        )}
        aria-label={state.isPinned ? "Unpin toast" : "Pin toast"}
        aria-pressed={state.isPinned}
      >
        <Pin size={14} strokeWidth={2.5} aria-hidden="true" />
      </button>
    )}
    {isCloseable && (
      <button
        type="button"
        onClick={handleDismiss}
        className={cn(
          "p-1.5 rounded-md transition-all duration-150 ease-out",
          "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
          "active:scale-95",
          "text-muted-foreground hover:text-foreground",
        )}
        aria-label="Dismiss toast"
      >
        <X size={14} strokeWidth={2.5} aria-hidden="true" />
      </button>
    )}
  </div>
</div>

{limitedActionIcons.length > 0 && (
  <div className="flex items-center justify-end gap-1.5 pt-1 border-t border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]">
    {limitedActionIcons.map((action, index) => (
      <button
        key={index}
        type="button"

```

```

        onClick={() => handleActionClick(action)}
        className={cn(
          "p-2 rounded-md transition-all duration-150 ease-out",
          "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
          "active:scale-95",
          "text-muted-foreground hover:text-foreground",
          "flex items-center justify-center",
        )}
        aria-label={action.tooltipText || `Action ${index + 1}`}
      >
        <action.icon size={16} strokeWidth={2.5} aria-hidden="true" />
      </button>
    )}}
  </div>
)}
</div>
);
}

export function toast(props: CustomToastProps) {
  const { id, ...rest } = props;
  return sonnerToast.custom(
    () => (
      <CustomToastCard
        id={id}
        {...rest}
        onDismiss={() => sonnerToast.dismiss(dismissId)}
      />
    ),
    {
      id,
      duration: Infinity,
    },
  );
}

export function dismiss(id?: string | number) {
  sonnerToast.dismiss(id);
}

export function dismissAll() {
  sonnerToast.dismiss();
}

interface CustomToasterProps {
  position?:
    | "top-left"
    | "top-right"
    | "bottom-left"
    | "bottom-right"
    | "top-center"
    | "bottom-center";
  className?: string;
}

export function CustomToaster({
  position = "top-right",
  className,

```

```

    }: CustomToasterProps) {
      return (
        <SonnerToaster
          position={position}
          className={cn("toaster-group", className)}
          toastOptions={{
            className: cn(
              "group",
              "bg-card/80 backdrop-blur-[var(--backdrop-blur)]",
              "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
              "rounded-[var(--radius-surface)]",
              "shadow-[var(--shadow-md)]",
              "text-text-main",
              "transition-all duration-200 ease-out",
            ),
            style: {
              "--normal-bg": "var(--card)",
              "--normal-text": "var(--foreground)",
              "--normal-border":
                "color-mix(in oklch, var(--border) 60%, transparent)",
            } as React.CSSProperties,
          }}
          closeButton={false}
          expand={false}
          richColors={false}
        />
      );
    }
  }

  export default CustomToaster;

```

```

FILE: report-plugins-core.ts
LOKASI: src\feedback\report-workspace

```

```

/**
 * Plugin Infrastructure Core for Report Workspace.
 * Clean decoupled plugin system with strict typing.
 * Zero `any` usage. Strict discriminated unions.
 */

import type {
  PluginEngine,
  PluginConfig,
  PluginId,
  PluginRegistry,
  PluginRegistryEntry,
  PluginInitContext,
  RenderContext,
  RenderResult,
  UpdateResult,
  PluginEvent,
  PluginEventType,
  DOMEngine,
  StyleEngine,
  BindingEngine,
  ReportContext,
  ElementId,
  ElementType,
  ReportEventEmitter,
  ReportEventType,
  ReportEventMap,
  ReportEventListener,
  ElementConfig,
  RenderedElement,
  ComputedStyle,
  ElementMeasurements,
  DataSourceId,
  ResolvedBinding,
  BindingConfig,
  ReportData,
  ReportParameters,
  ValueFormatter,
  ResolvedBindingMap,
  PageBreakReason,
  ElementCreationOptions,
  StyleContext,
  ThemeConfig,
  StyleRuleConfig,
} from "../report-types";

// =====
// Simple Event Emitter (avoids Node.js dependency)
// =====

type EventListener = (...args: unknown[]) => void;

```

```

class SimpleEventEmitter {
  private events = new Map<string, Set<EventListener>>();

  on(event: string, listener: EventListener): this {
    if (!this.events.has(event)) {
      this.events.set(event, new Set());
    }
    this.events.get(event)!.add(listener);
    return this;
  }

  off(event: string, listener: EventListener): this {
    this.events.get(event)?.delete(listener);
    return this;
  }

  emit(event: string, ...args: unknown[]): boolean {
    const listeners = this.events.get(event);
    if (!listeners) return false;
    for (const listener of listeners) {
      listener(...args);
    }
    return true;
  }

  once(event: string, listener: EventListener): this {
    const onceWrapper: EventListener = (...args: unknown[]) => {
      listener(...args);
      this.off(event, onceWrapper);
    };
    return this.on(event, onceWrapper);
  }

  listeners(event: string): EventListener[] {
    return Array.from(this.events.get(event) ?? []);
  }

  rawListeners(event: string): EventListener[] {
    return this.listeners(event);
  }
}

// =====
// Core Plugin Registry Implementation
// =====

/**
 * Default plugin registry implementation.
 * Thread-safe, supports enable/disable, filtering by element type.
 */
export class DefaultPluginRegistry implements PluginRegistry {
  private readonly registry = new Map<PluginId, PluginRegistryEntry>();
  private readonly byElementType = new Map<ElementType, Set<PluginId>>();

  register(engine: PluginEngine, config: PluginConfig): void {
    if (this.registry.has(config.id)) {
      throw new Error(`Plugin with id "${config.id}" already registered`);
    }
  }
}

```



```

const entry: PluginRegistryEntry = {
  engine,
  config,
  registeredAt: new Date(),
  enabled: config.enabled,
};

this.registry.set(config.id, entry);

// Index by element types
for (const elementType of engine.supportedElementTypes) {
  if (!this.byElementType.has(elementType)) {
    this.byElementType.set(elementType, new Set());
  }
  this.byElementType.get(elementType)!.add(config.id);
}
}

unregister(pluginId: PluginId): boolean {
  const entry = this.registry.get(pluginId);
  if (!entry) {
    return false;
  }

  // Remove from element type index
  for (const elementType of entry.engine.supportedElementTypes) {
    const set = this.byElementType.get(elementType);
    if (set) {
      set.delete(pluginId);
      if (set.size === 0) {
        this.byElementType.delete(elementType);
      }
    }
  }

  return this.registry.delete(pluginId);
}

get(pluginId: PluginId): PluginRegistryEntry | undefined {
  return this.registry.get(pluginId);
}

getAll(): PluginRegistryEntry[] {
  return Array.from(this.registry.values());
}

getByElementType(elementType: ElementType): PluginRegistryEntry[] {
  const pluginIds = this.byElementType.get(elementType);
  if (!pluginIds) {
    return [];
  }
  return Array.from(pluginIds)
    .map((id) => this.registry.get(id))
    .filter((entry): entry is PluginRegistryEntry => entry !== undefined);
}

isEnabled(pluginId: PluginId): boolean {

```

```

    return this.registry.get(pluginId)?.enabled ?? false;
  }

  setEnabled(pluginId: PluginId, enabled: boolean): boolean {
    const entry = this.registry.get(pluginId);
    if (!entry) {
      return false;
    }
    entry.enabled = enabled;
    return true;
  }
}

// =====
// Default Engine Implementations
// =====

/**
 * Default DOM Engine implementation using native browser APIs.
 * Single centralized source of truth for DOM queries.
 */
export class DefaultDOMEngine implements DOMEngine {
  private container: HTMLElement | null = null;

  setContainer(container: HTMLElement | null): void {
    this.container = container;
  }

  getContainer(): HTMLElement | null {
    return this.container;
  }

  createElement(tag: string, options?: ElementCreationOptions): HTMLElement {
    const element = document.createElement(tag);

    if (options?.id) {
      element.id = options.id;
    }
    if (options?.className) {
      element.className = options.className;
    }
    if (options?.style) {
      Object.assign(element.style, options.style);
    }
    if (options?.attributes) {
      for (const [name, value] of Object.entries(options.attributes)) {
        element.setAttribute(name, value);
      }
    }
    if (options?.dataset) {
      for (const [key, value] of Object.entries(options.dataset)) {
        element.dataset[key] = value;
      }
    }
    if (options?.children) {
      for (const child of options.children) {
        element.appendChild(child);
      }
    }
  }
}

```

```

    }

    return element;
}

createTextNode(text: string): Text {
    return document.createTextNode(text);
}

appendChild(parent: HTMLElement, child: Node): void {
    parent.appendChild(child);
}

removeChild(parent: HTMLElement, child: Node): void {
    parent.removeChild(child);
}

replaceChild(parent: HTMLElement, newChild: Node, oldChild: Node): void {
    parent.replaceChild(newChild, oldChild);
}

setAttribute(element: HTMLElement, name: string, value: string): void {
    element.setAttribute(name, value);
}

removeAttribute(element: HTMLElement, name: string): void {
    element.removeAttribute(name);
}

setStyle(
    element: HTMLElement,
    property: string,
    value: string | number,
): void {
    element.style.setProperty(property, String(value));
}

getComputedStyle(element: HTMLElement): ComputedStyle {
    const computed = window.getComputedStyle(element);
    const style: ComputedStyle = {};
    for (let i = 0; i < computed.length; i++) {
        const prop = computed[i];
        style[prop] = computed.getPropertyValue(prop);
    }
    return style;
}

measureElement(element: HTMLElement): ElementMeasurements {
    const rect = element.getBoundingClientRect();
    return {
        width: rect.width,
        height: rect.height,
        naturalWidth: (element as HTMLImageElement).naturalWidth ?? undefined,
        naturalHeight: (element as HTMLImageElement).naturalHeight ?? undefined,
    };
}

querySelector(selector: string, root?: HTMLElement): HTMLElement | null {

```

```

    const searchRoot = root ?? this.container ?? document;
    return searchRoot.querySelector<HTMLElement>(selector);
}

querySelectorAll(selector: string, root?: HTMLElement): HTMLElement[] {
    const searchRoot = root ?? this.container ?? document;
    return Array.from(searchRoot.querySelectorAll<HTMLElement>(selector));
}

addEventListener(
    element: HTMLElement,
    event: string,
    handler: (event: Event) => void,
): void {
    element.addEventListener(event, handler);
}

removeEventListener(
    element: HTMLElement,
    event: string,
    handler: (event: Event) => void,
): void {
    element.removeEventListener(event, handler);
}

// Centralized DOM query methods - single source of truth
getPages(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-report-page]");
}

findFooters(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-report-footer]");
}

findGroups(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-group]");
}

findLinks(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-link]");
}

measurePageHeight(page: HTMLElement): {
    scrollHeight: number;
    clientHeight: number;
} {

```

```

        return {
            scrollHeight: page.scrollHeight,
            clientHeight: page.clientHeight,
        };
    }
}

/**
 * Default Style Engine implementation.
 * Handles CSS computation, custom properties, and selector matching.
 */
export class DefaultStyleEngine implements StyleEngine {
    private theme: ThemeConfig = {
        name: "default",
        colors: {},
        fonts: {},
        spacing: {},
        breakpoints: {},
        customProperties: {},
    };

    setTheme(theme: ThemeConfig): void {
        this.theme = theme;
    }

    getTheme(): ThemeConfig {
        return this.theme;
    }

    computeStyle(
        element: RenderedElement,
        rules: StyleRuleConfig[],
    ): ComputedStyle {
        // Sort rules by priority (specificity)
        const sortedRules = [...rules].sort((a, b) => b.priority - a.priority);

        const computed: ComputedStyle = {};

        // Apply base element styles
        if (element.config.style) {
            Object.assign(computed, element.config.style);
        }

        // Apply matching rules
        for (const rule of sortedRules) {
            if (this.matchSelector(element, rule.selector)) {
                Object.assign(computed, rule.style);
            }
        }

        // Resolve custom properties
        return this.resolveCustomProperties(computed, {
            variables: this.theme.customProperties,
            theme: this.theme,
        });
    }

    applyStyle(element: HTMLElement, style: ComputedStyle): void {

```

```

    for (const [property, value] of Object.entries(style)) {
        if (value !== undefined) {
            element.style.setProperty(property, String(value));
        }
    }
}

resolveCustomProperties(
    style: ComputedStyle,
    context: StyleContext,
): ComputedStyle {
    const resolved: ComputedStyle = {};

    for (const [property, value] of Object.entries(style)) {
        if (typeof value === "string" && value.includes("var(")) {
            // Simple var() resolution
            resolved[property] = value.replace(
                /var\(--([^\)]+)\)/g,
                (_, varName) => context.variables[varName] ?? value,
            );
        } else {
            resolved[property] = value;
        }
    }

    return resolved;
}

matchSelector(element: RenderedElement, selector: string): boolean {
    if (!element.domElement) return false;
    try {
        return element.domElement.matches(selector);
    } catch {
        return false;
    }
}

getSpecificity(selector: string): number {
    // Simplified specificity calculation
    let score = 0;
    // ID selectors
    score += (selector.match(/#[\w-]+/g) || []).length * 100;
    // Class selectors
    score += (selector.match(/\.[\w-]+/g) || []).length * 10;
    // Element selectors
    score += (selector.match(/[a-zA-Z]+/g) || []).length;
    return score;
}
}

/**
 * Default Binding Engine implementation.
 * Handles data binding resolution and formatting.
 */
export class DefaultBindingEngine implements BindingEngine {
    private formatters = new Map<string, ValueFormatter>();

    constructor() {

```

```

    this.registerDefaultFormatters();
}

private registerDefaultFormatters(): void {
    this.registerFormatter("string", (value) => String(value ?? ""));
    this.registerFormatter("number", (value) => Number(value).toLocaleString());
    this.registerFormatter("currency", (value, args) => {
        const locale = (args?.locale as string) ?? "en-US";
        const currency = (args?.currency as string) ?? "USD";
        return new Intl.NumberFormat(locale, {
            style: "currency",
            currency,
        }).format(Number(value));
    });
    this.registerFormatter("date", (value, args) => {
        const date = value instanceof Date ? value : new Date(String(value));
        const locale = (args?.locale as string) ?? "en-US";
        const options = (args?.options as Intl.DateTimeFormatOptions) ?? {
            year: "numeric",
            month: "short",
            day: "numeric",
        };
        return new Intl.DateTimeFormat(locale, options).format(date);
    });
    this.registerFormatter("percentage", (value, args) => {
        const decimals = (args?.decimals as number) ?? 1;
        return `${(Number(value) * 100).toFixed(decimals)}%`;
    });
}

async resolveBinding(
    binding: BindingConfig,
    data: ReportData,
    parameters: ReportParameters,
): Promise<ResolvedBinding> {
    try {
        const dataSourceData = data[binding.dataSourceId] ?? [];
        let result: unknown = dataSourceData;

        // Apply query (simplified - in real impl would use query parser)
        if (binding.query) {
            result = this.applyQuery(result, binding.query, parameters);
        }

        // Apply transform
        if (binding.transform) {
            result = this.applyTransform(result, binding.transform);
        }

        // Format value
        let formattedValue: string | null = null;
        if (binding.parameters?.formatter) {
            formattedValue = this.formatValue(
                result,
                binding.parameters.formatter as string,
                binding.parameters.formatterArgs as Record<string, unknown>,
            );
        }
    }
}

```

```

        return {
            config: binding,
            data: result,
            formattedValue,
        };
    } catch (error) {
        return {
            config: binding,
            data: null,
            formattedValue: null,
            error: error instanceof Error ? error : new Error(String(error)),
        };
    }
}

async resolveBindings(
    bindings: BindingConfig[],
    data: ReportData,
    parameters: ReportParameters,
): Promise<ResolvedBindingMap> {
    const results = await Promise.all(
        bindings.map((binding) => this.resolveBinding(binding, data, parameters)),
    );

    const map = new Map<BindingConfig["id"], ResolvedBinding>();
    for (const result of results) {
        map.set(result.config.id, result);
    }
    return map;
}

formatValue(
    value: unknown,
    formatter: string,
    args?: Record<string, unknown>,
): string {
    const fn = this.formatters.get(formatter);
    if (!fn) {
        return String(value ?? "");
    }
    return fn(value, args);
}

registerFormatter(name: string, formatter: ValueFormatter): void {
    this.formatters.set(name, formatter);
}

unregisterFormatter(name: string): void {
    this.formatters.delete(name);
}

private applyQuery(
    data: unknown,
    query: string,
    parameters: ReportParameters,
): unknown {
    // Simplified query application - in real implementation would use a query engine

```



```

    // For now, return data as-is
    return data;
}

private applyTransform(
    data: unknown,
    transform: { type: string; config: Record<string, unknown> },
): unknown {
    // Simplified transform application
    return data;
}
}

// =====
// Report Event Emitter
// =====

/**
 * Typed event emitter for report events.
 * Uses discriminated unions for type-safe event handling.
 */
export class ReportEventEmitterImpl implements ReportEventEmitter {
    private emitter = new SimpleEventEmitter();

    on<T extends ReportEventType>(
        type: T,
        listener: ReportEventListener<T>,
    ): () => void {
        this.emitter.on(type, listener as EventListener);
        return () => this.off(type, listener);
    }

    off<T extends ReportEventType>(
        type: T,
        listener: ReportEventListener<T>,
    ): void {
        this.emitter.off(type, listener as EventListener);
    }

    async emit<T extends ReportEventType>(
        type: T,
        event: ReportEventMap[T],
    ): Promise<void> {
        this.emitter.emit(type, event);
        // Wait for any async listeners
        const listeners = this.emitter.listeners(type) as ReportEventListener<T>[];
        await Promise.all(listeners.map((l) => Promise.resolve(l(event))));
    }

    once<T extends ReportEventType>(
        type: T,
        listener: ReportEventListener<T>,
    ): () => void {
        this.emitter.once(type, listener as EventListener);
        return () => this.off(type, listener);
    }
}

```

```
// =====
// Plugin Manager
// =====

/**
 * High-level plugin manager that coordinates plugin lifecycle.
 */
export class PluginManager {
  private readonly registry: PluginRegistry;
  private readonly domEngine: DOMEngine;
  private readonly styleEngine: StyleEngine;
  private readonly bindingEngine: BindingEngine;
  private readonly eventEmitter: ReportEventEmitter;
  private readonly initializedPlugins = new Set<PluginId>();

  constructor(
    registry: PluginRegistry,
    domEngine: DOMEngine,
    styleEngine: StyleEngine,
    bindingEngine: BindingEngine,
    eventEmitter: ReportEventEmitter,
  ) {
    this.registry = registry;
    this.domEngine = domEngine;
    this.styleEngine = styleEngine;
    this.bindingEngine = bindingEngine;
    this.eventEmitter = eventEmitter;
  }

  async initializePlugin(
    pluginId: PluginId,
    context: ReportContext,
  ): Promise<void> {
    const entry = this.registry.get(pluginId);
    if (!entry || !entry.enabled) {
      throw new Error(`Plugin "${pluginId}" not found or disabled`);
    }

    if (this.initializedPlugins.has(pluginId)) {
      return; // Already initialized
    }

    const initContext: PluginInitContext = {
      reportContext: context,
      pluginConfig: entry.config,
      domEngine: this.domEngine,
      styleEngine: this.styleEngine,
      bindingEngine: this.bindingEngine,
    };

    await entry.engine.initialize(initContext);
    this.initializedPlugins.add(pluginId);

    await this.eventEmitter.emit("plugin:register", { plugin: entry });
  }

  async destroyPlugin(pluginId: PluginId): Promise<void> {
    const entry = this.registry.get(pluginId);
```

```

    if (!entry) {
        return;
    }

    await entry.engine.destroy();
    this.initializedPlugins.delete(pluginId);

    await this.eventEmitter.emit("plugin:unregister", { pluginId });
}

async renderElement(
    element: RenderedElement,
    context: RenderContext,
): Promise<RenderResult> {
    const pluginId = element.config.pluginId;
    if (!pluginId) {
        return {
            success: false,
            error: new Error("No plugin assigned to element"),
        };
    }

    const entry = this.registry.get(pluginId);
    if (!entry || !entry.enabled) {
        return {
            success: false,
            error: new Error(`Plugin "${pluginId}" not available`),
        };
    }

    await this.initializePlugin(pluginId, context.reportContext);

    const result = await entry.engine.renderElement(element, context);

    await this.eventEmitter.emit("element:render-complete", {
        element,
        result,
    });

    return result;
}

async updateElement(
    element: RenderedElement,
    context: RenderContext,
): Promise<UpdateResult> {
    const pluginId = element.config.pluginId;
    if (!pluginId) {
        return {
            success: false,
            changed: false,
            error: new Error("No plugin assigned"),
        };
    }

    const entry = this.registry.get(pluginId);
    if (!entry || !entry.enabled) {
        return {

```

```

        success: false,
        changed: false,
        error: new Error(`Plugin "${pluginId}" not available`),
    };
}

const result = await entry.engine.updateElement(element, context);

await this.eventEmitter.emit("element:update", {
    element,
    result,
});

return result;
}

getPluginForElement(
    elementType: ElementType,
): PluginRegistryEntry | undefined {
    const entries = this.registry.getByElementType(elementType);
    return entries.find((e) => e.enabled);
}

getRegistry(): PluginRegistry {
    return this.registry;
}

getEventEmitter(): ReportEventEmitter {
    return this.eventEmitter;
}
}

// =====
// Factory Functions
// =====

/**
 * Create a complete plugin infrastructure with default implementations.
 */
export function createPluginInfrastructure(): {
    registry: PluginRegistry;
    domEngine: DOMEngine;
    styleEngine: StyleEngine;
    bindingEngine: BindingEngine;
    eventEmitter: ReportEventEmitter;
    pluginManager: PluginManager;
} {
    const registry = new DefaultPluginRegistry();
    const domEngine = new DefaultDOMEngine();
    const styleEngine = new DefaultStyleEngine();
    const bindingEngine = new DefaultBindingEngine();
    const eventEmitter = new ReportEventEmitterImpl();
    const pluginManager = new PluginManager(
        registry,
        domEngine,
        styleEngine,
        bindingEngine,
        eventEmitter,
    );
}

```

```

    );

    return {
        registry,
        domEngine,
        styleEngine,
        bindingEngine,
        eventEmitter,
        pluginManager,
    };
}

/**
 * Create a minimal plugin engine for testing or simple plugins.
 */
export function createBasePluginEngine(
    id: PluginId,
    name: string,
    version: string,
    supportedElementTypes: readonly ElementType[],
): PluginEngine {
    return {
        id,
        name,
        version,
        supportedElementTypes,
        initialize: async () => {},
        renderElement: async () => ({
            success: false,
            error: new Error("Not implemented"),
        }),
        updateElement: async () => ({
            success: false,
            changed: false,
            error: new Error("Not implemented"),
        }),
        destroy: async () => {},
    };
}

// =====
// Re-export types for convenience
// =====

export type {
    PluginEngine,
    PluginConfig,
    PluginId,
    PluginRegistry,
    PluginRegistryEntry,
    PluginInitContext,
    RenderContext,
    RenderResult,
    UpdateResult,
    PluginEvent,
    PluginEventType,
    DOMEEngine,
    StyleEngine,

```

```
    BindingEngine,  
    ReportContext,  
    ElementId,  
    ElementType,  
    ReportEventEmitter,  
    ReportEventType,  
    ReportEventMap,  
    ReportEventListener,  
    ElementConfig,  
    RenderedElement,  
    ComputedStyle,  
    ElementMeasurements,  
    DataSourceId,  
    ResolvedBinding,  
    BindingConfig,  
    ReportData,  
    ReportParameters,  
    ValueFormatter,  
    ResolvedBindingMap,  
    PageBreakReason,  
    ElementCreationOptions,  
    StyleContext,  
    ThemeConfig,  
} from "../report-types";
```

```

FILE: report-types.ts
LOKASI: src\feedback\report-workspace

```

```

/**
 * Strict type contracts for the Report Workspace system.
 * Zero `any` usage. Strict discriminated unions for plugin types.
 * Zero `any` usage. Strict discriminated unions for plugin types.
 */

// =====
// Core Domain Types
// =====

/** Unique identifier for a report */
export type ReportId = string & { readonly __brand: unique symbol };

/** Unique identifier for a plugin */
export type PluginId = string & { readonly __brand: unique symbol };

/** Unique identifier for a DOM element within the report */
export type ElementId = string & { readonly __brand: unique symbol };

/** Unique identifier for a data source */
export type DataSourceId = string & { readonly __brand: unique symbol };

/** Unique identifier for a layout */
export type LayoutId = string & { readonly __brand: unique symbol };

/** Unique identifier for a style rule */
export type StyleRuleId = string & { readonly __brand: unique symbol };

/** Unique identifier for a data binding */
export type BindingId = string & { readonly __brand: unique symbol };

// =====
// Branding Helpers (internal use only)
// =====

/** @internal */
export const brand = {
  reportId: (id: string): ReportId => id as ReportId,
  pluginId: (id: string): PluginId => id as PluginId,
  elementId: (id: string): ElementId => id as ElementId,
  dataSourceId: (id: string): DataSourceId => id as DataSourceId,
  layoutId: (id: string): LayoutId => id as LayoutId,
  styleRuleId: (id: string): StyleRuleId => id as StyleRuleId,
  bindingId: (id: string): BindingId => id as BindingId,
} as const;

// =====
// Report Definition Types
// =====

/** Supported report output formats */
export type OutputFormat = "pdf" | "html" | "excel" | "csv" | "json";

```

```

/** Report page orientation */
export type PageOrientation = "portrait" | "landscape";

/** Paper size standards */
export type PaperSize = "A4" | "A3" | "Letter" | "Legal" | "Custom";

/** Custom paper dimensions */
export interface CustomPaperSize {
  width: number; // in millimeters
  height: number; // in millimeters
}

/** Paper configuration */
export type PaperConfig =
  | { type: Exclude<PaperSize, "Custom">; orientation: PageOrientation }
  | { type: "Custom"; size: CustomPaperSize; orientation: PageOrientation };

/** Page margin configuration (in millimeters) */
export interface PageMargins {
  top: number;
  right: number;
  bottom: number;
  left: number;
}

/** Report page configuration */
export interface PageConfig {
  paper: PaperConfig;
  margins: PageMargins;
  headerHeight?: number; // mm
  footerHeight?: number; // mm
}

/** Report metadata */
export interface ReportMetadata {
  id: ReportId;
  name: string;
  description?: string;
  version: string;
  author?: string;
  createdAt: Date;
  updatedAt: Date;
  tags?: string[];
}

/** Complete report definition */
export interface ReportDefinition {
  metadata: ReportMetadata;
  pageConfig: PageConfig;
  dataSources: DataSourceConfig[];
  layouts: LayoutConfig[];
  styleRules: StyleRuleConfig[];
  bindings: BindingConfig[];
  plugins: PluginConfig[];
}

/** Data source configuration */

```



```

export interface DataSourceConfig {
  id: DataSourceId;
  name: string;
  type: DataSourceType;
  config: Record<string, unknown>;
  refreshInterval?: number; // milliseconds, 0 = manual
}

/** Supported data source types */
export type DataSourceType =
  "sql" | "rest" | "graphql" | "csv" | "json" | "static" | "function";

/** Layout configuration */
export interface LayoutConfig {
  id: LayoutId;
  name: string;
  type: LayoutType;
  rootElement: ElementConfig;
  conditions?: LayoutCondition[];
}

/** Layout types */
export type LayoutType =
  "page" | "band" | "freeform" | "table" | "chart" | "custom";

/** Layout condition for conditional rendering */
export interface LayoutCondition {
  field: string;
  operator: ConditionOperator;
  value: unknown;
}

/** Condition operators */
export type ConditionOperator =
  | "eq"
  | "neq"
  | "gt"
  | "gte"
  | "lt"
  | "lte"
  | "contains"
  | "startsWith"
  | "endsWith"
  | "in"
  | "notIn"
  | "isNull"
  | "isNotNull"
  | "between";

/** Element configuration */
export interface ElementConfig {
  id: ElementId;
  type: ElementType;
  name?: string;
  style?: ElementStyleConfig;
  bindings?: BindingRef[];
  children?: ElementConfig[];
  conditions?: LayoutCondition[];
}

```

```

    pluginId?: PluginId;
    pluginConfig?: Record<string, unknown>;
}

```

```

/** Element types */
export type ElementType =
  | "container"
  | "text"
  | "image"
  | "table"
  | "chart"
  | "barcode"
  | "qr"
  | "chart"
  | "subreport"
  | "pageBreak"
  | "line"
  | "rectangle"
  | "custom";

```

```

/** Element style configuration */
export interface ElementStyleConfig {
  // Layout
  display?: "block" | "inline" | "flex" | "grid" | "none";
  position?: "static" | "relative" | "absolute" | "fixed" | "sticky";
  top?: string;
  right?: string;
  bottom?: string;
  left?: string;
  width?: string;
  height?: string;
  minWidth?: string;
  maxWidth?: string;
  minHeight?: string;
  maxHeight?: string;
  margin?: string;
  padding?: string;

  // Flexbox
  flexDirection?: "row" | "column" | "row-reverse" | "column-reverse";
  flexWrap?: "nowrap" | "wrap" | "wrap-reverse";
  justifyContent?:
    | "flex-start"
    | "flex-end"
    | "center"
    | "space-between"
    | "space-around"
    | "space-evenly";
  alignItems?: "stretch" | "flex-start" | "flex-end" | "center" | "baseline";
  alignContent?:
    | "stretch"
    | "flex-start"
    | "flex-end"
    | "center"
    | "space-between"
    | "space-around";
  flex?: string;
  flexGrow?: number;
}

```

```

flexShrink?: number;
flexBasis?: string;

// Grid
gridTemplateColumns?: string;
gridTemplateRows?: string;
gridColumn?: string;
gridRow?: string;

// Typography
fontFamily?: string;
fontSize?: string;
fontWeight?: string | number;
fontStyle?: "normal" | "italic" | "oblique";
lineHeight?: string | number;
letterSpacing?: string;
textAlign?: "left" | "right" | "center" | "justify" | "start" | "end";
textDecoration?: string;
textTransform?: "none" | "capitalize" | "uppercase" | "lowercase";
whiteSpace?: "normal" | "nowrap" | "pre" | "pre-wrap" | "pre-line";
wordBreak?: "normal" | "break-all" | "keep-all" | "break-word";
overflow?: "visible" | "hidden" | "scroll" | "auto" | "clip";
textOverflow?: "clip" | "ellipsis" | string;

// Colors
color?: string;
backgroundColor?: string;
backgroundImage?: string;
backgroundSize?: string;
backgroundPosition?: string;
backgroundRepeat?: string;
borderColor?: string;
borderWidth?: string;
borderStyle?:
  | "none"
  | "solid"
  | "dashed"
  | "dotted"
  | "double"
  | "groove"
  | "ridge"
  | "inset"
  | "outset";
borderRadius?: string;
boxShadow?: string;
opacity?: number;

// Print
pageBreakBefore?:
  "auto" | "always" | "avoid" | "left" | "right" | "recto" | "verso";
pageBreakAfter?:
  "auto" | "always" | "avoid" | "left" | "right" | "recto" | "verso";
pageBreakInside?: "auto" | "avoid";

// Custom properties (CSS custom properties)
customProperties?: Record<string, string>;
}

```

```

/** Binding reference */
export interface BindingRef {
  bindingId: BindingId;
  path?: string; // JSON path for nested data
  formatter?: string; // formatter name
  formatterArgs?: Record<string, unknown>;
}

/** Binding configuration */
export interface BindingConfig {
  id: BindingId;
  name: string;
  dataSourceId: DataSourceId;
  query: string; // SQL, GraphQL, REST path, etc.
  parameters?: Record<string, unknown>;
  transform?: BindingTransform;
}

/** Binding transformation */
export interface BindingTransform {
  type:
    "map" | "filter" | "reduce" | "groupBy" | "sort" | "paginate" | "custom";
  config: Record<string, unknown>;
}

/** Style rule configuration */
export interface StyleRuleConfig {
  id: StyleRuleId;
  name: string;
  selector: string; // CSS selector
  style: ElementStyleConfig;
  priority: number; // higher = more specific
  conditions?: LayoutCondition[];
}

/** Plugin configuration */
export interface PluginConfig {
  id: PluginId;
  name: string;
  version: string;
  enabled: boolean;
  config: Record<string, unknown>;
  elementTypes?: ElementType[]; // element types this plugin handles
}

// =====
// Runtime Types (Runtime State)
// =====

/** Report execution context */
export interface ReportContext {
  report: ReportDefinition;
  data: ReportData;
  parameters: ReportParameters;
  runtime: RuntimeContext;
}

/** Resolved report data */

```

```

export interface ReportData {
  [dataSourceId: DataSourceId]: unknown[];
}

/** Report parameters */
export interface ReportParameters {
  [key: string]: unknown;
}

/** Runtime context */
export interface RuntimeContext {
  reportId: ReportId;
  startTime: Date;
  currentPage: number;
  totalPages: number;
  variables: Record<string, unknown>;
  plugins: Map<PluginId, PluginInstance>;
  elements: Map<ElementId, RenderedElement>;
  styles: ComputedStyleMap;
  bindings: ResolvedBindingMap;
}

/** Rendered element in the DOM */
export interface RenderedElement {
  id: ElementId;
  config: ElementConfig;
  domElement: HTMLElement | null;
  computedStyle: ComputedStyle;
  bounds: ElementBounds | null;
  children: RenderedElement[];
  parent: RenderedElement | null;
  dataContext: unknown;
  bindings: ResolvedBinding[];
}

/** Computed style for an element */
export interface ComputedStyle {
  [property: string]: string | number | undefined;
}

/** Element bounding box */
export interface ElementBounds {
  x: number;
  y: number;
  width: number;
  height: number;
  pageNumber: number;
}

/** Computed style map */
export type ComputedStyleMap = Map<ElementId, ComputedStyle>;

/** Resolved binding with data */
export interface ResolvedBinding {
  config: BindingConfig;
  data: unknown;
  formattedValue: string | null;
  error?: Error;
}

```

```

}

/** Resolved binding map */
export type ResolvedBindingMap = Map<BindingId, ResolvedBinding>;

/** Plugin instance at runtime */
export interface PluginInstance {
  config: PluginConfig;
  instance: PluginEngine;
  elements: Map<ElementId, RenderedElement>;
  initialized: boolean;
  destroyed: boolean;
}

// =====
// Plugin System Types (Discriminated Unions - Zero `any`)
// =====

/** Base plugin engine interface */
export interface PluginEngine {
  readonly id: PluginId;
  readonly name: string;
  readonly version: string;
  readonly supportedElementTypes: readonly ElementType[];

  /** Initialize the plugin */
  initialize(context: PluginInitContext): Promise<void> | void;

  /** Render an element */
  renderElement(
    element: RenderedElement,
    context: RenderContext,
  ): Promise<RenderResult> | RenderResult;

  /** Update an element with new data */
  updateElement(
    element: RenderedElement,
    context: RenderContext,
  ): Promise<UpdateResult> | UpdateResult;

  /** Destroy/cleanup the plugin */
  destroy(): Promise<void> | void;

  /** Handle custom events */
  handleEvent?(event: PluginEvent): Promise<void> | void;
}

/** Plugin initialization context */
export interface PluginInitContext {
  reportContext: ReportContext;
  pluginConfig: PluginConfig;
  domEngine: DOMEngine;
  styleEngine: StyleEngine;
  bindingEngine: BindingEngine;
}

/** Render context for plugin rendering */
export interface RenderContext {

```

```

    element: RenderedElement;
    reportContext: ReportContext;
    domEngine: DOMEngine;
    styleEngine: StyleEngine;
    bindingEngine: BindingEngine;
    parentElement: HTMLElement | null;
}

```

```

/** Render result from plugin */

```

```

export interface RenderResult {
    success: boolean;
    domElement?: HTMLElement;
    error?: Error;
    warnings?: string[];
    measurements?: ElementMeasurements;
}

```

```

/** Element measurements */

```

```

export interface ElementMeasurements {
    width: number;
    height: number;
    naturalWidth?: number;
    naturalHeight?: number;
}

```

```

/** Update result from plugin */

```

```

export interface UpdateResult {
    success: boolean;
    changed: boolean;
    error?: Error;
    warnings?: string[];
}

```

```

/** Plugin event */

```

```

export interface PluginEvent {
    type: PluginEventType;
    source: PluginId;
    target?: PluginId;
    payload: unknown;
    timestamp: Date;
}

```

```

/** Plugin event types */

```

```

export type PluginEventType =
    | "element:render"
    | "element:update"
    | "element:destroy"
    | "data:refresh"
    | "page:break"
    | "page:render"
    | "style:change"
    | "binding:resolve"
    | "custom";

```

```

/** Plugin initialization context (alias for clarity) */

```

```

export type PluginInitContextAlias = PluginInitContext;

```

```

/** DOMEngine interface for DOM operations */

```

```

export interface DOMEngine {
  createElement(tag: string, options?: ElementCreationOptions): HTMLElement;
  createTextNode(text: string): Text;
  appendChild(parent: HTMLElement, child: Node): void;
  removeChild(parent: HTMLElement, child: Node): void;
  replaceChild(parent: HTMLElement, newChild: Node, oldChild: Node): void;
  setAttribute(element: HTMLElement, name: string, value: string): void;
  removeAttribute(element: HTMLElement, name: string): void;
  setStyle(
    element: HTMLElement,
    property: string,
    value: string | number,
  ): void;
  getComputedStyle(element: HTMLElement): ComputedStyle;
  measureElement(element: HTMLElement): ElementMeasurements;
  querySelector(selector: string, root?: HTMLElement): HTMLElement | null;
  querySelectorAll(selector: string, root?: HTMLElement): HTMLElement[];
  addEventListener(
    element: HTMLElement,
    event: string,
    handler: (event: Event) => void,
  ): void;
  removeEventListener(
    element: HTMLElement,
    event: string,
    handler: (event: Event) => void,
  ): void;
}

/** Element creation options */
export interface ElementCreationOptions {
  id?: string;
  className?: string;
  style?: Partial<CSSStyleDeclaration>;
  attributes?: Record<string, string>;
  dataset?: Record<string, string>;
  children?: (HTMLElement | Text)[];
}

/** StyleEngine interface */
export interface StyleEngine {
  computeStyle(
    element: RenderedElement,
    rules: StyleRuleConfig[],
  ): ComputedStyle;
  applyStyle(element: HTMLElement, style: ComputedStyle): void;
  resolveCustomProperties(
    style: ComputedStyle,
    context: StyleContext,
  ): ComputedStyle;
  matchSelector(element: RenderedElement, selector: string): boolean;
  getSpecificity(selector: string): number;
}

/** Style context for custom property resolution */
export interface StyleContext {
  variables: Record<string, string>;
  theme: ThemeConfig;
}

```



```

    parent?: StyleContext;
  }

  /** Theme configuration */
  export interface ThemeConfig {
    name: string;
    colors: Record<string, string>;
    fonts: Record<string, string>;
    spacing: Record<string, string>;
    breakpoints: Record<string, string>;
    customProperties: Record<string, string>;
  }

  /** BindingEngine interface */
  export interface BindingEngine {
    resolveBinding(
      binding: BindingConfig,
      data: ReportData,
      parameters: ReportParameters,
    ): Promise<ResolvedBinding>;
    resolveBindings(
      bindings: BindingConfig[],
      data: ReportData,
      parameters: ReportParameters,
    ): Promise<ResolvedBindingMap>;
    formatValue(
      value: unknown,
      formatter: string,
      args?: Record<string, unknown>,
    ): string;
    registerFormatter(name: string, formatter: ValueFormatter): void;
    unregisterFormatter(name: string): void;
  }

  /** Value formatter function */
  export type ValueFormatter = (
    value: unknown,
    args?: Record<string, unknown>,
  ) => string;

  /** Plugin event type (discriminated union for events) */
  export type PluginEventMap = {
    "element:render": { element: RenderedElement; context: RenderContext };
    "element:update": {
      element: RenderedElement;
      context: RenderContext;
      previousData: unknown;
    };
    "element:destroy": { element: RenderedElement };
    "data:refresh": { dataSourceId: DataSourceId; data: unknown[] };
    "page:break": { pageNumber: number; reason: PageBreakReason };
    "page:render": { pageNumber: number; elements: RenderedElement[] };
    "style:change": {
      element: RenderedElement;
      oldStyle: ComputedStyle;
      newStyle: ComputedStyle;
    };
    "binding:resolve": { binding: BindingConfig; result: ResolvedBinding };
  }

```

```

    custom: { eventName: string; payload: unknown };
};

/** Page break reasons */
export type PageBreakReason =
  | "content-overflow"
  | "explicit-break"
  | "page-break-before"
  | "page-break-after"
  | "keep-together-violation";

/** Plugin registry entry */
export interface PluginRegistryEntry {
  engine: PluginEngine;
  config: PluginConfig;
  registeredAt: Date;
  enabled: boolean;
}

/** Plugin registry */
export interface PluginRegistry {
  register(engine: PluginEngine, config: PluginConfig): void;
  unregister(pluginId: PluginId): boolean;
  get(pluginId: PluginId): PluginRegistryEntry | undefined;
  getAll(): PluginRegistryEntry[];
  getByElementType(elementType: ElementType): PluginRegistryEntry[];
  isEnabled(pluginId: PluginId): boolean;
  setEnabled(pluginId: PluginId, enabled: boolean): boolean;
}

// =====
// Engine Interfaces
// =====

/** Report engine interface */
export interface ReportEngine {
  readonly context: ReportContext;

  /** Initialize the report engine */
  initialize(definition: ReportDefinition): Promise<void>;

  /** Render the report */
  render(options?: RenderOptions): Promise<RenderOutput>;

  /** Update report data */
  updateData(dataSourceId: DataSourceId, data: unknown[]): Promise<void>;

  /** Update report parameters */
  updateParameters(parameters: ReportParameters): Promise<void>;

  /** Export report */
  export(format: OutputFormat, options?: ExportOptions): Promise<ExportResult>;

  /** Destroy the engine */
  destroy(): Promise<void>;

  /** Get plugin registry */
  getPluginRegistry(): PluginRegistry;

```

```

    /** Get DOM engine */
    getDOMEngine(): DOMEngine;

    /** Get Style engine */
    getStyleEngine(): StyleEngine;

    /** Get Binding engine */
    getBindingEngine(): BindingEngine;
}

/** Render options */
export interface RenderOptions {
    format?: OutputFormat;
    target?: "dom" | "pdf" | "string";
    container?: HTMLElement;
    pages?: number[]; // specific pages to render
    onProgress?: (progress: RenderProgress) => void;
}

/** Render progress */
export interface RenderProgress {
    phase: RenderPhase;
    currentPage: number;
    totalPages: number;
    processedElements: number;
    totalElements: number;
}

/** Render phases */
export type RenderPhase =
    | "preparing"
    | "layout"
    | "rendering"
    | "measuring"
    | "paginating"
    | "complete"
    | "error";

/** Render output */
export interface RenderOutput {
    success: boolean;
    output?: HTMLElement | string | Blob;
    pages: RenderedPage[];
    errors: RenderError[];
    warnings: RenderWarning[];
    metrics: RenderMetrics;
}

/** Rendered page */
export interface RenderedPage {
    pageNumber: number;
    element: HTMLElement;
    elements: RenderedElement[];
    bounds: PageBounds;
}

/** Page bounds */

```

```

export interface PageBounds {
  x: number;
  y: number;
  width: number;
  height: number;
}

/** Render error */
export interface RenderError {
  elementId?: ElementId;
  pluginId?: PluginId;
  message: string;
  error?: Error;
  severity: "error" | "warning" | "info";
}

/** Render warning */
export interface RenderWarning {
  elementId?: ElementId;
  pluginId?: PluginId;
  message: string;
  code?: string;
}

/** Render metrics */
export interface RenderMetrics {
  startTime: Date;
  endTime: Date;
  duration: number;
  pagesRendered: number;
  elementsRendered: number;
  bindingsResolved: number;
  stylesComputed: number;
  memoryUsed?: number;
}

/** Export options */
export interface ExportOptions {
  filename?: string;
  quality?: number; // for PDF/images
  includeFonts?: boolean;
  embedFonts?: boolean;
  metadata?: Record<string, string>;
}

/** Export result */
export interface ExportResult {
  success: boolean;
  blob?: Blob;
  url?: string;
  filename?: string;
  error?: Error;
  metadata?: ExportMetadata;
}

/** Export metadata */
export interface ExportMetadata {
  format: OutputFormat;
}

```

```

    pageCount: number;
    fileSize: number;
    generatedAt: Date;
    reportId: ReportId;
}

// =====
// Layout Engine Types
// =====

/** Layout engine interface */
export interface LayoutEngine {
    /** Calculate layout for a page */
    layoutPage(page: LayoutPage, context: LayoutContext): Promise<LayoutResult>;

    /** Calculate element position */
    calculatePosition(
        element: LayoutElement,
        context: LayoutContext,
    ): ElementPosition;

    /** Check if element fits in remaining space */
    fitsInSpace(element: LayoutElement, availableSpace: AvailableSpace): boolean;

    /** Split element across pages */
    splitElement(
        element: LayoutElement,
        availableSpace: AvailableSpace,
    ): SplitResult;
}

/** Layout page */
export interface LayoutPage {
    pageNumber: number;
    bounds: PageBounds;
    contentArea: ContentArea;
    elements: LayoutElement[];
}

/** Content area (page bounds minus margins) */
export interface ContentArea {
    x: number;
    y: number;
    width: number;
    height: number;
}

/** Layout element */
export interface LayoutElement {
    id: ElementId;
    config: ElementConfig;
    computedStyle: ComputedStyle;
    measuredSize: ElementMeasurements;
    children: LayoutElement[];
    dataContext: unknown;
}

/** Layout context */

```

```
export interface LayoutContext {
  pageConfig: PageConfig;
  currentPage: number;
  contentArea: ContentArea;
  currentPosition: Position;
  stack: LayoutStack[];
  variables: Record<string, unknown>;
}
```

```
/** Position */
```

```
export interface Position {
  x: number;
  y: number;
}
```

```
/** Layout stack entry */
```

```
export interface LayoutStack {
  element: LayoutElement;
  startPosition: Position;
  children: LayoutElement[];
}
```

```
/** Available space */
```

```
export interface AvailableSpace {
  width: number;
  height: number;
  x: number;
  y: number;
}
```

```
/** Element position result */
```

```
export interface ElementPosition {
  x: number;
  y: number;
  pageNumber: number;
  fits: boolean;
  overflow?: OverflowInfo;
}
```

```
/** Overflow information */
```

```
export interface OverflowInfo {
  overflowX: number;
  overflowY: number;
  canSplit: boolean;
}
```

```
/** Layout result */
```

```
export interface LayoutResult {
  success: boolean;
  pages: LayoutPage[];
  errors: LayoutError[];
  warnings: LayoutWarning[];
}
```

```
/** Layout error */
```

```
export interface LayoutError {
  elementId: ElementId;
  message: string;
}
```

```

    code: string;
  }

  /** Layout warning */
  export interface LayoutWarning {
    elementId: ElementId;
    message: string;
    code: string;
  }

  /** Split result */
  export interface SplitResult {
    firstPart: LayoutElement;
    secondPart: LayoutElement | null;
    splitPosition: number;
  }

  // =====
  // Event System Types
  // =====

  /** Report event types */
  export type ReportEventType =
    | "report:init"
    | "report:render-start"
    | "report:render-complete"
    | "report:render-error"
    | "report:data-changed"
    | "report:parameter-changed"
    | "report:export-start"
    | "report:export-complete"
    | "report:export-error"
    | "report:destroy"
    | "page:render-start"
    | "page:render-complete"
    | "page:break"
    | "element:render-start"
    | "element:render-complete"
    | "element:update"
    | "element:destroy"
    | "binding:resolve"
    | "binding:error"
    | "style:compute"
    | "style:apply"
    | "plugin:register"
    | "plugin:unregister"
    | "plugin:event";

  /** Report event map (discriminated union) */
  export type ReportEventMap = {
    "report:init": { report: ReportDefinition };
    "report:render-start": { options: RenderOptions };
    "report:render-complete": { output: RenderOutput };
    "report:render-error": { error: Error; options: RenderOptions };
    "report:data-changed": { dataSourceId: DataSourceId; data: unknown[] };
    "report:parameter-changed": { parameters: ReportParameters };
    "report:export-start": { format: OutputFormat; options: ExportOptions };
    "report:export-complete": { result: ExportResult };
  };

```

```

    "report:export-error": {
      error: Error;
      format: OutputFormat;
      options: ExportOptions;
    };
    "report:destroy": { reportId: ReportId };
    "page:render-start": { pageNumber: number };
    "page:render-complete": { page: RenderedPage };
    "page:break": { pageNumber: number; reason: PageBreakReason };
    "element:render-start": { element: RenderedElement };
    "element:render-complete": { element: RenderedElement; result: RenderResult };
    "element:update": { element: RenderedElement; result: UpdateResult };
    "element:destroy": { element: RenderedElement };
    "binding:resolve": { binding: BindingConfig; result: ResolvedBinding };
    "binding:error": { binding: BindingConfig; error: Error };
    "style:compute": { element: RenderedElement; style: ComputedStyle };
    "style:apply": { element: HTMLElement; style: ComputedStyle };
    "plugin:register": { plugin: PluginRegistryEntry };
    "plugin:unregister": { pluginId: PluginId };
    "plugin:event": { event: PluginEvent };
  };

```

```

/** Event listener type */
export type ReportEventListener<T extends ReportEventType> = (
  event: ReportEventMap[T],
) => void | Promise<void>;

```

```

/** Event emitter interface */
export interface ReportEventEmitter {
  on<T extends ReportEventType>(
    type: T,
    listener: ReportEventListener<T>,
  ): () => void;
  off<T extends ReportEventType>(
    type: T,
    listener: ReportEventListener<T>,
  ): void;
  emit<T extends ReportEventType>(
    type: T,
    event: ReportEventMap[T],
  ): Promise<void>;
  once<T extends ReportEventType>(
    type: T,
    listener: ReportEventListener<T>,
  ): () => void;
}

```

```

// =====
// Utility Types
// =====

```

```

/** Deep readonly */
export type DeepReadonly<T> = {
  readonly [P in keyof T]: DeepReadonly<T[P]>;
};

```

```

/** Make specific keys required */
export type RequireKeys<T, K extends keyof T> = T & { [P in K]-?: T[P] };

```



```

/** Make specific keys optional */
export type OptionalKeys<T, K extends keyof T> = Omit<T, K> &
  Partial<Pick<T, K>>;

/** Extract element type from config */
export type ElementTypeFromConfig<C extends ElementConfig> = C["type"];

/** Extract plugin ID from config */
export type PluginIdFromConfig<C extends PluginConfig> = C["id"];

/** Type guard for discriminated unions */
export type IsExact<T, U> =
  (<T>() => T extends U ? 1 : 2) extends <U>() => U extends T ? 1 : 2
    ? true
    : false;

// =====
// Type exports are already available via ES2015 module syntax above
// No namespace export needed - use direct imports instead
// =====

```

```

FILE: dom-engine.ts
LOKASI: src\feedback\report-workspace\engines

```

```

/**
 * DOM Engine - Single centralized source of truth for DOM query interactions.
 * No other engine or plugin is allowed to run querySelector loops directly.
 * Zero `any` usage. Strict typing throughout.
 */

import type {
  DOMEngine,
  ElementCreationOptions,
  ElementMeasurements,
  ComputedStyle,
} from "../report-types";

/**
 * Centralized DOM Engine implementation.
 * All DOM queries go through this single class.
 * No other engine or plugin should call querySelector directly.
 */
export class DOMEngineImpl implements DOMEngine {
  private container: HTMLElement | null = null;

  /**
   * Set the root container for all queries.
   * Must be called before any query methods.
   */
  public setContainer(el: HTMLElement | null): void {
    this.container = el;
  }

  /**
   * Get the current container element.
   */
  public getContainer(): HTMLElement | null {
    return this.container;
  }

  // =====
  // Element Creation & Manipulation
  // =====

  public createElement(
    tag: string,
    options?: ElementCreationOptions,
  ): HTMLElement {
    const element = document.createElement(tag);

    if (options?.id) {
      element.id = options.id;
    }
    if (options?.className) {
      element.className = options.className;
    }
  }

```

```

    if (options?.style) {
        Object.assign(element.style, options.style);
    }
    if (options?.attributes) {
        for (const [name, value] of Object.entries(options.attributes)) {
            element.setAttribute(name, value);
        }
    }
    if (options?.dataset) {
        for (const [key, value] of Object.entries(options.dataset)) {
            element.dataset[key] = value;
        }
    }
    if (options?.children) {
        for (const child of options.children) {
            element.appendChild(child);
        }
    }

    return element;
}

public createTextNode(text: string): Text {
    return document.createTextNode(text);
}

public appendChild(parent: HTMLElement, child: Node): void {
    parent.appendChild(child);
}

public removeChild(parent: HTMLElement, child: Node): void {
    parent.removeChild(child);
}

public replaceChild(
    parent: HTMLElement,
    newChild: Node,
    oldChild: Node,
): void {
    parent.replaceChild(newChild, oldChild);
}

// =====
// Attribute & Style Manipulation
// =====

public setAttribute(element: HTMLElement, name: string, value: string): void {
    element.setAttribute(name, value);
}

public removeAttribute(element: HTMLElement, name: string): void {
    element.removeAttribute(name);
}

public setStyle(
    element: HTMLElement,
    property: string,
    value: string | number,

```

```

): void {
    element.style.setProperty(property, String(value));
}

public getComputedStyle(element: HTMLElement): ComputedStyle {
    const computed = window.getComputedStyle(element);
    const style: ComputedStyle = {};
    for (let i = 0; i < computed.length; i++) {
        const prop = computed[i];
        style[prop] = computed.getPropertyValue(prop);
    }
    return style;
}

// =====
// Measurement
// =====

public measureElement(element: HTMLElement): ElementMeasurements {
    const rect = element.getBoundingClientRect();
    return {
        width: rect.width,
        height: rect.height,
        naturalWidth: (element as HTMLImageElement).naturalWidth ?? undefined,
        naturalHeight: (element as HTMLImageElement).naturalHeight ?? undefined,
    };
}

// =====
// Centralized DOM Query Methods - SINGLE SOURCE OF TRUTH
// =====

/**
 * Get all report pages.
 * Selector: [data-report-page]
 */
public getPages(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-report-page]");
}

/**
 * Get all report footers.
 * Selector: [data-report-footer]
 */
public findFooters(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-report-footer]");
}

/**
 * Get all group elements.
 * Selector: [data-group]
 */

```

```

public findGroups(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-group]");
}

/**
 * Get all link elements.
 * Selector: [data-link]
 */
public findLinks(): NodeListOf<HTMLElement> {
    if (!this.container) {
        return [] as unknown as NodeListOf<HTMLElement>;
    }
    return this.container.querySelectorAll<HTMLElement>("[data-link]");
}

/**
 * Measure page height for pagination calculations.
 */
public measurePageHeight(page: HTMLElement): {
    scrollHeight: number;
    clientHeight: number;
} {
    return {
        scrollHeight: page.scrollHeight,
        clientHeight: page.clientHeight,
    };
}

// =====
// Generic Query Methods (for plugins that need custom queries)
// =====

public querySelector(
    selector: string,
    root?: HTMLElement,
): HTMLElement | null {
    const searchRoot = root ?? this.container ?? document;
    return searchRoot.querySelector<HTMLElement>(selector);
}

public querySelectorAll(selector: string, root?: HTMLElement): HTMLElement[] {
    const searchRoot = root ?? this.container ?? document;
    return Array.from(searchRoot.querySelectorAll<HTMLElement>(selector));
}

// =====
// Event Handling
// =====

public addEventListener(
    element: HTMLElement,
    event: string,
    handler: EventListener,
): void {
    element.addEventListener(event, handler);
}

```

```

    }

    public removeEventListener(
        element: HTMLElement,
        event: string,
        handler: EventListener,
    ): void {
        element.removeEventListener(event, handler);
    }
}

/**
 * Factory function to create a DOM engine instance.
 */
export function createDOMEngine(): DOMEngineImpl {
    return new DOMEngineImpl();
}

// =====
// Type Re-exports for convenience
// =====

export type {
    DOMEngine,
    ElementCreationOptions,
    ElementMeasurements,
    ComputedStyle,
} from "../report-types";

```

FILE: avatar.tsx
LOKASI: src\general

```
import React, { useState } from "react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export type AvatarSize = "sm" | "md" | "lg" | "xl" | "2xl";
export type AvatarShape = "circle" | "square";

export interface AvatarProps extends React.HTMLAttributes<HTMLDivElement> {
  src?: string;
  alt?: string;
  name?: string;
  size?: AvatarSize;
  shape?: AvatarShape;
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

export function Avatar({
  src,
  alt,
  name = "System Operator",
  size = "md",
  shape = "circle",
  className,
  ...props
}: AvatarProps) {
  const [imageError, setImageError] = useState(false);
  const [isLoading, setIsLoaded] = useState(false);

  const initials = React.useMemo(() => {
    if (!name) return "";
    const parts = name.trim().split(/\s+/).filter(Boolean);
    if (parts.length === 1) {
      return parts[0].slice(0, 2).toUpperCase();
    }
    return (parts[0][0] + parts[parts.length - 1][0]).toUpperCase();
  }, [name]);

  const sizeClasses: Record<AvatarSize, string> = {
    sm: "w-8 h-8 text-xs",
    md: "w-10 h-10 text-sm",
    lg: "w-12 h-12 text-base",
    xl: "w-16 h-16 text-lg",
    "2xl": "w-20 h-20 text-xl",
  };

  const shapeClasses: Record<AvatarShape, string> = {
    circle: "rounded-full",
    square: "rounded-[var(--radius-surface)]",
  };
}
```

```

const containerClasses = cn(
  "relative inline-flex items-center justify-center overflow-hidden",
  "bg-muted/40",
  "border",
  "border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
  "hover:scale-105 transition-transform duration-200 ease-out",
  sizeClasses[size],
  shapeClasses[shape],
  className,
);

const imageClasses = cn(
  "absolute inset-0 w-full h-full object-cover",
  "transition-opacity duration-300 ease-out",
  isLoading && !imageError ? "opacity-100" : "opacity-0",
);

const fallbackClasses = cn(
  "relative z-10 flex items-center justify-center w-full h-full",
  "bg-muted/40",
  "text-muted-foreground",
  "font-medium",
  "transition-opacity duration-300 ease-out",
  !src || imageError ? "opacity-100" : "opacity-0",
);

const handleImageError = () => {
  setImageError(true);
};

const handleImageLoad = () => {
  setIsLoaded(true);
};

const { hint, popupMenu, ...avatarProps } = props;

return (
  <div className="inline-flex" onContextMenu={(e) => popupMenu?.trigger(e)}>
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    <div {...avatarProps} className={containerClasses}>
      {src && !imageError && (
        <img
          src={src}
          alt={alt || name}
          className={imageClasses}
          onError={handleImageError}
          onLoad={handleImageLoad}
        />
      )}
      <div className={fallbackClasses} aria-hidden={!src && !imageError}>
        {initials}
      </div>
    </div>
  </div>

```



```
    </div>  
  );  
}
```

```
FILE: button.tsx
LOKASI: src\general
```

```
import React, {
  forwardRef,
  useState,
  useEffect,
  useCallback,
  useMemo,
} from "react";
import { Loader2, LucideIcon } from "lucide-react";
import { cn } from "@lib/utills";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

// ðŸ›¸;ï¿½ LOCAL TYPE ISOLATION GATEWAY
export interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?:
    | "brand"
    | "secondary"
    | "outline"
    | "ghost"
    | "destructive"
    | "link"
    | "premium";
  size?: "xs" | "sm" | "md" | "lg" | "xl" | "icon";
  isLoading?: boolean;
  loadingText?: string;
  leftIcon?: LucideIcon;
  rightIcon?: LucideIcon;
  iconOnly?: boolean;
  fullWidth?: boolean;
  isDisabled?: boolean;
  asChild?: boolean;
  loadingPosition?: "left" | "right" | "center";
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

const variantStyles = {
  brand:
    "bg-brand-primary text-white hover:bg-brand-hover active:bg-brand-primary active:scale-[0.98] shadow-sm hover:shadow-md transition-[var(--transition-fast)]",
  secondary:
    "bg-secondary text-secondary-foreground hover:bg-secondary/80 active:bg-secondary active:scale-[0.98] border border-border transition-all duration-200 ease-out",
  outline:
    "border border-input bg-transparent hover:bg-accent hover:text-accent-foreground active:bg-accent active:scale-[0.98] transition-all duration-200 ease-out",
  ghost:
    "bg-transparent hover:bg-accent hover:text-accent-foreground active:bg-accent active:scale-[0.98] transition-all duration-200 ease-out",
  destructive:
    "bg-destructive text-destructive-foreground hover:bg-destructive/90 active:bg-destructive active:scale-[0.98] shadow-sm hover:shadow-md transition-all duration-200 ease-out",
}
```

```

link: "text-brand-primary underline-offset-4 hover:underline text-sm font-medium transition-colors
duration-200 ease-out",
premium:
  "bg-gradient-to-r from-brand-primary via-brand-primary/90 to-brand-primary/80 text-white
hover:from-brand-hover hover:via-brand-primary/80 hover:to-brand-primary/70 active:from-brand-primary
active:via-brand-primary/90 active:to-brand-primary active:scale-[0.98] shadow-lg hover:shadow-xl
active:shadow-lg transition-[var(--transition-fast)] relative overflow-hidden",
};

const sizeStyles = {
  xs: "h-7 px-2.5 text-xs gap-1",
  sm: "h-8 px-3 text-sm gap-1.5",
  md: "h-10 px-4 py-2 text-base gap-2",
  lg: "h-12 px-6 text-lg gap-2.5",
  xl: "h-14 px-8 text-xl gap-3",
  icon: "h-10 w-10 p-0",
};

const iconSizeStyles = {
  xs: "w-3 h-3",
  sm: "w-3.5 h-3.5",
  md: "w-4 h-4",
  lg: "w-5 h-5",
  xl: "w-6 h-6",
  icon: "w-5 h-5",
};

const loadingSizeStyles = {
  xs: "w-3 h-3",
  sm: "w-4 h-4",
  md: "w-5 h-5",
  lg: "w-6 h-6",
  xl: "w-7 h-7",
  icon: "w-5 h-5",
};

const focusStyles =
  "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 focus-
visible:ring-offset-background";

const disabledStyles =
  "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed";

const baseStyles =
  "inline-flex items-center justify-center font-sans font-medium leading-none rounded-surface
transition-all duration-200 ease-out select-none";

export const buttonVariants = variantStyles;
export const buttonSizes = sizeStyles;

export const Button = forwardRef<HTMLButtonElement, ButtonProps>(
  (
    {
      variant = "brand",
      size = "md",
      isLoading = false,
      loadingText,
      leftIcon: LeftIcon,

```

```

    rightIcon: RightIcon,
    iconOnly = false,
    fullWidth = false,
    isDisabled = false,
    loadingPosition = "left",
    className,
    children,
    disabled,
    ...props
  },
  ref,
) => {
  const isActuallyDisabled = disabled || isDisabled || isLoading;
  const hasLeftIcon = Boolean(LeftIcon) && !isLoading;
  const hasRightIcon = Boolean(RightIcon) && !isLoading;
  const showLoadingSpinner =
    isLoading && (loadingPosition !== "center" || !loadingText);
  const showLoadingText = isLoading && loadingText;

  const iconSize = useMemo(() => iconSizeStyles[size], [size]);
  const loadingSize = useMemo(() => loadingSizeStyles[size], [size]);

  const baseClasses = useMemo(
    () =>
      cn(
        baseStyles,
        variantStyles[variant],
        sizeStyles[size],
        focusStyles,
        disabledStyles,
        fullWidth && "w-full",
        iconOnly && "p-0",
        className,
      ),
    [variant, size, fullWidth, iconOnly, className],
  );

  const [ripple, setRipple] = useState<{
    x: number;
    y: number;
    size: number;
  } | null>(null);

  const handleMouseDown = useCallback(
    (e: React.MouseEvent<HTMLButtonElement>) => {
      if (isActuallyDisabled) return;
      const rect = e.currentTarget.getBoundingClientRect();
      const size = Math.max(rect.width, rect.height);
      setRipple({
        x: e.clientX - rect.left - size / 2,
        y: e.clientY - rect.top - size / 2,
        size,
      });
    },
    [isActuallyDisabled],
  );

  const handleMouseUp = useCallback(() => {

```

```

    setTimeout(() => setRipple(null), 300);
  }, []);

  const handleMouseLeave = useCallback(() => {
    setRipple(null);
  }, []);

  useEffect(() => {
    if (isLoading) {
      setRipple(null);
    }
  }, [isLoading]);

  const rippleStyles = useMemo(
    () =>
      ripple
        ? {
            width: ripple.size,
            height: ripple.size,
            left: ripple.x,
            top: ripple.y,
          }
        : {},
    [ripple],
  );

  const { hint, popupMenu, ...buttonProps } = props;

  return (
    <div className="inline-flex" onContextMenu={(e) => popupMenu?.trigger(e)}>
      {hint && (
        <HintBox content={hint} className="mb-1.5">
          <span className="text-sm text-text-muted" />
        </HintBox>
      )}
      <button
        ref={ref}
        className={baseClasses}
        disabled={isActuallyDisabled}
        onMouseDown={handleMouseDown}
        onMouseUp={handleMouseUp}
        onMouseLeave={handleMouseLeave}
        aria-busy={isLoading}
        aria-disabled={isActuallyDisabled}
        {...buttonProps}
      >
        {showLoadingSpinner && loadingPosition === "left" && (
          <Loader2
            className={cn(loadingSize, "animate-spin", "shrink-0")}
            aria-hidden="true"
          />
        )}
        {hasLeftIcon && LeftIcon && (
          <LeftIcon className={cn(iconSize, "shrink-0")} aria-hidden="true" />
        )}
        {!isLoading && children}
        {showLoadingText && <span>{loadingText}</span>}
        {hasRightIcon && RightIcon && (

```

```

        <RightIcon
            className={cn(iconSize, "shrink-0")}
            aria-hidden="true"
        />
    )}
    {showLoadingSpinner && loadingPosition === "right" && (
        <Loader2
            className={cn(loadingSize, "animate-spin", "shrink-0")}
            aria-hidden="true"
        />
    )}
    {ripple && (
        <span
            className="absolute rounded-full bg-white/30 pointer-events-none animate-
[ripple_600ms_ease-out]"
            style={rippleStyles}
            aria-hidden="true"
        />
    )}
    </button>
</div>
);
},
);

Button.displayName = "Button";

```

FILE: label.tsx
LOKASI: src\general

```
import React, { useState, useRef, useEffect, useCallback } from "react";
import { HelpCircle, LucideIcon } from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export interface LabelProps extends React.LabelHTMLAttributes<HTMLLabelElement> {
  isRequired?: boolean;
  error?: string;
  disabled?: boolean;
  hintContent?: React.ReactNode;
  hintIcon?: LucideIcon;
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

export const Label = React.forwardRef<HTMLLabelElement, LabelProps>(
  (
    {
      isRequired = false,
      error,
      disabled = false,
      hintContent,
      hintIcon: HintIcon = HelpCircle,
      className,
      children,
      ...props
    },
    ref,
  ) => {
    const hasError = Boolean(error);
    const isInteractive = !disabled && !hasError;
    const [isHintOpen, setIsHintOpen] = useState(false);
    const [currentSide, setCurrentSide] = useState<"top" | "bottom">("top");
    const triggerRef = useRef<HTMLButtonElement>(null);
    const contentRef = useRef<HTMLDivElement>(null);
    const enterTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const leaveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const positionStylesRef = useRef<React.CSSProperties>({});

    const DEFAULT_ENTER_DELAY = 200;
    const DEFAULT_LEAVE_DELAY = 150;

    const updatePosition = useCallback(() => {
      if (triggerRef.current && contentRef.current) {
        const trigger = triggerRef.current.getBoundingClientRect();
        const content = contentRef.current.getBoundingClientRect();
        const viewportWidth = window.innerWidth;
        const viewportHeight = window.innerHeight;
        const gap = 8;
        const viewportPadding = 8;

```

```

    let top = trigger.bottom + gap;
    let left = trigger.left + (trigger.width - content.width) / 2;
    let side: "top" | "bottom" = "bottom";

    if (top + content.height > viewportHeight - viewportPadding) {
      top = trigger.top - content.height - gap;
      side = "top";
    }
    if (left < viewportPadding) {
      left = viewportPadding;
    }
    if (left + content.width > viewportWidth - viewportPadding) {
      left = viewportWidth - content.width - viewportPadding;
    }

    setCurrentSide(side);

    positionStylesRef.current = {
      top: `${top}px`,
      left: `${left}px`,
      transform: "translateX(-50%)",
    };
  }
}, []);

useEffect(() => {
  if (isHintOpen) {
    updatePosition();
    window.addEventListener("resize", updatePosition);
    window.addEventListener("scroll", updatePosition, true);
  }
  return () => {
    window.removeEventListener("resize", updatePosition);
    window.removeEventListener("scroll", updatePosition, true);
  };
}, [isHintOpen, updatePosition]);

const clearTimeouts = useCallback(() => {
  if (enterTimeoutRef.current) {
    clearTimeout(enterTimeoutRef.current);
  }
  if (leaveTimeoutRef.current) {
    clearTimeout(leaveTimeoutRef.current);
  }
}, []);

const handleTriggerEnter = useCallback(() => {
  if (!isInteractive) return;
  clearTimeouts();
  enterTimeoutRef.current = setTimeout(() => {
    setIsHintOpen(true);
  }, DEFAULT_ENTER_DELAY);
}, [clearTimeouts, isInteractive]);

const handleTriggerLeave = useCallback(() => {
  clearTimeouts();
  leaveTimeoutRef.current = setTimeout(() => {

```



```

    setIsHintOpen(false);
  }, DEFAULT_LEAVE_DELAY);
}, [clearTimeouts]);

const handleContentEnter = useCallback(() => {
  clearTimeouts();
  setIsHintOpen(true);
}, [clearTimeouts]);

const handleContentLeave = useCallback(() => {
  clearTimeouts();
  leaveTimeoutRef.current = setTimeout(() => {
    setIsHintOpen(false);
  }, DEFAULT_LEAVE_DELAY);
}, [clearTimeouts]);

const handleKeyDown = useCallback((e: React.KeyboardEvent) => {
  if (e.key === "Escape") {
    setIsHintOpen(false);
  }
}, []);

const { hint, popupMenu, ...labelProps } = props;

return (
  <div className="inline-flex" onContextMenu={(e) => popupMenu?.trigger(e)}>
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    <label
      ref={ref}
      className={cn(
        "inline-flex items-center gap-1.5",
        "font-sans text-sm font-medium leading-none",
        "transition-colors duration-200 ease-out",
        "text-text-main",
        hasError && "text-destructive/90",
        disabled && "opacity-50 pointer-events-none cursor-not-allowed",
        className,
      )}
      {...labelProps}
    >
      {children}
      {isRequired && (
        <span
          className={cn(
            "inline-flex items-center justify-center",
            "text-destructive",
            "transition-opacity duration-200 ease-out",
            "aria-hidden",
          )}
          aria-hidden="true"
        >
          *
        </span>
      )}
    </div>
  )}

```

```

{hintContent && (
  <span className="relative inline-flex items-center justify-center">
    <button
      ref={triggerRef}
      type="button"
      className={cn(
        "relative flex items-center justify-center",
        "w-5 h-5 rounded-md",
        "text-muted-foreground/60 hover:text-foreground",
        "bg-transparent hover:bg-accent",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2",
        "disabled:opacity-50 disabled:pointer-events-none",
        !isInteractive &&
        "opacity-50 pointer-events-none cursor-not-allowed",
      )}
      aria-label="Help"
      aria-describedby={isHintOpen ? "hint-tooltip" : undefined}
      aria-expanded={isHintOpen}
      disabled={!isInteractive}
      onMouseEnter={handleTriggerEnter}
      onMouseLeave={handleTriggerLeave}
      onFocus={handleTriggerEnter}
      onBlur={handleTriggerLeave}
      onKeyDown={handleKeyDown}
    >
      <HintIcon className="w-3.5 h-3.5" aria-hidden="true" />
    </button>
    {isHintOpen && (
      <div
        ref={contentRef}
        id="hint-tooltip"
        role="tooltip"
        className={cn(
          "fixed z-50 pointer-events-auto",
          "bg-card backdrop-blur-[var(--backdrop-blur)]",
          "border border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
          "rounded-surface",
          "shadow-lg",
          "p-3",
          "max-w-xs",
          "text-sm text-card-foreground",
          "font-sans",
          "transition-all duration-200 ease-out",
          "opacity-0 scale-95 data-[state=open]:opacity-100 data-[state=open]:scale-100",
          "data-[side=bottom]:translate-y-2 data-[side=top]:-translate-y-2",
          "data-[side=left]:-translate-x-2 data-[side=right]:translate-x-2",
        )}
        data-state="open"
        data-side={currentSide}
        style={positionStylesRef.current}
        onMouseEnter={handleContentEnter}
        onMouseLeave={handleContentLeave}
      >
        {hintContent}
      </div>
    )}
  )}

```

```

        style={
          {
            width: 0,
            height: 0,
            borderLeft: "6px solid transparent",
            borderRight: "6px solid transparent",
            borderTop: currentSide === "top" ? "6px solid" : "none",
            borderBottom:
              currentSide === "bottom" ? "6px solid" : "none",
            bottom: currentSide === "top" ? "-6px" : "auto",
            top: currentSide === "bottom" ? "-6px" : "auto",
            left: "50%",
            transform: "translateX(-50%)",
            position: "absolute",
            pointerEvents: "none",
          } as React.CSSProperties
        }
        aria-hidden="true"
      />
    </div>
  )}
</span>
)}
</label>
</div>
);
},
);

Label.displayName = "Label";

```

FILE: progress.tsx
LOKASI: src\general

```
import React, { forwardRef } from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { HintBox } from "../feedback/hint-box";

// ðŸš€ LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// ðŸš¡ LOCAL TYPE ISOLATION GATEWAY
export type ProgressVariant = "brand" | "success" | "warning" | "danger";

export interface ProgressProps extends React.HTMLAttributes<HTMLDivElement> {
  value?: number;
  variant?: ProgressVariant;
  showLabel?: boolean;
  hint?: string;
}

export const Progress = forwardRef<HTMLDivElement, ProgressProps>(
  (
    {
      value = 0,
      variant = "brand",
      showLabel = false,
      className,
      hint,
      ...props
    },
    ref,
  ) => {
    const clampedValue = Math.max(0, Math.min(100, value));

    const variantStyles = {
      brand: "bg-brand-primary",
      success: "bg-emerald-500",
      warning: "bg-amber-500",
      danger: "bg-red-500",
    } as const satisfies Record<ProgressVariant, string>;

    const containerStyles = {
      brand: "bg-brand-primary/10",
      success: "bg-emerald-500/10",
      warning: "bg-amber-500/10",
      danger: "bg-red-500/10",
    } as const satisfies Record<ProgressVariant, string>;

    const labelStyles = {
      brand: "text-brand-primary",
      success: "text-emerald-600 dark:text-emerald-400",
      warning: "text-amber-600 dark:text-amber-400",
    } as const satisfies Record<ProgressVariant, string>;
```

```

    danger: "text-red-600 dark:text-red-400",
  } as const satisfies Record<ProgressVariant, string>;

  return (
    <>
      {hint && (
        <HintBox content={hint} className="mb-1.5">
          <span aria-hidden="true" style={{ display: "none" }} />
        </HintBox>
      )}
      <div
        ref={ref}
        role="progressbar"
        aria-valuenow={clampedValue}
        aria-valuemin={0}
        aria-valuemax={100}
        className={cn(
          "relative w-full h-2 rounded-full overflow-hidden",
          containerStyles[variant],
          className,
        )}
        {...props}
      >
        <div
          className={cn(
            "h-full rounded-full transition-all duration-500 ease-out",
            variantStyles[variant],
          )}
          style={{ width: `${clampedValue}%` }}
          aria-hidden="true"
        />
        {showLabel && (
          <span
            className={cn(
              "absolute right-0 top-1/2 -translate-y-1/2 pr-1.5 text-xs font-medium tabular-nums",
              labelStyles[variant],
            )}
            aria-hidden="true"
          >
            {clampedValue}%
          </span>
        )}
      </div>
    </>
  );
},
);
Progress.displayName = "Progress";

```

FILE: tabs.tsx
LOKASI: src\general

```
import React, {
  createContext,
  useContext,
  useState,
  useEffect,
  useRef,
  useCallback,
  useMemo,
} from "react";
import { X, Plus } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

// Æ°Å,Å;â,- LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// Æ°Å,Å;Å! LOCAL TYPE ISOLATION GATEWAY
export interface TabsRootProps extends React.HTMLAttributes<HTMLDivElement> {
  defaultValue: string;
  value?: string;
  onValueChange?: (value: string) => void;
  children: React.ReactNode;
  orientation?: "horizontal" | "vertical";
  activationMode?: "automatic" | "manual";
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

export interface TabsListProps extends React.HTMLAttributes<HTMLDivElement> {
  children: React.ReactNode;
  showAddButton?: boolean;
  onAddTab?: () => void;
}

export interface TabsTriggerProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  value: string;
  children: React.ReactNode;
  disabled?: boolean;
  isCloseable?: boolean;
  onClose?: (value: string) => void;
}

export interface TabsContentProps extends React.HTMLAttributes<HTMLDivElement> {
  value: string;
  children: React.ReactNode;
  forceMount?: boolean;
}
```

```

interface TabsContextValue {
  value: string;
  onValueChange: (value: string) => void;
  orientation: "horizontal" | "vertical";
  activationMode: "automatic" | "manual";
  registerTrigger: (value: string, ref: HTMLButtonElement | null) => void;
  unregisterTrigger: (value: string) => void;
  triggerRefs: Map<string, HTMLButtonElement | null>;
  activeTriggerRef: React.MutableRefObject<HTMLButtonElement | null>;
  setActiveTriggerRef: (ref: HTMLButtonElement | null) => void;
}

const TabsContext = createContext<TabsContextValue | null>(null);

function useTabsContext() {
  const context = useContext(TabsContext);
  if (!context) {
    throw new Error("Tabs compound components must be used within TabsRoot");
  }
  return context;
}

// Tabs ROOT - State Management & Context Provider
export const TabsRoot = React.forwardRef<HTMLDivElement, TabsRootProps>(
  (
    {
      defaultValue,
      value: controlledValue,
      onValueChange,
      children,
      orientation = "horizontal",
      activationMode = "automatic",
      className,
      hint,
      popupMenu,
      ...props
    },
    ref,
  ) => {
    const [uncontrolledValue, setUncontrolledValue] = useState(defaultValue);
    const isControlled = controlledValue !== undefined;
    const currentValue = isControlled ? controlledValue : uncontrolledValue;

    const triggerRefs = useRef<Map<string, HTMLButtonElement | null>>(
      new Map(),
    );
    const activeTriggerRef = useRef<HTMLButtonElement | null>(null);
    const [indicatorStyle, setIndicatorStyle] = useState<React.CSSProperties>(
      {},
    );

    const registerTrigger = useCallback(
      (value: string, ref: HTMLButtonElement | null) => {
        triggerRefs.current.set(value, ref);
      },
      [],
    );
  }
);

```

```

const unregisterTrigger = useCallback((value: string) => {
  triggerRefs.current.delete(value);
}, []);

const setActiveTriggerRef = useCallback((ref: HTMLButtonElement | null) => {
  activeTriggerRef.current = ref;
}, []);

const handleValueChange = useCallback(
  (value: string) => {
    if (!isControlled) {
      setUncontrolledValue(value);
    }
    onValueChange?.(value);
  },
  [isControlled, onValueChange],
);

const updateIndicatorPosition = useCallback(() => {
  const activeTrigger = activeTriggerRef.current;
  const tabsList = activeTrigger?.closest(
    "[data-tabs-list]",
  ) as HTMLElement | null;

  if (activeTrigger && tabsList) {
    const triggerRect = activeTrigger.getBoundingClientRect();
    const listRect = tabsList.getBoundingClientRect();

    if (orientation === "horizontal") {
      setIndicatorStyle({
        width: `${triggerRect.width}px`,
        transform: `translateX(${triggerRect.left - listRect.left}px)`,
        height: "2px",
      });
    } else {
      setIndicatorStyle({
        height: `${triggerRect.height}px`,
        transform: `translateY(${triggerRect.top - listRect.top}px)`,
        width: "2px",
      });
    }
  }
}, [orientation]);

useEffect(() => {
  updateIndicatorPosition();
  window.addEventListener("resize", updateIndicatorPosition);
  return () => {
    window.removeEventListener("resize", updateIndicatorPosition);
  };
}, [currentValue, updateIndicatorPosition]);

const contextValue = useMemo<TabsContextValue>(
  () => ({
    value: currentValue,
    onValueChange: handleValueChange,
    orientation,
    activationMode,
    registerTrigger,
  })

```



```

    unregisterTrigger,
    triggerRefs: triggerRefs.current,
    activeTriggerRef,
    setActiveTriggerRef,
  })),
  [
    currentValue,
    handleValueChange,
    orientation,
    activationMode,
    registerTrigger,
    unregisterTrigger,
    setActiveTriggerRef,
  ],
);

return (
  <TabsContext.Provider value={contextValue}>
    <div
      ref={ref}
      className={cn("relative", className)}
      data-tabs-root
      {...props}
      onContextMenu={(e) => popupMenu?.trigger(e)}
    >
      {hint && (
        <HintBox content={hint} className="mb-1.5">
          <span className="sr-only">hint</span>
        </HintBox>
      )}
      {children}
    <div
      className={cn(
        "absolute bg-brand-primary transition-all duration-300 ease-out pointer-events-none",
        orientation === "horizontal" ? "bottom-0" : "left-0",
      )}
      style={indicatorStyle}
      data-tabs-indicator
      aria-hidden="true"
    />
  </div>
</TabsContext.Provider>
);
},
);

```

```
TabsRoot.displayName = "TabsRoot";
```

```
// Ã°ÃÃ TABS LIST - Container for Triggers
```

```

export const TabsList = React.forwardRef<HTMLDivElement, TabsListProps>(
  ({ children, className, showAddButton = false, onAddTab, ...props }, ref) => {
    const { orientation, activationMode, value, onValueChange, triggerRefs } =
      useTabsContext();

    const triggerRefsArray = useMemo(
      () =>
        Array.from(triggerRefs.entries()).map(([value, ref]) => ({
          value,

```

```

    ref,
  })),
  [triggerRefs],
);

const handleKeyDown = useCallback(
  (event: React.KeyboardEvent) => {
    const triggers = triggerRefsArray.filter(
      ({ ref }) => ref && !ref.disabled,
    );
    if (triggers.length === 0) return;

    const currentIndex = triggers.findIndex(({ value: v }) => v === value);
    let nextIndex = currentIndex;

    const isHorizontal = orientation === "horizontal";

    switch (event.key) {
      case "ArrowRight":
        if (isHorizontal) {
          event.preventDefault();
          nextIndex = (currentIndex + 1) % triggers.length;
        }
        break;
      case "ArrowLeft":
        if (isHorizontal) {
          event.preventDefault();
          nextIndex =
            (currentIndex - 1 + triggers.length) % triggers.length;
        }
        break;
      case "ArrowDown":
        if (!isHorizontal) {
          event.preventDefault();
          nextIndex = (currentIndex + 1) % triggers.length;
        }
        break;
      case "ArrowUp":
        if (!isHorizontal) {
          event.preventDefault();
          nextIndex =
            (currentIndex - 1 + triggers.length) % triggers.length;
        }
        break;
      case "Home":
        event.preventDefault();
        nextIndex = 0;
        break;
      case "End":
        event.preventDefault();
        nextIndex = triggers.length - 1;
        break;
      default:
        return;
    }

    const nextTrigger = triggers[nextIndex];
    if (nextTrigger && nextTrigger.ref) {

```

```

        nextTrigger.ref.focus();
        if (activationMode === "automatic") {
            onValueChange(nextTrigger.value);
        }
    },
    [orientation, activationMode, value, onValueChange, triggerRefsArray],
);

return (
    <div
        ref={ref}
        role="tablist"
        aria-orientation={orientation}
        className={cn(
            "inline-flex items-center gap-1",
            "bg-muted p-1 rounded-surface",
            orientation === "horizontal" ? "flex-row" : "flex-col",
            className,
        )}
        data-tabs-list
        onKeyDown={handleKeyDown}
        {...props}
    >
        {children}
        {showAddButton && (
            <button
                type="button"
                onClick={onAddTab}
                className={cn(
                    "flex items-center justify-center",
                    "p-1.5 rounded-[calc(var(--radius)-2px)]",
                    "text-text-muted hover:text-text-main",
                    "hover:bg-accent/50",
                    "transition-all duration-150 ease-out",
                    "active:scale-95",
                    "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2",
                    orientation === "horizontal" ? "ml-auto" : "mt-auto",
                )}
                aria-label="Add new tab"
            >
                <Plus className="w-4 h-4" aria-hidden="true" />
            </button>
        )}
    </div>
);
},
);

TabsList.displayName = "TabsList";

// Ã°ÃÃÃ TABS TRIGGER - Individual Tab Button
export const TabsTrigger = React.forwardRef<
    HTMLButtonElement,
    TabsTriggerProps
>(
    (

```

```

    {
      value,
      children,
      disabled = false,
      isCloseable = false,
      onClose,
      className,
      ...props
    },
    ref,
  ) => {
    const {
      value: currentValue,
      onValueChange,
      orientation,
      activationMode,
      registerTrigger,
      unregisterTrigger,
      activeTriggerRef,
      setActiveTriggerRef,
    } = useTabsContext();

    const isActive = currentValue === value;
    const triggerRef = useRef<HTMLButtonElement>(null);

    const composedRef = useCallback(
      (node: HTMLButtonElement | null) => {
        triggerRef.current = node;
        registerTrigger(value, node);
        if (isActive) {
          setActiveTriggerRef(node);
          activeTriggerRef.current = node;
        }
        if (ref) {
          if (typeof ref === "function") {
            ref(node);
          } else {
            ref.current = node;
          }
        }
      },
      [value, registerTrigger, isActive, setActiveTriggerRef, ref],
    );

    useEffect(() => {
      return () => unregisterTrigger(value);
    }, [value, unregisterTrigger]);

    useEffect(() => {
      if (isActive && triggerRef.current) {
        setActiveTriggerRef(triggerRef.current);
        activeTriggerRef.current = triggerRef.current;
      }
    }, [isActive, setActiveTriggerRef]);

    const handleClick = useCallback(() => {
      if (!disabled) {
        onValueChange(value);
      }
    }, [disabled, onValueChange]);
  }
}

```

```

    }
  }, [disabled, onValueChange, value]);

  const handleFocus = useCallback(() => {
    if (activationMode === "automatic" && !disabled) {
      onValueChange(value);
    }
  }, [activationMode, disabled, onValueChange, value]);

  const handleClose = useCallback(
    (e: React.MouseEvent) => {
      e.stopPropagation();
      onClose?.(value);
    },
    [onClose, value],
  );

  return (
    <button
      ref={composedRef}
      role="tab"
      aria-selected={isActive}
      aria-controls={`tabs-content-${value}`}
      id={`tabs-trigger-${value}`}
      tabIndex={isActive ? 0 : -1}
      disabled={disabled}
      type="button"
      className={cn(
        "relative z-10 flex items-center justify-center gap-1.5",
        "font-medium text-sm leading-none",
        "rounded-[calc(var(--radius)-2px)]",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
        "focus-visible:ring-offset-background",
        "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
        "whitespace-nowrap",
        orientation === "horizontal"
          ? "px-4 py-2"
          : "px-3 py-2.5 w-full text-left",
        isActive
          ? "text-brand-primary font-semibold"
          : "text-text-main hover:bg-accent/50 hover:text-text-main",
        isCloseable && "pr-6",
        className,
      )}
      onClick={handleClick}
      onFocus={handleFocus}
      {...props}
    >
      {children}
      {isCloseable && (
        <button
          type="button"
          onClick={handleClose}
          className={cn(
            "absolute right-2 top-1/2 -translate-y-1/2",
            "flex items-center justify-center",

```

```

        "p-0.5 rounded-[calc(var(--radius)-2px)]",
        "text-text-muted hover:text-text-main",
        "hover:bg-accent/50",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2",
        "disabled:pointer-events-none disabled:opacity-50",
      )}
      aria-label={`Close tab ${value}`}
      tabIndex={-1}
    >
      <X className="w-3.5 h-3.5" aria-hidden="true" />
    </button>
  )}
</button>
);
},
);

TabsTrigger.displayName = "TabsTrigger";

// Ã°ÃÃ TABS CONTENT - Tab Panel
export const TabsContent = React.forwardRef<HTMLDivElement, TabsContentProps>(
  ({ value, children, forceMount = false, className, ...props }, ref) => {
    const { value: currentValue } = useTabsContext();

    const isActive = currentValue === value;

    if (!forceMount && !isActive) {
      return null;
    }

    return (
      <div
        ref={ref}
        role="tabpanel"
        id={`tabs-content-${value}`}
        aria-labelledby={`tabs-trigger-${value}`}
        hidden={!isActive}
        tabIndex={0}
        className={cn(
          "mt-4 focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2",
          "transition-all duration-200 ease-out",
          "opacity-0 data-[state=active]:opacity-100",
          "translate-y-1 data-[state=active]:translate-y-0",
          className,
        )}
        data-state={isActive ? "active" : "inactive"}
        data-tabs-content
        {...props}
      >
        {children}
      </div>
    );
  },
);

```

```
TabsContent.displayName = "TabsContent";

// Æ°Å, Å½Å- COMPOUND COMPONENT EXPORTS
export const Tabs = Object.assign(TabsRoot, {
  List: TabsList,
  Trigger: TabsTrigger,
  Content: TabsContent,
  Root: TabsRoot,
});

export type TabsOrientation = "horizontal" | "vertical";
export type TabsActivationMode = "automatic" | "manual";
```

```
FILE: thumbnail.tsx
LOKASI: src\general
```

```
import React, { forwardRef, useState } from "react";
import { FileText, ImageIcon, AlertCircle } from "lucide-react";
import { cn } from "@lib/utils";

// ðŸŸ¡ LOCAL TYPE ISOLATION GATEWAY
export interface ThumbnailProps extends React.HTMLAttributes<HTMLDivElement> {
  src?: string;
  alt?: string;
  fileType?: string;
  size?: "sm" | "md" | "lg";
}

export const Thumbnail = forwardRef<HTMLDivElement, ThumbnailProps>(
  (
    { src, alt = "Preview", fileType, size = "md", className, ...props },
    ref,
  ) => {
    const [imageError, setImageError] = useState(false);
    const [isLoading, setIsLoaded] = useState(false);

    const isImageType = fileType?.startsWith("image/") ?? false;
    const isPdfType = fileType === "application/pdf";

    const extensionMap: Record<string, string> = {
      "application/pdf": "PDF",
      "image/jpeg": "JPG",
      "image/jpg": "JPG",
      "image/png": "PNG",
      "image/gif": "GIF",
      "image/webp": "WEBP",
      "image/svg+xml": "SVG",
      "image/bmp": "BMP",
      "image/tiff": "TIFF",
      "image/x-icon": "ICO",
    };

    const getExtension = (type?: string): string => {
      if (!type) return "FILE";
      if (type.startsWith("image/")) return "IMG";
      return (
        extensionMap[type] ?? type.split("/").pop()?.toUpperCase() ?? "FILE"
      );
    };

    const extension = getExtension(fileType);

    const sizeClasses: Record<"sm" | "md" | "lg", string> = {
      sm: "w-8 h-8",
      md: "w-10 h-10",
      lg: "w-12 h-12",
    };
  }
);
```



```

const iconSizeClasses: Record<"sm" | "md" | "lg", string> = {
  sm: "w-4 h-4",
  md: "w-5 h-5",
  lg: "w-6 h-6",
};

const badgeSizeClasses: Record<"sm" | "md" | "lg", string> = {
  sm: "text-[9px] px-1 py-0.5",
  md: "text-xs px-1.5 py-0.5",
  lg: "text-xs px-2 py-0.5",
};

const containerClasses = cn(
  "relative inline-flex items-center justify-center overflow-hidden",
  "rounded-md",
  "bg-card/90 backdrop-blur",
  "border",
  "border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
  "active:scale-95 transition-transform duration-100",
  "hover:shadow-lg transition-shadow duration-200",
  sizeClasses[size],
  className,
);

const imageClasses = cn(
  "absolute inset-0 w-full h-full object-cover",
  "transition-opacity duration-200 ease-out",
  isLoading && !imageError ? "opacity-100" : "opacity-0",
);

const fallbackClasses = cn(
  "relative z-10 flex items-center justify-center w-full h-full",
  "bg-muted/40",
  "text-muted-foreground",
  "transition-opacity duration-200 ease-out",
  !src || imageError || !isImageType ? "opacity-100" : "opacity-0",
);

const handleImageError = () => {
  setImageError(true);
  setIsLoaded(false);
};

const handleImageLoad = () => {
  setIsLoaded(true);
  setImageError(false);
};

const shouldShowFallback = !src || imageError || !isImageType;

return (
  <div ref={ref} {...props} className={containerClasses}>
    {src && isImageType && !imageError && (
      <img
        src={src}
        alt={alt}
        className={imageClasses}
        onError={handleImageError}

```

```

        onLoad={handleImageLoad}
        loading="lazy"
      />
    )}
    <div className={fallbackClasses} aria-hidden={!shouldShowFallback}>
      {isPdfType ? (
        <FileText
          className={cn(iconSizeClasses[size], "text-muted-foreground/80")}
          aria-hidden="true"
        />
      ) : imageError ? (
        <AlertCircle
          className={cn(iconSizeClasses[size], "text-destructive/60")}
          aria-hidden="true"
        />
      ) : (
        <ImageIcon
          className={cn(iconSizeClasses[size], "text-muted-foreground/60")}
          aria-hidden="true"
        />
      )}
    <span
      className={cn(
        "absolute bottom-0 right-0 m-1",
        "rounded-[calc(var(--radius)-2px)]",
        "bg-background/95 backdrop-blur-sm",
        "border border-[color-mix(in_oklch,var(--color-border)_30%,transparent)]",
        "text-foreground/90 font-medium uppercase tracking-wider",
        "shadow-sm",
        badgeSizeClasses[size],
      )}
      aria-label={`File type: ${extension}`}
    >
      {extension}
    </span>
  </div>
</div>
);
},
);

```

```
Thumbnail.displayName = "Thumbnail";
```

FILE: checkbox.tsx
LOKASI: src\selector

```
"use client";

import React, {
  useEffect,
  useRef,
  createContext,
  useContext,
  forwardRef,
  useMemo,
} from "react";
import { Check, Minus } from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

// ðŸŒŠ! LOCAL TYPE ISOLATION GATEWAY
export interface CheckboxProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  isIndeterminate?: boolean;
  value?: string;
  checked?: boolean;
  onChange?: (checked: boolean | "indeterminate", value?: string) => void;
}

export interface CheckboxGroupProps {
  value: string[];
  onChange: (value: string[]) => void;
  maxChoice?: number;
  disabled?: boolean;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  children: React.ReactNode;
  className?: string;
}

interface CheckboxGroupContextValue {
  value: string[];
  onChange: (value: string[]) => void;
  maxChoice?: number;
  disabled?: boolean;
  error?: string;
}

const CheckboxGroupContext = createContext<CheckboxGroupContextValue | null>(
  null,
```

```

);

export const Checkbox = forwardRef<HTMLInputElement, CheckboxProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      isIndeterminate = false,
      value,
      checked,
      onChange,
      className,
      disabled,
      ...props
    },
    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const groupContext = useContext(CheckboxGroupContext);
    const isInGroup = !!groupContext;
    const isGroupDisabled = groupContext?.disabled ?? false;
    const isGroupError = groupContext?.error;
    const groupValue = groupContext?.value ?? [];
    const groupMaxChoice = groupContext?.maxChoice;
    const groupOnChange = groupContext?.onChange;

    const isActuallyDisabled = disabled || isGroupDisabled;
    const isChecked = isInGroup
      ? groupValue.includes(value ?? "")
      : (checked ?? false);
    const hasError = error || isGroupError;
    const isAtMaxChoice =
      groupMaxChoice !== undefined &&
      groupValue.length >= groupMaxChoice &&
      !isChecked;

    // hint and popupMenu are handled by CheckboxGroup
    void hint;
    void popupMenu;

    useEffect(() => {
      if (inputRef.current) {
        inputRef.current.indeterminate = isIndeterminate;
      }
    }, [isIndeterminate]);

    const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
      if (isActuallyDisabled || isAtMaxChoice) return;

      const newChecked = e.target.checked;
      const newValue = value ?? "";

      if (isInGroup && groupOnChange) {
        const newGroupValue = newChecked
          ? [...groupValue, newValue]
          : groupValue.filter((v) => v !== newValue);
      }
    };
  }
);

```

```

    groupOnChange(newGroupValue);
  }

  onChange?.(newChecked ? true : false, newValue);
};

const checkboxClasses = cn(
  "relative inline-flex items-center justify-center",
  "w-5 h-5 shrink-0",
  "rounded-md border-2",
  "border-border bg-background",
  "transition-all duration-200 ease-out",
  "active:scale-95",
  "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
  "focus-visible:ring-offset-background",
  "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
  "aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20",
  isChecked && !isIndeterminate
    ? "bg-brand-primary border-brand-primary text-white shadow-sm hover:shadow-md"
    : isIndeterminate
      ? "bg-brand-primary border-brand-primary text-white shadow-sm hover:shadow-md"
      : "hover:border-brand-primary/50 hover:bg-brand-primary/5",
  isAtMaxChoice && "opacity-50 cursor-not-allowed",
  hasError && "border-destructive focus-visible:ring-destructive",
  className,
);

const iconClasses = cn(
  "transition-all duration-200 ease-out",
  isChecked && !isIndeterminate
    ? "scale-100 opacity-100"
    : "scale-0 opacity-0",
  isIndeterminate ? "scale-100 opacity-100" : "scale-0 opacity-0",
);

const indeterminateIconClasses = cn(
  "transition-all duration-200 ease-out",
  isIndeterminate ? "scale-100 opacity-100" : "scale-0 opacity-0",
);

return (
  <label
    className={cn(
      "inline-flex items-center gap-2 cursor-pointer select-none",
      isActuallyDisabled &&
        "opacity-50 pointer-events-none cursor-not-allowed",
    )}
  >
    <input
      ref={(el) => {
        inputRef.current = el;
        if (ref) {
          if (typeof ref === "function") {
            ref(el);
          } else {
            ref.current = el;
          }
        }
      }}
    />
  </label>
);

```

```

    }}
    type="checkbox"
    checked={isChecked}
    disabled={isActuallyDisabled || isAtMaxChoice}
    onChange={handleChange}
    aria-checked={isIndeterminate ? "mixed" : isChecked}
    aria-invalid={hasError ? "true" : "false"}
    aria-disabled={isActuallyDisabled}
    aria-required={props.required}
    className="sr-only peer"
    {...props}
  />
  <div className={checkboxClasses} aria-hidden="true">
    {isIndeterminate ? (
      <Minus
        className={cn("w-3 h-3", indeterminateIconClasses)}
        aria-hidden="true"
      />
    ) : (
      <Check
        className={cn("w-3.5 h-3.5", iconClasses)}
        aria-hidden="true"
      />
    )}
  </div>
  {label && (
    <span
      className={cn(
        "text-sm font-medium leading-none",
        "transition-colors duration-200 ease-out",
        "text-text-main",
        hasError && "text-destructive/90",
        isActuallyDisabled && "opacity-50",
      )}
    >
      {label}
    </span>
  )}
</label>
);
},
);
Checkbox.displayName = "Checkbox";

```

```

export const CheckboxGroup = ({
  value,
  onChange,
  maxChoice,
  disabled,
  error,
  hint,
  popupMenu,
  children,
  className,
}: CheckboxGroupProps) => {
  const contextValue = useMemo(
    () => ({
      value,

```

```

        onChange,
        maxChoice,
        disabled,
        error,
    }},
    [value, onChange, maxChoice, disabled, error],
  );

  return (
    <CheckboxGroupContext.Provider value={contextValue}>
      <div
        className={cn("inline-flex flex-col gap-2", className)}
        role="group"
        aria-invalid={!error}
        aria-disabled={disabled}
        onContextMenu={(e) => popupMenu?.trigger(e)}
      >
        {hint && (
          <HintBox content={hint} className="mb-1.5">
            <span className="text-sm text-text-muted" />
          </HintBox>
        )}
        {children}
        {error && (
          <p
            className={cn(
              "text-sm",
              "text-destructive/90",
              "transition-colors duration-200 ease-out",
              "mt-1",
            )}
            role="alert"
            aria-live="polite"
          >
            {error}
          </p>
        )}
      </div>
    </CheckboxGroupContext.Provider>
  );
};

CheckboxGroup.displayName = "CheckboxGroup";

```

FILE: combo-box.tsx
LOKASI: src\selector

```
import React, {
  forwardRef,
  useState,
  useRef,
  useEffect,
  useCallback,
  useMemo,
} from "react";
import {
  ChevronDown,
  ChevronUp,
  X,
  Search,
  MoreHorizontal,
} from "lucide-react";
import { cn } from "@lib/utils";
import { Thumbnail } from "@components/general/thumbnail";
import {
  CUSTOM_DROPDOWN_CONFIGS,
  DropdownColumnLayout,
} from "@components/selector/custom-dropdown-column-config";
import { Button } from "../general/button";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";
import { THEME_CONFIG } from "@config/theme-constants";

export interface ComboBoxActionButtonConfig {
  icon: React.ReactNode;
  onClick: (
    e: React.MouseEvent<HTMLButtonElement>,
    value: string | number | null,
    fullRecord: ComboBoxOption | null,
  ) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface ComboBoxOption {
  [key: string]: unknown;
  id?: string | number;
  name?: string;
}

export interface ComboBoxProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value" | "defaultValue"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  innerIcons?: Array<{ icon: React.ReactNode; className?: string }>;
```



```

    actionButtons?: ComboBoxActionButtonConfig[];
    prefixIcon?: React.ReactNode;
    suffixIcon?: React.ReactNode;
    prefixText?: string;
    suffixText?: string;
    value?: string | number | null;
    onChange?: (
      selectedValue: string | number | null,
      fullRecordRow: ComboBoxOption | null,
    ) => void;
    options?: ComboBoxOption[];
    configKey?: string;
    displayKey?: string;
    valueKey?: string;
    thumbnailKey?: string;
    thumbnailTypeKey?: string;
    placeholder?: string;
    disabled?: boolean;
    required?: boolean;
    maxDropdownHeight?: number;
    searchable?: boolean;
    clearable?: boolean;
    maxActionButtons?: number;
    showInlineSearchRow?: boolean;
  }

export const ComboBox = forwardRef<HTMLInputElement, ComboBoxProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      innerIcons = [],
      actionButtons = [],
      prefixIcon,
      suffixIcon,
      prefixText,
      suffixText,
      value,
      onChange,
      options = [],
      configKey,
      displayKey = "name",
      valueKey = "id",
      thumbnailKey,
      thumbnailTypeKey,
      placeholder = "Select an option...",
      disabled = false,
      required = false,
      maxDropdownHeight = THEME_CONFIG.layout.dropdownMaxHeight,
      searchable = true,
      clearable = true,
      maxActionButtons = 4,
      showInlineSearchRow = false,
      className,
      ...props
    },
  ),

```

```

    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const dropdownRef = useRef<HTMLDivElement>(null);
    const [isOpen, setIsOpen] = useState(false);
    const [searchQuery, setSearchQuery] = useState("");
    const [highlightedIndex, setHighlightedIndex] = useState(-1);
    const [selectedRecord, setSelectedRecord] = useState<ComboBoxOption | null>(
      null,
    );
    const hasError = Boolean(error);

    const config = useMemo(() : DropdownColumnLayout[] => {
      if (configKey) {
        return CUSTOM_DROPDOWN_CONFIGS[configKey] || [];
      }
      return [];
    }, [configKey]);

    const filteredOptions = useMemo(() => {
      if (!searchable || searchQuery.trim().length < 3) return options;
      const query = searchQuery.toLowerCase().trim();
      return options.filter((option: unknown) => {
        const opt = option as Record<string, unknown>;
        const displayValue =
          (opt[displayKey] as string) ||
          (opt[valueKey] as string) ||
          String(option);
        return String(displayValue).toLowerCase().includes(query);
      });
    }, [options, searchQuery, searchable, displayKey, valueKey]);

    const handleRef = useCallback(
      (node: HTMLInputElement | null) => {
        inputRef.current = node;
        if (ref) {
          if (typeof ref === "function") {
            ref(node);
          } else {
            ref.current = node;
          }
        }
      },
      [ref],
    );

    const handleClickOutside = useCallback((event: MouseEvent) => {
      if (
        dropdownRef.current &&
        !dropdownRef.current.contains(event.target as Node)
      ) {
        if (
          inputRef.current &&
          !inputRef.current.contains(event.target as Node)
        ) {
          setIsOpen(false);
          setSearchQuery("");
          setHighlightedIndex(-1);
        }
      }
    });
  }
}

```

```

    }
  }
}, []);

useEffect(() => {
  document.addEventListener("mousedown", handleClickOutside);
  return () =>
    document.removeEventListener("mousedown", handleClickOutside);
}, [handleClickOutside]);

useEffect(() => {
  if (value !== undefined && options.length > 0) {
    const found = options.find((opt) => opt[valueKey] === value);
    if (found) {
      setSelectedRecord(found);
    } else if (typeof value === "string" || typeof value === "number") {
      setSelectedRecord({ [displayKey]: value, [valueKey]: value });
    }
  } else if (value === null || value === undefined || value === "") {
    setSelectedRecord(null);
  }
}, [value, options, valueKey, displayKey]);

const handleSelect = useCallback(
  (option: ComboBoxOption) => {
    const selectedValue =
      (option[valueKey] as string | number) ??
      (option[displayKey] as string | number) ??
      option;
    const fullRecord = option;
    setSelectedRecord(fullRecord);
    setSearchQuery("");
    setIsOpen(false);
    setHighlightedIndex(-1);
    onChange?.(selectedValue, fullRecord);
  },
  [onChange, valueKey, displayKey],
);

const handleClear = useCallback(
  (e: React.MouseEvent<HTMLButtonElement>) => {
    e.stopPropagation();
    setSelectedRecord(null);
    setSearchQuery("");
    setIsOpen(false);
    onChange?.(null, null);
  },
  [onChange],
);

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLInputElement>) => {
    if (
      !isOpen &&
      (e.key === "ArrowDown" || e.key === "ArrowUp" || e.key === "Enter")
    ) {
      e.preventDefault();
      setIsOpen(true);
    }
  },
  [isOpen],
);

```

```

        setHighlightedIndex(0);
        return;
    }

    if (!isOpen) return;

    switch (e.key) {
        case "ArrowDown":
            e.preventDefault();
            setHighlightedIndex((prev) =>
                Math.min(prev + 1, filteredOptions.length - 1),
            );
            break;
        case "ArrowUp":
            e.preventDefault();
            setHighlightedIndex((prev) => Math.max(prev - 1, -1));
            break;
        case "Enter":
            e.preventDefault();
            if (highlightedIndex >= 0 && filteredOptions[highlightedIndex]) {
                handleSelect(filteredOptions[highlightedIndex]);
            }
            break;
        case "Escape":
            setIsOpen(false);
            setSearchQuery("");
            setHighlightedIndex(-1);
            break;
    }
},
[isOpen, filteredOptions, highlightedIndex, handleSelect],
);

const limitedInnerIcons = innerIcons.slice(0, 4);
const limitedActionButtons = actionButtons.slice(0, maxActionButtons);

return (
    <div
        className={cn("w-full", className)}
        onContextMenu={(e) => popupMenu?.trigger(e)}
    >
        {hint && (
            <HintBox content={hint} className="mb-1.5">
                <span className="text-sm text-text-muted" />
            </HintBox>
        )}
        {label && (
            <label
                className={cn(
                    "block text-sm font-medium text-text-main mb-1.5",
                    disabled && "opacity-50",
                )}
            >
                {label}
                {required && (
                    <span className="text-destructive ml-0.5" aria-hidden="true">
                        *
                    </span>
                )}
            </label>
        )}
    >

```

```

    })
  </label>
})
<div className="flex items-center w-full gap-2">
  <div
    className={cn(
      "relative flex items-center flex-1 min-w-0",
      "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-surface",
      "text-text-main placeholder:text-muted-foreground/60",
      "focus-within:ring-2 focus-within:ring-amber-500/20 focus-within:border-amber-500 focus-
within:bg-amber-500/5",
      "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
      "transition-all duration-200 ease-out",
      hasError &&
        "border-destructive/50 focus-within:ring-destructive/50 focus-within:border-
destructive/50",
      disabled && "bg-muted/50",
    )}
  >
    {(prefixIcon || prefixText) && (
      <div
        className={cn(
          "flex items-center justify-center px-3",
          "text-muted-foreground/60",
          disabled && "opacity-50",
        )}
        aria-hidden="true"
      >
        {prefixIcon && (
          <span className="w-4 h-4" aria-hidden="true">
            {prefixIcon}
          </span>
        )}
        {prefixText && (
          <span className="text-sm font-medium">{prefixText}</span>
        )}
      </div>
    )}
    <input
      ref={handleRef}
      type="text"
      className={cn(
        "flex-1 bg-transparent border-none outline-none",
        "px-4 py-2.5",
        "text-text-main placeholder:text-muted-foreground/60",
        "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
        "w-full min-w-0",
        prefixIcon || prefixText ? "pl-0" : "pl-4",
        suffixIcon || suffixText || limitedInnerIcons.length > 0
          ? "pr-0"
          : "pr-4",
      )}
      disabled={disabled}
      required={required}
      placeholder={placeholder}
      value={

```

```

      selectedRecord
      ? ((selectedRecord[displayKey] as string) ??
        (selectedRecord[valueKey] as string) ??
        "")
      : ""
    }
    onClick={(e) => {
      e.stopPropagation();
      if (!disabled) setIsOpen(!isOpen);
    }}
    onKeyDown={handleKeyDown}
    readOnly
    {...props}
  />
{suffixIcon && (
  <div
    className={cn(
      "flex items-center justify-center px-3",
      "text-muted-foreground/60",
      disabled && "opacity-50",
    )}
    aria-hidden="true"
  >
    {suffixIcon}
  </div>
)}
{suffixText && (
  <div
    className={cn(
      "flex items-center justify-center px-3",
      "text-muted-foreground/60",
      disabled && "opacity-50",
    )}
    aria-hidden="true"
  >
    <span className="text-sm font-medium">{suffixText}</span>
  </div>
)}
{clearable && selectedRecord && !disabled && (
  <Button
    type="button"
    variant="ghost"
    size="icon"
    className="active:scale-95 transition-all duration-100"
    onClick={handleClear}
    aria-label="Clear selection"
  >
    <X className="w-4 h-4" aria-hidden="true" />
  </Button>
)}
<Button
  type="button"
  variant="ghost"
  size="icon"
  className="active:scale-95 transition-all duration-100 flex-shrink-0"
  disabled={disabled}
  onClick={() => setIsOpen(!isOpen)}
  aria-label={isOpen ? "Close dropdown" : "Open dropdown"}

```

```

    >
    {isOpen ? (
      <ChevronUp className="w-4 h-4" aria-hidden="true" />
    ) : (
      <ChevronDown className="w-4 h-4" aria-hidden="true" />
    )}
  </Button>
</div>
{limitedActionButtons.length > 0 && (
  <div className="flex items-center gap-1.5 shrink-0 ml-1.5">
    {limitedActionButtons.map((action, index) => (
      <Button
        key={index}
        variant="ghost"
        size="icon"
        disabled={disabled || action.disabled}
        onClick={(e) =>
          action.onClick(e, value ?? null, selectedRecord)
        }
        className="active:scale-95 transition-all duration-100"
        aria-label={action.tooltipText}
      >
        {action.icon}
      </Button>
    ))}
  </div>
)}

{isOpen && (
  <div
    ref={dropdownRef}
    className={cn(
      "absolute z-50 w-full mt-1.5",
      "bg-card/95 backdrop-blur-[var(--backdrop-blur)] border border-[color-
mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-surface shadow-lg",
      "overflow-auto",
      "transition-all duration-150 ease-out",
      isOpen
        ? "opacity-100 scale-100 translate-y-0"
        : "opacity-0 scale-95 -translate-y-1 pointer-events-none",
    )}
    style={{ maxHeight: THEME_CONFIG.layout.dropdownMaxHeight }}
    role="listbox"
    aria-label="Options"
  >
    {searchable && showInlineSearchRow && (
      <div className="p-2 border-b border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)]">
        <div className="relative">
          <Search
            className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-
foreground/60"
            aria-hidden="true"
          />
          <input
            type="text"
            className={cn(

```

```

        "w-full bg-transparent border border-input",
        "rounded-md px-9 py-2 text-sm",
        "placeholder:text-muted-foreground/60",
        "focus:outline-none focus:ring-2 focus:ring-amber-500/20 focus:border-amber-500
focus:bg-amber-500/5",
    )}
    placeholder="Search..."
    value={searchQuery}
    onChange={(e) => {
        setSearchQuery(e.target.value);
        setHighlightedIndex(0);
    }}
    onKeyDown={(e) => e.stopPropagation()}
    autoFocus
  />
</div>
</div>
)}

{config.length > 0 && (
  <div className="px-3 py-2 bg-card/40 border-b border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)] font-medium text-xs text-muted-foreground uppercase tracking-wider">
    <div className="flex" style={{ width: "100%" }}>
      {config.map((col) => (
        <div
          key={col.key}
          className="truncate"
          style={{ width: `${col.widthPercent}%` }}
        >
          {col.headerLabel}
        </div>
      ))}
    </div>
  </div>
)}

{filteredOptions.length === 0 ? (
  <div className="px-4 py-8 text-center text-muted-foreground text-sm">
    {searchQuery
      ? "No matching options found"
      : "No options available"}
  </div>
) : (
  <div role="listbox" aria-label="Options">
    {filteredOptions.map((option, index) => {
      const isHighlighted = index === highlightedIndex;
      const isSelected = Boolean(
        selectedRecord &&
        option[valueKey] === selectedRecord[valueKey],
      );

      const renderCell = (
        col: DropdownColumnLayout,
        opt: ComboBoxOption,
      ) => {
        if (
          col.renderType === "thumbnail" &&
          thumbnailKey &&

```



```

        thumbnailTypeKey
    ) {
        return (
            <Thumbnail
                key={col.key}
                src={opt[thumbnailKey] as string}
                fileType={opt[thumbnailTypeKey] as string}
                size="sm"
                alt={(opt[displayKey] as string) || "Preview"}
                className="mx-auto"
            />
        );
    }
    return (
        <span key={col.key} className="truncate block">
            {(opt[col.key] as string) ??
                (opt[displayKey] as string) ??
                ""}
        </span>
    );
};

return (
    <Button
        key={(option[valueKey] as string | number) ?? index}
        type="button"
        role="option"
        aria-selected={isSelected}
        aria-highlighted={isHighlighted}
        variant="ghost"
        size="sm"
        className={cn(
            "w-full px-3 py-2 text-left text-sm",
            "transition-colors duration-100",
            "active:scale-95",
            isHighlighted && "bg-brand-primary/5 text-text-main",
            isSelected &&
                "bg-brand-primary/10 text-brand-primary font-semibold",
            disabled && "opacity-50 pointer-events-none",
        )}
        onClick={() => handleSelect(option)}
        onMouseEnter={() => setHighlightedIndex(index)}
        disabled={disabled}
        style={{ maxHeight: maxDropdownHeight }}
    >
        {config.length > 0 ? (
            <div className="flex" style={{ width: "100%" }}>
                {config.map((col) => (
                    <div
                        key={col.key}
                        className="truncate px-1"
                        style={{ width: `${col.widthPercent}%` }}
                    >
                        {renderCell(col, option)}
                    </div>
                ))}
            </div>
        ) : (

```

```

        <span className="truncate block">
            {(option[displayKey] as string) ??
              (option[valueKey] as string) ??
              String(
                option[displayKey] ?? option[valueKey] ?? "",
              )}
        </span>
      )}
    </Button>
  );
  }}}
</div>
)}

{actionButtons.length > maxActionButtons && (
  <div className="border-t border-[color-mix(in_oklch,var(--color-
border)_40%,transparent)] p-2 bg-card/50 backdrop-blur-[var(--backdrop-blur)]">
    <Button
      type="button"
      variant="ghost"
      size="sm"
      className={cn(
        "w-full flex items-center justify-center gap-2 px-3 py-2 text-sm",
        "text-muted-foreground/70 hover:text-foreground",
        "active:scale-95 transition-all duration-100",
      )}
      onClick={() => {}}
    >
      <MoreHorizontal className="w-4 h-4" aria-hidden="true" />
      <span className="font-medium tracking-wide">
        More actions ({actionButtons.length - maxActionButtons})
      </span>
    </Button>
  </div>
)}
</div>
)}

{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>
</div>
);
},
);

ComboBox.displayName = "ComboBox";

```

FILE: custom-dropdown-column-config.ts
LOKASI: src\selector

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface DropdownColumnLayout {
  key: string;
  headerLabel: string;
  widthPercent: number;
  renderType?: "text" | "thumbnail";
}

export const CUSTOM_DROPDOWN_CONFIGS: Record<string, DropdownColumnLayout[]> =
  {};
```

FILE: list-box.tsx
LOKASI: src\selector

```
import React, {
  forwardRef,
  useState,
  useEffect,
  useRef,
  useCallback,
  useMemo,
} from "react";
import { LucideIcon } from "lucide-react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";
import { Checkbox } from "../checkbox";
import type { PopupMenuConfig } from "../feedback/popup-menu";

export interface ListBoxOption extends Record<string, unknown> {
  id?: string | number;
  name?: string;
}

export interface ListBoxRightActionConfig {
  icon: LucideIcon;
  onClick: (
    e: React.MouseEvent<HTMLButtonElement>,
    optionItem: ListBoxOption,
  ) => void;
  tooltipText?: string;
  className?: string;
}

export interface ListBoxProps extends Omit<
  React.HTMLAttributes<HTMLDivElement>,
  "onChange" | "value"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  options: ListBoxOption[];
  value?: unknown | unknown[];
  onChange?: (selectedValue: unknown, fullRecordRows: unknown) => void;
  multiple?: boolean;
  displayKey?: string;
  valueKey?: string;
  rightActionButtons?: ListBoxRightActionConfig[];
  disabled?: boolean;
}

export const ListBox = forwardRef<HTMLDivElement, ListBoxProps>(
  (
    {
      label,
      error,
```

```

    hint,
    popupMenu,
    options = [],
    value,
    onChange,
    multiple = false,
    displayKey = "name",
    valueKey = "id",
    rightActionButtons = [],
    disabled = false,
    className,
    ...props
  },
  ref,
) => {
  const containerRef = useRef<HTMLDivElement>(null);
  const [focusedIndex, setFocusedIndex] = useState<number>(-1);
  const [selectedValues, setSelectedValues] = useState<unknown[]>([]);

  const normalizedValue = useMemo(() => {
    if (value === undefined || value === null) return [];
    return Array.isArray(value) ? value : [value];
  }, [value]);

  useEffect(() => {
    setSelectedValues(normalizedValue);
  }, [normalizedValue]);

  const getOptionValue = useCallback(
    (option: ListBoxOption): unknown => {
      return option[valueKey] ?? option[displayKey] ?? option;
    },
    [valueKey, displayKey],
  );

  const getOptionLabel = useCallback(
    (option: ListBoxOption): string => {
      return String(option[displayKey] ?? option[valueKey] ?? "");
    },
    [displayKey, valueKey],
  );

  const isOptionSelected = useCallback(
    (option: ListBoxOption): boolean => {
      const optValue = getOptionValue(option);
      return selectedValues.some((v) => v === optValue);
    },
    [selectedValues, getOptionValue],
  );

  const handleSelectionChange = useCallback(
    (option: ListBoxOption) => {
      const optValue = getOptionValue(option);
      const isSelected = isOptionSelected(option);

      let newSelectedValues: unknown[];
      if (multiple) {
        newSelectedValues = isSelected

```

```

        ? selectedValues.filter((v) => v !== optValue)
        : [...selectedValues, optValue];
    } else {
        newSelectedValues = isSelected ? [] : [optValue];
    }

    setSelectedValues(newSelectedValues);
    onChange?.(multiple ? newSelectedValues : newSelectedValues[0], option);
  },
  [multiple, selectedValues, getOptionValue, isOptionSelected, onChange],
);

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLDivElement>, index: number) => {
    switch (e.key) {
      case "ArrowDown":
        e.preventDefault();
        setFocusedIndex((prev) => Math.min(prev + 1, options.length - 1));
        break;
      case "ArrowUp":
        e.preventDefault();
        setFocusedIndex((prev) => Math.max(prev - 1, 0));
        break;
      case "Enter":
      case " ":
        e.preventDefault();
        if (index >= 0 && index < options.length) {
          handleSelectionChange(options[index]);
        }
        break;
      case "Escape":
        setFocusedIndex(-1);
        break;
    }
  },
  [options, handleSelectionChange],
);

const handleClick = useCallback(
  (
    e: React.MouseEvent<HTMLButtonElement>,
    option: ListBoxOption,
    index: number,
  ) => {
    if (rightActionButtons.length > 0) {
      const target = e.target as HTMLElement;
      const isActionButton = target.closest('[data-action-button="true"]');
      if (!isActionButton) {
        handleSelectionChange(option);
      }
    } else {
      handleSelectionChange(option);
    }
    setFocusedIndex(index);
  },
  [rightActionButtons, handleSelectionChange],
);

```

```

const handleClick = useCallback(
  (
    e: React.MouseEvent<HTMLButtonElement>,
    action: ListBoxRightActionConfig,
    option: ListBoxOption,
  ) => {
    e.stopPropagation();
    action.onClick(e, option);
  },
  [],
);

const handleFocus = useCallback((index: number) => {
  setFocusedIndex(index);
}, []);

const handleBlur = useCallback(() => {
  setFocusedIndex(-1);
}, []);

const limitedRightActions = useMemo(
  () => rightActionButtons.slice(0, 2),
  [rightActionButtons],
);

const hasError = Boolean(error);

return (
  <div
    ref={ref}
    className={cn("w-full", className)}
    onContextMenu={(e) => popupMenu?.trigger(e)}
    {...props}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
      </label>
    )}
    <div
      ref={containerRef}
      className={cn(
        "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
        "border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
        "rounded-surface",
        "overflow-hidden",
        "focus-within:ring-2 focus-within:ring-amber-500/20 focus-within:border-amber-500 focus-within:bg-amber-500/5",
      )}
    >

```

```

      "disabled:opacity-50 disabled:pointer-events-none disabled:cursor-not-allowed",
      "transition-all duration-200 ease-out",
      hasError &&
        "border-destructive/50 focus-within:ring-destructive/50 focus-within:border-
destructive/50",
        disabled && "bg-muted/50",
      )}
      role="listbox"
      aria-multiselectable={multiple}
      tabIndex={disabled ? -1 : 0}
      onKeyDown={(e) => handleKeyDown(e, focusedIndex)}
      onFocus={() => handleFocus(0)}
      onBlur={handleBlur}
    >
    {options.length === 0 ? (
      <div className="px-4 py-8 text-center text-muted-foreground text-sm">
        No options available
      </div>
    ) : (
      <div role="listbox" aria-label="Options">
        {options.map((option, index) => {
          const isSelected = isOptionSelected(option);
          const isFocused = index === focusedIndex;
          const optValue = getOptionValue(option);
          const optLabel = getOptionLabel(option);

          return (
            <button
              key={String(optValue)}
              type="button"
              role="option"
              aria-selected={isSelected}
              aria-highlighted={isFocused}
              tabIndex={-1}
              onClick={(e) => handleRowClick(e, option, index)}
              onFocus={() => handleFocus(index)}
              onBlur={handleBlur}
              className={cn(
                "w-full px-4 py-3 text-left text-sm",
                "flex items-center gap-3",
                "transition-colors duration-100",
                "focus:outline-none focus:bg-brand-primary/5 hover:bg-brand-primary/5",
                isFocused && "bg-brand-primary/5 text-text-main",
                isSelected &&
                  "bg-brand-primary/10 text-brand-primary font-semibold border-1-2 border-brand-
primary",
                disabled &&
                  "opacity-50 pointer-events-none cursor-not-allowed",
              )}
            >
              <div>
                {optLabel}
              </div>
              <div>
                {optValue}
              </div>
            </button>
          )
        })}
      </div>
    )}
  >
  {multiple && (
    <Checkbox
      checked={isSelected}
      onChange={() => handleSelectionChange(option)}
      disabled={disabled}
      className="shrink-0"
      aria-hidden="true"
    />
  )}

```



```

    })
    <span
      className={cn(
        "truncate flex-1 min-w-0",
        isSelected && "font-semibold",
      )}
    >
      {optLabel}
    </span>
    {limitedRightActions.length > 0 && (
      <div
        className="flex items-center gap-1.5 shrink-0"
        data-action-buttons="true"
      >
        {limitedRightActions.map((action, actionIndex) => (
          <button
            key={actionIndex}
            type="button"
            data-action-button="true"
            className={cn(
              "relative flex items-center justify-center",
              "w-8 h-8 rounded-md",
              "text-muted-foreground/60 hover:text-foreground",
              "bg-transparent hover:bg-accent",
              "transition-transform duration-100",
              "active:scale-95",
              "focus:outline-none focus-visible:ring-2 focus-visible:ring-amber-500/20",
              "focus-visible:border-amber-500",
              "disabled:opacity-50 disabled:pointer-events-none",
              action.className,
            )}
            onClick={(e) =>
              handleActionClick(e, action, option)
            }
            aria-label={action.tooltipText || "Action"}
            disabled={disabled}
          >
            <action.icon
              className="w-4 h-4"
              aria-hidden="true"
            />
          </button>
        ))}
      </div>
    )}
  </button>
);
}}
</div>
)}
</div>
{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
  )}

```

```
        role="alert"
      >
        {error}
      </p>
    )}
  </div>
);
},
);
ListBox.displayName = "ListBox";
```

FILE: radio-button.tsx
LOKASI: src\selector

```
"use client";

import React, {
  createContext,
  useContext,
  useMemo,
  useRef,
  useEffect,
  useState,
  useCallback,
} from "react";
import { Circle } from "lucide-react";
import { cn } from "@lib/Utils";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

// Ã°Ã¡Ã¡! LOCAL TYPE ISOLATION GATEWAY
export interface RadioOptionItemConfig {
  value: string;
  label: string;
  disabled?: boolean;
}

export interface RadioGroupProps {
  name?: string;
  value: string;
  onChange: (value: string) => void;
  options: RadioOptionItemConfig[];
  allowClear?: boolean;
  disabled?: boolean;
  error?: string;
  orientation?: "horizontal" | "vertical";
  className?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
}

interface RadioGroupContextValue {
  name: string | undefined;
  value: string;
  onChange: (value: string) => void;
  allowClear: boolean;
  disabled: boolean;
  error: string | undefined;
  orientation: "horizontal" | "vertical";
  registerOption: (
    value: string,
    ref: React.RefObject<HTMLInputElement | null>,
  ) => void;
  unregisterOption: (value: string) => void;
  focusedValue: string | null;
  setFocusedValue: (value: string | null) => void;
```

```

    }

    const RadioGroupContext = createContext<RadioGroupContextValue | null>(null);

    function useRadioGroupContext() {
      const context = useContext(RadioGroupContext);
      if (!context) {
        throw new Error(
          "RadioGroup compound components must be used within RadioGroup",
        );
      }
      return context;
    }

    interface RadioOptionProps {
      option: RadioOptionItemConfig;
      className?: string;
    }

    const RadioOption = React.forwardRef<HTMLInputElement, RadioOptionProps>(
      ({ option, className }, ref) => {
        const {
          name,
          value,
          onChange,
          allowClear,
          disabled,
          error,
          registerOption,
          unregisterOption,
          setFocusedValue,
        } = useRadioGroupContext();

        const inputRef = useRef<HTMLInputElement>(null);
        const isChecked = value === option.value;
        const isDisabled = disabled || option.disabled;

        useEffect(() => {
          registerOption(option.value, inputRef);
          return () => unregisterOption(option.value);
        }, [option.value, registerOption, unregisterOption]);

        useEffect(() => {
          if (inputRef.current) {
            inputRef.current.checked = isChecked;
          }
        }, [isChecked]);

        const handleChange = useCallback(() => {
          if (isDisabled) return;

          if (allowClear && isChecked) {
            onChange("");
            return;
          }

          onChange(option.value);
        }, [isDisabled, allowClear, isChecked, onChange, option.value]);
      }
    );
  }

```

```

const handleKeyDown = useCallback(
  (event: React.KeyboardEvent<HTMLInputElement>) => {
    if (isDisabled) return;

    const options = Array.from(
      document.querySelectorAll<HTMLInputElement>(
        `input[name="${name}"][type="radio"]:not(:disabled)`
      ),
    );

    if (options.length === 0) return;

    const currentIndex = options.findIndex(
      (el) => el === event.currentTarget,
    );
    if (currentIndex === -1) return;

    let nextIndex = currentIndex;
    let shouldNavigate = false;

    if (event.key === "ArrowUp" || event.key === "ArrowLeft") {
      event.preventDefault();
      nextIndex = (currentIndex - 1 + options.length) % options.length;
      shouldNavigate = true;
    } else if (event.key === "ArrowDown" || event.key === "ArrowRight") {
      event.preventDefault();
      nextIndex = (currentIndex + 1) % options.length;
      shouldNavigate = true;
    } else if (event.key === "Home") {
      event.preventDefault();
      nextIndex = 0;
      shouldNavigate = true;
    } else if (event.key === "End") {
      event.preventDefault();
      nextIndex = options.length - 1;
      shouldNavigate = true;
    }

    if (shouldNavigate) {
      const nextOption = options[nextIndex];
      if (nextOption) {
        nextOption.focus();
        nextOption.click();
      }
    }
  },
  [isDisabled, name],
);

const handleFocus = useCallback(() => {
  setFocusedValue(option.value);
}, [option.value, setFocusedValue]);

const handleBlur = useCallback(() => {
  setFocusedValue(null);
}, [setFocusedValue]);

```

```

const radioClasses = cn(
  "relative inline-flex items-center justify-center",
  "w-5 h-5 shrink-0",
  "rounded-full border-2",
  "border-border bg-background",
  "transition-all duration-200 ease-out",
  "active:scale-95",
  "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
  "focus-visible:ring-offset-background",
  "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
  "aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20",
  isChecked
    ? "bg-brand-primary border-brand-primary text-white shadow-sm hover:shadow-md"
    : "hover:border-brand-primary/50 hover:bg-brand-primary/5",
  error && "border-destructive focus-visible:ring-destructive",
  className,
);

const innerDotClasses = cn(
  "transition-all duration-200 ease-out",
  isChecked ? "scale-100 opacity-100" : "scale-0 opacity-0",
);

const labelClasses = cn(
  "text-sm font-medium leading-none",
  "transition-colors duration-200 ease-out",
  "text-text-main",
  error && "text-destructive/90",
  isDisabled && "opacity-50",
  "ml-2",
);

const containerClasses = cn(
  "inline-flex items-center cursor-pointer select-none",
  isDisabled && "opacity-50 pointer-events-none cursor-not-allowed",
);

return (
  <label className={containerClasses}>
    <input
      ref={(el) => {
        inputRef.current = el;
        if (ref) {
          if (typeof ref === "function") {
            ref(el);
          } else {
            ref.current = el;
          }
        }
      }}
      type="radio"
      name={name}
      value={option.value}
      checked={isChecked}
      disabled={isDisabled}
      onChange={handleChange}
      onKeyDown={handleKeyDown}
      onFocus={handleFocus}
    />
  </label>
);

```

```

        onBlur={handleBlur}
        aria-checked={isChecked}
        aria-invalid={error ? "true" : "false"}
        aria-disabled={isDisabled}
        aria-required={true}
        className="sr-only peer"
      />
      <div className={radioClasses} aria-hidden="true">
        <Circle
          className={cn("w-2.5 h-2.5", innerDotClasses)}
          aria-hidden="true"
        />
      </div>
      <span className={labelClasses}>{option.label}</span>
    </label>
  );
},
);
RadioOption.displayName = "RadioOption";

export function RadioGroup({
  name,
  value,
  onChange,
  options,
  allowClear = false,
  disabled = false,
  error,
  orientation = "vertical",
  className,
  hint,
  popupMenu,
}: RadioGroupProps) {
  const [focusedValue, setFocusedValue] = useState<string | null>(null);
  const optionRefs = useRef<
    Map<string, React.RefObject<HTMLInputElement | null>>
  >(new Map());

  const registerOption = useCallback(
    (optionValue: string, ref: React.RefObject<HTMLInputElement | null>) => {
      optionRefs.current.set(optionValue, ref);
    },
    [],
  );

  const unregisterOption = useCallback((optionValue: string) => {
    optionRefs.current.delete(optionValue);
  }, []);

  const contextValue = useMemo<RadioGroupContextValue>(
    () => ({
      name,
      value,
      onChange,
      allowClear,
      disabled,
      error,
      orientation,

```

```

        registerOption,
        unregisterOption,
        focusedValue,
        setFocusedValue,
    )),
    [
        name,
        value,
        onChange,
        allowClear,
        disabled,
        error,
        orientation,
        registerOption,
        unregisterOption,
        focusedValue,
        setFocusedValue,
    ],
);

const groupClasses = cn(
    "inline-flex",
    orientation === "horizontal" ? "flex-row gap-6" : "flex-col gap-2",
    className,
);

return (
    <RadioGroupContext.Provider value={contextValue}>
        {hint && (
            <HintBox content={hint} className="mb-1.5">
                <span className="sr-only">hint</span>
            </HintBox>
        )}
        <div
            className={groupClasses}
            role="radiogroup"
            aria-label={name}
            aria-invalid={!!error}
            aria-disabled={disabled}
            aria-orientation={orientation}
            onContextMenu={(e) => popupMenu?.trigger(e)}
        >
            {options.map((option) => (
                <RadioOption key={option.value} option={option} />
            ))}
            {error && (
                <p
                    className={cn(
                        "text-sm",
                        "text-destructive/90",
                        "transition-colors duration-200 ease-out",
                        orientation === "horizontal" ? "ml-10 mt-1" : "mt-1 ml-7",
                    )}
                    role="alert"
                    aria-live="polite"
                >
                    {error}
                </p>
            )}
        </div>
    </RadioGroupContext.Provider>
);

```



```
    })  
  </div>  
</RadioGroupContext.Provider>  
);  
}  
RadioGroup.displayName = "RadioGroup";  
  
export { RadioGroupContext, RadioOption };
```

FILE: slider.tsx
LOKASI: src\selector

```
"use client";

import React, {
  forwardRef,
  useRef,
  useState,
  useEffect,
  useCallback,
  useMemo,
} from "react";
import { cn } from "@lib/utils";
import { HintBox } from "../../feedback/hint-box";
import type { PopupMenuConfig } from "../../feedback/popup-menu";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export interface SliderProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value" | "defaultValue" | "type"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  min?: number;
  max?: number;
  step?: number;
  value?: number | string;
  defaultValue?: number | string;
  stepsArray?: string[];
  onChange?: (value: number | string) => void;
  showValue?: boolean;
  orientation?: "horizontal" | "vertical";
}

export const Slider = forwardRef<HTMLInputElement, SliderProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      min = 0,
      max = 100,
      step = 1,
      value,
      defaultValue,
      stepsArray,
      onChange,
      className,
      disabled,
      showValue = true,
      orientation = "horizontal",
```

```

    ...props
  },
  ref,
) => {
  const sliderRef = useRef<HTMLDivElement>(null);
  const thumbRef = useRef<HTMLDivElement>(null);
  const trackRef = useRef<HTMLDivElement>(null);
  const isDraggingRef = useRef(false);
  const startXRef = useRef(0);
  const startValueRef = useRef(0);

  const isEnumMode = stepsArray && stepsArray.length > 0;
  const enumMin = 0;
  const enumMax = isEnumMode ? stepsArray.length - 1 : max;
  const enumStep = isEnumMode ? 1 : step;
  const effectiveMin = isEnumMode ? enumMin : min;
  const effectiveMax = isEnumMode ? enumMax : max;
  const effectiveStep = isEnumMode ? enumStep : step;

  const [internalValue, setInternalValue] = useState<number>(() => {
    if (value !== undefined) {
      if (isEnumMode && typeof value === "string") {
        return stepsArray!.indexOf(value);
      }
      return typeof value === "number"
        ? value
        : parseFloat(value as string) || effectiveMin;
    }
    if (defaultValue !== undefined) {
      if (isEnumMode && typeof defaultValue === "string") {
        return stepsArray!.indexOf(defaultValue);
      }
      return typeof defaultValue === "number"
        ? defaultValue
        : parseFloat(defaultValue as string) || effectiveMin;
    }
    return effectiveMin;
  });

  const [focused, setFocused] = useState(false);
  const [hovered, setHovered] = useState(false);

  const clampedValue = useMemo(() => {
    let val = internalValue;
    if (val < effectiveMin) val = effectiveMin;
    if (val > effectiveMax) val = effectiveMax;
    const stepped =
      Math.round((val - effectiveMin) / effectiveStep) * effectiveStep +
      effectiveMin;
    return Math.min(Math.max(stepped, effectiveMin), effectiveMax);
  }, [internalValue, effectiveMin, effectiveMax, effectiveStep]);

  const percentage = useMemo(() => {
    if (effectiveMax === effectiveMin) return 0;
    return (
      ((clampedValue - effectiveMin) / (effectiveMax - effectiveMin)) * 100
    );
  }, [clampedValue, effectiveMin, effectiveMax]);

```

```

const displayValue = useMemo(() => {
  if (isEnumMode) {
    return stepsArray[clampedValue] ?? "";
  }
  return clampedValue;
}, [isEnumMode, stepsArray, clampedValue]);

const isActuallyDisabled = disabled ?? false;
const hasError = !!error;

useEffect(() => {
  if (value !== undefined) {
    if (isEnumMode && typeof value === "string") {
      const idx = stepsArray!.indexOf(value);
      if (idx !== -1) setInternalValue(idx);
    } else {
      const numVal =
        typeof value === "number" ? value : parseFloat(value as string);
      if (!isNaN(numVal)) setInternalValue(numVal);
    }
  }
}, [value, isEnumMode, stepsArray]);

const notifyChange = useCallback(
  (newValue: number) => {
    const finalValue = isEnumMode ? stepsArray[newValue] : newValue;
    onChange?.(finalValue);
  },
  [isEnumMode, stepsArray, onChange],
);

const updateValueFromPosition = useCallback(
  (clientX: number, clientY: number) => {
    if (!trackRef.current) return;

    const rect = trackRef.current.getBoundingClientRect();
    let newPercentage: number;

    if (orientation === "horizontal") {
      newPercentage = ((clientX - rect.left) / rect.width) * 100;
    } else {
      newPercentage = ((rect.bottom - clientY) / rect.height) * 100;
    }

    newPercentage = Math.max(0, Math.min(100, newPercentage));
    const newValue =
      effectiveMin + (newPercentage / 100) * (effectiveMax - effectiveMin);
    const steppedValue =
      Math.round((newValue - effectiveMin) / effectiveStep) *
      effectiveStep +
      effectiveMin;
    const clamped = Math.min(
      Math.max(steppedValue, effectiveMin),
      effectiveMax,
    );

    setInternalValue(clamped);
  },
);

```

```

    notifyChange(clamped);
  },
  [effectiveMin, effectiveMax, effectiveStep, orientation, notifyChange],
);

const handleMouseDown = useCallback(
  (
    e: React.MouseEvent<HTMLDivElement> | React.TouchEvent<HTMLDivElement>,
  ) => {
    if (isActuallyDisabled) return;

    isDraggingRef.current = true;
    const clientX = "touches" in e ? e.touches[0].clientX : e.clientX;
    const clientY = "touches" in e ? e.touches[0].clientY : e.clientY;
    startXRef.current = clientX;
    startValueRef.current = clampedValue;

    e.preventDefault();
    updateValueFromPosition(clientX, clientY);
  },
  [isActuallyDisabled, clampedValue, updateValueFromPosition],
);

const handleMouseMove = useCallback(
  (e: MouseEvent | TouchEvent) => {
    if (!isDraggingRef.current || isActuallyDisabled) return;

    const clientX = "touches" in e ? e.touches[0].clientX : e.clientX;
    const clientY = "touches" in e ? e.touches[0].clientY : e.clientY;
    updateValueFromPosition(clientX, clientY);
  },
  [isActuallyDisabled, updateValueFromPosition],
);

const handleMouseUp = useCallback(() => {
  isDraggingRef.current = false;
}, []);

useEffect(() => {
  if (isDraggingRef.current) {
    document.addEventListener(
      "mousemove",
      handleMouseMove as EventListener,
    );
    document.addEventListener("mouseup", handleMouseUp);
    document.addEventListener(
      "touchmove",
      handleMouseMove as EventListener as EventListener,
      { passive: false },
    );
    document.addEventListener("touchend", handleMouseUp);
  }

  return () => {
    document.removeEventListener(
      "mousemove",
      handleMouseMove as EventListener,
    );
  };

```

```

document.removeEventListener("mouseup", handleMouseUp);
document.removeEventListener(
  "touchmove",
  handleMouseMove as EventListener,
);
document.removeEventListener("touchend", handleMouseUp);
};
}, [handleMouseMove, handleMouseUp]);

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLDivElement>) => {
    if (isActuallyDisabled) return;

    let newValue = clampedValue;
    let shouldUpdate = false;

    switch (e.key) {
      case "ArrowRight":
      case "ArrowUp":
        e.preventDefault();
        newValue = Math.min(clampedValue + effectiveStep, effectiveMax);
        shouldUpdate = true;
        break;
      case "ArrowLeft":
      case "ArrowDown":
        e.preventDefault();
        newValue = Math.max(clampedValue - effectiveStep, effectiveMin);
        shouldUpdate = true;
        break;
      case "Home":
        e.preventDefault();
        newValue = effectiveMin;
        shouldUpdate = true;
        break;
      case "End":
        e.preventDefault();
        newValue = effectiveMax;
        shouldUpdate = true;
        break;
      case "PageUp":
        e.preventDefault();
        newValue = Math.min(
          clampedValue + effectiveStep * 10,
          effectiveMax,
        );
        shouldUpdate = true;
        break;
      case "PageDown":
        e.preventDefault();
        newValue = Math.max(
          clampedValue - effectiveStep * 10,
          effectiveMin,
        );
        shouldUpdate = true;
        break;
    }

    if (shouldUpdate) {

```

```

        setInternalValue(newValue);
        notifyChange(newValue);
    }
},
[
    isActuallyDisabled,
    clampedValue,
    effectiveMin,
    effectiveMax,
    effectiveStep,
    notifyChange,
],
);

const handleTrackClick = useCallback(
    (e: React.MouseEvent<HTMLDivElement>) => {
        if (isActuallyDisabled) return;
        if (e.currentTarget === e.target) {
            updateValueFromPosition(e.clientX, e.clientY);
        }
    },
    [isActuallyDisabled, updateValueFromPosition],
);

const combinedRef = useCallback(
    (el: HTMLInputElement | null) => {
        if (ref) {
            if (typeof ref === "function") {
                ref(el);
            } else {
                ref.current = el;
            }
        }
    },
    [ref],
);

const trackClasses = cn(
    "relative",
    "w-full h-2",
    "rounded-full bg-border",
    "transition-colors duration-200 ease-out",
    "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
    "focus-visible:ring-offset-background",
    "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
    "aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20",
    hasError && "border-destructive focus-visible:ring-destructive",
    orientation === "vertical" && "w-2 h-full",
    className,
);

const progressClasses = cn(
    "absolute",
    "rounded-full",
    "bg-brand-primary",
    "transition-all duration-150 ease-out",
    orientation === "horizontal"
        ? "h-full left-0 top-0"

```

```

        : "w-full bottom-0 left-0",
    );

    const thumbClasses = cn(
        "absolute",
        "w-5 h-5",
        "rounded-full bg-white",
        "shadow-lg shadow-black/15",
        "border-2 border-brand-primary",
        "transition-all duration-150 ease-out",
        "active:scale-110",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-brand-primary focus-visible:ring-
offset-2 focus-visible:ring-offset-background",
        "hover:scale-110 hover:shadow-xl",
        "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
        "aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20",
        orientation === "horizontal"
            ? "top-1/2 -translate-y-1/2 -translate-x-1/2"
            : "left-1/2 -translate-x-1/2 -translate-y-1/2",
        focused &&
            "ring-2 ring-brand-primary ring-offset-2 ring-offset-background",
        hovered && "scale-110 shadow-xl",
    );

    const labelClasses = cn(
        "block text-sm font-medium leading-none",
        "transition-colors duration-200 ease-out",
        "text-text-main",
        hasError && "text-destructive/90",
        isActuallyDisabled && "opacity-50",
        "mb-2",
    );

    const valueDisplayClasses = cn(
        "absolute",
        "text-xs font-medium",
        "text-text-main",
        "transition-all duration-150 ease-out",
        "pointer-events-none",
        "white-space-nowrap",
        orientation === "horizontal"
            ? "bottom-full left-1/2 -translate-x-1/2 mb-1.5"
            : "right-full top-1/2 -translate-y-1/2 mr-2",
        isEnumMode && "min-w-[60px] text-center",
    );

    const containerClasses = cn(
        "inline-flex flex-col gap-2",
        orientation === "vertical" && "items-center",
        isActuallyDisabled && "opacity-50 pointer-events-none",
    );

    const errorClasses = cn(
        "text-sm",
        "text-destructive/90",
        "transition-colors duration-200 ease-out",
        "mt-1",
    );

```



```

return (
  <div
    className={containerClasses}
    role="group"
    aria-invalid={hasError}
    aria-disabled={isActuallyDisabled}
    onContextMenu={(e) => popupMenu?.trigger(e)}
  >
    {hint && (
      <HintBox content={hint} className="mb-1.5">
        <span className="text-sm text-text-muted" />
      </HintBox>
    )}
    {label && <label className={labelClasses}>{label}</label>}

    <div
      ref={sliderRef}
      className={cn(
        "relative",
        orientation === "horizontal" ? "w-full" : "h-64",
      )}
      onMouseDown={handleMouseDown}
      onTouchStart={
        handleMouseDown as React.TouchEventHandler<HTMLDivElement>
      }
      onKeyDown={handleKeyDown}
      onMouseEnter={() => setHovered(true)}
      onMouseLeave={() => setHovered(false)}
      onFocus={() => setFocused(true)}
      onBlur={() => setFocused(false)}
      tabIndex={isActuallyDisabled ? -1 : 0}
      role="slider"
      aria-label={label}
      aria-valuemin={effectiveMin}
      aria-valuemax={effectiveMax}
      aria-valuenow={clampedValue}
      aria-valuetext={String(displayValue)}
      aria-orientation={orientation}
      aria-disabled={isActuallyDisabled}
      aria-invalid={hasError ? "true" : "false"}
      aria-required={props.required}
    >
      <div
        ref={trackRef}
        className={trackClasses}
        onClick={handleTrackClick}
        role="presentation"
        aria-hidden="true"
      >
        <div
          className={progressClasses}
          style={
            {
              orientation === "horizontal"
                ? { width: `${percentage}%` }
                : { height: `${percentage}%` }
            }
          }
          role="presentation"
        >

```

```

        aria-hidden="true"
      />
    <div
      ref={thumbRef}
      className={thumbClasses}
      style={
        orientation === "horizontal"
          ? { left: `${percentage}%` }
          : { bottom: `${percentage}%` }
      }
      role="presentation"
      aria-hidden="true"
    />
  </div>

  {showValue && (
    <div
      className={valueDisplayClasses}
      style={
        orientation === "horizontal"
          ? { left: `${percentage}%` }
          : { bottom: `${percentage}%` }
      }
      aria-hidden="true"
    >
      {displayValue}
    </div>
  )}
</div>

{error && (
  <p className={errorClasses} role="alert" aria-live="polite">
    {error}
  </p>
)}

<input
  ref={combinedRef}
  type="hidden"
  value={isEnumMode ? displayValue : clampedValue}
  disabled={isActuallyDisabled}
  required={props.required}
  name={props.name}
/>
</div>
);
},
);

Slider.displayName = "Slider";

```

```
FILE: switch.tsx
LOKASI: src\selector
```

```
"use client";

import React, { forwardRef, useRef } from "react";
import { cn } from "@lib/utils";
import { HintBox } from "../feedback/hint-box";
import type { PopupMenuConfig } from "../feedback/popup-menu";

// ðŸš! LOCAL TYPE ISOLATION GATEWAY
export interface SwitchProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "type"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  checked?: boolean;
  onChange?: (checked: boolean) => void;
}

export const Switch = forwardRef<HTMLInputElement, SwitchProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      checked,
      onChange,
      className,
      disabled,
      ...props
    },
    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const isChecked = checked ?? false;
    const hasError = !!error;
    const isActuallyDisabled = disabled ?? false;

    const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
      if (isActuallyDisabled) return;
      const newChecked = e.target.checked;
      onChange?.(newChecked);
    };

    const handleKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) => {
      if (e.key === " " || e.key === "Enter") {
        e.preventDefault();
        if (!isActuallyDisabled) {
          onChange?.(!isChecked);
        }
      }
    };
  }
);
```

```

    }
  };

  const switchTrackClasses = cn(
    "relative inline-flex items-center",
    "w-11 h-6 shrink-0",
    "rounded-full border-2",
    "border-border bg-background",
    "transition-all duration-200 ease-out",
    "active:scale-95 duration-150 transition-all",
    "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
    "focus-visible:ring-offset-background",
    "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed",
    "aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20",
    isChecked
      ? "bg-brand-primary border-brand-primary shadow-sm hover:shadow-md"
      : "hover:border-brand-primary/50 hover:bg-brand-primary/5",
    hasError && "border-destructive focus-visible:ring-destructive",
    className,
  );

  const thumbClasses = cn(
    "relative inline-block",
    "w-5 h-5 shrink-0",
    "rounded-full bg-white",
    "shadow-md shadow-black/10",
    "transition-transform duration-200 ease-spring",
    "active:scale-95",
    isChecked ? "translate-x-full" : "translate-x-0",
    hasError && "shadow-destructive/20",
  );

  const labelClasses = cn(
    "inline-flex items-center gap-2 cursor-pointer select-none",
    "transition-colors duration-200 ease-out",
    "text-text-main",
    hasError && "text-destructive/90",
    isActuallyDisabled && "opacity-50 pointer-events-none cursor-not-allowed",
  );

  return (
    <div className="w-full" onContextMenu={(e) => popupMenu?.trigger(e)}>
      {hint && (
        <HintBox content={hint} className="mb-1.5">
          <span className="text-sm text-text-muted" />
        </HintBox>
      )}
      <label className={labelClasses}>
        <input
          ref={(el) => {
            inputRef.current = el;
            if (ref) {
              if (typeof ref === "function") {
                ref(el);
              } else {
                ref.current = el;
              }
            }
          }}
        />
      </label>
    </div>
  );

```

```

    }}
    type="checkbox"
    role="switch"
    checked={isChecked}
    disabled={isActuallyDisabled}
    onChange={handleChange}
    onKeyDown={handleKeyDown}
    aria-checked={isChecked}
    aria-invalid={hasError ? "true" : "false"}
    aria-disabled={isActuallyDisabled}
    aria-required={props.required}
    className="sr-only peer"
    {...props}
  />
  <div className={switchTrackClasses} aria-hidden="true">
    <span className={thumbClasses} aria-hidden="true" />
  </div>
  {label && (
    <span
      className={cn(
        "text-sm font-medium leading-none",
        "transition-colors duration-200 ease-out",
        "text-text-main",
        hasError && "text-destructive/90",
        isActuallyDisabled && "opacity-50",
      )}
    >
      {label}
    </span>
  )}
</label>
</div>
);
},
);
Switch.displayName = "Switch";

```

FILE: button.tsx
LOKASI: src\ui

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@lib/utils"

const buttonVariants = cva(
  "group/button inline-flex shrink-0 items-center justify-center rounded-lg border border-transparent bg-clip-padding text-sm font-medium whitespace-nowrap transition-all outline-none select-none focus-visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/50 active:not-aria-[haspopup]:translate-y-pixel disabled:pointer-events-none disabled:opacity-50 aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20 dark:aria-invalid:border-destructive/50 dark:aria-invalid:ring-destructive/40 [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/80",
        outline:
          "border-border bg-background hover:bg-muted hover:text-foreground aria-expanded:bg-muted aria-expanded:text-foreground dark:border-input dark:bg-input/30 dark:hover:bg-input/50",
        secondary:
          "bg-secondary text-secondary-foreground hover:bg-[color-mix(in_oklch,var(--secondary),var(--foreground)_5%)] aria-expanded:bg-secondary aria-expanded:text-secondary-foreground",
        ghost:
          "hover:bg-muted hover:text-foreground aria-expanded:bg-muted aria-expanded:text-foreground dark:hover:bg-muted/50",
        destructive:
          "bg-destructive/10 text-destructive hover:bg-destructive/20 focus-visible:border-destructive/40 focus-visible:ring-destructive/20 dark:bg-destructive/20 dark:hover:bg-destructive/30 dark:focus-visible:ring-destructive/40",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default:
          "h-8 gap-1.5 px-2.5 has-data-[icon=inline-end]:pr-2 has-data-[icon=inline-start]:pl-2",
        xs: "h-6 gap-1 rounded-[min(var(--radius-md),10px)] px-2 text-xs in-data-[slot=button-group]:rounded-lg has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 [&_svg:not([class*='size-'])]:size-3",
        sm: "h-7 gap-1 rounded-[min(var(--radius-md),12px)] px-2.5 text-[0.8rem] in-data-[slot=button-group]:rounded-lg has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 [&_svg:not([class*='size-'])]:size-3.5",
        lg: "h-9 gap-1.5 px-2.5 has-data-[icon=inline-end]:pr-2 has-data-[icon=inline-start]:pl-2",
        icon: "size-8",
        "icon-xs":
          "size-6 rounded-[min(var(--radius-md),10px)] in-data-[slot=button-group]:rounded-lg [&_svg:not([class*='size-'])]:size-3",
        "icon-sm":
          "size-7 rounded-[min(var(--radius-md),12px)] in-data-[slot=button-group]:rounded-lg",
        "icon-lg": "size-9",
      },
    },
  },
)
```

```

    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function Button({
  className,
  variant = "default",
  size = "default",
  asChild = false,
  ...props
}: React.ComponentProps<"button"> &
  VariantProps<typeof buttonVariants> & {
    asChild?: boolean
  }) {
  const Comp = asChild ? Slot.Root : "button"

  return (
    <Comp
      data-slot="button"
      data-variant={variant}
      data-size={size}
      className={cn(buttonVariants({ variant, size, className })))}
      {...props}
    />
  )
}

export { Button, buttonVariants }

```

LOKASI: src\ui

```
import * as React from "react"
```

```
import {
  DatePicker,
  getDefaultClassNames,
  type DayButton,
  type Locale,
} from "react-day-picker"
```

```
import { cn } from "@lib/utls"
import { Button, buttonVariants } from "@components/ui/button"
import { ChevronLeftIcon, ChevronRightIcon, ChevronDownIcon } from "lucide-react"
```

280/308


```

month: cn("flex w-full flex-col gap-4", defaultClassNames.month),
nav: cn(
  "absolute inset-x-0 top-0 flex w-full items-center justify-between gap-1",
  defaultClassNames.nav
),
button_previous: cn(
  buttonVariants({ variant: buttonVariant }),
  "size-(--cell-size) p-0 select-none aria-disabled:opacity-50",
  defaultClassNames.button_previous
),
button_next: cn(
  buttonVariants({ variant: buttonVariant }),
  "size-(--cell-size) p-0 select-none aria-disabled:opacity-50",
  defaultClassNames.button_next
),
month_caption: cn(
  "flex h-(--cell-size) w-full items-center justify-center px-(--cell-size)",
  defaultClassNames.month_caption
),
dropdowns: cn(
  "flex h-(--cell-size) w-full items-center justify-center gap-1.5 text-sm font-medium",
  defaultClassNames.dropdowns
),
dropdown_root: cn(
  "relative rounded-(--cell-radius)",
  defaultClassNames.dropdown_root
),
dropdown: cn(
  "absolute inset-0 bg-popover opacity-0",
  defaultClassNames.dropdown
),
caption_label: cn(
  "font-medium select-none",
  captionLayout === "label"
    ? "text-sm"
    : "flex items-center gap-1 rounded-(--cell-radius) text-sm [&svg]:size-3.5 [&svg]:text-
muted-foreground",
  defaultClassNames.caption_label
),
month_grid: cn("w-full border-collapse", defaultClassNames.month_grid),
weekdays: cn("flex", defaultClassNames.weekdays),
weekday: cn(
  "flex-1 rounded-(--cell-radius) text-[0.8rem] font-normal text-muted-foreground select-none",
  defaultClassNames.weekday
),
week: cn("mt-2 flex w-full", defaultClassNames.week),
week_number_header: cn(
  "w-(--cell-size) select-none",
  defaultClassNames.week_number_header
),
week_number: cn(
  "text-[0.8rem] text-muted-foreground select-none",
  defaultClassNames.week_number
),
day: cn(
  "group/day relative aspect-square h-full w-full rounded-(--cell-radius) p-0 text-center
select-none [&:last-child[data-selected=true]_button]:rounded-r-(--cell-radius)",
  props.showWeekNumber

```

```

      ? "[&:nth-child(2)[data-selected=true]_button]:rounded-l(--cell-radius)"
      : "[&:first-child[data-selected=true]_button]:rounded-l(--cell-radius)",
      defaultClassNames.day
    ),
    range_start: cn(
      "relative isolate z-0 rounded-l(--cell-radius) bg-muted after:absolute after:inset-y-0
after:right-0 after:w-4 after:bg-muted",
      defaultClassNames.range_start
    ),
    range_middle: cn("rounded-none", defaultClassNames.range_middle),
    range_end: cn(
      "relative isolate z-0 rounded-r(--cell-radius) bg-muted after:absolute after:inset-y-0
after:left-0 after:w-4 after:bg-muted",
      defaultClassNames.range_end
    ),
    today: cn(
      "rounded(--cell-radius) bg-muted text-foreground data-[selected=true]:rounded-none",
      defaultClassNames.today
    ),
    outside: cn(
      "text-muted-foreground aria-selected:text-muted-foreground",
      defaultClassNames.outside
    ),
    disabled: cn(
      "text-muted-foreground opacity-50",
      defaultClassNames.disabled
    ),
    hidden: cn("invisible", defaultClassNames.hidden),
    ...classNames,
  })
}
}
components={{
  Root: ({ className, rootRef, ...props }) => {
    return (
      <div
        data-slot="calendar"
        ref={rootRef}
        className={cn(className)}
        {...props}
      />
    )
  },
  Chevron: ({ className, orientation, ...props }) => {
    if (orientation === "left") {
      return (
        <ChevronLeftIcon className={cn("size-4", className)} {...props} />
      )
    }

    if (orientation === "right") {
      return (
        <ChevronRightIcon className={cn("size-4", className)} {...props} />
      )
    }

    return (
      <ChevronDownIcon className={cn("size-4", className)} {...props} />
    )
  },

```

```

    DayButton: ({ ...props }) => (
      <CalendarDayButton locale={locale} {...props} />
    ),
    WeekNumber: ({ children, ...props }) => {
      return (
        <td {...props}>
          <div className="flex size-(--cell-size) items-center justify-center text-center">
            {children}
          </div>
        </td>
      )
    },
    ...components,
  }}
  {...props}
/>
)
}

function CalendarDayButton({
  className,
  day,
  modifiers,
  locale,
  ...props
}): React.ComponentProps<typeof DayButton> & { locale?: Partial<Locale> } {
  const defaultClassNames = getDefaultClassNames()

  const ref = React.useRef<HTMLButtonElement>(null)
  React.useEffect(() => {
    if (modifiers.focused) ref.current?.focus()
  }, [modifiers.focused])

  return (
    <Button
      ref={ref}
      variant="ghost"
      size="icon"
      data-day={day.date.toLocaleDateString(locale?.code)}
      data-selected-single={
        modifiers.selected &&
        !modifiers.range_start &&
        !modifiers.range_end &&
        !modifiers.range_middle
      }
      data-range-start={modifiers.range_start}
      data-range-end={modifiers.range_end}
      data-range-middle={modifiers.range_middle}
      className={cn(
        "relative isolate z-10 flex aspect-square size-auto w-full min-w-(--cell-size) flex-col gap-1
border-0 leading-none font-normal group-data-[focused=true]/day:relative group-data-
[focused=true]/day:z-10 group-data-[focused=true]/day:border-ring group-data-[focused=true]/day:ring-
[3px] group-data-[focused=true]/day:ring-ring/50 data-[range-end=true]:rounded-(--cell-radius) data-
[range-end=true]:rounded-r-(--cell-radius) data-[range-end=true]:bg-primary data-[range-end=true]:text-
primary-foreground data-[range-middle=true]:rounded-none data-[range-middle=true]:bg-muted data-[range-
middle=true]:text-foreground data-[range-start=true]:rounded-(--cell-radius) data-[range-
start=true]:rounded-l-(--cell-radius) data-[range-start=true]:bg-primary data-[range-start=true]:text-
primary-foreground data-[selected-single=true]:bg-primary data-[selected-single=true]:text-primary-

```

```
    foreground dark: hover: text-foreground [&>span]:text-xs [&>span]:opacity-70",
    defaultClassNames.day,
    className
  )}
  {...props}
/>
)
}

export { Calendar, CalendarDayButton }
```

FILE: checkbox.tsx**LOKASI:** src\ui

```
"use client"
```

```
import * as React from "react"
```

```
import { Checkbox as CheckboxPrimitive } from "radix-ui"
```

```
import { cn } from "@/lib/utils"
```

```
import { CheckIcon } from "lucide-react"
```

```
function Checkbox({
```

```
  className,
```

```
  ...props
```

```
}: React.ComponentProps<typeof CheckboxPrimitive.Root>) {
```

```
  return (
```

```
    <CheckboxPrimitive.Root
```

```
      data-slot="checkbox"
```

```
      className={cn(
```

```
        "peer relative flex size-4 shrink-0 items-center justify-center rounded-[4px] border border-input transition-colors outline-none group-has-disabled/field:opacity-50 after:absolute after:-inset-x-3 after:-inset-y-2 focus-visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/50 disabled:cursor-not-allowed disabled:opacity-50 aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20 aria-invalid:aria-checked:border-primary dark:bg-input/30 dark:aria-invalid:border-destructive/50 dark:aria-invalid:ring-destructive/40 data-checked:border-primary data-checked:bg-primary data-checked:text-primary-foreground dark:data-checked:bg-primary",
```

```
        className
```

```
      ))
```

```
      {...props}
```

```
    >
```

```
      <CheckboxPrimitive.Indicator
```

```
        data-slot="checkbox-indicator"
```

```
        className="grid place-content-center text-current transition-none [&>svg]:size-3.5"
```

```
      >
```

```
        <CheckIcon
```

```
        />
```

```
      </CheckboxPrimitive.Indicator>
```

```
    </CheckboxPrimitive.Root>
```

```
  )
```

```
}
```

```
export { Checkbox }
```

FILE: command.tsx
LOKASI: src\ui

"use client"

```
import * as React from "react"
import { Command as CommandPrimitive } from "cmdk"

import { cn } from "@lib/Utils"
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog"
import {
  InputGroup,
  InputGroupAddon,
} from "@components/ui/input-group"
import { SearchIcon, CheckIcon } from "lucide-react"

function Command({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive>) {
  return (
    <CommandPrimitive
      data-slot="command"
      className={cn(
        "flex size-full flex-col overflow-hidden rounded-xl bg-popover p-1 text-popover-foreground",
        className
      )}
      {...props}
    />
  )
}

function CommandDialog({
  title = "Command Palette",
  description = "Search for a command to run...",
  children,
  className,
  showCloseButton = false,
  ...props
}: React.ComponentProps<typeof Dialog> & {
  title?: string
  description?: string
  className?: string
  showCloseButton?: boolean
}) {
  return (
    <Dialog {...props}>
      <DialogHeader className="sr-only">
        <DialogTitle>{title}</DialogTitle>
      </DialogHeader>
    </Dialog>
  )
}
```

```

    <DialogDescription>{description}</DialogDescription>
  </DialogHeader>
  <DialogContent
    className={cn(
      "top-1/3 translate-y-0 overflow-hidden rounded-xl! p-0",
      className
    )}
    showCloseButton={showCloseButton}
  >
    {children}
  </DialogContent>
</Dialog>
)
}

function CommandInput({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Input>) {
  return (
    <div data-slot="command-input-wrapper" className="p-1 pb-0">
      <InputGroup className="h-8! rounded-lg! border-input/30 bg-input/30 shadow-none! *:data-
[slot=input-group-addon]:pl-2!">
        <CommandPrimitive.Input
          data-slot="command-input"
          className={cn(
            "w-full text-sm outline-hidden disabled:cursor-not-allowed disabled:opacity-50",
            className
          )}
          {...props}
        />
        <InputGroupAddon>
          <SearchIcon className="size-4 shrink-0 opacity-50" />
        </InputGroupAddon>
      </InputGroup>
    </div>
  )
}

function CommandList({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.List>) {
  return (
    <CommandPrimitive.List
      data-slot="command-list"
      className={cn(
        "no-scrollbar max-h-72 scroll-py-1 overflow-x-hidden overflow-y-auto outline-none",
        className
      )}
      {...props}
    />
  )
}

function CommandEmpty({
  className,
  ...props

```

```

    }: React.ComponentProps<typeof CommandPrimitive.Empty>) {
      return (
        <CommandPrimitive.Empty
          data-slot="command-empty"
          className={cn("py-6 text-center text-sm", className)}
          {...props}
        />
      )
    }
  }

function CommandGroup({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Group>) {
  return (
    <CommandPrimitive.Group
      data-slot="command-group"
      className={cn(
        "overflow-hidden p-1 text-foreground **:[[cmdk-group-heading]]:px-2 **:[[cmdk-group-
heading]]:py-1.5 **:[[cmdk-group-heading]]:text-xs **:[[cmdk-group-heading]]:font-medium **:[[cmdk-
group-heading]]:text-muted-foreground",
        className
      )}
      {...props}
    />
  )
}

function CommandSeparator({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Separator>) {
  return (
    <CommandPrimitive.Separator
      data-slot="command-separator"
      className={cn("-mx-1 h-px bg-border", className)}
      {...props}
    />
  )
}

function CommandItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Item>) {
  return (
    <CommandPrimitive.Item
      data-slot="command-item"
      className={cn(
        "group/command-item relative flex cursor-default items-center gap-2 rounded-sm px-2 py-1.5
text-sm outline-hidden select-none in-data-[slot=dialog-content]:rounded-lg! data-
[disabled=true]:pointer-events-none data-[disabled=true]:opacity-50 data-selected:bg-muted data-
selected:text-foreground [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-
'])]:size-4 data-selected:*:[svg]:text-foreground",
        className
      )}
      {...props}
    />
  )
}

```



```

    >
    {children}
    <CheckIcon className="ml-auto opacity-0 group-has-data-[slot=command-shortcut]/command-
item:hidden group-data-[checked=true]/command-item:opacity-100" />
    </CommandPrimitive.Item>
  )
}

function CommandShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="command-shortcut"
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground group-data-selected/command-item:text-
foreground",
        className
      )}
      {...props}
    />
  )
}

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
}

```

FILE: dialog.tsx
LOKASI: src\ui

```
import * as React from "react"
import { Dialog as DialogPrimitive } from "radix-ui"

import { cn } from "@lib/utils"
import { Button } from "@components/ui/button"
import { XIcon } from "lucide-react"

function Dialog({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Root>) {
  return <DialogPrimitive.Root data-slot="dialog" {...props} />
}

function DialogTrigger({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Trigger>) {
  return <DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} />
}

function DialogPortal({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Portal>) {
  return <DialogPrimitive.Portal data-slot="dialog-portal" {...props} />
}

function DialogClose({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Close>) {
  return <DialogPrimitive.Close data-slot="dialog-close" {...props} />
}

function DialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Overlay>) {
  return (
    <DialogPrimitive.Overlay
      data-slot="dialog-overlay"
      className={cn(
        "fixed inset-0 isolate z-50 bg-black/10 duration-100 supports-backdrop-filter:backdrop-blur-xs
data-open:animate-in data-open:fade-in-0 data-closed:animate-out data-closed:fade-out-0",
        className
      )}
      {...props}
    />
  )
}

function DialogContent({
  className,
  children,
  showCloseButton = true,
```

```

    ...props
  }: React.ComponentProps<typeof DialogPrimitive.Content> & {
    showCloseButton?: boolean
  }) {
    return (
      <DialogPortal>
        <DialogOverlay />
        <DialogPrimitive.Content
          data-slot="dialog-content"
          className={cn(
            "fixed top-1/2 left-1/2 z-50 grid w-full max-w-[calc(100%-2rem)] -translate-x-1/2 -translate-y-1/2 gap-4 rounded-xl bg-popover p-4 text-sm text-popover-foreground ring-1 ring-foreground/10 duration-100 outline-none sm:max-w-sm data-open:animate-in data-open:fade-in-0 data-open:zoom-in-95 data-closed:animate-out data-closed:fade-out-0 data-closed:zoom-out-95",
            className
          )}
        >
          {children}
          {showCloseButton && (
            <DialogPrimitive.Close data-slot="dialog-close" asChild>
              <Button
                variant="ghost"
                className="absolute top-2 right-2"
                size="icon-sm"
              >
                <XIcon />
                <span className="sr-only">Close</span>
              </Button>
            </DialogPrimitive.Close>
          )}
        </DialogPrimitive.Content>
      </DialogPortal>
    )
  }

```

```

function DialogHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="dialog-header"
      className={cn("flex flex-col gap-2", className)}
      {...props}
    />
  )
}

```

```

function DialogFooter({
  className,
  showCloseButton = false,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  showCloseButton?: boolean
}) {
  return (
    <div
      data-slot="dialog-footer"

```

```

        className={cn(
            "-mx-4 -mb-4 flex flex-col-reverse gap-2 rounded-b-xl border-t bg-muted/50 p-4 sm:flex-row
sm:justify-end",
            className
        )}
        {...props}
    >
        {children}
        {showCloseButton && (
            <DialogPrimitive.Close asChild>
                <Button variant="outline">Close</Button>
            </DialogPrimitive.Close>
        )}
    </div>
)
}

```

```

function DialogTitle({
    className,
    ...props
}): React.ComponentProps<typeof DialogPrimitive.Title> {
    return (
        <DialogPrimitive.Title
            data-slot="dialog-title"
            className={cn(
                "font-heading text-base leading-none font-medium",
                className
            )}
            {...props}
        />
    )
}

```

```

function DialogDescription({
    className,
    ...props
}): React.ComponentProps<typeof DialogPrimitive.Description> {
    return (
        <DialogPrimitive.Description
            data-slot="dialog-description"
            className={cn(
                "text-sm text-muted-foreground *:[a]:underline *:[a]:underline-offset-3 *:[a]:hover:text-
foreground",
                className
            )}
            {...props}
        />
    )
}

```

```

export {
    Dialog,
    DialogClose,
    DialogContent,
    DialogDescription,
    DialogFooter,
    DialogHeader,
    DialogOverlay,
}

```

7/9/26, 3:29 PM

```
DialogPortal,  
DialogTitle,  
DialogTrigger,  
}
```

FILE: input-group.tsx
LOKASI: src\ui

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"
import { Button } from "@components/ui/button"
import { Input } from "@components/ui/input"
import { Textarea } from "@components/ui/textarea"

function InputGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="input-group"
      role="group"
      className={cn(
        "group/input-group relative flex h-8 w-full min-w-0 items-center rounded-lg border border-input
        transition-colors outline-none in-data-[slot=combobox-content]:focus-within:border-inherit in-data-
        [slot=combobox-content]:focus-within:ring-0 has-disabled:bg-input/50 has-disabled:opacity-50 has-
        [[data-slot=input-group-control]:focus-visible]:border-ring has-[[data-slot=input-group-control]:focus-
        visible]:ring-3 has-[[data-slot=input-group-control]:focus-visible]:ring-ring/50 has-[[data-slot][aria-
        invalid=true]]:border-destructive has-[[data-slot][aria-invalid=true]]:ring-3 has-[[data-slot][aria-
        invalid=true]]:ring-destructive/20 has-[[>[data-align=block-end]]:h-auto has-[[>[data-align=block-
        end]]:flex-col has-[[>[data-align=block-start]]:h-auto has-[[>[data-align=block-start]]:flex-col has-
        [[>textarea]:h-auto dark:bg-input/30 dark:has-disabled:bg-input/80 dark:has-[[data-slot][aria-
        invalid=true]]:ring-destructive/40 has-[[>[data-align=block-end]]:[&>input]:pt-3 has-[[>[data-
        align=block-start]]:[&>input]:pb-3 has-[[>[data-align=inline-end]]:[&>input]:pr-1.5 has-[[>[data-
        align=inline-start]]:[&>input]:pl-1.5",
        className
      )}
    >
      {...props}
    </div>
  )
}

const inputGroupAddonVariants = cva(
  "flex h-auto cursor-text items-center justify-center gap-2 py-1.5 text-sm font-medium text-muted-
  foreground select-none group-data-[disabled=true]/input-group:opacity-50 [&>kbd]:rounded-[calc(var(--
  radius)-5px)] [&>svg:not([class*='size-'])]:size-4",
  {
    variants: {
      align: {
        "inline-start":
          "order-first pl-2 has-[[>button]:ml-[-0.3rem] has-[[>kbd]:ml-[-0.15rem]",
        "inline-end":
          "order-last pr-2 has-[[>button]:mr-[-0.3rem] has-[[>kbd]:mr-[-0.15rem]",
        "block-start":
          "order-first w-full justify-start px-2.5 pt-2 group-has-[[>input]/input-group:pt-2 [.border-
          b]:pb-2",
        "block-end":
          "order-last w-full justify-start px-2.5 pb-2 group-has-[[>input]/input-group:pb-2 [.border-
          t]:pt-2",
      },
    },
  },
)
```

```

        defaultVariants: {
            align: "inline-start",
        },
    }
)

function InputGroupAddon({
    className,
    align = "inline-start",
    ...props
}: React.ComponentProps<"div"> & VariantProps<typeof inputGroupAddonVariants>) {
    return (
        <div
            role="group"
            data-slot="input-group-addon"
            data-align={align}
            className={cn(inputGroupAddonVariants({ align })), className}
            onClick={(e) => {
                if ((e.target as HTMLElement).closest("button")) {
                    return
                }
                e.currentTarget.parentElement?.querySelector("input")?.focus()
            }}
            {...props}
        />
    )
}

const inputGroupButtonVariants = cva(
    "flex items-center gap-2 text-sm shadow-none",
    {
        variants: {
            size: {
                xs: "h-6 gap-1 rounded-[calc(var(--radius)-3px)] px-1.5 [&>svg:not([class*='size-'])]:size-3.5",
                sm: "",
                "icon-xs":
                    "size-6 rounded-[calc(var(--radius)-3px)] p-0 has-[>svg]:p-0",
                "icon-sm": "size-8 p-0 has-[>svg]:p-0",
            },
        },
        defaultVariants: {
            size: "xs",
        },
    }
)

function InputGroupButton({
    className,
    type = "button",
    variant = "ghost",
    size = "xs",
    ...props
}: Omit<React.ComponentProps<typeof Button>, "size"> &
VariantProps<typeof inputGroupButtonVariants>) {
    return (
        <Button
            type={type}

```

```

        data-size={size}
        variant={variant}
        className={cn(inputGroupButtonVariants({ size }), className)}
        {...props}
      />
    )
  }

function InputGroupText({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      className={cn(
        "flex items-center gap-2 text-sm text-muted-foreground [&_svg]:pointer-events-none
[&_svg:not([class*='size-'])]:size-4",
        className
      )}
      {...props}
    />
  )
}

function InputGroupInput({
  className,
  ...props
}: React.ComponentProps<"input">) {
  return (
    <Input
      data-slot="input-group-control"
      className={cn(
        "flex-1 rounded-none border-0 bg-transparent shadow-none ring-0 focus-visible:ring-0
disabled:bg-transparent aria-invalid:ring-0 dark:bg-transparent dark:disabled:bg-transparent",
        className
      )}
      {...props}
    />
  )
}

function InputGroupTextarea({
  className,
  ...props
}: React.ComponentProps<"textarea">) {
  return (
    <Textarea
      data-slot="input-group-control"
      className={cn(
        "flex-1 resize-none rounded-none border-0 bg-transparent py-2 shadow-none ring-0 focus-
visible:ring-0 disabled:bg-transparent aria-invalid:ring-0 dark:bg-transparent dark:disabled:bg-
transparent",
        className
      )}
      {...props}
    />
  )
}

export {
  InputGroup,

```


7/9/26, 3:29 PM

```
    InputGroupAddon,  
    InputGroupButton,  
    InputGroupText,  
    InputGroupInput,  
    InputGroupTextarea,  
}
```

FILE: input.tsx
LOKASI: src\ui

```
import * as React from "react"

import { cn } from "@lib/utils"

function Input({ className, type, ...props }: React.ComponentProps<"input">) {
  return (
    <input
      type={type}
      data-slot="input"
      className={cn(
        "h-8 w-full min-w-0 rounded-lg border border-input bg-transparent px-2.5 py-1 text-base
        transition-colors outline-none file:inline-flex file:h-6 file:border-0 file:bg-transparent file:text-sm
        file:font-medium file:text-foreground placeholder:text-muted-foreground focus-visible:border-ring
        focus-visible:ring-3 focus-visible:ring-ring/50 disabled:pointer-events-none disabled:cursor-not-
        allowed disabled:bg-input/50 disabled:opacity-50 aria-invalid:border-destructive aria-invalid:ring-3
        aria-invalid:ring-destructive/20 md:text-sm dark:bg-input/30 dark:disabled:bg-input/80 dark:aria-
        invalid:border-destructive/50 dark:aria-invalid:ring-destructive/40",
        className
      )}
      {...props}
    />
  )
}

export { Input }
```

FILE: label.tsx
LOKASI: src\ui

```
import * as React from "react"
import { Label as LabelPrimitive } from "radix-ui"

import { cn } from "@lib/utils"

function Label({
  className,
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root>) {
  return (
    <LabelPrimitive.Root
      data-slot="label"
      className={cn(
        "flex items-center gap-2 text-sm leading-none font-medium select-none group-data-
[disabled=true]:pointer-events-none group-data-[disabled=true]:opacity-50 peer-disabled:cursor-not-
allowed peer-disabled:opacity-50",
        className
      )}
      {...props}
    />
  )
}

export { Label }
```

FILE: popover.tsx

LOKASI: src\ui

```
import * as React from "react"
import { Popover as PopoverPrimitive } from "radix-ui"

import { cn } from "@lib/utils"

function Popover({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Root>) {
  return <PopoverPrimitive.Root data-slot="popover" {...props} />
}

function PopoverTrigger({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Trigger>) {
  return <PopoverPrimitive.Trigger data-slot="popover-trigger" {...props} />
}

function PopoverContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Content>) {
  return (
    <PopoverPrimitive.Portal>
      <PopoverPrimitive.Content
        data-slot="popover-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "z-50 flex w-72 origin-(--radix-popover-content-transform-origin) flex-col gap-2.5 rounded-lg
bg-popover p-2.5 text-sm text-popover-foreground shadow-md ring-1 ring-foreground/10 outline-hidden
duration-100 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-
[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 data-open:animate-in data-
open:fade-in-0 data-open:zoom-in-95 data-closed:animate-out data-closed:fade-out-0 data-closed:zoom-
out-95",
          className
        )}
        {...props}
      />
    </PopoverPrimitive.Portal>
  )
}

function PopoverAnchor({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Anchor>) {
  return <PopoverPrimitive.Anchor data-slot="popover-anchor" {...props} />
}

function PopoverHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
```

```

    <div
      data-slot="popover-header"
      className={cn("flex flex-col gap-0.5 text-sm", className)}
      {...props}
    />
  )
}

function PopoverTitle({ className, ...props }: React.ComponentProps<"h2">) {
  return (
    <div
      data-slot="popover-title"
      className={cn("font-medium", className)}
      {...props}
    />
  )
}

function PopoverDescription({
  className,
  ...props
}: React.ComponentProps<"p">) {
  return (
    <p
      data-slot="popover-description"
      className={cn("text-muted-foreground", className)}
      {...props}
    />
  )
}

export {
  Popover,
  PopoverAnchor,
  PopoverContent,
  PopoverDescription,
  PopoverHeader,
  PopoverTitle,
  PopoverTrigger,
}

```

FILE: skeleton.tsx**LOKASI:** src\ui

```
import { cn } from "@lib/utils"

function Skeleton({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="skeleton"
      className={cn("animate-pulse rounded-md bg-muted", className)}
      {...props}
    />
  )
}

export { Skeleton }
```

FILE: slider.tsx

LOKASI: src\ui

```
import * as React from "react"
import { Slider as SliderPrimitive } from "radix-ui"

import { cn } from "@lib/Utils"

function Slider({
  className,
  defaultValue,
  value,
  min = 0,
  max = 100,
  ...props
}: React.ComponentProps<typeof SliderPrimitive.Root>) {
  const _values = React.useMemo(
    () =>
      Array.isArray(value)
        ? value
        : Array.isArray(defaultValue)
          ? defaultValue
          : [min, max],
    [value, defaultValue, min, max]
  )

  return (
    <SliderPrimitive.Root
      data-slot="slider"
      defaultValue={defaultValue}
      value={value}
      min={min}
      max={max}
      className={cn(
        "relative flex w-full touch-none items-center select-none data-disabled:opacity-50 data-
vertical:h-full data-vertical:min-h-40 data-vertical:w-auto data-vertical:flex-col",
        className
      )}
    >
      <SliderPrimitive.Track
        data-slot="slider-track"
        className="relative grow overflow-hidden rounded-full bg-muted data-horizontal:h-1 data-
horizontal:w-full data-vertical:h-full data-vertical:w-1"
      >
        <SliderPrimitive.Range
          data-slot="slider-range"
          className="absolute bg-primary select-none data-horizontal:h-full data-vertical:w-full"
        />
      </SliderPrimitive.Track>
      {Array.from({ length: _values.length }, (_, index) => (
        <SliderPrimitive.Thumb
          data-slot="slider-thumb"
          key={index}
          className="relative block size-3 shrink-0 rounded-full border border-ring bg-white ring-
```

```
ring/50 transition-[color,box-shadow] select-none after:absolute after:-inset-2 hover:ring-3 focus-
visible:ring-3 focus-visible:outline-hidden active:ring-3 disabled:pointer-events-none
disabled:opacity-50"
    />
  )}}
</SliderPrimitive.Root>
)
}

export { Slider }
```


FILE: switch.tsx
LOKASI: src\ui

"use client"

```
import * as React from "react"
import { Switch as SwitchPrimitive } from "radix-ui"

import { cn } from "@lib/utils"

function Switch({
  className,
  size = "default",
  ...props
}: React.ComponentProps<typeof SwitchPrimitive.Root> & {
  size?: "sm" | "default"
}) {
  return (
    <SwitchPrimitive.Root
      data-slot="switch"
      data-size={size}
      className={cn(
        "peer group/switch relative inline-flex shrink-0 items-center rounded-full border border-
transparent transition-all outline-none after:absolute after:-inset-x-3 after:-inset-y-2 focus-
visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/50 aria-invalid:border-destructive
aria-invalid:ring-3 aria-invalid:ring-destructive/20 data-[size=default]:h-[18.4px] data-
[size=default]:w-[32px] data-[size=sm]:h-[14px] data-[size=sm]:w-[24px] dark:aria-invalid:border-
destructive/50 dark:aria-invalid:ring-destructive/40 data-checked:bg-primary data-unchecked:bg-input
dark:data-unchecked:bg-input/80 data-disabled:cursor-not-allowed data-disabled:opacity-50",
        className
      )}
      {...props}
    >
      <SwitchPrimitive.Thumb
        data-slot="switch-thumb"
        className="pointer-events-none block rounded-full bg-background ring-0 transition-transform
group-data-[size=default]/switch:size-4 group-data-[size=sm]/switch:size-3 group-data-
[size=default]/switch:data-checked:translate-x-[calc(100%-2px)] group-data-[size=sm]/switch:data-
checked:translate-x-[calc(100%-2px)] dark:data-checked:bg-primary-foreground group-data-
[size=default]/switch:data-unchecked:translate-x-0 group-data-[size=sm]/switch:data-
unchecked:translate-x-0 dark:data-unchecked:bg-foreground"
      />
    </SwitchPrimitive.Root>
  )
}

export { Switch }
```

FILE: textarea.tsx**LOKASI:** src\ui

```

import * as React from "react"

import { cn } from "@lib/utils"

function Textarea({ className, ...props }: React.ComponentProps<"textarea">) {
  return (
    <textarea
      data-slot="textarea"
      className={cn(
        "flex field-sizing-content min-h-16 w-full rounded-lg border border-input bg-transparent px-2.5
py-2 text-base transition-colors outline-none placeholder:text-muted-foreground focus-visible:border-
ring focus-visible:ring-3 focus-visible:ring-ring/50 disabled:cursor-not-allowed disabled:bg-input/50
disabled:opacity-50 aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-
destructive/20 md:text-sm dark:bg-input/30 dark:disabled:bg-input/80 dark:aria-invalid:border-
destructive/50 dark:aria-invalid:ring-destructive/40",
        className
      )}
      {...props}
    />
  )
}

export { Textarea }

```

```
FILE: tooltip.tsx
LOKASI: src\ui
```

```
"use client"
```

```
import * as React from "react"
```

```
import { Tooltip as TooltipPrimitive } from "radix-ui"
```

```
import { cn } from "@/lib/utils"
```

```
function TooltipProvider({
  delayDuration = 0,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Provider>) {
  return (
    <TooltipPrimitive.Provider
      data-slot="tooltip-provider"
      delayDuration={delayDuration}
      {...props}
    />
  )
}
```

```
function Tooltip({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Root>) {
  return <TooltipPrimitive.Root data-slot="tooltip" {...props} />
}
```

```
function TooltipTrigger({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Trigger>) {
  return <TooltipPrimitive.Trigger data-slot="tooltip-trigger" {...props} />
}
```

```
function TooltipContent({
  className,
  sideOffset = 0,
  children,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Content>) {
  return (
    <TooltipPrimitive.Portal>
      <TooltipPrimitive.Content
        data-slot="tooltip-content"
        sideOffset={sideOffset}
        className={cn(
          "z-50 inline-flex w-fit max-w-xs origin(--radix-tooltip-content-transform-origin) items-
center gap-1.5 rounded-md bg-foreground px-3 py-1.5 text-xs text-background has-data-[slot=kbd]:pr-1.5
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-
in-from-left-2 data-[side=top]:slide-in-from-bottom-2 **:data-[slot=kbd]:relative **:data-
[slot=kbd]:isolate **:data-[slot=kbd]:z-50 **:data-[slot=kbd]:rounded-sm data-[state=delayed-
open]:animate-in data-[state=delayed-open]:fade-in-0 data-[state=delayed-open]:zoom-in-95 data-
open:animate-in data-open:fade-in-0 data-open:zoom-in-95 data-closed:animate-out data-closed:fade-out-0
data-closed:zoom-out-95",

```

```
        className
      )}
      {...props}
    >
      {children}
      <TooltipPrimitive.Arrow className="z-50 size-2.5 translate-y-[calc(-50%_-_2px)] rotate-45
rounded-[2px] bg-foreground fill-foreground" />
    </TooltipPrimitive.Content>
  </TooltipPrimitive.Portal>
)
}

export { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger }
```